



Visa Digital Solutions

API Reference Guide

Effective: 26 September 2023

Version 23.09

Important Note on Confidentiality and Copyright

The Visa Confidential label signifies that the information in this document is confidential and proprietary to Visa and is intended for use only by Visa Clients subject to the confidentiality restrictions in Visa's Operating Regulations, non-Client Third Party Processors that have an executed and valid Exhibit K on file with Visa, and other third parties that have a current nondisclosure agreement (NDA) or other agreement with Visa that covers disclosure of the information contained herein. This document is protected by copyright restricting its use, copying, distribution, and decompilation. No part of this document may be reproduced in any form by any means without prior written authorization of Visa. Visa and other trademarks are registered trademarks of Visa International Service Association, and are used under license by Visa. All other product names mentioned herein are the trademarks of their respective owners.

THIS PUBLICATION COULD INCLUDE TECHNICAL INACCURACIES OR TYPOGRAPHICAL ERRORS. CHANGES ARE PERIODICALLY ADDED TO THE INFORMATION HEREIN: THESE CHANGES WILL BE INCORPORATED IN NEW EDITIONS OF THE PUBLICATION. VISA MAY MAKE IMPROVEMENTS AND/OR CHANGES IN THE PRODUCT(S) AND/OR THE PROGRAM(S) DESCRIBED IN THIS PUBLICATION AT ANY TIME.

If you have technical questions or questions regarding a Visa service or capability, contact your Visa representative.

Contents

What’s New in This Version

Related Publications

Chapter 1 • API Overview

- Prerequisites. 37
- Introduction. 37
 - Token Active Account Management. 38
 - Token Life Cycle Management. 38
 - Card-On-File, ECOM, and HCE Use Cases. 38
 - HCE Client Actions. 39
 - Card-on-File/ECOM Client Actions. 39
 - Backward Compatibility. 39
 - VTs Data Type Information. 40
 - Endpoints. 41
 - JSON Web Encryption and Object Signing (JWE/JWS) 42
 - Encoding. 43
 - Hyphens in ID Fields. 43
 - API Samples. 43
 - Successful Responses. 44
- Errors. 44
 - Standard Error Responses. 44
 - Details. 45
 - Standard Errors. 45
 - Error Handling. 47

Chapter 2 • Enroll Device

- Enroll Device Introduction. 48

Enroll Device Request.	48
Endpoints.	48
Methods.	49
Required Headers.	49
Query Parameters.	51
Request Body.	51
Device Info.	52
Channel Security Context.	53
Device Init Params.	55
Enroll Device Response.	55
HTTP Response.	55
HTTP Response Header.	56
Response Body.	56
Device Init Params.	57
Enroll Device Errors.	57
Enroll Device Examples.	57
Enroll Device PKI-Based Secure Channel Example.	57
Enroll Device Opacity A Example.	59
Enroll Device Embedded Secure Element.	60

Chapter 3 • Enroll PAN

Enroll PAN Introduction.	62
Enroll PAN Request.	62
Endpoints.	62
Methods.	63
Required Headers.	63
Query Parameters.	65
Request Body.	65
Channel Security Context.	68
Encrypted Payment Instrument.	70

Expiration Date.	.71
Billing Address.	.71
Payment Instrument Provider.	.72
Encrypted Chip Data.	.79
Signed Payload for Nonce (sigNonce).	.79
Enroll PAN Response.	.80
HTTP Response.	.80
HTTP Response Header.	.80
Response Body.	.80
Card Metadata.	.81
AID Info Extended.	.85
Coresidence.	.88
Coresidence Alternate Networks.	.88
Payment Instrument.	.89
Provider.	.91
Enroll PAN Errors.	.93
Enroll PAN Examples.	.94
Enroll PAN Push Provisioning Example.	.94
Enroll PAN by TR-TSP Example.	.97
Enroll PAN Examples for VCEH.	.98

Chapter 4 • Provision Token—Given PAN Enrollment ID

Provision Token—Given PAN Enrollment ID Introduction.	.103
Provision Token—Given PAN Enrollment ID Request.	.104
Endpoints.	.104
Methods.	.104
Required Headers.	.104
Query Parameters.	.106
Request Body.	.106
Terms and Conditions.	.110

SSD Data.	110
Channel Security Context.	112
SE Data.	114
Provision Token—Given PAN Enrollment ID Response.	114
HTTP Response.	114
HTTP Response Header.	114
Response Body.	115
Payment Instrument.	115
Step-Up Request.	118
Token Info.	120
ODA Data.	132
ODA ICC CRT Private Key Field.	134
ODA ICC CRT Private Key.	134
Provision Token—Given PAN Enrollment ID Errors.	135
Provision Token—Given PAN Enrollment ID Examples.	136
Provision Token-Given PAN Enrollment ID Example.	136
Provision Token by TR-TSP Example.	145
Opacity xv2 Example.	146
JSON Request.	146
JSON Response.	147
Unencrypted eniccPrivateKey.	148
Unencrypted encRiskDataInfo.	149

Chapter 5 • Provision Token—Given PAN Data

Provision Token—Given PAN Data Introduction.	152
Provision Token—Given PAN Data Request.	153
Endpoints.	153
Methods.	153
Required Headers.	153
Query Parameters.	155

Request Body.	155
SSD Data.	160
Channel Security Context.	161
Passcode.	163
Encrypted Payment Instrument.	164
Expiration Date.	165
Billing Address.	165
Payment Instrument Provider.	166
Provision Token—Given PAN Data Response.	171
HTTP Response.	171
HTTP Response Header.	171
Response Body.	172
Payment Instrument.	173
Step-Up Request.	175
Token Info.	177
ODA Data.	185
Provision Token—Given PAN Data Errors.	186
Provision Token—Given PAN Data Examples.	187
Provision Token—Given Pan Data Example.	187
Elliptic Curve Cryptography ESE Request.	188
JWE encrypted encPaymentInstrument.	189
Unencrypted encRiskDataInfo.	189
Elliptic Curve Cryptography ESE Response.	191
Unencrypted encTokenInfo	192
Provision Token—Given Another Token Example.	193
Provision Token—VCES Applied Response Example.	194
Provision with Given PAN (async=true) Example.	196

Chapter 6 • Confirm Provisioning

Confirm Provisioning Introduction.	197
--	-----

Confirm Provisioning Request.	197
Endpoints.	197
Methods.	198
Required Headers.	198
Query Parameters.	200
Request Body.	200
APDU Response Code.	201
Confirm Provisioning Response.	201
HTTP Response.	201
HTTP Response Header.	201
Response Body.	201
Confirm Provisioning Errors.	201
Confirm Provisioning Examples.	202
Confirm Provisioning Example.	202

Chapter 7 • Active Account Management—Replenish

Active Account Management Replenish Introduction.	203
Active Account Management—Replenish Request.	204
Endpoints.	204
Methods.	204
Required Headers.	204
Query Parameters.	206
Request Body.	206
Active Account Management—Replenish Response.	207
HTTP Response.	207
HTTP Response Header.	208
Response Body.	208
Active Account Management—Replenish Errors.	209
Active Account Management—Replenish Examples.	209
JSON OPACITY Request.	209

Active Account Management-Replenish Example.	210
--	-----

Chapter 8 • Active Account Management—Confirm Replenishment

Active Account Management—Confirm Replenishment Introduction.	212
Active Account Management—Confirm Replenishment Request.	213
Endpoints.	213
Methods.	213
Required Headers.	213
Query Parameters.	215
Request Body.	215
Token Info.	215
Active Account Management—Confirm Replenishment Response.	216
HTTP Response.	216
HTTP Response Header.	216
Response Body.	216
Active Account Management—Confirm Replenishment Errors.	217
Active Account Management—Confirm Replenishment Errors.	217
Active Account Management—Confirm Replenishment Examples.	217
Active Account Management—Confirm Replenishment Example.	217

Chapter 9 • Replenish ODA Data

Replenish ODA Data Introduction.	219
Replenish ODA Data Request.	219
Endpoints.	219
Methods.	220
Required Headers.	220
Request Body.	222
Replenish ODA Data Response.	222
HTTP Response.	222
HTTP Response Header.	222

Response Body.	222
Replenish ODA Data Errors.	222
Replenish ODA Data Examples.	223
Replenish ODA Data Examples.	223

Chapter 10 • Submit ID&V Step-Up Method

Submit ID&V Step-Up Method Introduction.	224
Submit ID&V Step-Up Method Request.	224
Endpoints.	224
Methods.	225
Required Headers.	225
Query Parameters.	227
Request Body.	227
Submit ID&V Step-Up Method Response.	228
HTTP Response.	228
HTTP Response Header.	228
Response Body.	228
Submit ID&V Step-Up Method Errors.	229
Submit ID&V Step-Up Method Examples.	229
Submit ID&V Step-Up Method Examples.	229

Chapter 11 • Validate OTP

Validate OTP Request.	231
Endpoints.	231
Methods.	232
Required Headers.	232
Query Parameters.	234
Request Body.	234
Validate OTP Response.	235
HTTP Response.	235

HTTP Response Header.	235
Response Body.	236
Validate OTP Errors.	236
Validate OTP Examples.	236
Validate OTP Examples.	236

Chapter 12 • Validate Authentication Code

Validate Authentication Code Request.	238
Endpoints.	238
Methods.	239
Required Headers.	239
Query Parameters.	241
Request Body.	241
Validate Authentication Code Response.	242
HTTP Response.	242
HTTP Response Header.	242
Response Body.	242
Validate Authentication Code Errors.	243
Validate Authentication Code Examples.	243
Validate Authentication Code Example.	243

Chapter 13 • Get Step Up Options

Get Step Up Options Introduction.	244
Get Step Up Options Request.	245
Endpoints.	245
Method.	245
Required Headers.	245
Query Parameters.	247
Request Body.	248
Unencrypted Billing Info.	248

Billing Address.	248
Get Step Up Options Response.	250
HTTP Response.	250
HTTP Response Header.	250
Response Body	250
Step-Up Request.	250
Get Step Up Options Errors.	252
Get Step Up Options Examples.	253
Get Step-Up Options Examples.	253

Chapter 14 • Get Card Metadata

Get Card Metadata Introduction.	255
Get Card Metadata Request.	255
Endpoints.	255
Method.	256
Required Headers.	256
Query Parameters.	258
Request Body.	258
Get Card Metadata Response.	258
HTTP Response.	258
HTTP Response Header.	259
Response Body.	259
Coresidence.	260
Payment Instrument.	261
Provisioned Tokens.	263
Card Metadata.	264
Card Details.	268
Encrypted Payment Instrument.	268
Get Card Metadata Errors.	269
Get Card Metadata Examples.	269

Get Card Metadata Example.	269
Chapter 15 • Get Payment Data (Token & Cryptogram)—Given Token ID	
Get Payment Data (Token & Cryptogram)—Given Token ID Introduction.	272
Get Payment Data (Token & Cryptogram)—Given Token ID Request.	273
Endpoints.	273
Methods.	273
Required Headers.	273
Query Parameters.	275
Request Body.	275
Payment Request.	278
Risk Data.	279
Three DS Data.	279
Cryptogram Data.	279
Dynamic Data.	280
Get Payment Data (Token & Cryptogram)—Given Token ID Response.	284
HTTP Response.	284
HTTP Response Header.	284
Response Body.	284
Authentication Flag Status.	285
Payment Instrument.	285
Token Info.	286
Cryptogram Info.	287
QR Info.	287
Get Payment Data (Token & Cryptogram)—Given Token ID Errors.	288
Get Payment Data (Token & Cryptogram)—Given Token ID Example.	288
CTAVV Cryptogram Type.	288
No Cryptogram Type.	290
DTWV Cryptogram Type.	290
Get Payment Data Examples by Token Requestor.	291

Authentication Flags Response Example.	294
Get Payment Data Examples by Token Service Provider (TR-TSP).	294
Network Authentication Request Examples.	295

Chapter 16 • Get Payment Data for Autofill

Get Payment Data for Autofill Introduction.	297
Get Payment Data for Autofill Request.	298
Endpoints.	298
Methods.	298
Required Headers.	298
Query Parameters.	300
Request Body.	300
Payment Request.	301
Dynamic Data.	302
Device Binding Short Info.	305
Get Payment Data for Autofill Response.	305
HTTP Response.	305
HTTP Response Header.	306
Response Body.	306
Authentication Flag Status.	307
Payment Instrument.	307
Token Info.	308
Cryptogram Info.	309
QR Info.	309
Get Payment Data for Autofill Errors.	310
Get Payment Data for Autofill Example.	310
Unencrypted encTokenInfo	312

Chapter 17 • Get Transaction History

Get Transaction History Introduction.	313
---	-----

Get Transaction History Request.	314
Endpoints.	314
Method.	314
Required Headers.	314
Query Parameters.	316
Request Body.	316
Get Transaction History Response.	317
HTTP Response.	317
HTTP Response Header.	317
Response Body.	317
Transaction Details.	318
Get Transaction History Errors.	320
Get Transaction History Examples.	320
Get Transaction History Example.	320
Opacity Get Transaction History Response.	320

Chapter 18 • Retrieve Transaction History

Retrieve Transaction History Introduction.	322
Retrieve Transaction History Request.	323
Endpoints.	323
Methods.	323
Required Headers.	323
Query Parameters.	325
Request Body.	325
Channel Security Context.	325
Retrieve Transaction History Response.	327
HTTP Response.	327
HTTP Response Header.	327
Response Body.	328
Transaction Details.	328

Retrieve Transaction History Errors.	330
Retrieve Transaction History Examples.	331
Retrieve Transaction History Example.	331

Chapter 19 • Get Content

Get Content Introduction.	333
Get Content Request.	333
Endpoints.	333
Method.	334
Required Headers.	334
Request Body.	336
Get Content Response.	336
HTTP Response.	336
HTTP Response Header.	336
Response Body.	336
Content.	337
Get Content Errors.	337
Get Content Examples.	337
Get Content Example.	337

Chapter 20 • Get Token Status

Get Token Status Introduction.	339
Get Token Status Request.	339
Endpoints.	339
Method.	340
Required Headers.	340
Query Parameters.	342
Request Body.	342
Get Token Status Response.	342
Response Body.	342

Device Binding Info.	343
Token Info.	343
Expiration Date.	344
Get Token Status Errors.	345
Get Token Status Examples.	345
Get Token Status Example.	345
Get Token Status Response for Obsolete Token.	346

Chapter 21 • Token Life-Cycle Management—Suspend Token

Token Life-Cycle Management—Suspend Token Introduction.	347
Token Life-Cycle Management—Suspend Token Request.	348
Endpoints.	348
Methods.	348
Required Headers.	348
Query Parameters.	350
Request Body.	350
Update Reason.	350
Token Life-Cycle Management—Suspend Token Response.	351
HTTP Response.	351
HTTP Response Header.	351
Response Body.	351
Token Life-Cycle Management—Suspend Token Errors.	351
Token Life-Cycle Management—Suspend Token Examples.	352
Suspend Token Example.	352

Chapter 22 • Token Life-Cycle Management—Resume Token

Token Life-Cycle Management—Resume Token Introduction.	353
Token Life-Cycle Management—Resume Token Request.	354
Endpoints.	354
Methods.	354

Required Headers.	354
Query Parameters.	356
Request Body.	356
Update Reason.	356
Token Life-Cycle Management—Resume Token Response.	357
HTTP Response.	357
HTTP Response Header.	357
Response Body.	357
Token Life-Cycle Management—Resume Token Errors.	357
Token Life-Cycle Management—Resume Token Examples.	358
Resume Token Example.	358

Chapter 23 • Token Life-Cycle Management—Delete Token

Token Life-Cycle Management—Delete Token Introduction.	359
Token Life-Cycle Management—Delete Token Request.	360
Endpoints.	360
Methods.	360
Required Headers.	360
Query Parameters.	362
Request Body.	362
Update Reason.	362
Token Life-Cycle Management—Delete Token Response.	363
HTTP Response.	363
HTTP Response Header.	363
Response Body.	363
Token Life-Cycle Management—Delete Token Errors.	363
Token Life-Cycle Management—Delete Token Examples.	364
Delete Token Example.	364

Chapter 24 • Get PAN Data

Get PAN Data Introduction.	365
Get PAN Data Request.	366
Endpoints.	366
Methods.	366
Required Headers.	366
Query Parameters.	368
Request Body.	368
Get PAN Data Response.	368
HTTP Response.	368
HTTP Response Header.	368
Response Body.	369
Payment Instrument.	369
Get PAN Data Errors.	370
Get PAN Data Examples.	370
RSA-PKI Get Pan Data.	370
JSON RSA-PKI Get PAN Data Request.	370
JSON RSA-PKI Get PAN Data Response.	370
Opacity Get PAN Data.	371
JSON OPACITY Get PAN Data Request.	371
JSON OPACITY Get PAN Data Response.	371
Unencrypted Payment Instrument Example.	371

Chapter 25 • Confirm Token Association

Confirm Token Association Introduction.	373
Confirm Token Association Request.	374
Endpoints.	374
Methods.	374
Required Headers.	374
Query Parameters.	376
Request Body.	376

Confirm Token Association Response.	377
HTTP Response.	377
HTTP Response Header.	377
Confirm Token Association Response.	377
Confirm Token Association Errors.	377
Confirm Token Association Examples.	378
Confirm Token Association Example.	378

Chapter 26 • Re-Perso Token Data

Re-Perso Token Data Introduction.	380
Re-Perso Token Data Request.	380
Endpoints.	380
Methods.	381
Required Headers.	381
Query Parameters.	383
Request Body.	383
SSD Data Reperso.	384
Channel Security Context.	384
SE Data.	386
Re-Perso Token Data Response.	386
HTTP Response.	386
HTTP Response Header.	386
Response Body.	387
Payment Instrument.	387
Token Info.	388
ODA Data.	396
Re-Perso Token Data Errors.	397
Re-Perso Token Data Examples.	397
JSON Re-Perso Token Data Request.	397
JSON Re-Perso Token Data Response.	397

eSE Reperso for SCP02 Example.	400
--	-----

Chapter 27 • Validate JWE

Validate JWE Introduction.	403
Validate JWE Request.	403
Endpoints.	403
Methods.	404
Required Headers.	404
Query Parameters.	405
Request Body.	405
Unencrypted Payment Instrument	406
Validate JWE Response.	414
Response Body.	414
Validate JWE Errors.	415
Validate JWE Examples.	415
Validate JWE VCEH Example.	416
JWE encrypted encPaymentInstrument.	417

Chapter 28 • Generate Keys

Generate Keys Introduction.	419
Generate Keys Request.	419
Endpoints.	419
Methods.	420
Required Headers.	420
Query Parameters.	422
Request Body	422
Generate Keys Response	422
HTTP Response.	422
HTTP Response Header.	423
Response Body.	423

Generate Keys Errors.	423
Generate Keys Examples.	423
Generate Keys Request.	423
Generate Keys Response.	424

Chapter 29 • Initiate Re-Perso

Initiate Re-Perso Introduction.	425
Initiate Re-Perso Request.	425
Endpoints.	425
Methods.	426
Required Headers.	426
Query Parameters.	428
Request Body	428
Initiate Re-Perso Response.	428
HTTP Response.	428
HTTP Response Header.	428
Response Body.	428
Initiate Re-Perso Errors.	429
Initiate Re-Perso Examples.	429
Initiate Re-Perso Request.	429
Initiate Re-Perso Response.	429

Chapter 30 • Get Task Status

Get Task Status Introduction.	430
Get Task Status Request.	430
Endpoints.	430
Method.	431
URL Parameters.	431
Required Headers.	431
Query Parameters.	433

Request Body.	.433
Get Task Status Response	.433
HTTP Response.	.433
HTTP Response Header.	.433
Response Body.	.433
Get Task Status Errors.	.434
Get Task Status Example.	.434

Chapter 31 • Get Async Token Provisioning Result

Get Async Token Provisioning Result Introduction.	.435
Get Async Token Provisioning Result Request.	.436
Endpoint.	.436
Methods.	.436
URL Parameters.	.436
Required Headers.	.436
Query Parameters.	.438
Request Body.	.438
Get Async Token Provisioning Result Response	.438
HTTP Response.	.438
HTTP Response Header.	.438
Response Body.	.439
Task Provision Data.	.439
Get Async Token Provisioning Results Errors.	.449
Get Async Token Provisioning Example.	.449

Chapter 32 • Submit Job

Submit Job Introduction.	.452
Submit Job Request.	.452
Submit Job Endpoint.	.452
Methods.	.453

Required Headers.	.453
Query Parameters.	454
Request Body	455
Submit Job Response	.455
HTTP Response.	.455
HTTP Response Header.	455
Response Body.	455
Submit Job Errors.	.456
Submit Job Examples.	456

Chapter 33 • Get Job Status

Get Job Status Introduction.	457
Get Job Status Request.	457
Get Job Status Endpoint.	457
URL Parameters.	.458
Method.	458
Required Headers.	.458
Query Parameters.	460
Request Body.	.460
Get Job Status Response	460
HTTP Response.	.460
HTTP Response Header.	460
Response Body.	.461
Get Job Status Errors.	461
Get Job Status Examples.	461

Chapter 34 • Upload Job Data

Upload Job Data Introduction.	463
Upload Job Data Request.	463
Endpoint Upload Job Data.	463

URL Parameters.	.464
Methods.	464
Required Headers.	.464
Query Parameters.	466
Request Body.	.466
Provisioning Data.	467
Upload Job Data Response	470
HTTP Response.	.470
Response Body.	.470
Upload Job Data Errors.	470
Upload Job Data Examples.	471

Chapter 35 • Get Job Results

Get Job Results Introduction.	.472
Get Job Results Request.	.472
Endpoint Get Job Results.	472
URL Parameters.	.473
Method.	473
Required Headers.	.473
Query Parameters.	475
Request Body.	.475
Get Job Results Response	.475
HTTP Response.	.475
HTTP Response Header.	475
Response Body.	.476
Token Provisioning Batch Job Summary.	476
Get Job Results Errors.	.480
Get Job Results Example.	480

Chapter 36 • Request Device Token Binding

Request Device Token Binding Introduction.	.484
Request Device Token Binding Request.	.485
Endpoint	485
Methods.	485
Required Headers.	.485
Query Parameters.	487
Request Body.	.488
Merchant Identifier.	490
Risk Data.	491
Binding Request.	.491
Unencrypted Billing Info.	.492
Billing Address.	492
Request Device Token Binding Response	.493
HTTP Response.	.493
HTTP Response Header.	494
Response Body.	.494
Step-Up Request.	494
Request Device Token Binding Errors.	.496
Request Device Token Binding Example.	496

Chapter 37 • Delete Device Binding

Delete Device Binding Introduction.	.499
Delete Device Binding Request.	.499
Endpoint	499
Methods.	500
Required Headers.	.500
Query Parameters.	502
Request Body.	.502
Delete Device Binding Response	.502
HTTP Response.	.502

HTTP Response Header.	502
Response Body.	503
Delete Device Binding Errors.	503
Delete Device Binding Example.	503

Chapter 38 • Get Nonce

Get Nonce Introduction.	504
Get Nonce Request.	504
Endpoints.	504
Methods.	505
Required Headers.	505
Query Parameters.	507
Request Body	507
Get Nonce Response.	507
HTTP Response.	507
HTTP Response Header.	507
Response Body.	508
Signed Payload for Nonce (sigNonce).	508
Get Nonce Errors.	508
Get Nonce Example.	508

Chapter 39 • Onboarding

About the Onboarding API.	510
Onboarding Request.	510
Path and Endpoints.	510
Method.	511
Required Headers.	511
Query Parameters.	513
Client Request Body.	513
Acquirer Identifier.	522

Token Requestor Settings.	522
Tokenization Configuration.	522
Relationships.	523
Onboarding Response.	523
HTTP Responses.	523
HTTP Response Headers.	524
Response Body.	524
Tokenization Configuration.	527
Relationships.	528
Acquirer Identifier.	528
Onboarding Errors.	528
Onboarding Examples.	530

Chapter 40 • Get Push Enrollment Info

About the Get Push Enrollment Info API.	533
Get Push Enrollment Info Request.	533
Path and Endpoints.	533
Method.	534
Required Headers.	534
Query Parameters.	536
Request Body.	536
Get Push Enrollment Info Response.	536
Response Body.	536
Unencrypted Info.	536
Billing Information.	537
Get Push Enrollment Info Example.	538

Chapter 41 • Status Notifications

Status Notifications Summary.	540
SSL Certificate Requirements.	542

Task Status Event	543
Endpoint.	.543
Query Parameters.	.543
Request Body.	.544
Task Status Event Example.	.544
Token Status Updated Event.	.544
Endpoint.	.545
Query Parameters.	.545
Request Body.	.545
Token Status Updated Event Examples.	546
Payment Transaction Event.	547
Endpoint.	.547
Query Parameters.	.547
Request Body.	.548
Payment Transaction Event Examples.	.548
Card Metadata Update Event.	548
Endpoint.	.548
Query Parameters.	.548
Request Body.	.549
Card Metadata Update Event Examples.	.549
Token Push Provisioning Event.	549
Endpoint.	.549
Query Parameters.	.550
Request Body.	.550
Token Push Provisioning Notification Examples.	551
Bulk Provisioning Job Completion.	552
Endpoint.	.552
Query Parameters.	.552
Request Body.	.552

Bulk Provisioning Job Completion Example.	553
---	-----

Chapter 42 • VCEH Web Proxy

About the VCEH Web Proxy.	554
VCEH Push Card Enrollment to Token Requestor's App.	554
Token Requestor's App Returns to VCEH.	556
VCEH Web Proxy Examples.	558

Chapter 43 • Digital Authentication Framework (DAF)

Risk Data Attributes.	561
DAF Authentication Factor Requirements.	565

Appendix A • ID&V Risk Data Attributes

ID&V Risk Data Attributes.	567
------------------------------------	-----

Appendix B • Issuer Support for App-to-App Step Up Mode

Issuer Support for App-to-App Step Up Mode.	572
---	-----

Appendix C • Email Hashing Requirements

Email Hashing Requirements.	574
-------------------------------------	-----

Appendix D • Encrypting and Decrypting Data

Data Encryption and Decryption Summary.	577
Field Level Encryption/Decryption.	578
Sample JWE Using API Key/Shared Secret.	581
Sample JWE Using Opacity A.	584
Sample JWE Using RSA PKI.	586
Sample JWS Using RSA PKI.	589
Sample JWE Using RSA PKI2.	590
Sample JWS Using RSA PKI2.	593

Appendix E • Opacity A End-to-End Encryption Scheme

Opacity A End-to-End Encryption Scheme.	595
Acronyms.	596
Request Encryption.	596
Request Decryption.	596
Response Encryption.	597
Response Decryption.	597
Additional Information.	597

Appendix F • RSA PKI Encryption Requirements

RSA PKI Encryption Introduction.	599
Mutual SSL Support.	599
Signature for Sensitive Data Transmission Between Visa and the Device.	600
Symmetric Encryption.	600
Hybrid Encryption.	600
Data Classification.	601
Cryptographic Algorithms.	601
Wallet Provider Root Certificate.	601
Transmission of Keys Related to Encryption in the Server APIs.	602
Visa-Provided Certificates.	602
Sensitive Data Between MAP and VTS.	602
Mac of the TVL.	602

Appendix G • Notes on JWE

Notes on JWE.	603
-----------------------	-----

Appendix H • Frequently Asked Questions about JWE and JWS

Appendix I • API Key Separation for Signing and Encryption

API Key Separation for Signing and Encryption.	609
--	-----

Appendix J • Replenish FAQ

When is the Replenish API called?	612
What does the device do when making a request for a new LUK?	612
What does the device do after getting a new LUK?	613
What happens if the device requests a new LUK and does not receive it (either because of request not reaching VTS, or response not reaching back to device)?	613
What is the function of confirmReplenishment API?	614

Appendix K • What's New in Prior Versions

What's New in Version 23.08.	.615
What's New in Version 23.07.	.616
What's New in Version 23.06.	.617
What's New in Version 23.02.	.617
What's New in Version 23.01.	.618
What's New in Version 22.10.	.618
What's New in Version 22.09.	.619
What's New in Version 22.08.	.620
What's New in Version 22.04.	.620
What's New in Version 22.01.	.622
What's New in Version 21.10.	.622
What's New in Version 21.08.	.623
What's New in Version 21.07.	.624
What's New in Version 21.05.	.624
What's New in Version 21.04.	.625
What's New in Version 21.03.	.625
What's New in Version 21.02.	.625
What's New in Version 21.01.	.625
What's New in Version 20.10.	.626
What's New in Version 20.09.	.626
What's New in Version 20.08.	.626

What's New in Version 20.07.	.626
What's New in Version 20.03.	.627
What's New in Version 20.01.	.627
What's New in Version 19.10.	.628
What's New in Version 19.09.	.628
What's New in Version 19.08.	.628
What's New in Version 19.06.	.629
What's New in Version 19.04.	.630
What's New in Version 19.03.	.630
What's New in Version 19.02.	.630
What's New in Version 19.01.	.631
What's New in Version 6.8.	.631
What's New In Version 6.7.	.631
What's New in Version 6.6.	.632
What's New in Version 6.5.	.633
What's New in Version 6.4.	.633
What's New in Version 6.3.	.634
VCEH Support.	.634
What's New in Version 6.1.	.634
What's New in Version 6.0.	.634
What's New in Version 5.8.	.635
What's New in Version 5.5.	.635
What's New in Version 5.4.	.635
What's New in Version 5.3.	.635
What's New in Version 5.2.	.635
New API.	.636
New APIs.	.636
Other.	.636
What's New in Version 4.9.	.636

What’s New in Version 4.7.637

What’s New in Version 4.6.638

What’s New in Versions 3.8 to 4.5. 638

What's New in This Version

This document describes enhancements to public features for issuers, wallet providers, merchants and developers.

Documentation changes

- Added a restriction regarding the use of hyphens in ID fields, such as `clientDeviceID`. In ID fields where hyphens are allowed, the field content cannot contain consecutive hyphens; a single hyphen separated by another character is allowed. For example, `123-456` is allowed, but not `123--456`.

Related Publications

The following documents contain information that issuers, acquirers, merchants, and solution providers can use to obtain additional technical details, system processing requirements, and implementation information to support a Visa cloud-based payments program.

Title	Description
<i>Visa Cloud-Based Payments Contactless Specification</i>	Provides the specification for Visa payWave contactless using cloud-based payments.

Chapter 1

API Overview

Prerequisites

1. The client is onboarded as a Token Requester.
2. The client has access to the Visa Developer Center to obtain the correct API credentials (the API Key and the Shared Secret Key).

Introduction

The APIs in this guide are to be used by the clients (Mobile Wallet Providers) of the Visa Token Service. This API provides a set of methods which enables PAN tokenization. Given a PAN, it can return a token and use it for e-commerce and m-commerce payments. Tokens can be restricted by domain, channel, device, usage, amount, etc. criteria. There are a set of APIs, known as Active Account Management (AAM) APIs, which provide a way to replenish a set of dynamic parameters on the token thus increasing payments security. Another set of APIs, known as Lifecycle Management (LCM) APIs, can further help with discovery and management of the token state. The back-end also provides a set of notifications you can subscribe to for any token state changes.

Token Active Account Management

Active account management APIs enable management of the token dynamic parameters on the device. These are parameters which have a limited use and have to be changed periodically. They also enable the back-end to be aware of the set of parameters deployed on the device and issue corresponding notifications when needed.

The AAM APIs are as follows:

- Replenish
- Confirm Replenishment

Token Life Cycle Management

Mobile wallet providers use these APIs to perform Token Life Cycle operations on a token. The result of this operation will lead to a token status update. This changed token status should be propagated to the device.

The following states are currently supported:

- Active
- Suspended
- Deleted

Card-On-File, ECOM, and HCE Use Cases

All of the APIs in this document can be used by HCE clients, with the exception of Provision Token—Given PAN Data. The Card-on-File and ECOM (also known as e-commerce) use cases only allow the following APIs to be used:

- Provision Token Given PAN Data
- Get Card Metadata
- Get Payment Data (Token & Cryptogram)—Given Token ID
- Get PAN Data
- Get Content
- Get Token Status
- Confirm Token Association
- Token Life Cycle Management Delete Token API

HCE Client Actions

Typically, an HCE client enrolls a device, enrolls a PAN, provisions a token given a PAN enrollment ID, and confirms the provisioning. The client then can replenish the token and confirm, replenish ODA data, submit an ID&V step-up method, validate one-time passwords, validate authentication codes, get step-up methods, get payment data, and manage tokens (e.g. suspend, resume, or delete them).

HCE clients must handle event notifications related to token status (created, updated, key changed), PAN metadata, and payment transactions. Specifically, the client should call the Token Status API when it receives a token status notification, call Get Card Metadata when the PAN metadata changes, and call Get Transaction History when it receives a payment transaction notification.

Card-on-File/ECOM Client Actions

Typically, a card-on-file or ECOM client provisions a token given a PAN, gets the card metadata, and manages tokens, e.g. deletes them.

A card-on-file, or ECOM client, must handle event notifications related to token status (created, updated, key changed), PAN metadata, and payment transactions. Specifically, the client should call the Token Status API when it receives a token status notification and call Get Card Metadata when the PAN metadata changes.

Backward Compatibility

The following changes are considered backwards compatible:

- Adding a new API resource.
- Adding a new, optional, request parameter to an existing API resource. The parameter can be added as part of the request body, as a URL parameter, or an HTTP header field.
- Adding a new `enum` value. The value is added either in the request or in the response.
- Relaxing some of the constraints on an existing request parameter. For example, making it optional.
- Adding a new response parameter to the API response.
- Changing the order in which parameters are returned in existing API responses.
- Changing the format of opaque string identifiers.

In simple terms these rules mean:

- Ignore any unknown fields or `enum` values received as part of API responses.
- Do not make any assumptions about opaque string identifiers, except for length.
- You can add new optional parameters to existing APIs and relax constraints on existing request parameters at any time.

VTS Data Type Information

Type	Description
String	Wherever String type is used it is assumed that the format will be [A-Z][a-z][0-9], no wild characters and any variation to this rule will be mentioned explicitly. Format: Alphabetic, numeric.
Enum	Only a list of specific values are allowed. The list is defined as part of the API specification. All enum values by default are assumed to have maximum size of 255 ASCII characters. Format: String.
Boolean	It is one of the following values: • true • false
Hexadecimal	Hexadecimal representation. Format: String. [A-Z][a-z][0-9].
HexInteger	Big integers are represented using Hexadecimal format of the binary representation of an integer, using Big Endian. For example, the number 2,000,000 will be represented as: "1E8480", where "80" is the least significant byte. The leading zero bytes can be omitted. Format: Hexadecimal.
Identifier	Generic identifier. Format: String. [A-Z][a-z][0-9].
Identifier2	Generic identifier, accepting '_' character. Format: String. [0-9,A-Z,a-z,-,_].
State	State or province code in ISO 3166-2 format, e.g. "NY" Format: String. Size: 3.
Country	Country String size: 2 Country in ISO 3166-1 alpha-2 format, e.g. "US". Format: String. Size: 2.
EpochDateTime	UTC Format in epoch seconds. Example: "date": 6763725673 Format: Numeric.
Base64	Base64 encoded string, per RFC4648. Note: The filler character is the equal (=) sign. Format: String. [A-Z][a-z][0-9]=.

Type	Description
Base64URL	Base64URL encoded string. The encoding is the same as Base64, but uses '-' character instead of '+' and '_' character instead of '/' character. Format: String.
CurrencyCode	ISO 4217 format
Amount	This type is used to represent an amount. Both Digits before and after the '.' are required. Format: String. Digits '.' Digits.
Digits	A sequence of digits represented as a string. Example: "122345898".
SystemId	Format: String. [A-Z][a-z][0-9]+,/,=.
encryptionMetaData	(Conditional) Cryptographic payload required for end-to-end encryption of sensitive fields. To be used in messages participating in end-to-end encryption scheme. Composition: base64url (ECC Public Key). ECC Public Key will be shared out-of-band with the wallet providers. Format: size: 1-512.

Note: Unless otherwise specified, a field is a string that contains either alphabetic [A-Z] [a-z] characters, numeric [0-9] digits, or alphanumeric characters, which may contain alphabetic characters, numeric digits, and other specified characters, such as underscores.

Endpoints

There are two publicly available endpoints. One is used for the sandbox and the other for production. All of the REST APIs defined here will use only the relative path. The complete URL is a concatenation of the endpoint defined here and the REST API path defined in the concrete API.

Endpoint	Description
https://api.visa.com	Visa production endpoint.
https://cert.api.visa.com	Visa sandbox environment endpoint. Use this environment for integration testing.

JSON Web Encryption and Object Signing (JWE/JWS)

Object signing and encryption conforms to the IETF JSON Object Signing and Encryption (JOSE) standard, which provides a general approach for encrypting and signing any content. It includes the use of JSON Web Encryption (JWE) for encryption and JSON Web Signature (JWS) for signing. Visa uses it to protect sensitive information, such as personal information about cardholders and their accounts, e.g. account numbers (PANs).

Clients use Field- and Message-Level Encryption (FLE and MLE) to encrypt sensitive data when making an API request to Visa. Field-level encryption encrypts specific data fields, while Message-level encryption encrypts the entire message. API responses may also contain Visa-encrypted data, which the client must decrypt.

Note: In many cases, the use of field-level encryption is indicated by the **enc** prefix on the field name, e.g. **encPaymentInstrument**.

For additional information about encryption and decryption, see the following appendixes:

- Encrypting and Decrypting Data
- Opacity A End-to-End Encryption Scheme
- RSA PKI Encryption Requirements
- Notes on JWE
- Frequently Asked Questions about JWE and JWS

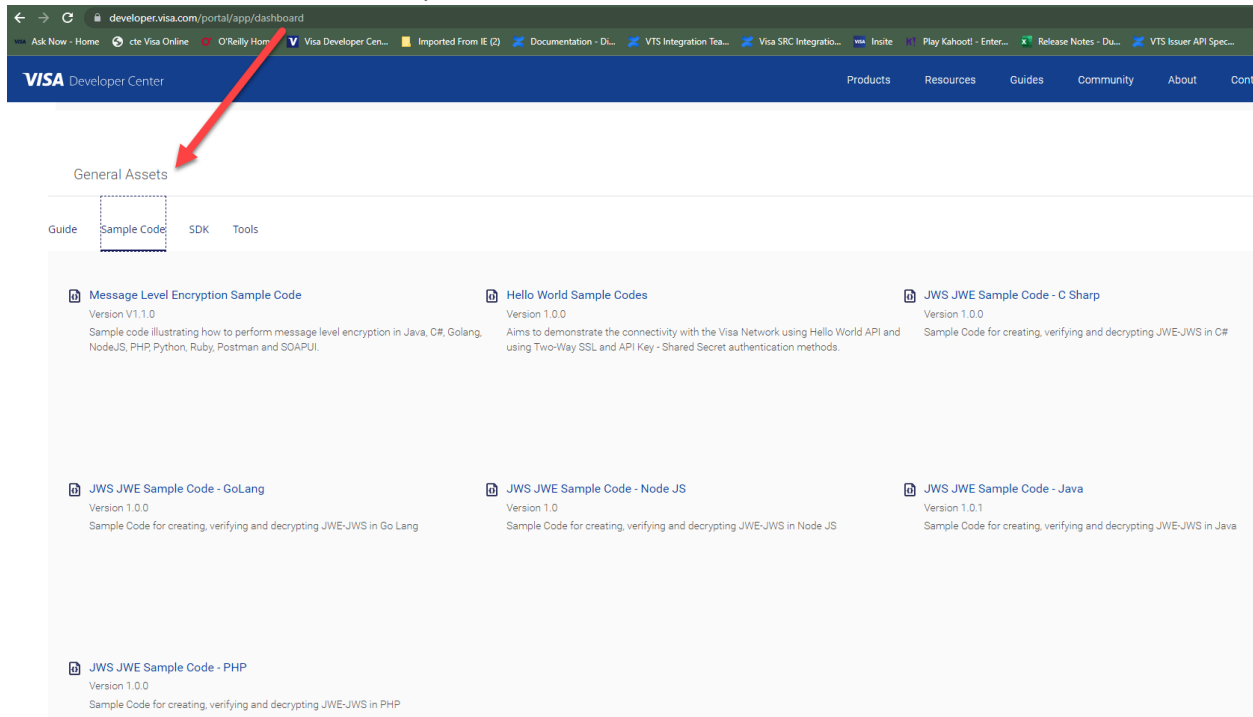
API Key Separation for Signing and Encryption

For additional information, see the API Key Separation for Signing and Encryption appendix.

JSON Web Encryption and JSON Web Signature Code Samples

Visa provides sample code in various languages, such as C#, Go Lang, Node JS, and Java, for handling JSON Web Encryption (JWE) for encryption and JSON Web Signatures (JWS) for object signing. To access these code samples, create an account on the Visa Developer Portal (developer.visa.com), log in, and scroll past **My Projects** on your dashboard to the **General**

Assets, which include code samples for download:



Encoding

Special characters (<, > and &) in all fields are HTML encoded and will need to be decoded by the API consumer.

Hyphens in ID Fields

In ID fields that allow hyphens in the field content, such as the `clientDeviceID` field, a single hyphen must be separated by a non-hyphen character. Consecutive hyphens are not permitted. For example, `xxx-xxx` may be allowed, but `xxx--xxx` is not permitted.

Note: Consecutive underscores (`. . . __ . . .`) can be used as an alternative to consecutive hyphens, if necessary.

API Samples

The API samples provided in this document are for illustrative purposes only. In case of confusion between the sample and the specification, the specification should be considered the source of truth.

Successful Responses

Successful API responses will return an HTTP status code in the 2xx range. The response body (if available) will contain the response of the API as described in this document.

Errors

API errors are reported using a Standard Error Response structure. Logically, all errors are categorized into Standard Errors and Business errors. Standard errors can be returned by any API and should be handled in a common way. Business errors are returned by only some APIs and are described in the Errors section for each API.

Standard Error Responses

Unless noted elsewhere, the Standard Response Structure (shown below) will be returned in case of an error. Use the `errorResponse.reason` field to drive your error handling logic. Errors are also provided with a human-readable description in the `errorResponse.message` field. This field should be used only to understand the problem. In rare cases, depending on the type of error, this error structure might not be returned and you should rely on the HTTP status code semantics.

```
{
  "errorResponse": {
    "message": "Missing or invalid value for Pan ReferenceID.",
    "reason": "cardVerificationFailed",
    "details": [
      {
        "location": "panReferenceId",
        "message": "Should be a numeric value"
      }
    ]
  }
}
```

Field	Description
message	Free-form text message describing the error condition. Do not show this message to the end user. Do not use this field to drive error handling logic. Use the <code>reason</code> field instead. The server can change this message at any point in time. Format: String.

<code>reason</code>	Use this field to drive your error handling business logic. Format: String.
<code>details</code>	Array of fields, which failed validation Format: String.

Details

Field	Description
<code>location</code>	The value of this field is using the XPATH expression to point to the field, that failed validation. Format: String.
<code>message</code>	The specific error for this field. Format: String.

Standard Errors

Standard errors can be returned by any API and should be handled consistently in a unified way. These errors are not documented as part of each individual API description.

Error Code	Reason Code	Description
400	<code>invalidParameter</code>	The value provided for one or more request parameters is considered invalid. Examples: Missing required field. The value exceeds the length specified for this field. Field formatting does not match the expected format. The <code>"details.location"</code> field of the error response may provide a list of fields which failed data validation constraints.

400	<code>invalidRequest</code>	The API server could not understand the request. This sometimes occurs when a data field is not in the appropriate/particular format. Examples: Base64URL decoding failed. The field is not in a particular format. The message field may provide additional clarification of what part/field of the request is considered incorrect. Refer to the API specification for the structure, format and constraints of the API request.
400	<code>incompleteRequest</code>	Missing required field. Refer to the <code>details.location</code> field to determine which fields failed data validation.
400	<code>trConfigIssue</code>	Token Requestor Configuration error. There is a problem with the Token Requestor's configuration. Please verify your configuration at Visa.
401	<code>authError</code>	API Credentials and the encryption keys used do not match, or the API Key was invalid or does not exist.
401	<code>inputValidationError</code>	Authentication error. The API assertion/token is either not provided or invalid. Refer to API specification how to obtain valid credentials and format security assertion for authentication.
403	<code>cardVerificationFailed</code>	Invalid field. Invalid payment instrument or data associated with the payment instrument. The "location" field may point to the field failing the validation. Ask users to check that all information was entered correctly.
403	<code>cardVerificationNotSupported</code>	Visa can't validate this card. Provision in another way.
403	<code>tokenUsageViolation</code>	The purpose of the token does not match the intended usage of the token. Please, refer to the API specification about the expected usage of the token.
403	<code>cardNotEligible</code>	This card cannot be used for tokenization at this moment. Please try again later or choose another Visa card.

500	<code>serviceError</code>	An error happened on the server. Either show a generic message, or retry the same request again. The number of retries should be limited. Consider implementing Exponential Backoff. In this algorithm the delay before you retry is defined as: $\text{retryDelay} = (2 \wedge N) * 1000 + \text{randomDelayMs}.$ <code>retryDelay</code> is the retry delay in milliseconds, <code>N</code> is your retry count (0, 1, 2, 3, ...), and <code>randomDelayMs</code> is the random delay in milliseconds (0 .. 1000).
503	<code>notReady</code>	The server is currently unable to handle the request due to a temporary overloading or maintenance of the server. If known, the length of the delay MAY be indicated in a Retry-After header.

Error Handling

This section contains guidelines about how to properly handle API errors for the purpose of simplifying API consumer error handling logic and achieving backward compatibility with the newer versions of the APIs. It is assumed the API consumer will follow these guidelines.

- Use the HTTP status codes ranges to determine if this is a successful API execution (2xx). If not, then who is at fault - either the API consumer (4XX), or the API (5XX)?
- Consider the API Standard Error Structure as optional. In very rare cases, it is possible that the API response does not contain an API Standard Error Structure. If this happens, rely on the HTTP status code semantics.
- Drive your error handling logic based on the value of the `errorResponse.reason` field only. The field `errorResponse.message` is for informational purpose only. The API is free to change this message at any point of time. Do not parse this field to drive your error handling logic. Do not expose this field to the end user.
- It is assumed the there is a default "catch all" type of error handling logic. Any 4XX HTTP status codes with unknown reason code field are considered as integration errors. You can choose to show a message, or you can simply fail to process this operation.

Chapter 2

Enroll Device

Enroll Device Introduction

Enrolls a device with VTS in order to subsequently be able to provision tokens to the device. Partners must call Enroll Device to register the device prior to provisioning device tokens or prior to sending a Device Binding request.

Enroll Device Request

Endpoints

Resource Path

`vtS/clients/{vClientID}/devices/{clientDeviceID}`

Sandbox Path

`https://cert.api.visa.com/vts/clients/{vClientID}/devices/
{clientDeviceID}?apiKey=key&relationshipID=id`

Live Path

`https://api.visa.com/vts/clients/{vClientID}/devices/{clientDeviceID}?
apiKey=key&relationshipID=id`

URL Parameters

Parameter	Description
<code>{vClientID}</code>	(Required) Visa-defined unique ID identifying the payment application provider (wallet provider) entity in the system. Provided during the onboarding process. Format: String. Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 36 characters.
<code>{clientDeviceID}</code>	(Required) This is a stable and unique device identifier set by the payment application provider (wallet provider). This could be a computer identifier or an ID tied to hardware, such as a TEE_ID or an SE_ID. Format: String. Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; 24 characters.

Methods

PUT

Required Headers

Header	Description
<code>x-request-id</code>	(Required) Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
<code>x-pay-token</code>	(Required) A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as: <code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
<code>apiKey</code>	(<i>Required</i>) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
<code>relationshipID</code>	(<i>Optional</i>) The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

Field	Description
<code>deviceInfo</code>	(<i>Required</i>) Device information. Format: A <code>deviceInfo</code> structure.
<code>channelSecurityContext</code>	(<i>Conditional</i>) Present in the case of a PAN entry on a companion device with an encryption scheme using device certificates, or with an OPACITY_A encryption scheme. This is required only when using end-to-end encryption and for all manually entered PANs. Format: A <code>channelSecurityContext</code> structure.
<code>deviceInitParams</code>	(<i>Conditional</i>) Device initialization parameters; required for use cases where the SDK is implemented using software-based secure storage. Format: A <code>deviceInitParams</code> structure.

Device Info

Field	Description
osType	(<i>Required</i>) Operating system type. Format: It is one of the following values: • ANDROID • IOS • WINDOWS • BLACKBERRY • TIZEN • OTHER
osVersion	(<i>Optional</i>) The version of the operating system running on the device. Format: Numeric, period (.); maximum 32 characters. Example: 4.4.4
osBuildID	(<i>Optional</i>) The operating system build identifier. Format: Alphanumeric; maximum 32 characters. Example: KTU84M.
deviceType	(<i>Required</i>) Device type. Format: It is one of the following values: • PHONE • TABLET • PHONE_TABLET • PC • WATCH • TV • WEARABLE_DEVICE • AUTOMOBILE_DEVICE • HOUSEHOLD_DEVICE
deviceIDType	(<i>Optional</i>) An attribute that indicates the source of the Device ID value (clientDeviceID). Format: It is one of the following values: • SecureElement • TEE • Derived

deviceManufacturer	(Optional) The manufacturer of the device (i.e., “Samsung”, “HTC”, “Apple”, “Nokia”, “Blackberry”). Format: Alphanumeric; maximum 32 characters.
deviceBrand	(Optional) The brand name of the device (i.e., “Galaxy”, “Nexus”, “Lumia”, “Bold”). Format: Alphanumeric; maximum 32 characters.
deviceModel	(Optional) The specific model of the device (i.e., SGH-T999, ME571KL NA K009). Format: Alphanumeric, numeric, hyphens (-), underscores (_), and spaces; maximum 32 characters.
deviceName	(Required) A readable name for the device. Ideally, this name would be entered by the consumer. It can be used to identify the device in customer support calls. Base64 URL-encoded since some platforms allow emoticons for the device name. Note: This field will support internationalization using UTF-8 characters. Format: Base64 URL-encoded string; maximum 256 characters.
deviceProfile	(Optional) The profile set up for the device. Format: It is one of the following values: • CDCVM_SUPPORTED • CDCVM_NOTSUPPORTED Note: The default value for this field is <code>CDCVM_SUPPORTED</code>.
hostDeviceID	(Optional) Client device ID for the device that will be paired with the host device. (The host device is already known to VTS because it has previously been enrolled.) This will be needed when enrolling secondary devices such as wearables. Format: Alphanumeric, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 24 characters.
phoneNumber	(Optional) Phone number. Format: Numeric with an optional leading plus sign (+); maximum 32 digits.

Channel Security Context

Field	Description
channelInfo	Security Channel used to communicate between the device and the Visa Token Service back-end. Format: A <code>channelInfo</code> structure.

<code>vtsCerts</code>	(Conditional) Visa certificate IDs. Format: An array of <code>vtsCerts</code> structures.
<code>deviceCerts</code>	(Conditional) Device certificates. Not required for autofill. Format: An array of <code>deviceCerts</code> structures.
<code>keyData</code>	(Conditional) Key data; required for OPACITY_A; otherwise, it is not used. In the enroll PAN and Provision with PAN APIs, <code>keyData</code> and <code>encryptionMetaData</code> must now be present together. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 512 characters.

Channel Info

Field	Description
<code>encryptionScheme</code>	(Conditional) The security scheme used to secure communication between the device and the Visa Token Service back-end. Note: For Secure Copy (SCP) encryption schemes, the encryption version and the type are concatenated to create a string in the form of <code>scp_version/type</code> as per Global Platform Amendment A. Format: It is one of the following values: • <code>RSA_PKI</code> • <code>OPACITY_A</code> • <code>SCP_02/PULL</code> • For ECC, <code>SCP_03/ecka</code>

Visa Token Service Certs

Field	Description
<code>vCertificateID</code>	(Required) Unique ID for Visa Certificate. This will be given to the wallet provider during onboarding. Devices can use this certificate ID while communicating with Visa Token Service . This will be the first 4 bytes of the Visa Public Key. Format: String. Size: 32.
<code>certUsage</code>	(Conditional) The usage of this certificate. Format: It is one of the following values: • <code>CONFIDENTIALITY</code> • <code>INTEGRITY</code>

Device Certs

Field	Description
certUsage	<i>(Conditional)</i> The usage of this certificate. Format: It is one of the following values: • CONFIDENTIALITY • INTEGRITY • DEVICE_ROOT
certFormat	<i>(Conditional)</i> X509 certificate structure. Format: String [A-Za-z0-9]+. Size: 16.
certValue	<i>(Conditional)</i> In PEM format/Base64URL safe encoded. Format: String[A-Za-z0-9]+. Size: 8192.

Device Init Params

Field	Description
sdkUsageType	<i>(Conditional)</i> Server-side nonce created during Enroll Device. Depending on the scheme selected, more data may be required. This field is required only if <code>channelInfo</code> is specified. Format: It is one of the following values: • None • Android_With_WBC • Android

Enroll Device Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
<code>requestID</code>	The ID specified in the <code>x-request-id</code> header.
<code>vClientID</code>	This is a Visa-defined unique ID identifying the payment application provider (wallet provider) entity in the system. This will be the same as what was provided during the onboarding process. Format: String. Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 36 characters.
<code>clientDeviceID</code>	This is a stable and unique device identifier set by the payment application provider (wallet provider). This could be a computer identifier or an ID tied to hardware, such as a TEE_ ID or an SE_ID. Format: String. Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; 24 characters.
<code>deviceID</code>	Internal; do not use Format: GUID
<code>deviceInitParams</code>	Applicable only to use cases where the SDK is implemented using software-based secure storage. Conditional if the SDK indicates <code>deviceInitParams</code> are needed. Format: A <code>deviceInitParams</code> structure.
<code>vServerNonce</code>	Server-side nonce created during Enroll Device. Depending on the scheme selected, more data may be required. This field is required only if <code>channelInfo</code> is specified. Note: This field is critical from an SDK perspective. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.

Device Init Params

Field	Description
sdkUsageType	(Conditional) Server-side nonce created during Enroll Device. Depending on the scheme selected, more data may be required. This field is required only if <code>channelInfo</code> is specified. Format: It is one of the following values: • None • Android_With_WBC • Android

Enroll Device Errors

This API returns Standard Errors.

Enroll Device Examples

Enroll Device PKI-Based Secure Channel Example

Request

Selected Headers:

```
x-request-id=taj...Sr
Accept=application/json
```

Endpoint:

```
https://cert.api.visa.com/vts/clients/217...bb/
devices/lzD...N9?apiKey=GPN...3I
```

Request Body:

```
{
  "channelSecurityContext": {
    "channelInfo": {
      "encryptionScheme": "RSA_PKI"
    }
  }
}
```

```
    },
    "deviceCerts": [
      {
        "certFormat": "X509",
        "certUsage": "CONFIDENTIALITY",
        "certValue": "LS0...0K"
      },
      {
        "certFormat": "X509",
        "certUsage": "INTEGRITY",
        "certValue": "LS0...0K"
      },
      {
        "certFormat": "X509",
        "certUsage": "DEVICE_ROOT",
        "certValue": "LS0...Qo"
      }
    ],
    "vtsCerts": [
      {
        "certUsage": "CONFIDENTIALITY",
        "vCertificateID": "215452c7"
      },
      {
        "certUsage": "INTEGRITY",
        "vCertificateID": "8302bc7f"
      }
    ]
  },
  "deviceInfo": {
    "deviceBrand": "AndroidBrand",
    "deviceIDType": "TEE",
    "deviceManufacturer": "Samsung",
    "deviceModel": "ANDROID-999",
    "deviceName": "TX1XYXRjaA",
    "phoneNumber": "6505551212",
    "deviceType": "WATCH",
    "hostDeviceID": "3e8...62",
    "osBuildID": "KTU84M",
    "osType": "ANDROID",
    "osVersion": "4.4.4"
  }
}
```

Response

Selected Headers:

```
"x-response-id": "gvC...yB"
```

Response Body:

```
{
  "requestID": "taj...Sr",
  "vClientID": "217...bb",
  "clientDeviceID": "lzD...N9",
}
```

```
"deviceID":"ebe...02",  
"vServerNonce":"5565a9f69410447bacdb4ce7799ad4ad3133"}
```

Enroll Device Opacity A Example

Request

Selected Headers:

```
x-request-id=XXX-1674835150  
Accept=application/json
```

Endpoint:

```
PUT https://cert.api.visa.com/vts/clients/d42...01/devices/XXX10122022?  
apiKey=2LX...7M&relationshipID=100...12
```

Request Body:

```
{  
  "channelSecurityContext": {  
    "channelInfo": {  
      "encryptionScheme": "OPACITY_A"  
    },  
    "vtsCerts": [  
      {  
        "certUsage": "CONFIDENTIALITY",  
        "vCertificateID": "e3577291"  
      }  
    ]  
  },  
  "deviceInfo": {  
    "deviceBrand": "Galaxy",  
    "deviceIDType": "Derived",  
    "deviceManufacturer": "Samsung",  
    "deviceModel": "ANDROID-999",  
    "deviceName": "TX1XYXRjaA",  
    "phoneNumber": "6505551212",  
    "deviceType": "PHONE",  
    "hostDeviceID": "zNa20bhT",  
    "osBuildID": "KTU84M",  
    "osType": "ANDROID",  
    "osVersion": "4.4.7"  
  }  
}
```

Response

Selected Headers:

```
"x-response-id": "xxx...yB"
```

Response Body:

```
{
  "requestID": "XXX-1674835150",
  "vClientID": "3b6.85",
  "clientDeviceID": "oPk...zQ",
  "deviceID": "7e0...01",
  "vServerNonce": "515...a0"
}
```

Enroll Device Embedded Secure Element

Request

```
{
  "deviceInfo": {
    "deviceBrand": "AndroidBrand",
    "deviceIDType": "TEE",
    "deviceManufacturer": "Samsung",
    "deviceModel": "ANDROID-999",
    "deviceName": "TXlXYXRjaA",
    "phoneNumber": "6504325678",
    "deviceType": "WEARABLE_DEVICE",
    "hostDeviceID": "3e89278f175403c4fc4e062",
    "osBuildID": "KTU84M",
    "osType": "TIZEN",
    "osVersion": "4.4.4"
  },
  "channelSecurityContext": {
    "channelInfo": {
      "encryptionScheme": "SCP_02/PULL"
    }
  }
}
```

Response

```
{
  "requestID": "...",
  "clientDeviceID": "...",
  "deviceID": "...",
  "vClientID": "...",
}
```

```
}    "vServerNonce": "..."
```

Chapter 3

Enroll PAN

Enroll PAN Introduction

Enrolls a payment instrument. You can use the PAN enrollment ID (`vPanEnrollmentID`) in the response to provision additional tokens,, monitor PAN metadata, and perform other operations.

Enroll PAN Request

Endpoints

Resource Path

`vts/panEnrollments`

Sandbox Path

`https://cert.api.visa.com/vts/panEnrollments?apiKey=key&relationshipID=id`

Live Path

`https://api.visa.com/vts/panEnrollments?apiKey=key&relationshipID=id`

Methods

POST

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as: <code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
relationshipID	(Optional) The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

Field	Description
clientAppID	(Required) Unique identifier for the client application. It is obtained from the wallet provider. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
clientWalletAccountID	(Required) Client-provided consumer ID that identifies the wallet account holder entity. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 24 characters
clientDeviceID	(Conditional) Stable and unique device identification set by the wallet provider. By default, this field should be considered required. It is optional for the following presentation types: ECOM, MERCHANT_PRESENTED_QR, QR_CONSUMER_CLOUD, and RECEIVE_ONLY. An Enroll PAN request for a card on file will not have a device ID Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; 24 characters
locale	(Required) Language in which the application communicates with the consumer. Based on ISO format for language (ISO 639-1) and an alpha-2 country code (ISO 3166-2 alpha-2). The language must be lowercase, and the country must be uppercase. The language and country should be separated using an underscore (_), e.g. en_US. Format: Alphabetic, underscores (_), e.g. spaces are not allowed; maximum 5 characters.

panSource	(<i>Required</i>) Source of the payment instrument (PAN). This enum value specifies the source and the method used to provide the payment instrument data. Format: It is one of the following values: • ONFILE – The wallet provider already has the PAN data for the customer in their records. • MANUALLYENTERED – The customer is expected to type the card number or capture a picture of the card. • ISSUER_PUSH_PROVISION – The PAN data was provided by the issuer of the PAN. • TOKEN_FOR_TOKEN – Token was used for source of the PAN data. • CHIP_DIP – PAN data provided when physical card was inserted. • CONTACTLESS_TAP – PAN data provided when physical card was tapped.
consumerEntryMode	(<i>Conditional</i>) The method of consumer entry. This field is required when the PAN source is manually entered. Not relevant for token-for-token use case. Format: It is one of the following values: • KEYENTERED • CAMERACAPTURED • UNKNOWN
encPaymentInstrument	(<i>Conditional</i>) BLOB field containing all payment instrument details (URL-safe base64 encoded) that represents the encrypted payload for a payment instrument. It is required except for Visa as a Hop. Do not specify both <code>encPaymentInstrument</code> and <code>paymentInstrumentReference</code>. Format: JSON Web Encrypted string using the shared secret. For more information, see "JSON Web Encryption and Object Signing." Base64URL encoded string, per RFC4648. The encoding is the same as Base64, but uses '-' character instead of '+' and '_' character instead of '/' character.> (schemas) [A-Za-z0-9=-_]+. Max size: 8192.
paymentInstrumentReference	(<i>Conditional</i>) A reference to the payment instrument or PAN provided by the issuer. You must provide this reference instead of an encrypted payment instrument for Visa as a Hop. Do not specify both <code>encPaymentInstrument</code> and <code>paymentInstrumentReference</code>. Format: Alphanumeric; maximum 32 characters.

encryptionMetaData	(Conditional) Cryptographic payload required for end-to-end encryption of sensitive fields. To be used in messages participating in end-to-end encryption schemes. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 512 characters.
platformType	(Conditional) Present if presentationType is NFC-SE or NFC-HCE. The step-up option app-to-app details in the response is based on the platform type. Format: It is one of the following values: • iOS – iOS app • ANDROID – Android app • WINDOWS – Windows app • WEB – Browser-based app
passcode	(Optional) Passcode will be matched against the passcode in the encPaymentInstrument, if present in Visa Card Enrollment Hub (VCEH). Format: String.
channelSecurityContext	(Conditional) Present in case of PAN entry on a companion device with encryption using device certificates or an OPACITY_A scheme. This is required for all manually entered PANs. Format: A channelSecurityContext structure.
intent	(Optional) The value in this field overrides the intent field in encPaymentInstrument. It allows the token requestor to override the issuer intent encrypted in encPaymentInstrument. The intent value PUSH_PROV_MOBILE in encPaymentInstrument can be overridden by PUSH_PROV_CROSS_DEVICE here. Format: String.

encChipData	(Conditional) BLOB field (URL-safe base64 encoded) containing all encrypted chip data details that represents the encrypted payload for a card's chip data. Required when panSource is CHIP_DIP or CONTACTLESS_TAP. Format: JSON Web Encrypted string using the shared secret. Base64URL encoded string, per RFC4648. The encoding is the same as Base64, but uses '-' character instead of '+' and '_' character instead of '/' character [A-Za-z0-9=-_]+. For more information, see "JSON Web Encryption and Object Signing." Max size: 8096 characters.
sigNonce	(Conditional) The JWS payload returned by the Get Nonce API. Required for non-EMVCo-certified terminals when panSource is CHIP_DIP or CONTACTLESS_TAP. Note: The nonce must be provided within 60 seconds of its creation. Format: String [A-Za-z0-9]+; max length 1024 characters

Channel Security Context

Field	Description
channelInfo	Security Channel used to communicate between the device and the Visa Token Service back-end. Format: A channelInfo structure.
vtscerts	(Conditional) Visa certificate IDs. Format: An array of vtscerts structures.
deviceCerts	(Conditional) Device certificates. Not required for autofill. Format: An array of deviceCerts structures.
keyData	(Conditional) Key data; required for OPACITY_A; otherwise, it is not used. In the enroll PAN and Provision with PAN APIs, keyData and encryptionMetaData must now be present together. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 512 characters.

Channel Info

Field	Description
encryptionScheme	(Conditional) The security scheme used to secure communication between the device and the Visa Token Service back-end. Note: For Secure Copy (SCP) encryption schemes, the encryption version and the type are concatenated to create a string in the form of <code>scp_version/type</code> as per Global Platform Amendment A. Format: It is one of the following values: • RSA_PKI • OPACITY_A • SCP_02/PULL

Visa Token Service Certs

Field	Description
vCertificateID	(Required) Unique ID for Visa Certificate. This will be given to the wallet provider during onboarding. Devices can use this certificate ID while communicating with Visa Token Service . This will be the first 4 bytes of the Visa Public Key. Format: String. Size: 32.
certUsage	(Conditional) The usage of this certificate. Format: It is one of the following values: • CONFIDENTIALITY • INTEGRITY

Device Certs

Field	Description
certUsage	(Conditional) The usage of this certificate. Format: It is one of the following values: • CONFIDENTIALITY • INTEGRITY • DEVICE_ROOT

certFormat	(Conditional) X509 certificate structure. Format: String [A-Za-z0-9]+. Size: 16.
certValue	(Conditional) In PEM format/Base64URL safe encoded. Format: String [A-Za-z0-9]+. Size: 8192.

Encrypted Payment Instrument

Field	Description
accountNumber	(Conditional) PAN of the card to be enrolled and provisioned. This field is required if an IBAN is not provided. Format: Numeric; 13-19 digits.
iban	(Conditional) This field is the IBAN of the card to be enrolled and provisioned. This field is required if a PAN is not provided. Format: Alphabetic, numeric; maximum 34 characters
token	(Conditional) EMV Co Token number. This is required if there is no accountNumber or iban. Format: Alphabetic, numeric; 12–19 characters..
cvv2	(Optional) CVV2 value associated with the PAN on the card. Note: For CVV2 to be considered during token provisioning, the CVV2 must also be provided within the encRiskDataInfo structure of the Provision Token request. Format: Numeric; 3-4 digits.
name	(Conditional) The full name on the Visa card associated with the enrolled payment instrument. This field is required if: • IBAN is provided AND no sessionIdIdentifier is provided for push provisioning. • IBAN is provided for normal pull provisioning This field is optional if a PAN is provided. Format: Alphanumeric, including spaces; maximum 256 characters.
expirationDate	(Conditional) Payment instrument's expiration date. This field is required unless the expiration date is not printed on the card. Format: An expirationDate structure.

Field	Description
billingAddress	(Optional) Billing address associated with the payment instrument. Format: A <code>billingAddress</code> structure.
provider	(Conditional) Information about the provider of the payment instrument and the context in which it is provided. This field is required if <code>panSource</code> equals <code>ISSUER_PUSH_PROVISION</code> . Format: A <code>provider</code> structure.

Expiration Date

Note:

The expiration date is optional; however, all fields must have a value if the date is provided.

Field	Description
month	(Required) The month the Visa card is set to expire. Format: Numeric; 2 digits.
year	(Required) The year the Visa card is set to expire. Format: Numeric; 4 digits.

Billing Address

Field	Description
line1	(Optional) Street 1 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line2	(Optional) Street 2 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line3	(Optional) Street 3 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.

line4	(Optional) Street 4 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line5	(Optional) Street 5 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
city	(Optional) The city associated with the enrolled payment instrument. Format: Alphabetic, single quote ('), hyphen (-), and whitespace.; maximum 100 characters.
state	(Optional) State or province code in ISO 3166-2 format, e.g. NY. Format: Alphabetic; maximum 3 characters.
country	(Optional) Country in ISO 3166-1 alpha-2 format, e.g. US. Format: Alphabetic; maximum 2 characters.
countryDigits	(Optional) ISO 3166-1 numeric Country Code. Format: Numeric; maximum 3 digits.
postalCode	(Optional) The postal code associated with the enrolled payment instrument. Note: This field does not support spaces (" ") in the middle of characters. Format: Alphabetic, numeric, e.g. spaces are not allowed; maximum 3–16 characters.

Payment Instrument Provider

This object allows the payment instrument provider to specify the intent under which the PAN should be used for token provisioning. Some of the fields provided here will be matched with the corresponding fields from the API payload. This structure applies only to Visa Card Enrollment Hub (VCEH) or legacy direct push provisioning.

Field	Description
intent	(Required) The intent of the encryptor, which specifies what the encryptor of the data trying to do. Format: It is one of the following values: PUSH_PROV_MOBILE - Push the PAN data to a Token Requestor to provision a token for the card holder on a specific device for a specific wallet/account. The push initiating application and the push receiving (token requestor) application are accessible on the same device. PUSH_PROV_CROSS_USER - Push the PAN data to a Token Requestor to provision a token for a secondary/authorized user. The Issuer is providing the PAN for the purpose of provisioning a token for an authorized consumer that is different from the push initiating card holder. This is only applicable to only to Visa Card Enrollment Hub (VCEH). PUSH_PROV_CROSS_DEVICE - Push the PAN data to a Token Requestor to provision a token for the cardholder on a different device. The issuer is providing the PAN for the purpose of provisioning a token for the consumer of a token requestor application accessible on a device that is different from the device from which the push is being initiated. This is only applicable to only to Visa Card Enrollment Hub (VCEH). PUSH_PROV_ONFILE - Push the PAN data to a Card On File merchant/Token Requestor to provision a token for the cardholder. The issuer is providing the PAN for the purpose of provisioning a token for the consumer into a card on file merchant. The push initiating application and the push receiving application(token requestor) are accessible on the same device.
clientWalletProvider	(Required) Token requestor ID. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed. Size: maximum 50 characters.
clientWalletAccountID	(Optional) Client provided consumer ID that identifies the Wallet Account Holder entity. It must have the same value as clientWalletAccountID field from the API payload. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed. Size: maximum 24 characters.
clientAppID	(Conditional) Unique Identifier for the client application, used to provide some of the encrypted values. Required if intent is PUSH_PROV_MOBILE. Example: Issuer's AppID used to select the PAN and the wallet. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed. Size: maximum 36 characters.

isIDnV	(Optional) Specifies whether the issuer wants ID&V to be performed. Format: It is one of the following values: • true - perform ID&V • false - do not perform ID&V
issuerAccountID	(Optional) Uniquely represents “pushing” account from issuer system. May be different from PAN holder account. It applies only to Visa Card Enrollment Hub (VCEH). Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 24 characters.
issuerClientInformation	(Optional) Client information shared from issuer to token requestor from information captured in the issuer systems about the PAN's owner. It applies only to Visa Card Enrollment Hub (VCEH). This information will be decrypted and shared with the token requestor. Format: A <code>clientInformation</code> structure.
passcode	(Optional) OTP generated by the issuer. It applies only to Visa Card Enrollment Hub (VCEH). Added to Enroll PAN and Provision + PAN data. Format: A <code>passcode</code> structure.
returnURIType	(Optional) The type of URI for the return app. It applies only to Visa Card Enrollment Hub (VCEH). Format: It is one of the following values: • IOS - iOS app • ANDROID - Android app • WEB - browser-based app
returnURI	(Optional) URI provided by the issuer to the token requestor to return control to the issuer app. This can be an app or web URL. Format: Alphanumeric. Size: maximum 128 characters.

<code>issuerTraceID</code>	(Optional) An ID, preferably a Globally Unique Identifier (GUID), provided by the issuer in order to query token status after sending the JWE to a token requestor. It applies only to Visa Card Enrollment Hub (VCEH). Format: Alphanumeric. Size: maximum 32 characters.
<code>isTsAndCsAccepted</code>	(Optional) This field indicates whether or not the customer has already accepted the issuer's Terms and Conditions. Note: If this value is set to <code>true</code>, the wallet provider can opt to not display the Terms and Conditions to the customer. They would submit the Terms and Conditions GUID to Visa. The default value for this field is <code>false</code>. Format: It is one of the following values: • <code>true</code> • <code>false</code>

Client Information

Client information applies only to Visa Card Enrollment Hub (VCEH).

Field	Description
<code>source</code>	(Optional) The source of the information. Format: It is one of the following values: • <code>ISSUER</code> • <code>TOKEN_REQUESTOR</code>
<code>clientWalletAccountID</code>	(Optional) Client provided consumer ID that identifies the wallet account holder entity. It must have the same value as <code>clientWalletAccountID</code> field from the API payload. Format: Alphanumeric, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 24 characters.
<code>firstName</code>	(Required) First name of the token requestor client that received the token. The issuer will populate with the information they have for the client. Format: Alphanumeric; between 2–80 characters, inclusive. Escape Characters, including tab entries, are not supported.
<code>middleName</code>	(Optional) Middle name of the token requestor client that received the token. The issuer will populate with the information they have for the client. Format: Alphanumeric; between 2–80 characters, inclusive. Escape Characters, including tab entries, are not supported.

Field	Description
lastName	(Required) Last name of the token requestor client that received the token. The issuer will populate with the information they have for the client. Format: Alphanumeric; between 2–80 characters, inclusive. Escape Characters, including tab entries, are not supported.
issuerAccountID	(Optional) The issuer account ID provided by the issuer to the token requestor. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 24 characters.
clientDeviceID	(Conditional) It is a stable and unique identifier set by the wallet provider for the device. This field is required when it is returned to the issuer by the token requestor. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 24 characters.
tokenReferenceID	(Conditional) It is a unique ID associated with a single token. The ID is created upon initial provisioning of a token to a device. This field is required when returned to the issuer by the token requestor. Format: Alphabetic, numeric; maximum 32 characters.
paymentAccountReference	(Optional) Payment Account Reference (PAR) if available for this payment instrument. Format: Alphanumeric; maximum 29 characters.
phoneNumber	(Conditional) Either phone number or email must be passed when intent is PUSH_PROV_CROSS_DEVICE or PUSH_PROV_CROSS_USER. Mobile phone number of the client as per issuer records. Format: A valid phone number; maximum 32 bytes.
contactEmail	(Conditional) Either phone number or email must be passed when intent is PUSH_PROV_CROSS_DEVICE or PUSH_PROV_CROSS_USER. Email address of client as per issuer records. Format: A valid email address; maximum 32 bytes. Note: This field will support internationalization using UTF-8 characters.
country	(Optional) The client's country code. Format: An alpha 2 country code (ISO 3166-1 alpha-2).

Field	Description
sessionIdentifier	<i>(Conditional)</i> Optional session identifier that can be used by the issuer to uniquely identify the cardholder. This field is required if IBAN and Push Provisioning and no cardholder name was provided. Format: Alphabetic, numeric, and underscores (_), e.g. spaces are not allowed; maximum 64 characters.
locale	<i>(Required)</i> Language in which the app communicates with the customer. Based on ISO format for language (ISO 639-1) and alpha 2 country code (ISO 3166-1 alpha-2). The language and country should be separated using an underscore (_). e.g. en_US. Format: Alphabetic, underscores (_), e.g. spaces are not allowed; maximum 5 characters.
campaignID	<i>(Optional)</i> Cobrand campaign identifier. Format: Alphabetic, numeric, hyphens (-), and underscores (_); maximum 100 characters.
campaignDescription	<i>(Optional)</i> Cobrand campaign description. Format: Alphabetic, numeric, hyphens (-), and underscores (_); maximum 250 characters.
userPersonalID	<i>(Optional)</i> An identifier that uniquely identifies a person. Format: Alphabetic or numeric; maximum 15 characters.

Passcode

Passcode fields are only applicable to Visa Card Enrollment Hub (VCEH).

Field	Description
type	<i>(Required)</i> Passcode type. Format: It is one of the following values: PASSCODE
value	<i>(Required)</i> The OTP / Passcode / Passphrase generated by the user or issuer that should be validated by Visa prior to PAN Enrollment. Format: Alphanumeric, maximum 50 characters.

<code>passCodeConversationID</code>	(<i>Conditional</i>) This is returned when <code>passcode</code> is required. The <code>passCodeConversationID</code> should be returned to Visa by the token requestor in the subsequent Enroll PAN call. This attribute should not be sent by the issuer. If the issuer sends a value for this attribute in the encrypted payment instrument, it should be ignored. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 32 characters
<code>maxRetries</code>	(<i>Optional</i>) The number of times the OTP can be retried before a risk code is sent to VIP indicating that the OTP did not match. This value is provided by the issuer when sending the payment instrument to the wallet provider. It should not be populated by the wallet provider during enrollment. If the wallet provider sends this attribute in the request, it is rejected (invalid request – bad parameter). Format: String. Digits [0-9].

Device Init Params

Field	Description
<code>sdkUsageType</code>	(<i>Conditional</i>) Server-side nonce created during Enroll Device. Depending on the scheme selected, more data may be required. This field is required only if <code>channelInfo</code> is specified. Format: It is one of the following values: • None • Android_With_WBC • Android

Encrypted Chip Data

Field	Description
terminalData	<i>(Required)</i> Tag value data from the terminal. Format: Array of tag-value data pairs; maximum 24 pairs
cardData	<i>(Required)</i> Tag value data from the card that was stored in record/file marked as <i>not signed</i> in the application file locator (AFL). Format: Array of tag-value data pairs; maximum 36 pairs
cardSignedData	<i>(Optional)</i> Transaction Verification Log (TVL) data from the card that was stored in record/file marked as <i>signed</i> in the application file locator (AFL). Format: String; maximum 1024 hexadecimal digits.

Tag-Value Data

Field	Description
tag	<i>(Required)</i> Tag value data from the terminal. Format: String; maximum 4 hexadecimal digits
value	<i>(Required)</i> Tag value data from the card that was stored in record/file marked as <i>not signed</i> in the AFL. Format: String; maximum 128 hexadecimal digits

Signed Payload for Nonce (sigNonce)

Field	Description
nonce	<i>(Required)</i> Nonce, which is a random number. Format: String; maximum 8 characters; 4 hexadecimal digits.
clientAppID	<i>(Required)</i> Unique Identifier for the client application. It is obtained from the wallet provider. Format: String; UUID, maximum length 36 characters
clientId	<i>(Required)</i> Unique ID of the client allowed to use the nonce. Format: String; UUID, maximum length 36 characters

Enroll PAN Response

HTTP Response

HTTP Response	Description
201	Enrollment artifacts created.

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
<code>vPanEnrollmentID</code>	Unique ID for enrolled PAN. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed. Size: 36.
<code>coresidenceInfo</code>	Contains data about the card and whether or not it is co-badged. This information can be used by Wallet Providers who support <code>coresidenceInfo</code> to determine the alternate networks available to use with this PAN. Format: A <code>coresidenceInfo</code> structure.
<code>serviceEnrollmentStatuses</code>	An array of <code>serviceEnrollmentStatuses</code> objects showing the status of enrollment for each service attempted for enrollment. If the wallet account is linked to the Visa Checkout account, the PAN enrollment in to the “VisaCheckout” service is attempted and the status is included as well. Format: A <code>serviceEnrollmentStatuses</code> array structure.
<code>paymentInstrument</code>	Summary of the Payment Instrument. Format: A <code>paymentInstrument</code> structure.

Field	Description
cardMetaData	Consumer contact information, label foreground and background colors, terms and conditions, etc. Format: A cardMetaData structure.
aidInfo	Present for NFC-SE use cases only. AID information is needed for the wallet provider to create AID identifiers for the SD Testts created in the secure element chip. Format: An aidInfo structure.

Card Metadata

Field	Description
backgroundColor	Background color of the mobile wallet UI entry (“space”) for the card. Specified as CSS style hex format, e.g., 0x0e19d2. Format: CSS style hex format, e.g. 0x0e19d2; maximum 32 characters.
foregroundColor	Foreground color of the mobile wallet entry for the card. Specified as CSS style hex format, e.g., 0x0e19d2. Format: CSS style hex format. E.g. 0x0e19d2; maximum 32 characters.
labelColor	Label color of the mobile wallet entry for the card. Specified as CSS style hex format, e.g. 0x0e19d2. Format: Alphabetic, numeric; maximum 32 characters.
contactWebsite	Customer service website of the issuing bank. It is recommended that this be a specific landing page created for cards that are provisioned into a specific wallet program. Format: Alphabetic, numeric; maximum 256 characters. Note: This field will support internationalization using UTF-8 characters.
contactEmail	Customer service email address of the issuing bank. Format: CSS style hex format, e.g. 0x0e19d2; maximum 32 characters. Note: This field will support internationalization using UTF-8 characters.
contactNumber	Customer service phone number of the issuing bank. Format: Alphabetic, numeric.

contactName	Name of the issuing bank. It may be used in combination with other text strings, e.g. if <code>contactName</code> is “XYZ Bank”, it may be used to construct a string “Contact XYZ Bank” or “XYZ Bank Contact Information”. Format: Alphabetic, numeric, and spaces (); maximum 32 characters. Note: This field will support internationalization using UTF-8 characters.
bankAppName	Name of banking application for display. Format: Alphabetic, numeric; maximum 64 characters. Note: This field will support internationalization using UTF-8 characters.
bankAppAddress	The package name for the destination mobile application. Example: <code>com.sampleissuer.thebankapp</code> Format: Alphabetic, numeric, and periods (.); maximum 1024 characters.
privacyPolicyURL	URL pointing to issuing bank privacy policy. This refers to the issuer website. Format: Valid URL; maximum 128 characters.
termsAndConditionsURL	URL pointing to issuing bank Terms and Conditions for card. Format: Valid URL; maximum 128 characters.
termsAndConditionsID	Terms and Conditions identifier in GUID format. If the GUID is changed from the earlier version, the consumer must be presented a new Terms and Conditions at time of provisioning. Format: Alphabetic, numeric; maximum 32 characters.
shortDescription	A short description of the card to be used for notifications on the mobile device (on the lock screen), e.g. <code>The Bank Card</code>. Format: Alphabetic, numeric; maximum 32 characters. Note: This field will support internationalization using UTF-8 characters.
longDescription	A longer description of the card that could appear inside the mobile wallet e.g., “The Bank Card Platinum Rewards”. The <code>shortDescription</code> will be used if <code>longDescription</code> is not specified. Format: Alphabetic, numeric; maximum 64 characters. Note: This field will support internationalization using UTF-8 characters.

unsupportedPresentationTypes	The presentation types that are not supported such as "MSR" (for example). This is an advisory field to tell the wallet provider that they should not include the unsupported presentation type returned in this field in the provisioning request. Note: - If the issuer is not enrolled for HCE, this field will contain: - NFC-HCE - INAPP - MERCHANT_PRESENTED_QR - If the issuer has opted out of ECOM, this field will contain ECOM presentation types. - If the issuer has opted out of ESE, this field will contain NFC-SE presentation types. Format: An array; maximum 256 characters.
unsupportedCardVerificationTypes	The card verification types that are not supported such as "AVS" (for example). This is an advisory field to let the wallet provider know which card verification services are not supported for a given payment instrument. The wallet provider can use this information to relax any field validations on the Customer UI (such as Address). Format: An array; maximum 256 characters.
cardData	Card data. Format: An array of structures.
issuerFlags	Issuer flags. Format: An issuerFlags structure.

Card Data

Field	Description
guid	GUID for the content. Each guid can have an array of content attributes. Content array will carry metadata related to the content. Format: Alphanumeric; maximum 32 characters.

contentType	Type for the content. Format: It is one of the following values: - digitalCardArt - digitalCardArtBackground - cardSymbol - termsAndConditions
content	Format: A content structure.

Content

Field	Description
mimeType	MIME type of the content. Format: It is one of the following values: - image/png - image/pdf - text/plain - text/html
width	Width of image, in pixels. Value between 1 and 4096 for image/png; the value is 0 for image/pdf and text/html. Format: String; maximum 32 characters.
height	Height of image, in pixels. Value between 1 and 4096; the value is 0 for image/pdf and text/html. Format: String; maximum 32 characters.

Issuer Flags

Field	Description
deviceBinding	Device binding. Format: It is one of the following values: - true - false
cardholderVerification	Whether the issuer participating in step-up flow. Format: It is one of the following values: - true - false

Field	Description
trustedBeneficiaryEnrollment	Whether this is a trusted beneficiary enrollment. Format: It is one of the following values: • true • false
delegatedAuthenticationSupported	Whether issuer supports delegated authentication. Format: It is one of the following values: • true • false
oboDeviceBinding	Whether issuer supports on behalf of device binding. Format: It is one of the following values: • true • false

AID Info Extended

Field	Description
applicationLabel	Mnemonic associated with the AID according to [ISO 7816- 5]. Used in application selection. Application Label is optional in the File Control Information (FCI) of an Application Definition File (ADF) and optional in an ADF directory entry for VCPS. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; 8-32 characters.
aid	Identifies the application as described in ISO 7816-5. It may be extended to partnering merchants and third-party wallet providers. However, as of now, it is specific to issuing banks. Source: Assigned by Visa during issuer onboarding. Note: aidInfo is an array of aids and related data. Review the provision response JSON example at the end of this section. Format: Alphabetic, numeric; 10-32 characters.
priority	Priority Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 2 characters.

Field	Description
CVMrequired	Indicates if CVM is required. It is one of the following values: - Y - CVM is required - N - CVM is not required Format: Alphabetic; maximum 1 character.
cap	Indicates card processing requirements and preferences such as type of cardholder verification methods supported by the issuer. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 8 characters
asrpd	The field describes the European Economic Area Interchange Fee Regulation Classification for any tokens provisioned in Europe. This value can be passed as part of the personalization script for SE tokens, but will have to be populated here for HCE tokens. The field contains token personalization information as described below: 9F0A0400010101 if the funding source is a debit card and the card product type is debit 9F0A0400010102 if the funding source is a credit card, charge, or a deferred debit and the card product type is credit 9F0A0400010103 if the card product type is commercial 9F0A0400010104 if the funding source is a prepaid card, and the card product type is prepaid The information can be interpreted, left-to-right, as follows: 9F0A - EMVCo assigned tag 04 - Length of all data within tag 0001 - Identifier assigned by EMVCo for European regulatory usage 01 - Length of the Product ID 01 through 04 - Values to identify debit, credit, commercial or prepaid products (above) Format: Alphabetic, numeric; maximum 14 characters.
appPrgrmID	This is a Visa proprietary data element identifying the Application Program ID of the card application. When personalized, the Application Program ID is returned in the FCI Issuer Discretionary Data of the Select response (tag BF0C). qVSDC readers that support Dynamic Reader Limits (DRL) functionality examine the Application Program ID to determine the Reader Limit Set to apply. Format: Alphabetic, numeric; 10–32 characters.

Field	Description
mst	MST is only applicable if the request was for the <code>MST</code> presentation type. Will be returned for NFC-HCE use case only, and not applicable for NFC-SE. Format: An <code>msr</code> data structure.
kernelIdentifier	Returned for Durbin (US Debit). Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 8 characters.
cardHolderNameVCPCS	Cardholder name per ISO 7813. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; 2-26 characters.
pdol	List of terminal/reader- related data objects (tags and lengths) requested by the card to be transmitted in the GET PROCESSING OPTIONS command. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 48 characters.
countrycode5F55	Returned for Durbin (US Debit) Source: Issuer Configuration File. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 4 characters
issuerIdentification Number	Returned for Durbin (US Debit). Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 6 characters.
track2DataDec	Format: A <code>track2DataDec</code> structure.
MsdData	Format: A <code>MsdData</code> structure.
track2DataNotDec	Format: A <code>track2DataNotDec</code> structure.
qVSDCData	Format: A <code>qVSDCData</code> structure.
ODADData	ODADData will be returned if the token can be used for Offline Data Authorization (ODA). If ODADData is returned, all the fields in ODADData will be returned. Format: An <code>ODADData</code> structure.

Field	Description
issuerApplicationDiscretionaryData	Issuer Application Discretionary Data (AID IDD). Format: String. Size: 15 bytes, hex-encoded.
issuerDiscretionaryData	Dynamic issuer perso data to be injected into FCC_IDD data. It is hex-encoded. Format: String. Size: 128 characters.

Coresidence

Field	Description
cardImageMode	Identifies the card image usage for the alternate networks available to the wallet provider. Format: It is one of the following values: • SHARED—Single co-residence CardArt image provided by one of the networks. The wallet provider will use this image (card Data element having digitalCardArt as the contentType) across all co-residence networks. • PER_NETWORK—A CardArt image for co-residence per network. Wallet will use the CardArt image provided by the respective network.
alternateNetworks	Array of available alternate networks. Format: An array of <code>alternateNetworks</code> structures.

Coresidence Alternate Networks

Field	Description
name	Alternate network name. Format: Alphabetic, numeric; maximum 30 characters.
id	EMV® Sepcification RID, a <i>registered application provider identifier</i> (RID) which is issued by the ISO/IEC 7816-5 registration authority. Format: Alphabetic, numeric; maximum 30 characters.

Payment Instrument

Field	Description
last4	Last 4 digits of the account. Note: If an IBAN was provided in the provisioning request, this is the last 4 digits of the IBAN. Format: String, numeric; maximum 4 digits.
expirationDate	Payment instrument's expiration date. Format: An <code>expirationDate</code> structure.
accountStatus	<i>Future:</i> Is the account closed? To be returned with Get Card Metadata API only. Format: It is one of the following values: - Y – Account closed - N – Account not closed
isTokenizable	<i>Future:</i> Is this account eligible for tokenization? To be returned with Get Card Metadata API only. Format: It is one of the following values: - Y – Eligible - N – Not Eligible
indicators	These are indicators associated with the Payment Instrument. If the field is not returned, or empty, then there are no indicators associated with it. Expected values are: PRIVATE_LABEL - This Payment Instrument is for a Private Label PAN. Format: String
expDatePrintedInd	Indicates if the expiration date is printed on the payment instrument. Format: It is one of the following String values: - Y - Printed - N - Not printed
cvv2PrintedInd	Indicates if the CVV2 is printed on the payment instrument. Format: It is one of the following String values: - Y - Printed - N - Not printed
paymentAccountReference	Payment Account Reference (PAR) if available for this payment instrument. Format: String, alphabetic or numeric; maximum 29 characters.

Field	Description
enabledServices	Enabled services, such as scan-and-pay. Format: An enabledServices structure.
verificationResults	Payment instrument verification results. Present only for ECOM presentation type when the CVV2 and/or address fields appear in the provision request. Format: A verificationResults object.

Expiration Date

Field	Description
month	The month upon which the payment instrument is set to expire. Example: "month": "9" Format: String; numeric; max length 2 digits.
year	The year upon which the payment instrument is set to expire. Example: "year": "2024" Format: String; numeric; length 4 digits.

Verification Results

Field	Description
cvv2VerificationCode	The CVV2 verification result. This field is present when CVV2 verification data is present in the request. Format: It is one of the following values: • M – verified, CVV2 data match • N – verified, negative data match • U – verification not possible (issuer does not participate) • D – verification not performed due to duplicate request
addressVerificationCode	The address verification result. This field is present when address verification data is present in the request. Format: It is one of the following values: • Y – verified, address and postal code data match • P – verified, partial match (either address or postal code data match, but no result from the other) • B – verified, partial match (either address or postal code data match, but the other could not be verified due to incompatible format) • N – verified, neither address nor postal code data match • U – verification not possible (issuer does not participate) • D – verification not performed due to duplicate request

Enabled Services

Field	Description
merchantPresentedQR	Whether the issuer enabled the PAN for the scan-and-pay use case. Format: It is one of the following String values: • Y – Enabled • N – Disabled

Provider

The `provider` object allows the payment instrument provider to specify the intent under which the PAN should be used for token provisioning. Some of the fields provided here will be matched with the corresponding fields from the API payload. This structure applies only to Visa Card Enrollment Hub (VCEH) or legacy direct push provisioning.

Field	Description
intent	The intent of the encryptor, which specifies what the encryptor of the data trying to do. Format: It is one of the following values: PUSH_PROV_MOBILE - Push the PAN data to a Token Requestor to provision a token for the card holder on a specific device for a specific wallet/account. The push initiating application and the push receiving (token requestor) application are accessible on the same device. PUSH_PROV_CROSS_USER - Push the PAN data to a Token Requestor to provision a token for a secondary/authorized user. The Issuer is providing the PAN for the purpose of provisioning a token for an authorized consumer that is different from the push initiating card holder. This is only applicable to only to Visa Card Enrollment Hub (VCEH). PUSH_PROV_CROSS_DEVICE - Push the PAN data to a Token Requestor to provision a token for the cardholder on a different device. The issuer is providing the PAN for the purpose of provisioning a token for the consumer of a token requestor application accessible on a device that is different from the device from which the push is being initiated. This is only applicable to only to Visa Card Enrollment Hub (VCEH). PUSH_PROV_ONFILE - Push the PAN data to a Card On File merchant/Token Requestor to provision a token for the cardholder. The issuer is providing the PAN for the purpose of provisioning a token for the consumer into a card on file merchant. The push initiating application and the push receiving application(token requestor) are accessible on the same device.
clientWalletProvider	Token requestor ID. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed. Size: maximum 50 characters.
clientWalletAccountID	Client provided consumer ID that identifies the Wallet Account Holder entity. It must have the same value as clientWalletAccountID field from the API payload. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed. Size: maximum 24 characters.
clientAppID	Unique Identifier for the client application, used to provide some of the encrypted values. Required if intent is PUSH_PROV_MOBILE. Example: Issuer's AppID used to select the PAN and the wallet. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed. Size: maximum 36 characters.

<code>isIDnV</code>	Specifies whether the issuer wants ID&V to be performed. Format: It is one of the following values: • <code>true</code> - perform ID&V • <code>false</code> - do not perform ID&V
<code>issuerAccountID</code>	Uniquely represents “pushing” account from issuer system. May be different from PAN holder account. It applies only to Visa Card Enrollment Hub (VCEH). Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 24 characters.
<code>passcode</code>	OTP generated by the issuer. It applies only to Visa Card Enrollment Hub (VCEH). Added to Enroll PAN and Provision + PAN data. Format: A <code>passcode</code> structure.
<code>returnURIType</code>	The type of URI for the return app. It applies only to Visa Card Enrollment Hub (VCEH). Format: It is one of the following values: • <code>IOS</code> - iOS app • <code>ANDROID</code> - Android app • <code>WEB</code> - browser-based app
<code>returnURI</code>	URI provided by the issuer to the token requestor to return control to the issuer app. This can be an app or web URL. Format: Alphanumeric. Size: maximum 128 characters.
<code>issuerTraceID</code>	An ID, preferably a Globally Unique Identifier (GUID), provided by the issuer in order to query token status after sending the JWE to a token requestor. It applies only to Visa Card Enrollment Hub (VCEH). Format: Alphanumeric. Size: maximum 32 characters.

Enroll PAN Errors

Error Code	Reason Code	Description
403	<code>cardVerification Failed</code>	Invalid field. Invalid payment instrument or data associated with the payment instrument. The <code>location</code> field may point to the field failing the validation. Ask users to check that all information was entered correctly.

403	cardNotEligible	This card cannot be used for tokenization at this moment. Please try again later or choose another Visa card.
403	cardNotAllowed	The requested action is not allowed for a given PAN. This card is valid but cannot be used for tokenization. This is most likely for Visa-specific reasons. Ask the consumer to enter a different card.
403	Declined	This card is not eligible for tokenization at this moment. Try to enroll/provision again for this PAN. If you encounter the same message after the new enrollment/provision request, ask the customer to either add a different card or to reach out to their issuer.

This API also returns Standard Errors.

Enroll PAN Examples

Enroll PAN Push Provisioning Example

Request

Endpoint:

```
https://cert.api.visa.com/vts/panEnrollments?apikey=GPN...3I
```

Request Body:

```
{
  "clientAppID": "CompanyABC",
  "clientWalletAccountID": "AiO...I6",
  "locale": "en_US",
  "panSource": "ONFILE",
  "consumerEntryMode": "KEYENTERED",
  "encPaymentInstrument": "eyJ...FQ"
}
```

Unencrypted Payment Instrument:

```
{
  "accountNumber": "462...33",
  "cvv2": "123",
  "expirationDate": {
    "month": "12",
    "year": "2022"
  },
}
```

```

"name": "vts",
"billingAddress": {
  "line1": "901 Metro Center Blvd",
  "line2": "901 Metro Center Blvd Line 2",
  "line3": "901 Metro Center Blvd Line 3",
  "line4": "901 Metro Center Blvd Line 4",
  "line5": "901 Metro Center Blvd Line 5",
  "city": "Foster City",
  "postalCode": "94404",
  "state": "CA",
  "country": "US"
},
"provider": {
  "intent": "PUSH_PROV_MOBILE",
  "clientWalletProvider": "400...95",
  "clientWalletAccountID": "Boz...id",
  "clientDeviceID": "boz...v1",
  "clientAppID": "CompanyABC",
  "isIDnV": true,
  "issuerAccountID": "TEST1",
  "issuerClientInformation": {
    "source": "ISSUER",
    "clientWalletAccountID": "Boz...id",
    "firstName": "Firstname",
    "middleName": "Middlename",
    "lastName": "Lastname",
    "issuerAccountID": "TEST1",
    "clientDeviceID": "boz...v1",
    "tokenReferenceID": "testtokenReferenceID",
    "paymentAccountReference": "testpaymentAccountReference",
    "phoneNumber": "650-555-1212",
    "contactEmail": "...",
    "country": "US",
    "locale": "us_EN",
    "campaignID": "testcampaignID",
    "campaignDescription": "testcampaignDescription",
    "userPersonalID": "testuserPersonalID"
  },
  "passcode": {
    "passCodeConversationID": "5as...jf",
    "maxRetries": "5",
    "value": "12345678905",
    "type": "PASSCODE"
  },
  "returnURIType": "ANDROID",
  "returnURI": "aHR...Ju",
  "issuerTraceID": "1529527725",
  "isTsAndCsAccepted": true
}
}

```

Response

```

{
  "vPanEnrollmentID": "350..02",
  "paymentInstrument": {

```

```
"expirationDate": {
  "month": "12",
  "year": "2022"
},
"last4": "0933",
"cvv2PrintedInd": "Y",
"expDatePrintedInd": "Y",
"enabledServices": {
  "merchantPresentedQR": "N"
}
},
"cardMetaData": {
  "backgroundColor": "0x000000",
  "foregroundColor": "0x7dffff",
  "labelColor": "0x000000",
  "contactWebsite": "https://www.visatest.com",
  "contactEmail": "test@visa.com",
  "contactNumber": "1800768999",
  "contactName": "visaTest",
  "privacyPolicyURL": "https://prefix.html",
  "termsAndConditionsURL": "https://terms.html",
  "shortDescription": "Visa Company Pay",
  "longDescription": "Visa Test Only Card",
  "cardData": [
    {
      "guid": "a0b72f0e60284f4f9809c9375557a77c",
      "contentType": "cardSymbol",
      "content": [
        {
          "mimeType": "image/png",
          "width": "100",
          "height": "100"
        }
      ]
    }
  ],
  {
    "guid": "388...b7",
    "contentType": "digitalCardArt",
    "content": [
      {
        "mimeType": "image/png",
        "width": "1536",
        "height": "969"
      }
    ]
  }
]
}
}
```


Enroll PAN by TR-TSP Example

Request

Endpoint:

```
https://cert.api.visa.com/vts/panEnrollments?  
apikey=PK2...xE&relationshipID=111...45
```

Request Body:

```
{  
  "consumerEntryMode": "KEYENTERED",  
  "panSource": "ONFILE",  
  "encPaymentInstrument": "eyJ...hw",  
  "locale": "en_US",  
  "clientWalletAccountID": "nnPFFig",  
  "clientAppID": "Trtsp2TR1Profile"  
}
```

Response

```
{  
  "vPanEnrollmentID": "605bb65fa32dd05e511a14b9c3fb8201",  
  "paymentInstrument": {  
    "expirationDate": {  
      "month": "10",  
      "year": "2023"  
    },  
    "last4": "0032",  
    "cvv2PrintedInd": "Y",  
    "expDatePrintedInd": "Y",  
    "enabledServices": {  
      "merchantPresentedQR": "N"  
    }  
  },  
  "cardMetaData": {  
    "backgroundColor": "0x7aff54",  
    "foregroundColor": "0x7aff54",  
    "labelColor": "0x7aff54",  
    "contactWebsite": "https://....com",  
    "contactEmail": "...@....com",  
    "contactNumber": "1082345678",  
    "contactName": "binding",  
    "privacyPolicyURL": "http://test.com/pp",  
    "termsAndConditionsURL": "http://test.com/tc",  
    "shortDescription": "1-2-3 REWARDS Visa Card",  
    "cardData": [  
      {  
        "guid": "be3...a6",  
        "contentType": "cardSymbol",  
        "content": [  

```

```
{
  {
    "mimeType": "image/png",
    "width": "100",
    "height": "100"
  }
],
},
{
  "guid": "e09...3e",
  "contentType": "digitalCardArt",
  "content": [
    {
      "mimeType": "image/png",
      "width": "1536",
      "height": "969"
    }
  ]
}
],
"unsupportedPresentationTypes": [
  "NFC-HCE",
  "INAPP",
  "QR_CONSUMER_DEVICE_HCE",
  "QR_CONSUMER_CLOUD",
  "MERCHANT_PRESENTED_QR",
  "MSR"
],
"issuerFlags": {
  "deviceBinding": true,
  "cardholderVerification": true,
  "trustedBeneficiaryEnrollment": false,
  "delegatedAuthenticationSupported": false
}
}
}
```

Enroll PAN Examples for VCEH

Request

Selected Headers:

```
x-request-id= muj...77
Accept= application/json
x-pay-token= xv2:153...09
```

Endpoint:

```
https://cert.api.visa.com/vts/panEnrollments?apikey=8EA...zM
```

Request Body:

```
{
  "consumerEntryMode": "KEYENTERED",
  "panSource": "ISSUER_PUSH_PROVISION",
  "clientDeviceID": "test",
  "encPaymentInstrument": "eyJ...bA",
  "locale": "en_US",
  "clientWalletAccountID": "clientWalletAccountID",
  "clientAppID": "clientAppID",
  "passcode": {
    "value": "12345678905",
    "type": "PASSCODE",
    "passCodeConversationID": "5d9...01"
  }
}
```

Unencrypted Payment Instrument:

This example shows the contents of the payment instrument before encryption.

```
{
  "accountNumber": "462...33",
  "cvv2": "478",
  "expirationDate": {
    "month": "12",
    "year": "2022"
  },
  "name": "vts",
  "billingAddress": {
    "state": "CA",
    "line1": "line1",
    "line2": "line2",
    "postalCode": "94404",
    "country": "US",
    "city": "A City"
  },
  "provider": {
    "clientAppID": "TestNHPP",
    "issuerTraceID": "152\u2026625",
    "isIDnV": true,
    "isTsAndCsAccepted": true,
    "clientWalletProvider": "400\u2026695",
    "intent": "PUSH_PROV_MOBILE",
    "issuerAccountID": "TEST1",
    "issuerClientInformation": {
      "source": "ISSUER",
      "firstName": "Firstname",
      "middleName": "Middlename",
      "lastName": "Lastname",
      "locale": "us_EN",
      "country": "US",
      "issuerAccountID": "TEST1",
      "phoneNumber": "650-555-1212",
      "contactEmail": "username@email.com"
    }
  },
  "passcode": {
```

```
    "passCodeConversationID": "5as...jf",
    "maxRetries": "5",
    "value": "12345678905",
    "type": "PASSCODE"
  },
  "returnURIType": "ANDROID",
  "returnURI": "aHR\u2026Ju"
}
```

Response

```
{
  "vPanEnrollmentID": "a10\u202602",
  "paymentInstrument": {
    "expirationDate": {
      "month": "12",
      "year": "2022"
    },
    "last4": "1121",
    "cvv2PrintedInd": "Y",
    "expDatePrintedInd": "Y",
    "paymentAccountReference": "V00\u202623",
    "enabledServices": {
      "merchantPresentedQR": "N"
    }
  },
  "cardMetaData": {
    "backgroundColor": "0x000000",
    "foregroundColor": "0x7dffff",
    "labelColor": "0x000000",
    "shortDescription": "Visa Test Pay",
    "longDescription": "Visa Test Only Card",
    "cardData": [
      {
        "guid": "a0b...7c",
        "contentType": "cardSymbol",
        "content": [
          {
            "mimeType": "image/png",
            "width": "100",
            "height": "100"
          }
        ]
      },
      {
        "guid": "388...b7",
        "contentType": "digitalCardArt",
        "content": [
          {
            "mimeType": "image/png",
            "width": "1536",
            "height": "969"
          }
        ]
      }
    ]
  }
}
```

```
]
}
}
```

Request for Visa as a Hop

Endpoint:

```
https://cert.digital.visa.com/vts/panEnrollments?apikey=GPN...3I
```

Request Body:

```
{
  "consumerEntryMode": "KEYENTERED",
  "panSource": "ISSUER_PUSH_PROVISION",
  "locale": "en_US",
  "clientWalletAccountID": "PttzUu",
  "clientAppID": "AppID1234",
  "clientDeviceID": "xCwr4_nkZQ",
  "passcode": {
    "value": "123456",
    "type": "PASSCODE",
    "passCodeConversationID": "746...02"
  },
  "paymentInstrumentReference": "8fc...02"
}
```

Response for Visa as a Hop

```
{
  "vPanEnrollmentID": "f0f...02",
  "paymentInstrument": {
    "expirationDate": {
      "month": "12",
      "year": "2022"
    },
    "last4": "9170",
    "cvv2PrintedInd": "Y",
    "expDatePrintedInd": "Y",
    "enabledServices": {
      "merchantPresentedQR": "Y"
    }
  },
  "cardMetaData": {
    "backgroundColor": "0x7aff54",
    "foregroundColor": "0x7aff54",
    "labelColor": "0x7aff54",
    "contactWebsite": "https://test.com",
    "contactNumber": "123-555-1212",
    "contactName": "VTS Test Value",
    "privacyPolicyURL": "https://test.com",
    "termsAndConditionsURL": "https://termsandconditions.com",
    "termsAndConditionsID": "11f5bdb02748464caa38b2d13f0c1a58",
    "shortDescription": "Test Short 2",
  }
}
```

```
"longDescription": "Test Long",
"cardData": [
  {
    "guid": "2a7dfefab1df4ccc926d8be690219b94",
    "contentType": "cardSymbol",
    "content": [
      {
        "mimeType": "image/png",
        "width": "1344",
        "height": "440"
      }
    ]
  },
  {
    "guid": "68ee773dc2dd45518e254f8faebec7e0",
    "contentType": "digitalCardArt",
    "content": [
      {
        "mimeType": "image/png",
        "width": "1536",
        "height": "969"
      }
    ]
  },
  {
    "guid": "11f...58",
    "contentType": "termsAndConditions",
    "content": [
      {
        "mimeType": "text/plain",
        "width": "0",
        "height": "0"
      }
    ]
  }
],
"unsupportedPresentationTypes": [
  "RECEIVE_ONLY",
  "ECOM"
],
"issuerFlags": {
  "deviceBinding": false,
  "cardholderVerification": false,
  "trustedBeneficiaryEnrollment": false,
  "delegatedAuthenticationSupported": true
}
}
```

Chapter 4

Provision Token—Given PAN Enrollment ID

Provision Token—Given PAN Enrollment ID Introduction

Call Provision Token—Given PAN Enrollment ID to provision additional tokens on an enrolled PAN (`vPanEnrollmentID`) and monitor PAN metadata (Get Card Metadata) without the need to re-enter PAN credentials. Tokens are bound to a Visa-branded payment instrument (`vPanEnrollmentID`), a known device (the device ID specified by the wallet provider as `clientDeviceID`), and a supported presentation method (`presentationType`), such as NFC-HCE, INAPP, ECOM, etc.

Provision Token—Given PAN Enrollment ID Request

Endpoints

Resource Path

vts/panEnrollments/{vPanEnrollmentID}/provisionedTokens

Sandbox Path

```
https://cert.api.visa.com/vts/panEnrollments/{vPanEnrollmentID}/provisionedTokens?apiKey=key&relationshipID=id
```

Live Path

```
https://api.visa.com/vts/panEnrollments/{vPanEnrollmentID}/provisionedTokens?apiKey=key&relationshipID=id
```

URL Parameters

Parameter	Description
{vPanEnrollmentID}	<i>(Required)</i> Unique ID for enrolled PAN. Format: String. Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.

Methods

POST

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as: <code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	<i>(Required)</i> Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
relationshipID	<i>(Optional)</i> The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

Field	Description
encryptionMetaData	<i>(Conditional)</i> Cryptographic payload required for end-to-end encryption of sensitive fields. To be used in messages participating in end-to-end encryption scheme. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 512 characters.
clientAppID	<i>(Required)</i> Unique identifier for the client application provided by the wallet provider. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
clientDeviceID	<i>(Conditional)</i> Stable device identification set by wallet provider. It is required for all requests except for the following presentation types: ECOM, MERCHANT_PRESENTED_QR, QR_CONSUMER_CLOUD, and RECEIVE_ONLY. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; 24 characters.
clientWalletAccountID	<i>(Required)</i> Client-provided consumer ID that identifies the wallet account holder entity. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 24 characters.

Field	Description
ip4address	(Optional) IP address of the device requesting token provisioning in dot-decimal notation consisting of four decimal numbers, each ranging from 0 to 255, separated by dots. Each part represents a group of 8 bits (an octet) of the address. Example: "ip4address": "89.24.143.232" Format: Numeric and periods (.); maximum 15 characters.
location	(Optional) Location of the device up to 4-digits of precision when it is being provisioned. Information will be supplied with latitude/longitude. Example: +xxx.xxxx/-xxx.xxxx. Precision can be rounded to the nearest whole digit. Example: +37/-121. Format: Alphanumeric; maximum 25 characters.
locationSource	(Optional) Source used to identify consumer location. Format: It is one of the following values: • WIFI • CELLULAR • GPS • OTHER
issuerAuthCode	(Conditional) Authentication code provided and encrypted by issuer. It is required if <code>presentationType</code> is <code>QR_CONSUMER_CLOUD</code> and optional for <code>ECOM</code>. The wallet provider should pass this value if it is provided to them by the issuer. Format: Standard base64 encoding per RFC4648.
clientWalletAccountEmailAddressHash	(Required) Hash of email address linked to their wallet account login. Hash algorithm to be provided by Visa. base64 URL-encoded. Format: Base64 encoded string, per RFC4648, with the following exceptions: hyphens (-) replace plus signs (+) and underscores (_) replace forward slash (/) characters; maximum 48 characters.
clientWalletAccountEmailAddress	(Optional) Client-provided email address linked to their wallet account login. Used for consumer service operations as well as risk operations. Format: Valid email address; maximum 48 characters.

Field	Description
protectionType	(Required) Attribute that describes how the token credentials are protected on the device. Format: It is one of the following values: • SOFTWARE • TEE • SE • CLOUD (applicable to e-commerce use-cases only)
presentationType	(Required) Token presentation mode. Format: It is an array containing any of the following enum elements: • NFC-HCE - To be used on the device for NFC payments • NFC-SE - To be used for NFC payments with devices supporting secure element. • ECOM - e-commerce presentation type • INAPP - In App presentation type • MSR - MSR presentation type • MERCHANT_PRESENTED_QR - Used when token needs to be used for scan-and-pay / merchant presented QR use case. • QR_CONSUMER_CLOUD - The token supports presenting a QR Code on the device. The token is stored in the cloud. • RECEIVE_ONLY - This token supports one-way person to micro-business transactions. GetPaymentData calls will be rejected. • QR_CONSUMER_DEVICE_HCE - This token supports presenting a QR code on the consumer's device. The token is stored on the device. It is an HCE token; therefore: – LUK is managed by the device, – ATC is managed by the device, and – Cryptogram and QR are generated by the device. • AUTO_FILL Note: Shared Secret is always used to encrypt the API response fields when the presentation type is: • ECOM • QR_CONSUMER_CLOUD • RECEIVE_ONLY
termsAndConditions	(Conditional) Terms and Conditions that were accepted by the consumer. Format: A termsAndConditions structure.

Field	Description
accountType	(Conditional) An attribute that describes the type of account. Applicable in e-commerce use cases. Format: It is one of the following values: • GUEST • WALLET
encRiskDataInfo	(Optional) To be used to convey data related to risk decision. Refer to the Addendum for a full description. Actual risk data elements will be shared separately outside of the technical specification. (Maximum number of Name/Value pairs allowed - 100). riskData will be encrypted using API Key/Shared Secret. For more information, see "JSON Web Encryption and Object Signing." Format: Alphanumeric; maximum 10240 characters.
ssdData	(Conditional) Container for all data that is specific to the SD in the chip. Required if presentationType is NFC-SE, SSD Data needs to be provided. Format: An ssdData structure.
channelSecurityContext	(Conditional) Present in case of a PAN entry on the companion device with encryption using device certs or OPACITY_A scheme. Required if presentationType is NFC-HCE or NFC-SE, except it must not be present for NFC-SE in the case of Provision with Pan Enroll ID. Format: A channelSecurityContext structure.
platformType	(Conditional) The step-up option app-to-app details in the response is based on the platform type. Required if presentationType is NFC-SE. Format: It is one of the following values: • iOS – iOS app • ANDROID – Android app • WINDOWS – Windows app • WEB – Browser-based app
seData	(Conditional) This field is needed when the caller asks for seData instead of APDU. Format: An seData object.

Terms and Conditions

Field	Description
id	(Optional) ID for Terms and Conditions that were accepted by the consumer. It is not returned for Card on File. Format: Alphabetic, numeric; maximum 36 characters.
date	(Optional) Timestamp when Terms and Conditions are acted upon by consumer. It is not returned for Card on File. Format: Date. UTC Format in epoch seconds The UNIX_UTC_timestamp is a UNIX Epoch timestamp. The value is in seconds.

SSD Data

Field	Description
VMPAVersion	(Required) Denotes the version of the VMPA on the chip from where the provisioning request is originating. Format: Hex String. Size: 128.
sdScript	(Conditional) For tokens sharing same SD, send only for first time provisioning. For tokens not sharing same SD, send always. Both <code>sdScript</code> and <code>sdAID</code> must not be passed in the same request. VMPA Applet Instantiation with appropriate AID and other APDU commands should be sent in a single script. It is the APDU response blob containing encrypted SCP Key(s) in case of PULL scenario or the response to the generate keys command in case of ECKA scenario. Format: String. Size: 2048.
aidArray	(Required) Each element contains AID + Test sequence number. Format: Size: 32 per array element.
keyVersionNumber	(Required) Key Version Number of the SCP key in the SD. Format: Numeric; maximum 2 digits.
sequenceCounter	(Required) Sequence number for the SCP key in the SD, hex-encoded integer padded left with zeros (0). Format: Numeric; maximum 6 digits. It is a hex-encoded integer. Note: For SCP02, the max size is 4. For SCP03, the max size is 6.

Field	Description
CASDCert	(Conditional) Required for tokens sharing the same SD. Send only for first time provisioning; otherwise, for tokens not sharing same SD, send always. CASD Cert • Supports only 2048-bit RSA key based cert or ECC-P256. • APDU response blob containing CASD cert • Key identifier for the root public key of the CA • Contains the signature as per ISO-9796-2 • Certificate must follow tags style format as per GlobalPlatform specifications. May contain several certificates with tag 7F21. The applicable casd cert can be identified using tag 95. Format: String. Size: 20K.
vCertificateID	(Conditional) Unique ID for Visa Certificate. Required for tokens sharing same SD. Send only for first time provisioning; otherwise, for tokens not sharing same SD, send always. Not present for the ECKA use case. This will be given to Wallet Provider at onboarding time. Devices can use this certificate ID while communicating with VTS. This will be first 4 bytes of the Visa Public Key modulus. The key size will be of 2048 bit length. Format: Alphabetic, numeric; maximum 32 characters.
vEphemeralVisaPublicKeyID	(Conditional) Key ID generated by VTS. Required only for the ECKA use case. This is an identifier that is sent the first time for a token sharing the same SD and each time for any tokens that do not share the same SD. Format: Identifier; max size: 36 bytes

Field	Description
<code>sdAID</code>	(Required) For SCP_02/PULL, BOTH the following conditions apply: • Required when the chip is using the single SD model and wants to send an indicator. • Required to be passed whenever at least one card has been provisioned already and additional cards are now being provisioned. It is not passed for the initial card provisioning, but only for provisioning subsequent cards. When provisioning the first card, the <code>sdAID</code> will be passed in the <code>sdScript</code>. Both <code>sdScript</code> and <code>sdAID</code> cannot be passed in the same request. For SCP_03/ECKA, always specify a value. Format: Alphabetic, numeric; maximum 32 bytes.
<code>generateScpKeysSdScript</code>	(Conditional) Required only for the ECKA use case. This script is sent the first time for a token sharing the same SD and each time for any tokens that do not share the same SD. Format: String. Maximum 2048 characters.

Channel Security Context

Field	Description
<code>channelInfo</code>	Security Channel used to communicate between the device and the Visa Token Service back-end. Format: A <code>channelInfo</code> structure.
<code>vtcCerts</code>	(Conditional) Visa certificate IDs. Format: An array of <code>vtcCerts</code> structures.
<code>deviceCerts</code>	(Conditional) Device certificates. Not required for autofill. Format: An array of <code>deviceCerts</code> structures.
<code>keyData</code>	(Conditional) Key data; required for OPACITY_A; otherwise, it is not used. In the enroll PAN and Provision with PAN APIs, <code>keyData</code> and <code>encryptionMetaData</code> must now be present together. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 512 characters.

Channel Info

Field	Description
encryptionScheme	(Conditional) The security scheme used to secure communication between the device and the Visa Token Service back-end. Note: For Secure Copy (SCP) encryption schemes, the encryption version and the type are concatenated to create a string in the form of <code>scp_version/type</code> as per Global Platform Amendment A. Format: It is one of the following values: • RSA_PKI • OPACITY_A • SCP_02/PULL

Visa Token Service Certs

Field	Description
vCertificateID	(Required) Unique ID for Visa Certificate. This will be given to the wallet provider during onboarding. Devices can use this certificate ID while communicating with Visa Token Service . This will be the first 4 bytes of the Visa Public Key. Format: String. Size: 32.
certUsage	(Conditional) The usage of this certificate. Format: It is one of the following values: • CONFIDENTIALITY • INTEGRITY

Device Certs

Field	Description
certUsage	(Conditional) The usage of this certificate. Format: It is one of the following values: • CONFIDENTIALITY • INTEGRITY • DEVICE_ROOT

certFormat	(Conditional) X509 certificate structure. Format: String [A-Za-z0-9]+. Size: 16.
certValue	(Conditional) In PEM format/Base64URL safe encoded. Format: String[A-Za-z0-9]+. Size: 8192.

SE Data

Field	Description
VMPAVersion	(Required) This field denotes the version of the VMPA on the chip from where the provisioning request originates. Format: Hex string; maximum 128 characters.

Provision Token—Given PAN Enrollment ID Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
vProvisionedTokenID	Unique ID for token. Format: String. Alphabetic, numeric [A-Z][a-z][0-9;-], and hyphens (-), e.g. spaces are not allowed; maximum 36.
paymentInstrument	Last 4 digits and expiration date of the PAN associated with the enrolled account. Format: A <code>PaymentInstrument</code> structure.
stepUpRequest	Step-up request. Format: A <code>StepUpRequest</code> structure.
tokenInfo	Data associated with the token. Format: A <code>tokenInfo</code> structure.
encryptionMetaData	Cryptographic payload required for end-to-end encryption of sensitive fields. This field is conditional if the encryption scheme is OPACITY. Format: Alphabetic, numeric; maximum 512 characters.
ODadata	Returned if the token can be used for Offline Data Authorization (ODA). If <code>ODadata</code> is returned, then all the fields in <code>ODadata</code> will be returned. Format: A <code>ODadata</code> structure.

Payment Instrument

Field	Description
last4	Last 4 digits of the account. Note: This will be the last 4 digits of the IBAN if an IBAN was provided in the provisioning request. Format: Numeric; maximum 4 digits.
expirationDate	Payment instrument's expiration date. Format: An <code>expirationDate</code> structure.
accountStatus	<i>Future:</i> Is the account closed? To be returned with Get Card Meta Data API only. Format: It is one of the following values: • Y – Account closed • N – Account not closed

Field	Description
<code>isTokenizable</code>	<i>Future:</i> Is this account eligible for tokenization? Format: It is one of the following values: • Y – Eligible • N – Not Eligible
<code>indicators</code>	These are indicators (Boolean values) associated with the Payment Instrument. If the array is not returned, or empty, then there are no indicators associated with it. Expected values are: • PRIVATE_LABEL - This Payment Instrument is for a Private Label PAN. Format: String array.
<code>expDatePrintedInd</code>	Indicates if the expiration date is printed on the payment instrument. Format: It is one of the following values: • Y - Printed • N - Not printed
<code>cvv2PrintedInd</code>	Indicates if the CVV2 is printed on the payment instrument. Format: It is one of the following values: • Y - Printed • N - Not printed
<code>paymentAccountReference</code>	Payment Account Reference (PAR) if available for this payment instrument. Format: Alphanumeric; maximum 29 characters.
<code>verificationResults</code>	Payment instrument verification results. Present only for <code>ECOM</code> presentation type when the CVV2 and/or address fields appear in the provision request. Format: A <code>verificationResults</code> object.

Expiration Date

Field	Description
month	The month upon which the payment instrument is set to expire. Example: "month": "9" Format: String; numeric; max length 2 digits.
year	The year upon which the payment instrument is set to expire. Example: "year": "2024" Format: String; numeric; length 4 digits.

Enabled Services

Field	Description
merchantPresentedQR	Whether the issuer enabled the PAN for the scan-and-pay use case. Format: It is one of the following String values: • Y – Enabled • N – Disabled

Verification Results

Field	Description
<code>cvv2VerificationCode</code>	The CVV2 verification result. This field is present when CVV2 verification data is present in the request. Format: It is one of the following values: • M – verified, CVV2 data match • N – verified, negative data match • U – verification not possible (issuer does not participate) • D – verification not performed due to duplicate request
<code>addressVerificationCode</code>	The address verification result. This field is present when address verification data is present in the request. Format: It is one of the following values: • Y – verified, address and postal code data match • P – verified, partial match (either address or postal code data match, but no result from the other) • B – verified, partial match (either address or postal code data match, but the other could not be verified due to incompatible format) • N – verified, neither address nor postal code data match • U – verification not possible (issuer does not participate) • D – verification not performed due to duplicate request

Step-Up Request

`stepUpRequest` is conditional. It is not required for the Ecommerce token type. Items 2, 3, and 4 below are applicable to push provisioning only.

Note: This is only applicable when:

- `TokenStatus` is `INACTIVE`.
- Device ID used during push provisioning does not match the `DeviceID` used during enrollment.
- Issuer indicated a preference for IDnV in the JWE by setting `isIDnV=Y`.
- Push Provisioning expired.

Field	Description
method	ID&V step-up method. Format: It is one of the following values: • OTPEMAIL • OTPONLINEBANKING • OTPSMS • CUSTOMERSERVICE • APP-TO-APP • OUTBOUNDCALL
value	This could be a masked value depending on the ID&V step-up method. Email looks like 'ip****@email.com' bank_app looks like 'Mobile Banking App' customer_service looks like '1-800-555-1212' Note: This field will support internationalization using UTF-8 characters. Format: Alphanumeric, UTF-8 whitespace; maximum 64 characters.
source	The issuer source of contact for this ID&V step-up challenge. In case of OTP, the source contains the issuer contact that will reach out to the consumer. In the case of banking-app step-up, the source is a concatenation of Application ID (<i>id</i>) and Intent Handler (<i>h</i>). This field will be defined by the issuer. Note: Method handler of Banking App designed to respond to authentication code handling. For the Android platform this will be an explicit intent with fully name-spaced class name. Visa requires the issuer to use the a2a name for this intent handler. Example: "source": "{id} {h}", where <i>id</i> is required (size 60, [a-z,A-Z,0-9,_,.]), and <i>h</i> is optional (size 3, [a-z][A-Z][0-9]). Format: Alphanumeric; maximum 1024 characters.
identifier	Unique and opaque identifier for this method. A subsequent step-up request will only provide the identifier. The wallet provider must send this value "as is" to subsequent step-up related APIs. Format: Alphabetic, numeric, plus signs (+), slashes (/), and equal signs (=), spaces are not allowed; maximum 96 characters.

Field	Description
requestPayload	The request payload to be sent to the banking application on behalf of Visa. This field is opaque to wallet providers. Note: This is only present if the step-up type is app-to-app. Format: Base64 encoded string, maximum 1024 characters.
platformType	Platform type, which is present in cases where the Token Requestor has opted in to 3DS Web Step-Up. Format: It is one of the following values: • WEB
subMethod	Flavor of the step-up method. Format: It is one of the following values: • THREE-DS

Token Info

Field	Description
tokenStatus	Token status. If a token was deleted, no information will be returned for the token. Format: It is one of the following values: • INACTIVE • ACTIVE • SUSPENDED
tokenRequestorID	Unique ID that identifies the entity for which the token request is being conducted. Format: Numeric; maximum 11 digits.
tokenReferenceID	Unique ID associated with a single token. The ID is created upon initial provisioning of a token to a device. Format: Alphabetic, numeric; maximum 32 characters.
last4	Last 4 digits of the token. Format: Numeric; maximum 4 digits.
expirationDate	Token expiration date in UTC format. Format: An <code>expirationDate</code> structure.

Field	Description
appPrgrmID	Visa proprietary data element identifying the application program ID of the card application; not applicable to NFC-SE. When personalized, the application program ID is returned in the FCI issuer discretionary data of the select response (tag 'BF0C'). qVSDC readers that support Dynamic Reader Limits (DRL) functionality examine the application program ID to determine the reader limit set to apply. Format: Alphabetic, numeric; 10–32 characters.
encTokenInfo	Encrypted token payload. For more information, see "JSON Web Encryption and Object Signing." This field is not returned for NFC-SE presentation types. Note: The unencrypted field contains a token as a 13-16 character string. Format: Alphabetic, numeric; maximum 4096 characters.
hceData	HCE data. Only applicable for NFC-HCE use case; not applicable for NFC-SE use case. Format: An <code>hceData</code> structure.
mst	MST data. Only applicable for NFC-HCE use case when the <code>presentationType</code> is <code>MST</code>; not applicable for NFC-SE use case. Format: An <code>mst</code> structure.
seCardPerso	APDU commands. Present in the case of NFC-SE use cases except when APDU script is handled by another entity. Format: An APDU JSON structure.
sePersoData	Perso application data. Present when APDU script is handled by another entity. Format: An <code>sePersoData</code> structure.
atc	Transaction counter; present in HCE tokens. Format: Numeric; 5 digits.

Expiration Date

Field	Description
<code>month</code>	The month upon which the payment instrument is set to expire. Example: "month": "9" Format: String; numeric; max length 2 digits.
<code>year</code>	The year upon which the payment instrument is set to expire. Example: "year": "2024" Format: String; numeric; length 4 digits.

hceData

Field	Description
<code>dynParams</code>	HCE data dynamic parameters. Format: A <code>dynParams</code> structure.
<code>staticParams</code>	HCE data static parameters. Format: A <code>staticParams</code> structure.

Static Parameters

Field	Description
<code>aidInfo</code>	Format: An array of <code>aidInfo</code> structures.
<code>kernelIdentifier</code>	Returned for Durbin (US Debit). Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 8 characters.
<code>cardHolderNameVCPCS</code>	Cardholder name per ISO 7813. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 26 characters.
<code>pdol</code>	List of terminal/reader- related data objects (tags and lengths) requested by the card to be transmitted in the GET PROCESSING OPTIONS command. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 48 characters.

Field	Description
countrycode5F55	Returned for Durbin (US Debit). Source: Issuer Configuration File. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 4 characters.
issuerIdentification Number	Returned for Durbin (US Debit). Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Numeric; maximum 6 digits.
MsdData	Format: A MsdData structure.
qVSDCData	Format: A qVSDCData structure.
track2DataDec	Format: A track2DataDec structure.
track2DataNotDec	Format: A track2DataNotDec structure.
qrConsumerDeviceData	Format: A qrConsumerDeviceData structure

Note:

Provisioning response payload related to qVSDC and MSD profiles. Paywave elements have conditional use, qVSDC response will have track2datanotdec whereas MSD/qVSDC will have msdData and track2datadec fields populated.

AID Info Extended

Field	Description
applicationLabel	Mnemonic associated with the AID according to [ISO 7816- 5]. Used in application selection. Application Label is optional in the File Control Information (FCI) of an Application Definition File (ADF) and optional in an ADF directory entry for VCPS. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; 8-32 characters.
aid	Identifies the application as described in ISO 7816-5. It may be extended to partnering merchants and third-party wallet providers. However, as of now, it is specific to issuing banks. Source: Assigned by Visa during issuer onboarding. Note: aidInfo is an array of aids and related data. Review the provision response JSON example at the end of this section. Format: Alphabetic, numeric; 10-32 characters.

Field	Description
priority	Priority Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 2 characters.
CVMrequired	Indicates if CVM is required. It is one of the following values: • Y - CVM is required • N - CVM is not required Format: Alphabetic; maximum 1 character.
cap	Indicates card processing requirements and preferences such as type of cardholder verification methods supported by the issuer. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 8 characters
asrpd	The field describes the European Economic Area Interchange Fee Regulation Classification for any tokens provisioned in Europe. This value can be passed as part of the personalization script for SE tokens, but will have to be populated here for HCE tokens. The field contains token personalization information as described below: • 9F0A0400010101 if the funding source is a debit card and the card product type is debit • 9F0A0400010102 if the funding source is a credit card, charge, or a deferred debit and the card product type is credit • 9F0A0400010103 if the card product type is commercial • 9F0A0400010104 if the funding source is a prepaid card, and the card product type is prepaid The information can be interpreted, left-to-right, as follows: • 9F0A - EMVCo assigned tag • 04 - Length of all data within tag • 0001 - Identifier assigned by EMVCo for European regulatory usage • 01 - Length of the Product ID • 01 through 04 - Values to identify debit, credit, commercial or prepaid products (above) Format: Alphabetic, numeric; maximum 14 characters.

Field	Description
appPrgrmID	This is a Visa proprietary data element identifying the Application Program ID of the card application. When personalized, the Application Program ID is returned in the FCI Issuer Discretionary Data of the Select response (tag BF0C). qVSDC readers that support Dynamic Reader Limits (DRL) functionality examine the Application Program ID to determine the Reader Limit Set to apply. Format: Alphabetic, numeric; 10–32 characters.
mst	MST is only applicable if the request was for the MST presentation type. Will be returned for NFC-HCE use case only, and not applicable for NFC-SE. Format: An msr data structure.
kernelIdentifier	Returned for Durbin (US Debit). Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 8 characters.
cardHolderNameVCPCS	Cardholder name per ISO 7813. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; 2-26 characters.
pdol	List of terminal/reader- related data objects (tags and lengths) requested by the card to be transmitted in the GET PROCESSING OPTIONS command. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 48 characters.
countrycode5F55	Returned for Durbin (US Debit) Source: Issuer Configuration File. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 4 characters
issuerIdentification Number	Returned for Durbin (US Debit). Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 6 characters.
track2DataDec	Format: A track2DataDec structure.
MsdData	Format: A MsdData structure.
track2DataNotDec	Format: A track2DataNotDec structure.
qVSDCData	Format: A qVSDCData structure.

Field	Description
ODADData	ODADData will be returned if the token can be used for Offline Data Authorization (ODA). If ODADData is returned, all the fields in ODADData will be returned. Format: An ODADData structure.
issuerApplication DiscretionaryData	Issuer Application Discretionary Data (AID IDD). Format: String. Size: 15 bytes, hex-encoded.
issuerDiscretionaryData	Dynamic issuer perso data to be injected into FCC_IDD data. It is hex-encoded. Format: String. Size: 128 characters.

Track2 Data Dec

Field	Description
svcCode	Service code that identifies the components necessary to compile Track 2 equivalent data per <i>Visa Contactless Payment Specification</i> . Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 3 characters.

MSD Data

Field	Description
aip	Application Interchange Profile that contains information about the technology the card supports, such as MSD or qVSDC. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; 2 hexadecimal digits. For hex representation, two characters are used to represent one hex byte.
afl	Application file locator that indicates the location (SFI, range of records) of the AEFs related to a given application. As with all data elements returned in the GPO response, the card application supports personalization of different AFL data elements for use in the following GPO responses: • qVSDC Offline GPO Response (if offline implemented) • qVSDC Online with ODA GPO Response • qVSDC Online/Decline without ODA GPO Response • MSD CVN17 with ODA GPO Response • MSD CVN17 without ODA GPO Response • MSD Legacy GPO Response Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 8 characters.

Track2 Data Not Dec

Field	Description
svcCode	Service code that identifies the components necessary to compile Track 2 equivalent data per the <i>Visa Contactless Payment Specification</i>. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 3 characters.
pinVerField	Necessary to compile Track 2 Equivalent data for the <i>Visa Payment Contactless Specification</i> Format: Alphabetic, numeric; maximum 5 characters.
track2DiscData	Track 2 disc data. Discretionary Data (Partly iCCV data). Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; 8-13 characters.

qVSDC Data

Field	Description
ctq	Card transaction identifiers used by the device to determine the card CVM requirements, issuer preferences and card capabilities. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 4 characters.
ced	Customer exclusive data that is transmitted to issuer. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 64 characters.
ffi	Form factor indicating the kind of consumer payment device and contactless interface over which the transaction was conducted. This information is made available to the issuer. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 8 characters
auc	Application usage control which is an ISO 3166 code identifying the issuer's country. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 4 characters.
psn	PAN sequence number used in encryption. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Numeric; maximum 2 digits.
cvn	Most significant nibble: '4': this IAD format. Least significant nibble: '3': Limited Use Key Cryptogram. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 2 characters.
digitalWalletID	VTS-defined unique ID that identifies the wallet provider entity from which the external request is being sent. This ID will be provided to the wallet provider during onboarding to VTS. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 8 characters.

Field	Description
countryCode	An ISO 3166 code identifying the issuer's country. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 4 characters.
cid	Indicates the type of cryptogram (TC, ARQC, or AAC) returned by the card and the actions to be performed by the reader. Source: Issuer configuration file. Format: Alphabetic, numeric; maximum 2 characters.
qVSDCWithoutODA	Format: A qVSDCWithoutODA structure.

qVSDC Without ODA

Field	Description
aip	Application Interchange Profile which contains information about the technology the card supports, such as MSD or qVSDC. Format: Alphabetic, numeric; maximum 4 characters.
afl	Application File Locator, which indicates the location (SFI, range of records) of the AEFs related to a given application. As with all data elements returned in the GPO response, the card application supports personalization of different AFL data elements for use in the following GPO responses: • qVSDC Offline GPO Response (if offline implemented) • qVSDC Online with ODA GPO Response • qVSDC Online/Decline without ODA GPO Response • MSD CVN17 with ODA GPO Response • MSD CVN17 without ODA GPO Response • MSD Legacy GPO Response Format: Alphabetic, numeric; maximum 8 characters.

QR Consumer Device Data

Field	Description
ced	Customer exclusive data that is transmitted to the issuer. Source: Issuer Configuration File. This data is configured by the issuer during VTS onboarding. Format: Alphabetic, numeric; maximum 64 characters.
cvn	'44' Format: Numeric; maximum 2 characters.

Field	Description
digitalWalletID	VTs-defined unique ID that identifies the wallet provider entity from which the external request is being sent. This ID is provided to the wallet provider after onboarding onto the VTS platform. Source: Issuer Configuration file. This data is configured by the issuer during VTS onboarding. Format: Alphabetic, numeric; maximum 8 characters.
aip	Application Interchange Profile that contains information about the technology the card supports such as MSD or qVSDC. Format: Alphabetic, numeric; maximum 2 characters.

Dynamic Parameters

Field	Description
encKeyInfo	Encrypted new, limited-use key for a given provision token ID. This value is used for cryptogram generation and is updated during replenishment requests. It is encrypted using the wallet provider's preferred encryption scheme (for example, _A). Format: Alphabetic, numeric; maximum 32 characters. For more information, see "JSON Web Encryption and Object Signing."
maxPmts	Maximum number of payments per provisioned token before replenishment is required Source: VTS. Format: Numeric; maximum 99999 characters.
api	Account parameter index used to generate cryptograms. Source: Generated by VTS and associated with token request/replenishment. Format is defined by VCPCS 1.8. Format: Alphabetic, numeric; maximum 8 characters.
sc	Sequence counter, which is used to generate cryptograms. Tracked in SDK, the sequence counter will be incremented upon replenishment call. Source: VTS generated. Format: Alphabetic, numeric; maximum 65535 characters.

Field	Description
keyExpTS	Expiration timestamp associated with the LUK time-to-live. Source: VTS. Format: Numeric, UTC Format in epoch seconds.
dki	Derived key indicator defining the key index used to calculate the cryptogram. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 2 characters.

MSR Data

Field	Description
svcCode	Service code which identifies the components necessary to compile Track 2 equivalent data per the <i>Visa Contactless Payment Specification</i> . Format: Alphabetic, numeric; maximum 3 characters.
cvv	Card verification value. Format: Numeric; 3-4 digits.

Note:

Provisioning response payload related to qVSDC and MSD profiles. Paywave elements have conditional use, qVSDC response will have track2datanotdec whereas MSD/qVSDC will have msdData and track2datadec fields populated.

APDU JSON

This is the `seCardPerso` structure.

Field	Description
appletInstanceAID	Full AID of the card instance or application. Format: String; max 20 characters.
priorityCode	Priority for the AID. Format: String; max 4 characters.
apduPerso	APDU commands. Format: An array of <code>ApduLines</code> structures.

APDU Lines

Field	Description
apduID	SerialID for each APDU command. Format: String; max 20 characters.
apdu	APDU command. Format: String; max 4 characters.

Perso Application Data

Field	Description
persoApplicationData	AID data. Format: An array of <code>PersoElement</code> structures.

Perso Element

Field	Description
dataTag	EMV/VMPA tag. Format: String; max 4 characters.
dataName	Data element name. Format: String; max 512 characters.
dataValue	Perso data element. Format: String; max 1024 characters.

ODA Data

This data will only be returned for NFC-HCE use cases. ODA data for NFC-SE will be part of the `seCardPerso` script. Will be returned if the token is enabled for Offline Data Authorization (ODA).

Field	Description
appFileLocator	Application File Locator. Format: Alphabetic, numeric; maximum 10 characters.
appProfile	Application Interchange Profile. Format: Alphabetic, numeric; maximum 5 characters.

caPubKeyIndex	Certificate Authority Public Key Index. Index to identify the payment system public key used for dynamic data authentication (for ODA). Format: Alphabetic, numeric; maximum 3 characters.
iccPubKeyCert	Integrated Circuit Card (ICC) Public Key Certificate.
tokenBinPubKeyCert	Token-BIN Public Key certificate. This is also known as Issuer Public Key Certificate.
encicccPrivateKey	JWE/JWS formatted encrypted data. Format: For more information, see "JSON Web Encryption and Object Signing."

ODA ICC Public Key Cert

Field	Description
certificate	Public Key certified by the Visa CA for offline data authorization. Format: Alphabetic, numeric, plus sign (+), and equal sign (=), e.g. spaces are not allowed; 236-352 characters.
exponent	Public Key Exponent used for the generation of the Signed Dynamic Application Data. Format: Alphabetic, numeric; 2-6 characters.
remainder	Digits of the Public Key Modulus which do not fit within the Public Key Certificate. Format: Alphabetic, numeric, plus sign (+), and equal sign (=), e.g. spaces are not allowed; maximum 84 characters.
expirationDate	Date after which the Public Key cert expires. Format: An <code>expriationDate</code> structure.

ODA Token Bin Public Key Cert

Field	Description
certificate	Public Key certified by the Visa CA for offline data authorization. Format: Alphabetic, numeric, plus sign (+), and equal sign (=), e.g. spaces are not allowed; 236-352 characters.
exponent	Public Key Exponent used for the generation of the Signed Dynamic Application Data Format: Hexadecimal representation (binary) of an integer, using Big Endian format. 2-6 characters.

remainder	Digits of the Public Key Modulus which do not fit within the Public Key Certificate. Format: Alphabetic, numeric, plus sign (+), and equal sign (=), e.g. spaces are not allowed; maximum 84 characters.
expirationDate	Date after which the Public Key cert expires. Format: An <code>expriationDate</code> structure.

Expiration Date

Field	Description
month	The month upon which the payment instrument is set to expire. Example: "month": "9" Format: String; numeric; max length 2 digits.
year	The year upon which the payment instrumet is set to expire. Example: "year": "2024" Format: String; numeric; length 4 digits.

ODA ICC CRT Private Key Field

Field	Description
iccCRTPrivateKey	Decrypted Chinese Remainder Theorem (CRT) ICC Private Key. If <code>enciccPrivateKey</code> is provided, this field is required. All the fields are used as an input into Chinese Remainder Theorem algorithm required for generation & verification of SDAD for ODA. Format: Alphabetic, numeric; 192–308 characters.

ODA ICC CRT Private Key

Field	Description
modulus	ICC Private Key CRT modulus. Format: Hexadecimal representation (binary) of an integer, using Big Endian format. 192-308 characters.
exponent	ICC Private Key CRT Exponent. Format: Hexadecimal representation (binary) of an integer, using Big Endian format. 192-308 characters.

primeP	ICC Private Key CRT constant prime factor p. Format: Hexadecimal representation (binary) of an integer, using Big Endian format. 96-154 characters.
primeQ	ICC Private Key CRT constant prime factor q. Format: Hexadecimal representation (binary) of an integer, using Big Endian format. 96-154 characters.
CoefDmodP	ICC Private Key CRT constant d mod (p-1). Format: Hexadecimal representation (binary) of an integer, using Big Endian format. 96-154 characters.
CoefDmodQ	ICC Private Key CRT constant d mod (q-1). Format: Hexadecimal representation (binary) of an integer, using Big Endian format. 96-154 characters.
CoefQinvModP	ICC Private Key CRT constant q-inverse mod p. Format: Hexadecimal representation (binary) of an integer, using Big Endian format. 96-154 characters.

Provision Token—Given PAN Enrollment ID Errors

Error Code	Reason Code	Message
400	msrNotSupported	MSR presentation type is not supported for this PAN. Remove this presentation type and retry the request.
400	provisionDataExpired	The PAN information provided is considered stale. If this is a Push Provisioning flow, which allows the user to choose a card from their banking application or a web page, the user should be asked to choose the card again. If <code>encPaymentInstrument</code> was prepared by the wallet provider, then the wallet provider should create it again and call the Enroll PAN API before invoking the Provision Token API.
403	cardVerificationFailed	Invalid field. Invalid payment instrument or data associated with the payment instrument. The “location” field may point to the field failing the validation. Ask users to check that all information was entered correctly.
403	cardNotEligible	This card cannot be used for tokenization at this moment. Please try again later or choose another Visa card.

Error Code	Reason Code	Message
403	cardNotAllowed	The requested action is not allowed for a given PAN. This card is valid but cannot be used for tokenization. This is most likely for Visa-specific reasons. Ask the consumer to enter a different card.
403	declined	This card is considered not eligible for tokenization at this time. Try enroll/provision again for this PAN. If you encounter the same message for the new enrollment/provision request, ask the customer to either add a different card or to reach out to their issuer.
409	staleCardMetadata	Card Metadata is stale. Please request the latest card metadata and retry.
409	notAllowed	Further operations for this card are no longer allowed. Contact your bank to resolve this issue.

Provision Token—Given PAN Enrollment ID Examples

Provision Token-Given PAN Enrollment ID Example

Request

Endpoint:

```
https://cert.api.visa.com/vts/panEnrollments/  
e53...01/provisionedTokens?apikey=GPN...3I
```

Request Body:

```
{  
  "clientAppID": "AppID1234",  
  "termsAndConditions": {  
    "id": "13c5e2ba-eb5c-432d-a027-97ddaf109af1",  
    "date": "111115"  
  },  
  "clientDeviceID": "1zDc22PW8c9gU8Ui17Z5jrN9",  
  "clientWalletAccountID": "1234",  
  "presentationType": [  
    "NFC-HCE",  
    "INAPP"  
  ]  
}
```



```
],  
"encRiskDataInfo": "eyJ...iQ",  
"protectionType": "SOFTWARE",  
"clientWalletAccountEmailAddress": "vtstests@visa.com",  
"clientWalletAccountEmailAddressHash": "2DX...L4"  
}
```

Response

```
{  
  "paymentInstrument": {  
    "last4": "0800",  
    "expirationDate": {  
      "ignore02field": "ECAA8E42",  
      "month": "12",  
      "year": "2022"  
    },  
    "cvv2PrintedInd": "Y",  
    "ignore01field": "3522B46F",  
    "expDatePrintedInd": "Y",  
    "paymentAccountReference": "V00...60",  
    "enabledServices": {  
      "merchantPresentedQR": "Y"  
    }  
  },  
  "vProvisionedTokenID": "8ec...02",  
  "ignore00field": "4484FB17",  
  "tokenInfo": {  
    "appPrgrmID": "1008400840",  
    "encTokenInfo": "eyJ...3g",  
    "hceData": {  
      "staticParams": {  
        "qVSDCData": {  
          "ctq": "0100",  
          "ffi": "238C0000",  
          "auc": "FF80",  
          "psn": "00",  
          "cvn": "43",  
          "digitalWalletID": "000000000",  
          "countryCode": "0840",  
          "cid": "80",  
          "qVSDCWithoutODA": {  
            "afl": "08030300",  
            "aip": "0040"  
          }  
        }  
      },  
      "kernelIdentifier": "03",  
      "cardHolderNameVCPCS": "",  
      "pdol": "9F6...04",  
      "msdData": {  
        "aip": "00C0",  
        "afl": "08010100"  
      },  
      "track2DataDec": {  
        "svcCode": "201"  
      }  
    },  
  },  
}
```

```
"countrycode5F55":"5553",
"issuerIdentificationNumber":"489537",
"aidInfo":[
  {
    "qVSDCData":{
      "qVSDCWithODA":{
        "afl":"10020400",
        "aip":"2040"
      },
      "ctq":"0100",
      "ffi":"238C0000",
      "auc":"FF80",
      "psn":"00",
      "cvn":"43",
      "digitalWalletID":"00000000",
      "countryCode":"0840",
      "cid":"80",
      "qVSDCWithoutODA":{
        "afl":"08030300",
        "aip":"0040"
      }
    },
    "appPrgrmID":"1008400840",
    "kernelIdentifier":"03",
    "cardHolderNameVCPCS":"",
    "pdol":"9F66049F02069F03069F1A0295055F2A029A039C019F3704",
    "countrycode5F55":"5553",
    "issuerIdentificationNumber":"489537",
    "track2DataDec":{
      "svcCode":"201"
    },
    "msdData":{
      "aip":"00C0",
      "afl":"08010100"
    },
    "ODADData":{
      "appFileLocator":"10020400",
      "appProfile":"2040",
      "caPubKeyIndex":"89",
      "encicccPrivateKey":"eyJ...Ew",
      "iccPubKeyCert":{
        "certificate":"bLn...ho=",
        "exponent":"03",
        "remainder":"",
        "expirationDate":{
          "month":"10",
          "year":"2018"
        }
      },
      "tokenBinPubKeyCert":{
        "certificate":"pJI+...vc",
        "exponent":"03",
        "remainder":"DYz...0s=",
        "expirationDate":{
          "month":"12",
          "year":"2023"
        }
      }
    }
  }
]
```

```
,
"applicationLabel":"555...74",
"aid":"A0000000980840",
"priority":"02",
"cap":"0000C000",
"CVMrequired":"N"
},
{
  "qVSDCData":{
    "qVSDCWithODA":{
      "afl":"10020400",
      "aip":"2040"
    },
    "ctq":"0100",
    "ffi":"238C0000",
    "auc":"0080",
    "psn":"00",
    "cvn":"43",
    "digitalWalletID":"00000000",
    "countryCode":"0840",
    "cid":"80",
    "qVSDCWithoutODA":{
      "afl":"08030300",
      "aip":"0040"
    }
  },
  "appPrgrmID":"1008400840",
  "kernelIdentifier":"03",
  "cardHolderNameVCPCS":"",
  "pdol":"9F6...04",
  "countrycode5F55":"5553",
  "issuerIdentificationNumber":"489537",
  "track2DataDec":{
    "svcCode":"201"
  },
  "msdData":{
    "aip":"00C0",
    "afl":"08010100"
  },
  "ODADData":{
    "appFileLocator":"10020400",
    "appProfile":"2040",
    "caPubKeyIndex":"89",
    "encicccPrivateKey":"eyJ...Ew",
    "icccPubKeyCert":{
      "certificate":"bLn...ho=",
      "exponent":"03",
      "remainder":"",
      "expirationDate":{
        "month":"10",
        "year":"2018"
      }
    },
    "tokenBinPubKeyCert":{
      "certificate":"pJI...vc",
      "exponent":"03",
      "remainder":"DYz...0s=",
      "expirationDate":{
```

```
        "month": "12",
        "year": "2023"
      }
    },
    {
      "applicationLabel": "566...74",
      "aid": "A0000000031010",
      "priority": "01",
      "cap": "00001800",
      "CVMrequired": "N"
    }
  ],
  "dynParams": {
    "encKeyInfo": "eyJ...DA",
    "maxPmts": 5,
    "api": "08581800",
    "sc": 1,
    "keyExpTS": 1537004378,
    "dki": "01"
  },
  "tokenRequestorID": "400...52",
  "tokenReferenceID": "DNI...48",
  "tokenStatus": "ACTIVE",
  "last4": "6472",
  "expirationDate": {
    "month": "12",
    "year": "2025"
  },
  "ODADData": {
    "appFileLocator": "10020400",
    "appProfile": "2040",
    "caPubKeyIndex": "89",
    "encicccPrivateKey": "eyJ...Ew",
    "iccPubKeyCert": {
      "certificate": "bLn...ho=",
      "exponent": "03",
      "remainder": "",
      "expirationDate": {
        "month": "10",
        "year": "2018"
      }
    },
    "tokenBinPubKeyCert": {
      "certificate": "pJI+...vc",
      "exponent": "03",
      "remainder": "DY...0s=",
      "expirationDate": {
        "month": "12",
        "year": "2023"
      }
    }
  }
}
```

Request for Visa as a Hop

Call after receiving a Token Push Provision Provisioning notification from VTS.

Endpoint:

```
https://cert.digital.visa.com/vts/panEnrollments/f0f...02/  
provisionedTokens  
?apikey=GPN...3I
```

Request Body:

```
{  
  "clientWalletAccountEmailAddress": "q?8me@3mail.com",  
  "protectionType": "SOFTWARE",  
  "clientWalletAccountID": "PttzUu",  
  "clientAppID": "AppID1234",  
  "clientWalletAccountEmailAddressHash": "CMS...oc",  
  "termsAndConditions": {  
    "id": "11f...58",  
    "date": "1570097091"  
  },  
  "encRiskDataInfo": "eyJ...Aw",  
  "clientDeviceID": "xCwr4_nkZQ",  
  "presentationType": [  
    "NFC-HCE",  
    "INAPP"  
  ]  
}
```

Response for Visa as a Hop

```
{  
  "paymentInstrument": {  
    "last4": "9170",  
    "expirationDate": {  
      "ignore02field": "51896D59",  
      "month": "12",  
      "year": "2022"  
    },  
    "cvv2PrintedInd": "Y",  
    "ignore01field": "93C06296",  
    "expDatePrintedInd": "Y",  
    "paymentAccountReference": "V00...46",  
    "enabledServices": {  
      "merchantPresentedQR": "Y"  
    }  
  },  
  "vProvisionedTokenID": "1eb...02",  
  "ignore00field": "BDFF4B60",  
  "tokenInfo": {  
    "appPrgrmID": "1008400840",  
    "encTokenInfo": "eyJ..._Q",  
    "hceData": {  
      "staticParams": {
```

```
"qVSDCData": {
  "ctq": "0000",
  "ffi": "238C0000",
  "auc": "0080",
  "psn": "00",
  "cvn": "43",
  "digitalWalletID": "000000000",
  "countryCode": "0840",
  "cid": "80",
  "qVSDCWithoutODA": {
    "afl": "08030300",
    "aip": "0040"
  }
},
"kernelIdentifier": "03",
"cardHolderNameVCPCS": "",
"pdol": "9F6...04",
"msdData": {
  "aip": "00C0",
  "afl": "08010100"
},
"track2DataDec": {
  "svcCode": "201"
},
"countrycode5F55": "5553",
"issuerIdentificationNumber": "489537",
"aidInfo": [
  {
    "qVSDCData": {
      "qVSDCWithODA": {
        "afl": "10020400",
        "aip": "2040"
      },
      "ctq": "0000",
      "ffi": "238C0000",
      "auc": "0080",
      "psn": "00",
      "cvn": "43",
      "digitalWalletID": "000000000",
      "countryCode": "0840",
      "cid": "80",
      "qVSDCWithoutODA": {
        "afl": "08030300",
        "aip": "0040"
      }
    },
    "appPrgrmID": "1008400840",
    "kernelIdentifier": "03",
    "cardHolderNameVCPCS": "",
    "pdol": "9F6...04",
    "countrycode5F55": "5553",
    "issuerIdentificationNumber": "489537",
    "track2DataDec": {
      "svcCode": "201"
    },
    "msdData": {
      "aip": "00C0",
      "afl": "08010100"
    }
  }
]
```

```
    },
    "ODADData": {
      "appFileLocator": "10020400",
      "appProfile": "2040",
      "caPubKeyIndex": "89",
      "encicccPrivateKey": "eyJ...pQ",
      "icccPubKeyCert": {
        "certificate": "KCJ...U=",
        "exponent": "03",
        "remainder": "",
        "expirationDate": {
          "month": "12",
          "year": "2019"
        }
      }
    },
    "tokenBinPubKeyCert": {
      "certificate": "pJI...vc",
      "exponent": "03",
      "remainder": "DYz...s=",
      "expirationDate": {
        "month": "12",
        "year": "2023"
      }
    }
  },
  "applicationLabel": "566...74",
  "aid": "A0000000031010",
  "priority": "01",
  "cap": "00001800",
  "CVMrequired": "N"
},
{
  "qVSDCData": {
    "qVSDCWithODA": {
      "afl": "10020400",
      "aip": "2040"
    },
    "ctq": "0000",
    "ffi": "238C0000",
    "auc": "FF80",
    "psn": "00",
    "cvn": "43",
    "digitalWalletID": "00000000",
    "countryCode": "0840",
    "cid": "80",
    "qVSDCWithoutODA": {
      "afl": "08030300",
      "aip": "0040"
    }
  },
  "appPrgrmID": "1008400840",
  "kernelIdentifier": "03",
  "cardHolderNameVCPCS": "",
  "pdol": "9F6...04",
  "countrycode5F55": "5553",
  "issuerIdentificationNumber": "489537",
  "track2DataDec": {
    "svcCode": "201"
  }
}
```

```
    },
    "msdData": {
      "aip": "00C0",
      "afl": "08010100"
    },
    "ODADData": {
      "appFileLocator": "10020400",
      "appProfile": "2040",
      "caPubKeyIndex": "89",
      "encicccPrivateKey": "eyJ...pQ",
      "icccPubKeyCert": {
        "certificate": "KCJ...=",
        "exponent": "03",
        "remainder": "",
        "expirationDate": {
          "month": "12",
          "year": "2019"
        }
      }
    },
    "tokenBinPubKeyCert": {
      "certificate": "pJI...vc",
      "exponent": "03",
      "remainder": "DYz...s=",
      "expirationDate": {
        "month": "12",
        "year": "2023"
      }
    }
  },
  "applicationLabel": "555...74",
  "aid": "A0000000980840",
  "priority": "02",
  "cap": "0000C000",
  "CVMrequired": "N"
}
]
},
"dynParams": {
  "encKeyInfo": "eyJ...qg",
  "maxPmts": 5,
  "api": "09661100",
  "sc": 1,
  "keyExpTS": 1571393092,
  "dki": "01"
}
},
"tokenRequestorID": "400...52",
"tokenReferenceID": "DNI...00",
"tokenStatus": "ACTIVE",
"last4": "2788",
"expirationDate": {
  "month": "12",
  "year": "2023"
}
},
"ODADData": {
  "appFileLocator": "10020400",
  "appProfile": "2040",
```



```
"caPubKeyIndex": "89",
"encicccPrivateKey": "eyJ...pQ",
"iccPubKeyCert": {
  "certificate": "KCJ...U=",
  "exponent": "03",
  "remainder": "",
  "expirationDate": {
    "month": "12",
    "year": "2019"
  }
},
"tokenBinPubKeyCert": {
  "certificate": "pJI...vc",
  "exponent": "03",
  "remainder": "DYz...s=",
  "expirationDate": {
    "month": "12",
    "year": "2023"
  }
}
}
```

Provision Token by TR-TSP Example

Request

Endpoint:

```
https://cert.api.visa.com/vts/panEnrollments/
e53...01/provisionedTokens?apikey=PK2...xE&relationshipID=111...45
```

Request Body:

```
{
  "clientWalletAccountEmailAddress": "7-h- .L-j+@3mail.com",
  "protectionType": "SOFTWARE",
  "clientWalletAccountID": "nnPFIg",
  "clientAppID": "Trtsp2TR1Profile",
  "clientWalletAccountEmailAddressHash": "510...gE",
  "termsAndConditions": {
    "id": "be3...a6",
    "date": "1664754882"
  },
  "encRiskDataInfo": "eyJ...BA",
  "presentationType": [
    "ECOM"
  ],
  "platformType": "ANDROID"
}
```

Response

```
{
  "paymentInstrument": {
    "expirationDate": {
      "year": "2023",
      "month": "10"
    },
    "verificationResults": {
      "cvv2VerificationCode": "M",
      "addressVerificationCode": "U"
    },
    "last4": "0032",
    "cvv2PrintedInd": "Y",
    "ignore01field": "...",
    "expDatePrintedInd": "Y",
    "paymentAccountReference": "V00...05",
    "enabledServices": {
      "merchantPresentedQR": "N"
    }
  },
  "vProvisionedTokenID": "c12...01",
  "ignore00field": "9E1C610D",
  "tokenInfo": {
    "encTokenInfo": "eyJ...iA",
    "tokenRequestorID": "400...45",
    "tokenReferenceID": "DNI...48",
    "tokenStatus": "ACTIVE",
    "last4": "5245",
    "expirationDate": {
      "month": "10",
      "year": "2023"
    }
  }
}
```

Opacity xv2 Example

JSON Request

```
{
  "clientAppID": "CompanyABC",
  "encryptionMetaData": "D5D...AE",
  "termsAndConditions": {
    "id": "13c5e2ba-eb5c-432d-a027-97ddaf109af1",
    "date": "111115"
  },
  "clientDeviceID": "-a-test_99",
  "clientWalletAccountID": "xyz_K_1",
  "presentationType": ["NFC-HCE", "MSR", "INAPP"],
  "encRiskDataInfo": "eyJ...w3",
  "protectionType": "SOFTWARE",
  "clientWalletAccountEmailAddress": "mujeeb@visa.com",
}
```

```
    "clientWalletAccountEmailAddressHash": "zmIoP0aEmZTgjK25dalVZc..."
  }
```

JSON Response

```
{
  "vProvisionedTokenID" : "ac9...01",
  "paymentInstrument" : {
    "expirationDate" : {
      "month" : "12",
      "year" : "2017"
    },
    "last4" : "2049",
    "paymentAccountReference", "V00...96"
  },
  "stepUpRequest" : [
    {
      "method" : "OTPONLINEBANKING",
      "value" : "https://test1.com/",
      "source" : "0.03|USD",
      "identifier" : "MWJ...DE="
    },
    {
      "method" : "CUSTOMERSERVICE",
      "value" : "1-800-555-1212",
      "identifier" : "ZWN...E="
    },
    {
      "method" : "APP-TO-APP",
      "value" : "Verify Online with Bank",
      "source" : "https://...",
      "identifier" : "NjZ...I=",
      "requestPayload" : "",
      "platformType" : "WEB"
    }
  ],
  "tokenInfo" : {
    "appPrgrmID" : "1008400840",
    "encTokenInfo" : "eyJ...2I",
    "hceData" : {
      "staticParams" : {
        "qVSDCData" : {
          "ctq" : "0100",
          "ffi" : "238C0000",
          "auc" : "FF80",
          "psn" : "00",

```

```
{
  "vProvisionedTokenID" : "ac9...01",
  "paymentInstrument" : {
    "expirationDate" : {
      "month" : "12",
      "year" : "2017"
    },
    "last4" : "2049",
    "paymentAccountReference", "V00...96"
  },

```

```
    },
    "stepUpRequest" : [
      {
        "method" : "OTPONLINEBANKING",
        "value" : "https://test1.com/",
        "source" : "0.03|USD",
        "identifier" : "MWJ...E="
      },
      {
        "method" : "CUSTOMERSERVICE",
        "value" : "1-800-555-1212",
        "identifier" : "ZWN...E="
      },
      {
        "method" : "APP-TO-APP",
        "value" : "VTS Testing Bank App name",
        "source" : "https://test.com/|a2a",
        "identifier" : "YjM...E=",
        "requestPayload" : "eyJ...5N"
      }
    ],
    "tokenInfo" : {
      "appPrgrmID" : "1008400840",
      "encTokenInfo" : "eyJ...W5",
      "hceData" : {
        "staticParams" : {
          "qVSDCData" : {
            "ctq" : "0100",
            "ffi" : "238C0000",
            "auc" : "FF80",
            "psn" : "00",
            "tokenStatus" : "INACTIVE",
            "last4" : "5200",
            "expirationDate" : {
              "month" : "12",
              "year" : "2020"
            }
          }
        }
      }
    }
  }
}
```

Unencrypted eniccPrivateKey

```
{
  "iccCRTPrivateKey": {
    "modulus": "ABC3F40589...",
    "exponent": "1E23F4040...",
    "primeP": "1E23F40407542...",
    "primeQ": "87ACFB9037917341731231251070707...",
    "CoefDmodP": "90A07512CDEF728947927129748...",
    "CoefDmodQ": "655766910989898913412...",
    "CoefQinvModP": "12CD3979B56E54754752757..."
  }
}
```

```
}  
}
```

Unencrypted encRiskDataInfo

```
[  
  {  
    "name": "cvv2",  
    "value": "242"  
  },  
  {  
    "name": "billingZip",  
    "value": "94404"  
  },  
  {  
    "name": "line1",  
    "value": "801MetroCenterBlvd"  
  },  
  {  
    "name": "line2",  
    "value": "..."  
  },  
  {  
    "name": "accountScore",  
    "value": "3"  
  },  
  {  
    "name": "deviceSerialNumber",  
    "value": "123444"  
  },  
  {  
    "name": "walletProviderPANAge",  
    "value": "35"  
  },  
  {  
    "name": "daysSinceConsumerDataLastAccountChange",  
    "value": "15"  
  },  
  {  
    "name": "walletAccountFirstCreated",  
    "value": "15"  
  },  
  {  
    "name": "userAccountFirstCreated",  
    "value": "15"  
  },  
  {  
    "name": "walletAccountHolderCardNameMatch",  
    "value": "Y"  
  },  
  {  
    "name": "accountHolderName",  
    "value": "Bill Evans"  
  }  
]
```

```
{
  {
    "name": "walletProviderAccountToDeviceBindingAge",
    "value": "34"
  },
  {
    "name": "provisioningAttemptsOnDevicein24hours",
    "value": "5"
  },
  {
    "name": "deviceCountry",
    "value": "US"
  },
  {
    "name": "walletAccountCountry",
    "value": "US"
  },
  {
    "name": "previousFraudLinked",
    "value": "AB"
  },
  {
    "name": "suspendedCardsInAccount",
    "value": "1"
  },
  {
    "name": "daysSinceLastAccountActivity",
    "value": "180"
  },
  {
    "name": "deviceLostMode",
    "value": "5"
  },
  {
    "name": "numberOfTransactionsInLast12months",
    "value": "3"
  },
  {
    "name": "numberOfActiveTokens",
    "value": "5"
  },
  {
    "name": "numberOfDevicesWithSameUseridWithToken",
    "value": "3"
  },
  {
    "name": "totalNumberOfActiveTokenOnAllDevicesForAccount",
    "value": "20"
  },
  {
    "name": "billingCountryCode",
    "value": "94404"
  },
  {
    "name": "deviceScore",
    "value": "4"
  },
  {

```

```
    "name": "distinctCardholderNames",  
    "value": "3"  
  },  
  {  
    "name": "paymentInstrument.expirationDate.month",  
    "value": "12"  
  },  
  {  
    "name": "paymentInstrument.expirationDate.year",  
    "value": "2022"  
  }  
]
```

Chapter 5

Provision Token—Given PAN Data

Provision Token—Given PAN Data Introduction

This method allows a wallet provider to provision a token using PAN data or using another token as input. If provisioning from another token, the input token must be a device token and it may belong either to the requesting token requestor or to another token requestor. For Cloud Token Framework (CTF), the final token being provisioned is an ECOM/COF token.

Provision Token—Given PAN Data Request

Endpoints

Resource Path

vts/provisionedTokens

Sandbox Path

```
https://cert.api.visa.com/vts/provisionedTokens?apiKey=key  
[&async=true|false]&relationshipID=id
```

Live Path

```
https://api.visa.com/vts/provisionedTokens?apiKey=key  
[&async=true|false]&relationshipID=id
```

Methods

POST

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
async	(Optional) Asynchronous provisioning. Format: It is one of the following values: • true • false - default
relationshipID	(Conditional) The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

Field	Description
encryptionMetaData	(Conditional) Cryptographic payload required for end-to-end encryption of sensitive fields. To be used in messages participating in end-to-end encryption scheme. This field is required if the encryption scheme is OPACITY_A. Format: Alphabetic, numeric; maximum 512 characters.
clientAppID	(Required) Unique identifier for the client application. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
clientDeviceID	(Conditional) Stable device identification set by wallet provider. It is required for all requests except for the following presentation types: ECOM, MERCHANT_PRESENTED_QR, QR_CONSUMER_CLOUD, and RECEIVE_ONLY. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; 24 characters.
clientWalletAccountID	(Required) Client-provided consumer ID that identifies the wallet account holder entity. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 24 characters.

Field	Description
ip4address	(Optional) IP address of the device requesting token provisioning in dot-decimal notation consisting of four decimal numbers, each ranging from 0 to 255, separated by dots. Each part represents a group of 8 bits (an octet) of the address. Example: "ip4address": "89.24.143.232" Format: Numeric and periods (.); maximum 15 characters.
location	(Optional) Location of the device up to 4-digits of precision when it is being provisioned. Information will be supplied with latitude/longitude. Example: +xxx.xxxx/-xxx.xxxx. Precision may be rounded to the nearest whole digit. Example: +37/-121. Format: Alphabetic, numeric, plus signs (+), hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 25 characters.
locale	(Required) Language in which the application communicates with the customer. Based on ISO format for language (ISO 639-1) and alpha-2 country code (ISO 3166-1 alpha-2). The language must be lowercase, and the country must be uppercase. The language and country should be separated using an underscore (_). e.g. en_US. Format: Alphabetic, underscores (_), e.g. spaces are not allowed; maximum 5 characters.
issuerAuthCode	(Conditional) Authentication code provided and encrypted by issuer. It is required if presentationType is QR_CONSUMER_CLOUD and optional for ECOM. The wallet provider should pass this value if it is provided to them by the issuer. Format: Standard base64 encoding per RFC4648; maximum 4096 characters.
clientWalletAccountEmailAddressHash	(Required) Hash of email address linked to their wallet account login. Hash algorithm to be provided by Visa. Base64 URL-encoded. Format: Base64 encoded string, per RFC4648, with the following exceptions: hyphens (-) replace plus signs (+) and underscores (_) replace forward slash (/) characters; maximum 48 characters.
clientWalletAccountEmailAddress	(Optional) Client-provided email address linked to their wallet account login. Used for consumer service operations as well as risk operations. Format: Valid email address; maximum 48 characters.

Field	Description
protectionType	<i>(Required)</i> An attribute that describes how the token credentials are protected on the device. Format: It is one of the following values: • SOFTWARE • TEE • SE • CLOUD (applicable to Ecommerce use-cases only)
panSource	<i>(Conditional)</i> Source of the payment instrument (PAN). This enum value specifies the source and the method used to provide the payment instrument data. Not required for Visa as a Hop. Format: It is one of the following values: • ONFILE – The wallet provider already has the PAN data for the customer in their records. • MANUALLYENTERED – The customer is expected to type the card number or capture a picture of the card. • ISSUER_PUSH_PROVISION – The PAN data was provided by the issuer of the PAN. • TOKEN_FOR_TOKEN – Token was used for source of the PAN data.
consumerEntryMode	<i>(Conditional)</i> The method of consumer entry. This field is required when the PAN source is manually entered. Format: It is one of the following values: • KEYENTERED • CAMERACAPTURED • UNKNOWN
encPaymentInstrument	<i>(Conditional)</i> BLOB field containing all payment instrument details (URL-safe base64 encoded) that represents the encrypted payload for a payment instrument. It is required except for Visa as a Hop. Do not specify both <code>encPaymentInstrument</code> and <code>paymentInstrumentReference</code>. Format: JSON Web Encrypted string using the shared secret. For more information, see "JSON Web Encryption and Object Signing." Base64URL encoded string, per RFC4648. The encoding is the same as Base64, but uses '-' character instead of '+' and '_' character instead of '/' character.> (schemas) [A-Za-z0-9- _]+. Max size: 8192.

Field	Description
paymentInstrumentReference	(Conditional) A reference to the payment instrument or PAN provided by the issuer. You must provide this reference instead of an encrypted payment instrument for Visa as a Hop. Do not specify both <code>encPaymentInstrument</code> and <code>paymentInstrumentReference</code>. Format: Alphanumeric; maximum 32 characters.
presentationType	(Required) Token presentation mode. Format: It is an array containing any of the following enum elements: • NFC-HCE - To be used on the device for NFC payments • NFC-SE - To be used for NFC payments with devices supporting secure element. • ECOM - Ecommerce presentation type • INAPP - In App presentation type • MSR - MSR presentation type • MERCHANT_PRESENTED_QR - Used when token needs to be used for scan-and-pay / merchant presented QR use case. • QR_CONSUMER_CLOUD - The token supports presenting a QR Code on the device. The token is stored in the cloud. • RECEIVE_ONLY - This token supports one-way person to micro-business transactions. GetPaymentData calls will be rejected. • QR_CONSUMER_DEVICE_HCE - This token supports presenting a QR Code on the consumer's device. The token is stored on the device. It is an HCE token; therefore: – LUK is managed by the device, – ATC is managed by the device, and – Cryptogram and QR are generated by the device. • AUTO_FILL Note: Shared Secret is always used to encrypt the API response fields when the presentation type is: • ECOM • QR_CONSUMER_CLOUD • RECEIVE_ONLY
accountType	(Conditional) An attribute that describes the type of account. Applicable in Ecommerce use cases. Format: It is one of the following values: • GUEST • WALLET

Field	Description
<code>encRiskDataInfo</code>	(Optional) To be used to convey data related to risk decision. Refer to the Addendum for a full description. Actual risk data elements will be shared separately outside of the technical specification. (Maximum number of Name/Value pairs allowed - 100). riskData will be encrypted using API Key/Shared Secret. For more information, see "JSON Web Encryption and Object Signing." Format: Alphanumeric; maximum 10240 characters.
<code>ssdData</code>	(Conditional) Container for all data that is specific to the SD in the chip. Required if <code>presentationType</code> is NFC-SE. Format: An <code>ssdData</code> structure.
<code>channelSecurityContext</code>	(Conditional) Present in case of PAN entry on the companion device with encryption using device certs or OPACITY_A scheme. Required if <code>presentationType</code> is NFC-HCE or NFC-SE, except it must not be present for NFC-SE in the case of Provision with Pan Enroll ID. Format: A <code>channelSecurityContext</code> structure.
<code>platformType</code>	(Conditional) The step-up option app-to-app details in the response is based on the platform type. Required if <code>presentationType</code> is NFC-SE. Format: It is one of the following values: • <code>iOS</code> – iOS app • <code>ANDROID</code> – Android app • <code>WINDOWS</code> – Windows app • <code>WEB</code> – Browser-based app
<code>passcode</code>	(Optional) OTP generated at issuer. Added to Enroll PAN and Provision + PAN data. Format: A <code>passcode</code> structure.
<code>intent</code>	(Optional) The value in this field overrides the <code>intent</code> field in <code>encPaymentInstrument</code>. It allows the token requestor to override the issuer <code>intent</code> encrypted in <code>encPaymentInstrument</code>. The intent value <code>PUSH_PROV_MOBILE</code> in <code>encPaymentInstrument</code> can be overridden by <code>PUSH_PROV_CROSS_DEVICE</code> here.
<code>suppressTokenCreateNotification</code>	(Optional) Specifies whether to suppress the VTS <code>TOKEN_CREATED</code> notification. Format: It is one of the following values: • <code>true</code> – suppress the notification • <code>false</code> – don't suppress the notification (default)

SSD Data

Field	Description
VMPAVersion	(Required) Denotes the version of the VMPA on the chip from where the provisioning request is originating. Format: Hex String. Size: 128.
sdScript	(Conditional) For tokens sharing same SD, send only for first time provisioning. For tokens not sharing same SD, send always. Both <code>sdScript</code> and <code>sdAID</code> must not be passed in the same request. VMPA Applet Instantiation with appropriate AID and other APDU commands should be sent in a single script. It is the APDU response blob containing encrypted SCP Key(s) in case of PULL scenario or the response to the generate keys command in case of ECKA scenario. Format: String. Size: 2048.
aidArray	(Required) Each element contains AID + Test sequence number. Format: Size: 32 per array element.
keyVersionNumber	(Required) Key Version Number of the SCP key in the SD. Format: Numeric; maximum 2 digits.
sequenceCounter	(Required) Sequence number for the SCP key in the SD, hex-encoded integer padded left with zeros (0). Format: Numeric; maximum 6 digits. It is a hex-encoded integer. Note: For SCP02, the max size is 4. For SCP03, the max size is 6.
CASDCert	(Conditional) Required for tokens sharing the same SD. Send only for first time provisioning; otherwise, for tokens not sharing same SD, send always. CASD Cert • Supports only 2048-bit RSA key based cert or ECC-P256. • APDU response blob containing CASD cert • Key identifier for the root public key of the CA • Contains the signature as per ISO-9796-2 • Certificate must follow tags style format as per GlobalPlatform specifications. May contain several certificates with tag 7F21. The applicable casd cert can be identified using tag 95. Format: String. Size: 20K.

Field	Description
<code>vCertificateID</code>	(Conditional) Unique ID for Visa Certificate. Required for tokens sharing same SD. Send only for first time provisioning; otherwise, for tokens not sharing same SD, send always. Not present for the ECKA use case. This will be given to Wallet Provider at onboarding time. Devices can use this certificate ID while communicating with VTS. This will be first 4 bytes of the Visa Public Key modulus. The key size will be of 2048 bit length. Format: Alphabetic, numeric; maximum 32 characters.
<code>vEphemeralVisaPublicKeyID</code>	(Conditional) Key ID generated by VTS. Required only for the ECKA use case. This is an identifier that is sent the first time for a token sharing the same SD and each time for any tokens that do not share the same SD. Format: Identifier; max size: 36 bytes
<code>sdAID</code>	(Required) For SCP_02/PULL, BOTH the following conditions apply: • Required when the chip is using the single SD model and wants to send an indicator. • Required to be passed whenever at least one card has been provisioned already and additional cards are now being provisioned. It is not passed for the initial card provisioning, but only for provisioning subsequent cards. When provisioning the first card, the <code>sdAID</code> will be passed in the <code>sdScript</code>. Both <code>sdScript</code> and <code>sdAID</code> cannot be passed in the same request. For SCP_03/ECKA, always specify a value. Format: Alphabetic, numeric; maximum 32 bytes.
<code>generateScpKeysSdScript</code>	(Conditional) Required only for the ECKA use case. This script is sent the first time for a token sharing the same SD and each time for any tokens that do not share the same SD. Format: String. Maximum 2048 characters.

Channel Security Context

Field	Description
<code>channelInfo</code>	Security Channel used to communicate between the device and the Visa Token Service back-end. Format: A <code>channelInfo</code> structure.
<code>vtsCerts</code>	(Conditional) Visa certificate IDs. Format: An array of <code>vtsCerts</code> structures.

deviceCerts	(Conditional) Device certificates. Not required for autofill. Format: An array of deviceCerts structures.
keyData	(Conditional) Key data; required for OPACITY_A; otherwise, it is not used. In the enroll PAN and Provision with PAN APIs, keyData and encryptionMetaData must now be present together. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 512 characters.

Channel Info

Field	Description
encryptionScheme	(Conditional) The security scheme used to secure communication between the device and the Visa Token Service back-end. Note: For Secure Copy (SCP) encryption schemes, the encryption version and the type are concatenated to create a string in the form of scp_version/type as per Global Platform Amendment A. Format: It is one of the following values: • RSA_PKI • OPACITY_A • SCP_02/PULL

Visa Token Service Certs

Field	Description
vCertificateID	(Required) Unique ID for Visa Certificate. This will be given to the wallet provider during onboarding. Devices can use this certificate ID while communicating with Visa Token Service . This will be the first 4 bytes of the Visa Public Key. Format: String. Size: 32.
certUsage	(Conditional) The usage of this certificate. Format: It is one of the following values: • CONFIDENTIALITY • INTEGRITY

Device Certs

Field	Description
certUsage	(Conditional) The usage of this certificate. Format: It is one of the following values: • CONFIDENTIALITY • INTEGRITY • DEVICE_ROOT
certFormat	(Conditional) X509 certificate structure. Format: String [A-Za-z0-9]+. Size: 16.
certValue	(Conditional) In PEM format/Base64URL safe encoded. Format: String[A-Za-z0-9]+. Size: 8192.

Passcode

Passcode fields are only applicable to Visa Card Enrollment Hub (VCEH).

Field	Description
type	(Required) Passcode type. Format: It is one of the following values: • PASSCODE
value	(Required) The OTP / Passcode / Passphrase generated by the user or issuer that should be validated by Visa prior to PAN Enrollment. Format: Alphanumeric, maximum 50 characters.
passCodeConversationID	(Conditional) This is returned when <code>passcode</code> is required. The <code>passCodeConversationID</code> should be returned to Visa by the token requestor in the subsequent Enroll PAN call. This attribute should not be sent by the issuer. If the issuer sends a value for this attribute in the encrypted payment instrument, it should be ignored. Format: Alphanumeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 32 characters
maxRetries	(Optional) The number of times the OTP can be retried before a risk code is sent to VIP indicating that the OTP did not match. This value is provided by the issuer when sending the payment instrument to the wallet provider. It should not be populated by the wallet provider during enrollment. If the wallet provider sends this attribute in the request, it is rejected (invalid request – bad parameter). Format: String. Digits [0-9].

Encrypted Payment Instrument

Field	Description
accountNumber	(Conditional) PAN of the card to be enrolled and provisioned. This field is required if an IBAN is not provided. Format: Numeric; 13-19 digits.
iban	(Conditional) This field is the IBAN of the card to be enrolled and provisioned. This field is required if a PAN is not provided. Format: Alphabetic, numeric; maximum 34 characters
token	(Conditional) EMV Co Token number. This is required if there is no accountNumber or iban. Format: Alphabetic, numeric; 12-19 characters..
cvv2	(Optional) CVV2 value associated with the PAN on the card. Note: For CVV2 to be considered during token provisioning, the CVV2 must also be provided within the encRiskDataInfo structure of the Provision Token request. Format: Numeric; 3-4 digits.
name	(Conditional) The full name on the Visa card associated with the enrolled payment instrument. This field is required if: • IBAN is provided AND no sessionId is provided for push provisioning. • IBAN is provided for normal pull provisioning This field is optional if a PAN is provided. Format: Alphanumeric, including spaces; maximum 256 characters.
expirationDate	(Conditional) Payment instrument's expiration date. This field is required unless the expiration date is not printed on the card. Format: An expirationDate structure.
billingAddress	(Optional) Billing address associated with the payment instrument. Format: A billingAddress structure.
provider	(Conditional) Information about the provider of the payment instrument and the context in which it is provided. This field is required if panSource equals ISSUER_PUSH_PROVISION. Format: A provider structure.

Expiration Date

Note:

The expiration date is optional; however, all fields must have a value if the date is provided.

Field	Description
month	<i>(Required)</i> The month the Visa card is set to expire. Format: Numeric; 2 digits.
year	<i>(Required)</i> The year the Visa card is set to expire. Format: Numeric; 4 digits.

Billing Address

Field	Description
line1	<i>(Optional)</i> Street 1 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line2	<i>(Optional)</i> Street 2 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line3	<i>(Optional)</i> Street 3 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line4	<i>(Optional)</i> Street 4 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line5	<i>(Optional)</i> Street 5 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.

city	<i>(Optional)</i> The city associated with the enrolled payment instrument. Format: Alphabetic, single quote ('), hyphen (-), and whitespace.; maximum 100 characters.
state	<i>(Optional)</i> State or province code in ISO 3166-2 format, e.g. NY. Format: Alphabetic; maximum 3 characters.
country	<i>(Optional)</i> Country in ISO 3166-1 alpha-2 format, e.g. US. Format: Alphabetic; maximum 2 characters.
countryDigits	<i>(Optional)</i> ISO 3166-1 numeric Country Code. Format: Numeric; maximum 3 digits.
postalCode	<i>(Optional)</i> The postal code associated with the enrolled payment instrument. Note: This field does not support spaces (" ") in the middle of characters. Format: Alphabetic, numeric, e.g. spaces are not allowed; maximum 3–16 characters.

Payment Instrument Provider

This object allows the payment instrument provider to specify the intent under which the PAN should be used for token provisioning. Some of the fields provided here will be matched with the corresponding fields from the API payload. This structure applies only to Visa Card Enrollment Hub (VCEH) or legacy direct push provisioning.

Field	Description
intent	(Required) The intent of the encryptor, which specifies what the encryptor of the data trying to do. Format: It is one of the following values: PUSH_PROV_MOBILE - Push the PAN data to a Token Requestor to provision a token for the card holder on a specific device for a specific wallet/account. The push initiating application and the push receiving (token requestor) application are accessible on the same device. PUSH_PROV_CROSS_USER - Push the PAN data to a Token Requestor to provision a token for a secondary/authorized user. The Issuer is providing the PAN for the purpose of provisioning a token for an authorized consumer that is different from the push initiating card holder. This is only applicable to only to Visa Card Enrollment Hub (VCEH). PUSH_PROV_CROSS_DEVICE - Push the PAN data to a Token Requestor to provision a token for the cardholder on a different device. The issuer is providing the PAN for the purpose of provisioning a token for the consumer of a token requestor application accessible on a device that is different from the device from which the push is being initiated. This is only applicable to only to Visa Card Enrollment Hub (VCEH). PUSH_PROV_ONFILE - Push the PAN data to a Card On File merchant/Token Requestor to provision a token for the cardholder. The issuer is providing the PAN for the purpose of provisioning a token for the consumer into a card on file merchant. The push initiating application and the push receiving application(token requestor) are accessible on the same device.
clientWalletProvider	(Required) Token requestor ID. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed. Size: maximum 50 characters.
clientWalletAccountID	(Optional) Client provided consumer ID that identifies the Wallet Account Holder entity. It must have the same value as clientWalletAccountID field from the API payload. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed. Size: maximum 24 characters.
clientAppID	(Conditional) Unique Identifier for the client application, used to provide some of the encrypted values. Required if intent is PUSH_PROV_MOBILE. Example: Issuer's AppID used to select the PAN and the wallet. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed. Size: maximum 36 characters.

isIDnV	(Optional) Specifies whether the issuer wants ID&V to be performed. Format: It is one of the following values: • true - perform ID&V • false - do not perform ID&V
issuerAccountID	(Optional) Uniquely represents “pushing” account from issuer system. May be different from PAN holder account. It applies only to Visa Card Enrollment Hub (VCEH). Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 24 characters.
issuerClientInformation	(Optional) Client information shared from issuer to token requestor from information captured in the issuer systems about the PAN's owner. It applies only to Visa Card Enrollment Hub (VCEH). This information will be decrypted and shared with the token requestor. Format: A <code>clientInformation</code> structure.
passcode	(Optional) OTP generated by the issuer. It applies only to Visa Card Enrollment Hub (VCEH). Added to Enroll PAN and Provision + PAN data. Format: A <code>passcode</code> structure.
returnURIType	(Optional) The type of URI for the return app. It applies only to Visa Card Enrollment Hub (VCEH). Format: It is one of the following values: • IOS - iOS app • ANDROID - Android app • WEB - browser-based app
returnURI	(Optional) URI provided by the issuer to the token requestor to return control to the issuer app. This can be an app or web URL. Format: Alphanumeric. Size: maximum 128 characters.

<code>issuerTraceID</code>	(Optional) An ID, preferably a Globally Unique Identifier (GUID), provided by the issuer in order to query token status after sending the JWE to a token requestor. It applies only to Visa Card Enrollment Hub (VCEH). Format: Alphanumeric. Size: maximum 32 characters.
<code>isTsAndCsAccepted</code>	(Optional) This field indicates whether or not the customer has already accepted the issuer's Terms and Conditions. Note: If this value is set to <code>true</code>, the wallet provider can opt to not display the Terms and Conditions to the customer. They would submit the Terms and Conditions GUID to Visa. The default value for this field is <code>false</code>. Format: It is one of the following values: • <code>true</code> • <code>false</code>

Client Information

Client information applies only to Visa Card Enrollment Hub (VCEH).

Field	Description
<code>source</code>	(Optional) The source of the information. Format: It is one of the following values: • <code>ISSUER</code> • <code>TOKEN_REQUESTOR</code>
<code>clientWalletAccountID</code>	(Optional) Client provided consumer ID that identifies the wallet account holder entity. It must have the same value as <code>clientWalletAccountID</code> field from the API payload. Format: Alphanumeric, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 24 characters.
<code>firstName</code>	(Required) First name of the token requestor client that received the token. The issuer will populate with the information they have for the client. Format: Alphanumeric; between 2–80 characters, inclusive. Escape Characters, including tab entries, are not supported.
<code>middleName</code>	(Optional) Middle name of the token requestor client that received the token. The issuer will populate with the information they have for the client. Format: Alphanumeric; between 2–80 characters, inclusive. Escape Characters, including tab entries, are not supported.

Field	Description
lastName	(Required) Last name of the token requestor client that received the token. The issuer will populate with the information they have for the client. Format: Alphanumeric; between 2–80 characters, inclusive. Escape Characters, including tab entries, are not supported.
issuerAccountID	(Optional) The issuer account ID provided by the issuer to the token requestor. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 24 characters.
clientDeviceID	(Conditional) It is a stable and unique identifier set by the wallet provider for the device. This field is required when it is returned to the issuer by the token requestor. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 24 characters.
tokenReferenceID	(Conditional) It is a unique ID associated with a single token. The ID is created upon initial provisioning of a token to a device. This field is required when returned to the issuer by the token requestor. Format: Alphabetic, numeric; maximum 32 characters.
paymentAccountReference	(Optional) Payment Account Reference (PAR) if available for this payment instrument. Format: Alphanumeric; maximum 29 characters.
phoneNumber	(Conditional) Either phone number or email must be passed when intent is PUSH_PROV_CROSS_DEVICE or PUSH_PROV_CROSS_USER. Mobile phone number of the client as per issuer records. Format: A valid phone number; maximum 32 bytes.
contactEmail	(Conditional) Either phone number or email must be passed when intent is PUSH_PROV_CROSS_DEVICE or PUSH_PROV_CROSS_USER. Email address of client as per issuer records. Format: A valid email address; maximum 32 bytes. Note: This field will support internationalization using UTF-8 characters.
country	(Optional) The client's country code. Format: An alpha 2 country code (ISO 3166-1 alpha-2).

Field	Description
<code>sessionIdentifier</code>	(Conditional) Optional session identifier that can be used by the issuer to uniquely identify the cardholder. This field is required if IBAN and Push Provisioning and no cardholder name was provided. Format: Alphabetic, numeric, and underscores (_), e.g. spaces are not allowed; maximum 64 characters.
<code>locale</code>	(Required) Language in which the app communicates with the customer. Based on ISO format for language (ISO 639-1) and alpha 2 country code (ISO 3166-1 alpha-2). The language and country should be separated using an underscore (_). e.g. en_US. Format: Alphabetic, underscores (_), e.g. spaces are not allowed; maximum 5 characters.
<code>campaignID</code>	(Optional) Cobrand campaign identifier. Format: Alphabetic, numeric, hyphens (-), and underscores (_); maximum 100 characters.
<code>campaignDescription</code>	(Optional) Cobrand campaign description. Format: Alphabetic, numeric, hyphens (-), and underscores (_); maximum 250 characters.
<code>userPersonalID</code>	(Optional) An identifier that uniquely identifies a person. Format: Alphabetic or numeric; maximum 15 characters.

Provision Token—Given PAN Data Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
<code>vPanEnrollmentID</code>	Unique id to identify this PAN enrollment. Another provision token API call on the same PAN is not guaranteed to return the same <code>vPanEnrollmentID</code>. Note: This is the only additional parameter returned as this API response, compared to the Provision Token Given PAN Enrollment ID API response. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
<code>vProvisionedTokenID</code>	Unique ID for token. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.
<code>paymentInstrument</code>	Payment instrument. Format: A <code>paymentInstrument</code> structure.
<code>stepUpRequest</code>	Step up request info. Not applicable to Ecommerce token type. Note: This is applicable only when the <code>tokenStatus</code> is <code>INACTIVE</code>. Format: A <code>stepUpRequest</code> structure.
<code>tokenInfo</code>	Token information. Format: A <code>tokenInfo</code> structure.
<code>encryptionMetaData</code>	Cryptographic payload for end-to-end encryption of sensitive fields. It is required for the Opacity_A protocol. Format: Alphabetic, numeric; maximum 512 characters.
<code>ODAdData</code>	Returned if the token can be used for Offline Data Authorization (ODA). If <code>ODAdData</code> is returned, then all the fields in <code>ODAdData</code> will be returned. Format: A <code>ODAdData</code> structure.
<code>appliedPrograms</code>	A program that applied an action to a token; otherwise, not present in the response. Format: An array of one or more of the following values: • "VCES"
<code>vTaskID</code>	Datacenter aware VTS-generated task ID for async provisioning. Provided when <code>async</code> query parameter is <code>true</code>. If provided, it is the only field provided in the response. Format:String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 40.

Payment Instrument

Field	Description
last4	Last 4 digits of the account. Note: If an IBAN was provided in the provisioning request, this is the last 4 digits of the IBAN. Format: String, numeric; maximum 4 digits.
expirationDate	Payment instrument's expiration date. Format: An <code>expirationDate</code> structure.
accountStatus	<i>Future:</i> Is the account closed? To be returned with Get Card Metadata API only. Format: It is one of the following values: - Y – Account closed - N – Account not closed
isTokenizable	<i>Future:</i> Is this account eligible for tokenization? To be returned with Get Card Metadata API only. Format: It is one of the following values: - Y – Eligible - N – Not Eligible
indicators	These are indicators associated with the Payment Instrument. If the field is not returned, or empty, then there are no indicators associated with it. Expected values are: PRIVATE_LABEL - This Payment Instrument is for a Private Label PAN. Format: String
expDatePrintedInd	Indicates if the expiration date is printed on the payment instrument. Format: It is one of the following String values: - Y - Printed - N - Not printed
cvv2PrintedInd	Indicates if the CVV2 is printed on the payment instrument. Format: It is one of the following String values: - Y - Printed - N - Not printed
paymentAccountReference	Payment Account Reference (PAR) if available for this payment instrument. Format: String, alphabetic or numeric; maximum 29 characters.

Field	Description
<code>enabledServices</code>	Enabled services, such as scan-and-pay. Format: An <code>enabledServices</code> structure.
<code>verificationResults</code>	Payment instrument verification results. Present only for <code>ECOM</code> presentation type when the CVV2 and/or address fields appear in the provision request. Format: A <code>verificationResults</code> object.

Expiration Date

Field	Description
<code>month</code>	The month upon which the payment instrument is set to expire. Example: <code>"month": "9"</code> Format: String; numeric; max length 2 digits.
<code>year</code>	The year upon which the payment instrument is set to expire. Example: <code>"year": "2024"</code> Format: String; numeric; length 4 digits.

Enabled Services

Field	Description
<code>merchantPresentedQR</code>	Whether the issuer enabled the PAN for the scan-and-pay use case. Format: It is one of the following String values: • Y – Enabled • N – Disabled

Verification Results

Field	Description
<code>cvv2VerificationCode</code>	The CVV2 verification result. This field is present when CVV2 verification data is present in the request. Format: It is one of the following values: • M – verified, CVV2 data match • N – verified, negative data match • U – verification not possible (issuer does not participate) • D – verification not performed due to duplicate request
<code>addressVerificationCode</code>	The address verification result. This field is present when address verification data is present in the request. Format: It is one of the following values: • Y – verified, address and postal code data match • P – verified, partial match (either address or postal code data match, but no result from the other) • B – verified, partial match (either address or postal code data match, but the other could not be verified due to incompatible format) • N – verified, neither address nor postal code data match • U – verification not possible (issuer does not participate) • D – verification not performed due to duplicate request

Step-Up Request

`stepUpRequest` is conditional. It is not required for the Ecommerce token type. Items 2, 3, and 4 below are applicable to push provisioning only.

Note: This is only applicable when:

- `TokenStatus` is `INACTIVE`.
- Device ID used during push provisioning does not match the `DeviceID` used during enrollment.
- Issuer indicated a preference for IDnV in the JWE by setting `isIDnV=Y`.
- Push Provisioning expired.

Field	Description
method	ID&V step-up method. Format: It is one of the following values: • OTPEMAIL • OTPONLINEBANKING • OTPSMS • CUSTOMERSERVICE • APP-TO-APP • OUTBOUNDCALL
value	This could be a masked value depending on the ID&V step-up method. Email looks like 'ip****@email.com' bank_app looks like 'Mobile Banking App' customer_service looks like '1-800-555-1212' Note: This field will support internationalization using UTF-8 characters. Format: Alphanumeric, UTF-8 whitespace; maximum 64 characters.
source	The issuer source of contact for this ID&V step-up challenge. In case of OTP, the source contains the issuer contact that will reach out to the consumer. In the case of banking-app step-up, the source is a concatenation of Application ID (<i>id</i>) and Intent Handler (<i>h</i>). This field will be defined by the issuer. Note: Method handler of Banking App designed to respond to authentication code handling. For the Android platform this will be an explicit intent with fully name-spaced class name. Visa requires the issuer to use the a2a name for this intent handler. Example: "source": "{id} {h}", where <i>id</i> is required (size 60, [a-z,A-Z,0-9,_,.]), and <i>h</i> is optional (size 3, [a-z][A-Z][0-9]). Format: Alphanumeric; maximum 1024 characters.
identifier	Unique and opaque identifier for this method. A subsequent step-up request will only provide the identifier. The wallet provider must send this value "as is" to subsequent step-up related APIs. Format: Alphabetic, numeric, plus signs (+), slashes (/), and equal signs (=), spaces are not allowed; maximum 96 characters.

Field	Description
requestPayload	The request payload to be sent to the banking application on behalf of Visa. This field is opaque to wallet providers. Note: This is only present if the step-up type is app-to-app. Format: Base64 encoded string, maximum 1024 characters.
platformType	Platform type, which is present in cases where the Token Requestor has opted in to 3DS Web Step-Up. Format: It is one of the following values: • WEB
subMethod	Flavor of the step-up method. Format: It is one of the following values: • THREE-DS

Token Info

Field	Description
tokenStatus	Token status. If a token was deleted, no information will be returned for the token. Format: It is one of the following values: • INACTIVE • ACTIVE • SUSPENDED
tokenRequestorID	Unique ID that identifies the entity for which the token request is being conducted. Format: Numeric; maximum 11 digits.
tokenReferenceID	Unique ID associated with a single token. The ID is created upon initial provisioning of a token to a device. Format: Alphanumeric; maximum 32 characters.
last4	Last 4 digits of the token. Format: Numeric; maximum 4 digits.
expirationDate	Token expiration date in UTC format. Format: An <code>expirationDate</code> structure.

Field	Description
appPrgrmID	Visa proprietary data element identifying the application program ID of the card application; not applicable to NFC-SE. When personalized, the application program ID is returned in the FCI issuer discretionary data of the select response (tag 'BF0C'). qVSDC readers that support Dynamic Reader Limits (DRL) functionality examine the application program ID to determine the reader limit set to apply. Format: Alphabetic, numeric; 10–32 characters.
encTokenInfo	Encrypted token payload. For more information, see "JSON Web Encryption and Object Signing." This field is not returned for NFC-SE presentation types. Note: The unencrypted field contains a token as a 13-16 character string. Format: Alphabetic, numeric; maximum 4096 characters.
hceData	HCE data. Only applicable for NFC-HCE use case; not applicable for NFC-SE use case. Format: An <code>hceData</code> structure.
mst	MST data. Only applicable for NFC-HCE use case when the <code>presentationType</code> is <code>MST</code>; not applicable for NFC-SE use case. Format: An <code>mst</code> structure.
seCardPerso	APDU commands. Present in the case of NFC-SE use cases except when APDU script is handled by another entity. Format: An APDU JSON structure.
sePersoData	Perso application data. Present when APDU script is handled by another entity. Format: An <code>sePersoData</code> structure.
atc	Transaction counter; present in HCE tokens. Format: Numeric; 5 digits.

Expiration Date

Field	Description
<code>month</code>	The month upon which the payment instrument is set to expire. Example: "month": "9" Format: String; numeric; max length 2 digits.
<code>year</code>	The year upon which the payment instrument is set to expire. Example: "year": "2024" Format: String; numeric; length 4 digits.

HCE Data

Field	Description
<code>dynParams</code>	HCE data dynamic parameters. Format: A <code>dynParams</code> structure.
<code>staticParams</code>	HCE data static parameters. Format: A <code>staticParams</code> structure.

Dynamic Parameters

Field	Description
<code>encKeyInfo</code>	Encrypted new, limited-use key for a given provision token ID. This value is used for cryptogram generation and is updated during replenishment requests. It is encrypted using the wallet provider's preferred encryption scheme (for example, <code>_A</code>). Format: Alphabetic, numeric; maximum 32 characters. For more information, see "JSON Web Encryption and Object Signing."
<code>maxPmts</code>	Maximum number of payments per provisioned token before replenishment is required Source: VTS. Format: Numeric; maximum 99999 characters.
<code>api</code>	Account parameter index used to generate cryptograms. Source: Generated by VTS and associated with token request/replenishment. Format is defined by VCPCS 1.8. Format: Alphabetic, numeric; maximum 8 characters.

Field	Description
sc	Sequence counter, which is used to generate cryptograms. Tracked in SDK, the sequence counter will be incremented upon replenishment call. Source: VTS generated. Format: Alphabetic, numeric; maximum 65535 characters.
keyExpTS	Expiration timestamp associated with the LUK time-to-live. Source: VTS. Format: Numeric, UTC Format in epoch seconds.
dki	Derived key indicator defining the key index used to calculate the cryptogram. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 2 characters.

Static Parameters

Field	Description
aidInfo	Format: An aidInfo structure.
kernelIdentifier	Returned for Durbin (US Debit). Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 8 characters.
cardHolderNameVCPCS	Cardholder name per ISO 7813. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 26 characters.
pdol	List of terminal/reader- related data objects (tags and lengths) requested by the card to be transmitted in the GET PROCESSING OPTIONS command. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 48 characters.
countrycode5F55	Returned for Durbin (US Debit). Source: Issuer Configuration File. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 4 characters.

Field	Description
issuerIdentificationNumber	Returned for Durbin (US Debit). Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Numeric; maximum 6 digits.
msdData	Format: A <code>MsdData</code> structure.
qVSDCData	Format: A <code>qVSDCData</code> structure.
track2DataDec	Format: A <code>track2DataDec</code> structure.
track2DataNotDec	Format: A <code>track2DataNotDec</code> structure.

Note:

Provisioning response payload related to qVSDC and MSD profiles. Paywave elements have conditional use, qVSDC response will have track2datanotdec whereas MSD/qVSDC will have msdData and track2datadec fields populated.

MSD Data

Field	Description
aip	Application Interchange Profile that contains information about the technology the card supports, such as MSD or qVSDC. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; 2 hexadecimal digits. For hex representation, two characters are used to represent one hex byte.
afl	Application file locator that indicates the location (SFI, range of records) of the AEFs related to a given application. As with all data elements returned in the GPO response, the card application supports personalization of different AFL data elements for use in the following GPO responses: • qVSDC Offline GPO Response (if offline implemented) • qVSDC Online with ODA GPO Response • qVSDC Online/Decline without ODA GPO Response • MSD CVN17 with ODA GPO Response • MSD CVN17 without ODA GPO Response • MSD Legacy GPO Response Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 8 characters.

qVSDC Data

Field	Description
ctq	Card transaction identifiers used by the device to determine the card CVM requirements, issuer preferences and card capabilities. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 4 characters.
ced	Customer exclusive data that is transmitted to issuer. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 64 characters.
ffi	Form factor indicating the kind of consumer payment device and contactless interface over which the transaction was conducted. This information is made available to the issuer. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 8 characters
auc	Application usage control which is an ISO 3166 code identifying the issuer's country. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 4 characters.
psn	PAN sequence number used in encryption. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Numeric; maximum 2 digits.
cvn	Most significant nibble: '4': this IAD format. Least significant nibble: '3': Limited Use Key Cryptogram. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 2 characters.
digitalWalletID	VTS-defined unique ID that identifies the wallet provider entity from which the external request is being sent. This ID will be provided to the wallet provider during onboarding to VTS. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 8 characters.

Field	Description
countryCode	An ISO 3166 code identifying the issuer's country. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 4 characters.
cid	Indicates the type of cryptogram (TC, ARQC, or AAC) returned by the card and the actions to be performed by the reader. Source: Issuer configuration file. Format: Alphabetic, numeric; maximum 2 characters.
qVSDCWithoutODA	Format: A qVSDCWithoutODA structure.

qVSDC Without ODA

Field	Description
aip	Application Interchange Profile which contains information about the technology the card supports, such as MSD or qVSDC. Format: Alphabetic, numeric; maximum 4 characters.
afl	Application File Locator, which indicates the location (SFI, range of records) of the AEFs related to a given application. As with all data elements returned in the GPO response, the card application supports personalization of different AFL data elements for use in the following GPO responses: • qVSDC Offline GPO Response (if offline implemented) • qVSDC Online with ODA GPO Response • qVSDC Online/Decline without ODA GPO Response • MSD CVN17 with ODA GPO Response • MSD CVN17 without ODA GPO Response • MSD Legacy GPO Response Format: Alphabetic, numeric; maximum 8 characters.

Track2 Data Dec

Field	Description
svcCode	Service code that identifies the components necessary to compile Track 2 equivalent data per <i>Visa Contactless Payment Specification</i> . Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 3 characters.

Track2 Data Not Dec

Field	Description
svcCode	Service code that identifies the components necessary to compile Track 2 equivalent data per the <i>Visa Contactless Payment Specification</i> . Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 3 characters.
pinVerField	Necessary to compile Track 2 Equivalent data for the <i>Visa Payment Contactless Specification</i> Format: Alphabetic, numeric; maximum 5 characters.
track2DiscData	Track 2 disc data. Discretionary Data (Partly ICCV data). Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; 8-13 characters.

MSR

Field	Description
svcCode	Service code which identifies the components necessary to compile Track 2 equivalent data per the <i>Visa Contactless Payment Specification</i> . Format: Alphabetic, numeric; maximum 3 characters.
cvv	Card verification value. Format: Numeric; 3-4 digits.

APDU JSON

This is the `seCardPerso` structure.

Field	Description
appletInstanceAID	Full AID of the card instance or application. Format: String; max 20 characters.
priorityCode	Priority for the AID. Format: String; max 4 characters.
apduPerso	APDU commands. Format: An array of <code>ApduLines</code> structures.

APDU Lines

Field	Description
apduID	SerialID for each APDU command. Format: String; max 20 characters.
apdu	APDU command. Format: String; max 4 characters.

Perso Application Data

Field	Description
persoApplicationData	AID data. Format: An array of <code>PersoElement</code> structures.

Perso Element

Field	Description
dataTag	EMV/VMPA tag. Format: String; max 4 characters.
dataName	Data element name. Format: String; max 512 characters.
dataValue	Perso data element. Format: String; max 1024 characters.

ODA Data

This data will only be returned for NFC-HCE use cases. ODA data for NFC-SE will be part of the `seCardPerso` script. Will be returned if the token is enabled for Offline Data Authorization (ODA).

Field	Description
appFileLocator	Application File Locator. Format: Alphabetic, numeric; maximum 10 characters.
appProfile	Application Interchange Profile. Format: Alphabetic, numeric; maximum 5 characters.

<code>caPubKeyIndex</code>	Certificate Authority Public Key Index. Index to identify the payment system public key used for dynamic data authentication (for ODA). Format: Alphabetic, numeric; maximum 3 characters.
<code>iccPubKeyCert</code>	Integrated Circuit Card (ICC) Public Key Certificate.
<code>tokenBinPubKeyCert</code>	Token-BIN Public Key certificate. This is also known as Issuer Public Key Certificate.
<code>enciccPrivateKey</code>	JWE/JWS formatted encrypted data. Format: For more information, see "JSON Web Encryption and Object Signing."

Provision Token—Given PAN Data Errors

Error Code	Reason Code	Message
400	<code>provisionDataExpired</code>	The PAN information provided is considered stale. If this is a Push Provisioning flow, which allows the user to choose a card from their banking application or a web page, the user should be asked to choose the card again. If the <code>encPaymentInstrument</code> field was prepared by the wallet provider, then the wallet provider should create it again and call the Enroll PAN API before invoking the Provision Token API.
403	<code>cardVerificationFailed</code>	Invalid field. Invalid payment instrument or data associated with the payment instrument. The "location" field may point to the field failing the validation. Ask users to check that all information was entered correctly.
403	<code>cardNotEligible</code>	This card cannot be used for tokenization at this moment. Please try again later or choose another Visa card.
403	<code>cardNotAllowed</code>	The requested action is not allowed for a given PAN. This card is valid but cannot be used for tokenization. This is most likely for Visa-specific reasons. Ask the consumer to enter a different card.

Error Code	Reason Code	Message
403	declined	This card is considered not eligible for tokenization at this time. Try enroll/provision again for this PAN. If you encounter the same message for the new enrollment/provision request, ask the customer to either add a different card or to reach out to their issuer.
409	notAllowed	Further operations for this card are no longer allowed. Contact your bank to resolve this issue.

Provision Token—Given PAN Data Examples

Provision Token—Given Pan Data Example

Request

```
{
  "encRiskDataInfo": "eyJ...RQ",
  "clientAppID": "...",
  "clientWalletAccountID": "1234",
  "clientWalletAccountEmailAddressHash": "pq0...oo",
  "locale": "en_US",
  "protectionType": "SOFTWARE",
  "panSource": "ONFILE",
  "encPaymentInstrument": "eyJ...nA",
  "presentationType": [
    "ECOM"
  ]
}
```

Response

```
{
  "vProvisionedTokenID": "0d7...02",
  "vPanEnrollmentID": "24e...02",
  "ignore00field": "A84CCAD0",
  "paymentInstrument": {
    "expirationDate": {
      "year": "2022",
      "month": "12"
    },
  },
  "verificationResults": {"cvv2VerificationCode": "M"},
  "last4": "0818",
  "cvv2PrintedInd": "Y",
}
```

```
    "ignore01field": "8F644728",
    "expDatePrintedInd": "Y",
    "paymentAccountReference": "V00...62",
    "enabledServices": {"merchantPresentedQR": "Y"}
  },
  "tokenInfo": {
    "encTokenInfo": "eyJ...xQ",
    "tokenRequestorID": "400...44",
    "tokenReferenceID": "DNI...72",
    "tokenStatus": "ACTIVE",
    "last4": "1767",
    "expirationDate": {
      "month": "12",
      "year": "2022"
    }
  }
}
```

Elliptic Curve Cryptography ESE Request

```
{
  "clientWalletAccountEmailAddress": "óUnN@3mail.com",
  "protectionType": "SOFTWARE",
  "clientWalletAccountID": "Rwt3tJek_k1JxivcwbPHjKdk",
  "clientAppID": "jioAppId",
  "clientWalletAccountEmailAddressHash": "eX2AJ8EOod4FC1jRPxP01o...",
  "termsAndConditions": {
    "id": "30aaeeffb0356480ab3ddefbf69c59e92",
    "date": "1530221427"
  },
  "encRiskDataInfo": "eyJhbGciOiJBMjU2R0NNSlciLCJpdj...",
  "clientDeviceID": "DeviceIDZiIG3a0DgF4mQfkC",
  "presentationType": [
    "NFC-SE",
    "INAPP"
  ],
  "ssdData": {
    "aidArray": [
      "A000000003101001"
    ],
    "sequenceCounter": "000001",
    "keyVersionNumber": "01",
    "vCertificateID": "c1285d52",
    "sdScript": "85101fa6abee78a33eb4b6a0636954541802861066bb4f...",
    "generateScpKeysSdScript": "80E209001B00A618A61690020303950...",
    "vEphemeralVisaPublicKeyID": "fd447de827d4e28b3300a211a79bbd01",
    "sdAID": "A00000007701060010100003101000",
    "VMPAVersion": "31373033333120564D5041204346475F31372031343430",
    "CASDCert": "7F2181BD7F2181B9931004073203494280017193013806..."
  },
  "platformType": "ANDROID"
}
```

JWE encrypted encPaymentInstrument

The `encPaymentInstrument` is JWE-encrypted, which includes:

```
{
  "accountNumber": "...",
  "cvv2": "758",
  "name": "BillEvans",
  "expirationDate": {
    "month": "10",
    "year": "2015"
  },
  "billingAddress": {
    "line1": "801MetroCenterBlvd",
    "line2": null,
    "city": "FosterCity",
    "state": "CA",
    "country": "US",
    "postalCode": "94404"
  },
  "provider": {
    "clientAppID": "APP1",
    "clientDeviceID": "DEiOiJBMjU2R0NNS1ciLCJiI",
    "clientWalletProvider": "sampleIssuer",
    "clientWalletAccountID": "AiOiJBMjU2R0NNS1ciLCJiI6",
    "Intent": "PUSH_PROV_MOBILE",
    "isIDnV": false
  }
}
```

Unencrypted encRiskDataInfo

```
[
  {
    "name": "cvv2",
    "value": "242"
  },
  {
    "name": "billingZip",
    "value": "94404"
  },
  {
    "name": "line1",
    "value": "801MetroCenterBlvd"
  },
  {
    "name": "line2",
    "value": "..."
  },
  {
    "name": "accountScore",
```

```
    "value": "3"
  },
  {
    "name": "deviceSerialNumber",
    "value": "123444"
  },
  {
    "name": "walletProviderPANAge",
    "value": "35"
  },
  {
    "name": "daysSinceConsumerDataLastAccountChange",
    "value": "15"
  },
  {
    "name": "walletAccountFirstCreated",
    "value": "15"
  },
  {
    "name": "userAccountFirstCreated",
    "value": "15"
  },
  {
    "name": "walletAccountHolderCardNameMatch",
    "value": "Y"
  },
  {
    "name": "accountHolderName",
    "value": "Bill Evans"
  },
  {
    "name": "walletProviderAccountToDeviceBindingAge",
    "value": "34"
  },
  {
    "name": "provisioningAttemptsOnDevicein24hours",
    "value": "5"
  },
  {
    "name": "deviceCountry",
    "value": "US"
  },
  {
    "name": "walletAccountCountry",
    "value": "US"
  },
  {
    "name": "previousFraudLinked",
    "value": "AB"
  },
  {
    "name": "suspendedCardsInAccount",
    "value": "1"
  },
  {
    "name": "daysSinceLastAccountActivity",
    "value": "180"
  },
  },
```

```
{
  "name": "deviceLostMode",
  "value": "5"
},
{
  "name": "numberOfTransactionsInLast12months",
  "value": "3"
},
{
  "name": "numberOfActiveTokens",
  "value": "5"
},
{
  "name": "numberOfDevicesWithSameUserIdWithToken",
  "value": "3"
},
{
  "name": "totalNumberOfActiveTokenOnAllDevicesForAccount",
  "value": "20"
},
{
  "name": "billingCountryCode",
  "value": "94404"
},
{
  "name": "deviceScore",
  "value": "4"
},
{
  "name": "distinctCardholderNames",
  "value": "3"
},
{
  "name": "paymentInstrument.expirationDate.month",
  "value": "12"
},
{
  "name": "paymentInstrument.expirationDate.year",
  "value": "2022"
}
]
```

Elliptic Curve Cryptography ESE Response

```
{
  "paymentInstrument": {
    "last4": "3003",
    "expirationDate": {
      "ignore02field": "DC568E36",
      "month": "12",
      "year": "2020"
    },
    "cvv2PrintedInd": "Y",
    "ignore01field": "44FCFE1F",
  }
}
```

```
"expDatePrintedInd": "Y",
"paymentAccountReference": "V00...47",
"enabledServices": {
  "merchantPresentedQR": "Y"
},
},
"vProvisionedTokenID": "12f...01",
"ignore00field": "2831D0CF",
"tokenInfo": {
  "tokenRequestorID": "400...51",
  "tokenReferenceID": "DNI...98",
  "seCardPerso": [
    {
      "TesttInstanceAID": "A00...01",
      "priorityCode": "01",
      "apduPerso": {
        "apduLines": [
          {
            "apduID": "01",
            "apdu": "00A...00"
          },
          {
            "apduID": "02",
            "apdu": "805...00"
          },
          ...
          {
            "apduID": "19",
            "apdu": "84E..."
          }
        ]
      }
    }
  ],
  "tokenStatus": "ACTIVE",
  "last4": "0006",
  "expirationDate": {
    "month": "12",
    "year": "2023"
  }
}
}
```

Unencrypted encTokenInfo

```
"token": "..."
```


Provision Token—Given Another Token Example

Request

Selected Headers:

```
x-request-id=ARA...ml
Accept=application/json
x-pay-token=x:1573073327:726...dd
```

Endpoint:

```
https://cert.api.visa.com/vts/provisionedTokens?apikey=...
```

Request Body:

```
{
  "consumerEntryMode": "KEYENTERED",
  "panSource": "TOKEN_FOR_TOKEN",
  "ip4address": "...",
  "presentationType": [
    "ECOM"
  ],
  "location": ".../...",
  "protectionType": "SOFTWARE",
  "locale": "en_US",
  "clientAppID": "...",
  "clientWalletAccountEmailAddressHash": "...",
  "clientWalletAccountEmailAddress": "...@...",
  "encPaymentInstrument": "...",
  "clientWalletAccountID": "..."
}
```

Response

```
{
  "vProvisionedTokenID": "189...01",
  "vPanEnrollmentID": "4d3...01",
  "ignore00field": "6BAC6AA2",
  "paymentInstrument": {
    "expirationDate": {
      "year": "2020",
      "month": "12"
    },
  },
  "verificationResults": {
    "cvv2VerificationCode": "M"
  },
  "last4": "3287",
  "cvv2PrintedInd": "Y",
  "ignore01field": "F7650999",
}
```

```
"expDatePrintedInd": "Y",
"paymentAccountReference": "V00...62",
"enabledServices": {
  "merchantPresentedQR": "N"
},
},
"tokenInfo": {
  "encTokenInfo": "...",
  "tokenRequestorID": "40...45",
  "tokenReferenceID": "DNI...41",
  "tokenStatus": "ACTIVE",
  "last4": "9103",
  "expirationDate": {
    "month": "12",
    "year": "2020"
  }
}
}
```

Provision Token—VCES Applied Response Example

Request

Selected Headers:

```
x-request-id=ruB...IF
Accept=application/json
x-pay-token=xv2:1690406651:187...99
```

Endpoint:

```
https://cert.api.visa.com/vts/provisionedTokens?apikey=...
```

Request Body:

```
{
  "consumerEntryMode": "KEYENTERED",
  "panSource": "ONFILE",
  "ip4address": "10.240.2.123",
  "presentationType": [
    "ECOM"
  ],
  "location": ".../...",
  "protectionType": "SOFTWARE",
  "locale": "en_US",
  "clientAppID": "ecomAppId",
  "clientWalletAccountEmailAddressHash": "...",
  "clientWalletAccountEmailAddress": "...",
  "encPaymentInstrument": "eyJ...Nw",
```

```
"clientWalletAccountID": "1B...r"
}
```

Response

```
{
  "appliedPrograms": [
    "VCES"
  ],
  "vPanEnrollmentID": "13d...01",
  "vProvisionedTokenID": "ebe...01",
  "ignore00field": "EE7F4360",
  "paymentInstrument": {
    "verificationResults": {
      "ignore02field": "7940EE4D",
      "cvv2VerificationCode": "M",
      "addressVerificationCode": "U"
    },
    "expirationDate": {
      "month": "07",
      "year": "2024"
    },
    "last4": "5366",
    "cvv2PrintedInd": "Y",
    "ignore01field": "B252216A",
    "expDatePrintedInd": "Y",
    "paymentAccountReference": "V00...84",
    "enabledServices": {
      "merchantPresentedQR": "N"
    }
  },
  "tokenInfo": {
    "encTokenInfo": "eyJ...iw",
    "tokenRequestorID": "400...45",
    "tokenReferenceID": "DNI...00",
    "tokenStatus": "ACTIVE",
    "last4": "0009",
    "expirationDate": {
      "month": "07",
      "year": "2026"
    }
  }
}
```

Provision with Given PAN (async=true) Example

Request

Selected Headers:

```
"x-request-id": "gvC...yB"  
"Accept": "application/json"
```

Endpoint:

```
POST https://cert.api.visa.com/vts/provisionedTokens?  
apiKey=2Y2...Ds&async=true&relationshipID=100...D3
```

Request Body

```
{  
  "clientAppID": "...",  
  "clientWalletAccountEmailAddressHash": "Fp2wvWMe",  
  "clientWalletAccountID": "Kbw2SgtO",  
  "encPaymentInstrument": "eyJ6g",  
  "locale": "en_US",  
  "panSource": "ONFILE",  
  "presentationType": [  
    "ECOM"  
  ],  
  "protectionType": "CLOUD"  
}
```

Response

Response Body:

```
{"vTaskID": "000...02"}
```

Chapter 6

Confirm Provisioning

Confirm Provisioning Introduction

Confirms the success or failure of a token provisioning event.

Confirm Provisioning Request

Endpoints

Resource Path

`vts/provisionedTokens/{vProvisionedTokenID}/confirmProvisioning`

Sandbox Path

```
https://cert.api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}
/confirmProvisioning?apiKey=key
```

Live Path

```
https://api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}  
/confirmProvisioning?apiKey=key
```

URL Parameters

Parameter	Description
{vProvisionedTokenID}	<i>(Required)</i> Unique ID for the provision being confirmed. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.

Methods

PUT

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
<code>apiKey</code>	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.

Request Body

Field	Description
<code>api</code>	(Conditional) Account Parameter Index (YHHHCC) value, which is required if the token was provisioned for NFC-HCE; otherwise, it is not used. This data element is utilized during cryptogram generation. Format: Alphabetic, numeric; maximum 8 characters
<code>provisioningStatus</code>	(Required) Status flag for provisioning call. Format: It is one of the following values: • SUCCESS • FAILURE
<code>failureReason</code>	(Optional) Reason description if available during a provision failure for NFC-HCE or NFC-SE. In current version this value will be defaulted to "Processing Failure" when <code>provisioningStatus</code> is FAILURE. Format: Alphabetic, numeric; maximum 32 characters
<code>apduReasonCode</code>	(Conditional) An array containing the APDU line numbers and reason codes; used only for NFC-SE errors Format: An array of <code>apduReasonCode</code> structures
<code>reperso</code>	(Optional) Whether the confirmation is for reperso. Format: It is one of the following values: • <code>true</code> – Confirmation is for reperso • <code>false</code> (default) – Confirmation is for provisioning.
<code>vNotificationID</code>	(Conditional) Reperso NotificationID—Required when <code>reperso</code> parameter is set to <code>true</code>. Otherwise, it is not needed. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.

APDU Response Code

The `apduReasonCode` structure contains the following fields:

Field	Description
<code>apduLine</code>	<i>(Required)</i> The APDU line identifier. Format: Alphabetic, numeric; maximum 36 characters
<code>reasonCode</code>	<i>(Required)</i> GP-compliant APDU reason code. Format: Alphabetic, numeric; maximum 10 characters

Confirm Provisioning Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

The response does not have a body.

Confirm Provisioning Errors

Error Code	Reason Code	Message
409	<code>notAllowed</code>	Further operations for this card are no longer allowed. Contact your bank to resolve this issue.

This API also returns Standard Errors.

Confirm Provisioning Examples

Confirm Provisioning Example

Request

Endpoint:

```
https://cert.api.visa.com/vts/provisionedTokens/  
8ec...02/confirmProvisioning?apikey=GPN...3I
```

Request Body:

```
{  
  "api": "08581800",  
  "": "SUCCESS"  
}
```

Response

The response does not have a body.

Chapter 7

Active Account Management— Replenish

Active Account Management Replenish Introduction

Requests new dynamic parameters for a token. Call this API if dynamic parameters have expired, or are about to expire.

Active Account Management— Replenish Request

Endpoints

Resource Path

`vts/provisionedTokens/{vProvisionedTokenID}/replenish`

Sandbox Path

```
https://cert.api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}
/replenish?apiKey=key
```

Live Path

```
https://api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}
/replenish?apiKey=key
```

URL Parameters

Parameter	Description
<code>{vProvisionedTokenID}</code>	<i>(Required)</i> Unique ID for the provision being confirmed. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.

Methods

POST

Required Headers

Header	Description
<code>x-request-id</code>	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
<code>x-pay-token</code>	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.

Request Body

Field	Description
mac	(Required) Digital code generated using a cryptogram algorithm which uses the limited use key to generate this signature. This signature helps confirm the presence of the key. Format: Alphabetic, numeric; maximum 16 characters
api	(Required) Account Parameter Index (YHHHCC), which is used to generate cryptograms. Generated by VTS and associated with token request/replenishment. Format is defined by VCPCS 1.8 Format: Alphabetic, numeric; maximum 8 characters
sc	(Required) Sequence counter will be incremented upon each replenishment call. Generated by VTS. Format: Numeric. Size: 65535 Max value

Field	Description
<code>tv1</code>	(Conditional) A consolidation of data elements associated with previous transactions conducted on the device utilizing a Token/LUK combination (UTC unpredictable number channel); max 100 TVL records per request. Required for subsequent Replenish calls if <code>tv1</code> was initially used to generate the MAC for MAC validation. The log contains entries since the first time the specified token was provisioned or since the latest replenishment, whichever is more recent. Each log entry contains the following information: • Timestamp • Unpredictable number • Application transaction count (ATC), which is the number of transactions for which the token has been used • Transaction type, which is one of the following values: – <code>Q</code> for qVSDC – <code>M</code> for MSD – <code>S</code> for MSR – <code>I</code> for INAPP, meaning unpredictable number, which must be null Source: Device Format: Alphabetic, numeric; 15-34 characters, UTC Format in epoch seconds Example: 1472567789 5 I
<code>encryptionMetaData</code>	(Conditional) Cryptographic payload for end-to-end encryption of sensitive fields. It is required for the OPACITY_A encryption scheme. Format: Alphabetic, numeric; maximum 512 characters.

Active Account Management— Replenish Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
<code>tokenInfo</code>	Token info. Format: A <code>tokenInfo</code> structure.

Token Info

Field	Description
<code>hceData</code>	HCE data. This field is populated for HCE tokens only. Format: An <code>hceData</code> structure.

HCE Data

Field	Description
<code>dynParams</code>	Dynamic parameters. Format: A <code>dynParams</code> structure.

Dynamic Parameters

Field	Description
<code>encKeyInfo</code>	Encrypted new, limited-use key for a given provision token ID. This value is used for cryptogram generation and is updated during replenishment requests. It is encrypted using the wallet provider's preferred encryption scheme (for example, <code>_A</code>). Format: Alphabetic, numeric; maximum 32 characters. For more information, see "JSON Web Encryption and Object Signing."
<code>maxPmts</code>	The maximum number of transactions that a token can be used before requiring replenishment Format: Numeric. Maximum value: 9999.
<code>api</code>	Account Parameter Index (YHHHHCC) value. This data element is used during cryptogram generation. Format: Alphabetic, numeric; maximum 8 characters.

Field	Description
keyExpTS	Expiration Timestamp associated with the LUK time to live. Once past the date, the token must be replenished. Format: Date. The UNIX_UTC_Timestamp is a UNIX Epoch timestamp. The value is in seconds.
dki	Derived key indicator defining the key index used to calculate the cryptogram. The issuer bank configures this data. Format: Alphabetic, numeric; maximum 2 characters.
encryptionMetaData	Cryptographic payload for end-to-end encryption of sensitive fields. It is required for the OPACITY_A encryption scheme. Format: Alphabetic, numeric; 256-512 characters.

Active Account Management— Replenish Errors

Error Code	Reason Code	Message
403	cardNotEligible	Replenish cannot be performed due to a problem with the issuer's configuration.
409	notAllowed	Further operations for this card are no longer allowed. Contact your bank to resolve this issue.

Active Account Management— Replenish Examples

JSON OPACITY Request

```
{
  "signature": {
    "mac": "714...7B"
  },
  "encryptionMetaData": "D5D...",
  "tokenInfo": {
    "hceData": {
      "dynParams": {
        "api": "06084400",
        "sc": "1",
        "tv1": ["143...|Q"]
      }
    }
  }
}
```

```
    }  
  }  
}
```

Active Account Management-Replenish Example

Request

Endpoint:

```
https://cert.api.visa.com/vts/provisionedTokens/  
8ec...02/replenish?apikey=GPN...3I
```

Header:

```
x-request-id: kkWp...uz  
x-pay-token: xv2:1544731560:a4e...ee  
Content-Type: application/json  
Host: cert.api.visa.com
```

Request Body

```
{  
  "signature" : {  
    "mac" : "9CF...57"  
  },  
  "tokenInfo" : {  
    "hceData" : {  
      "dynParams" : {  
        "api" : "00302100",  
        "sc" : "1",  
        "tv1" : [  
          "158...47||1|M"  
        ]  
      }  
    }  
  }  
}
```

Response

```
{  
  "tokenInfo" : {  
    "hceData" : {  
      "dynParams" : {  
        "encKeyInfo" : "eyJ...3w",  
        "maxPmts" : 5,  
      }  
    }  
  }  
}
```

```
    "api" : "00302101",  
    "keyExpTS" : 159...59,  
    "dki" : "ff"  
  }  
}  
}
```

Chapter 8

Active Account Management— Confirm Replenishment

Active Account Management— Confirm Replenishment Introduction

Call Confirm Replenishment to confirm that the dynamic parameters associated with the token were successfully deployed to the token deployment point (in most cases, this will be a mobile device).

Active Account Management— Confirm Replenishment Request

Endpoints

Resource Path

`vts/provisionedTokens/{vProvisionedTokenID}/confirmReplenishment`

Sandbox Path

```
https://cert.api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}
/confirmReplenishment?apiKey=key
```

Live Path

```
https://api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}
/confirmReplenishment?apiKey=key
```

URL Parameters

Parameter	Description
<code>{vProvisionedTokenID}</code>	<i>(Required)</i> Unique ID for the provision being confirmed. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.

Methods

PUT

Required Headers

Header	Description
<code>x-request-id</code>	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
<code>x-pay-token</code>	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	<i>(Required)</i> Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.

Request Body

Field	Description
tokenInfo	<i>(Required)</i> Format: A tokenInfo structure.

Token Info

Field	Description
hceData	<i>(Required)</i> Format: An hceData structure.

HCE Data

Field	Description
dynParams	<i>(Required)</i> Format: A dynParams structure.

Dynamic Parameters

Field	Description
api	(Required) Account parameter index, which is used to generate cryptograms. Source: Generated by VTS and associated with token request/replenishment. Format is defined by VCPCS 1.8. Format: Alphabetic, numeric; maximum 8 characters.
sc	(Required) Sequence counter which is used to generate cryptograms. Tracked in SDK, Sequence Counter will be incremented upon replenishment call. Source: VTS generated. Format: Alphabetic, numeric; maximum 65535 characters.

Active Account Management— Confirm Replenishment Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

The response does not have a body.

Active Account Management— Confirm Replenishment Errors

Active Account Management—Confirm Replenishment Errors

Error Code	Reason Code	Message
409	notAllowed	Further operations for this card are no longer allowed. Contact your bank to resolve this issue.

This API also returns Standard Errors.

Active Account Management— Confirm Replenishment Examples

Active Account Management—Confirm Replenishment Example

Request

Endpoint:

```
https://cert.api.visa.com/vts/provisionedTokens/  
8ec...02/confirmReplenishment?apikey=GPN...3I
```

Request Body:

```
{  
  "tokenInfo": {  
    "hceData": {  
      "dynParams": {  
        "api": "08581801",  
        "sc": "2"  
      }  
    }  
  }  
}
```

Response

The response does not have a body.

Chapter 9

Replenish ODA Data

Replenish ODA Data Introduction

This operation returns the ODA Data `iccPubKeyCert` field and supports certificate renewal. For example, it will automatically return a new certificate if the certificate has expired.

Replenish ODA Data Request

Endpoints

Resource Path

`vts/provisionedTokens/{vProvisionedTokenID}/replenishODA`

Sandbox Path

```
https://cert.api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}  
/replenishODA?apiKey=key
```

Live Path

```
https://api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}  
/replenishODA?apiKey=key
```

URL Parameters

Parameter	Description
{vProvisionedTokenID}	<i>(Required)</i> Unique ID for the provision being confirmed. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.

Methods

POST

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Request Body

None. The request does not have a body.

Replenish ODA Data Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
ODAData	(Required) Will be returned if the token can be used for ODA purposes. Refer to ODA Data. It will return only the <code>iccPubKeyCert</code> field of the ODA Data.

Replenish ODA Data Errors

Error Code	Reason Code	Message
400	notSupported	This token currently does not support ODA. Ignore this error and do not invoke this API until the token starts supporting ODA.
409	notAllowed	The requested action is not allowed for a given token. Token status is invalid for this operation. Most likely, the token is not in an “active” state. Check the token details and perform the required/appropriate LCM action.

This API also returns Standard Errors.

Replenish ODA Data Examples

Replenish ODA Data Examples

Request

Selected Headers:

```
"x-request-id": "gvC...yB"  
"Accept": "application/json"
```

Endpoint:

```
https://cert.api.visa.com/vts/provisionedTokens/50c...01/  
replenishODA?apikey=KJP...aQ
```

Request Body

There is no request body.

Response

Selected Headers:

```
"x-response-id": "gvC...yB"
```

Response Body:

```
{  
  "ignore00field": "B0AF015D",  
  "ODADData": {  
    "iccPubKeyCert": {  
      "exponent": "03",  
      "certificate": "Bip...M=",  
      "remainder": "",  
      "expirationDate": {  
        "month": "11",  
        "year": "2018"  
      }  
    }  
  }  
}
```

Chapter 10

Submit ID&V Step-Up Method

Submit ID&V Step-Up Method Introduction

This API tells VTS the consumer's choice of ID&V step-up request method. The request manages any critical metadata between VTS and issuers.

Submit ID&V Step-Up Method Request

Endpoints

Resource Path

`vts/provisionedTokens/{vProvisionedTokenID}/stepUpOptions/method`

Sandbox Path

```
https://cert.api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}  
/stepUpOptions/method?apiKey=key&relationshipID=id
```

Live Path

```
https://api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}  
/stepUpOptions/method?apiKey=key&relationshipID=id
```

URL Parameter

Parameter	Description
{vProvisionedTokenID}	<i>(Required)</i> Unique ID for the provision being confirmed. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.

Methods

PUT

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	<i>(Required)</i> Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
relationshipID	<i>(Optional)</i> The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

Field	Description
stepUpRequestID	<i>(Required)</i> ID of step-up option selected by the consumer. This is the same field as <code>stepUpRequest.identifier</code> of the provision API. The wallet provider must send this value "as is" to subsequent step-up related APIs. Format: Alphabetic, numeric, plus signs (+), slashes (/), and equal signs (=), spaces are not allowed; maximum 96 characters.
date	<i>(Required)</i> Time stamp when consumer inputs step up choice. This time stamp is in UTC format. Format: EpochDateTime.
clientReferenceId	<i>(Conditional)</i> Unique ID for a payment transaction, which is required when the <code>reasonCode</code> is <code>PAYMENT</code> , <code>DEVICE_BINDING</code> , or <code>TRUSTED_LISTING</code> in the step-up options request. Do not specify for token provisioning. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
clientAppID	<i>(Conditional)</i> Unique identifier for the client application; required when the <code>relationshipID</code> query parameter is specified. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.

Submit ID&V Step-Up Method Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
<code>maxOTPVerificationAllowed</code>	<i>(Required)</i> The number of retries a consumer can enter OTP before they must request a new one. Format: Numeric; maximum 3 digits.
<code>maxOTPRequestsAllowed</code>	<i>(Required)</i> The number of times the consumer can request a new OTP for a token. After the number of attempts is exceeded, step up authentication via OTP is disabled for this token. Format: Numeric; maximum 3 digits.
<code>codeExpiration</code>	<i>(Required)</i> OTP validation will fail if the consumer submits OTP after this period. The value is set in minutes. Format: Numeric; maximum 10 digits.

Submit ID&V Step-Up Method Errors

Error Code	Reason Code	Message
400	cvmNotAvailable	Requested CVM method is not available at this time. Choose another CVM method. The end user should be presented with the option to choose from one of the other CVM methods.
403	declined	This card is considered not eligible for tokenization at this moment. Try enroll/provision again for this PAN. If you encounter the same message even for the new enrollment/provision request then ask the customer to either add a different card or to reach out to their issuer.
409	notAllowed	The requested action is not allowed for a given token. Token status is invalid for this operation. Check the token details and perform the required/appropriate LCM action.

This API also returns Standard Errors.

Submit ID&V Step-Up Method Examples

Submit ID&V Step-Up Method Examples

Request

Selected Headers:

```
x-pay-token: xv2:1588709957:11f...29  
x-request-id: muj...36}
```

Endpoint:

```
https://cert.api.visa.com/vts/provisionedTokens/28d...01/  
stepUpOptions/method?apikey=KJ...aQ
```

Request Body

```
{  
  "stepUpRequestID": "NjhiNWQ2NzljYmY5M2E5MjQ2MDAxNDk4MmFmNGI0MDE=",
```

```
"date": "1441828471"  
}
```

Response

Selected Headers:

```
x-response-id: muj...36
```

Response Body:

```
{  
  "maxOTPRequestsAllowed": 3,  
  "maxOTPVerificationAllowed": 3,  
  "codeExpiration": 2159  
}
```

Chapter 11

Validate OTP

Validate OTP Request

This API allows clients to validate the One-Time Password (OTP) entered by the consumer as part of Identification and Verification (ID&V).

Endpoints

Resource Path

`vts/provisionedTokens/{vProvisionedTokenID}/stepUpOptions/validateOTP`

Sandbox Path

```
https://cert.api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}
/stepUpOptions/validateOTP?apiKey=key&relationshipID=id
```

Live Path

```
https://api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}  
/stepUpOptions/validateOTP?apiKey=key&relationshipID=id
```

URL Parameters

Parameter	Description
{vProvisionedTokenID}	<i>(Required)</i> Unique ID for the provision being confirmed. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.

Methods

POST

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
relationshipID	(Optional) The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

Field	Description
otpValue	(Conditional) Value of OTP entered by consumer. Required when the wallet provider (WP) determines that multi-factor authentication is needed for the transaction and initiates consumer authentication prior to calling the Get Payment Data API. If this field is not present, <code>encOtpValue</code> must be provided. Format: Numeric; maximum 8 digits.
date	(Required) Date Format: Date. The UNIX_UTC_Timestamp is a UNIX Epoch Timestamp. The value is in seconds.
clientReferenceId	(Required) Unique ID for the transaction. This field should not be present for token provisioning. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.

Field	Description
<code>encOtpValue</code>	(Conditional) JSON payload encrypted with Visa public key and signed by the device key. The payload contains: • <code>otp</code> that identifies the OTP value • <code>certID</code> that identifies the certificate ID; it is the first 4 bytes of the device integrity public key's modulus If this field is not present, <code>otpValue</code> must be provided. Format: BLOB; standard Base64 Encoding per RFC4648; maximum 8192 characters. JWE: RSAES-OAEP with SHA-256 as hash and MGF JWS: RSA-PSS with SHA-256 as hash and MGF For more information, see "JSON Web Encryption and Object Signing." Example: Unencrypted payload: <pre>{ "otp": "...", "certID": "..." }</pre>
<code>clientAppID</code>	(Conditional) Unique identifier for the client application; required when the <code>relationshipID</code> query parameter is specified. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.

Validate OTP Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

The response does not have a body.

Validate OTP Errors

Error Code	Reason Code	Message
400	cvmNotAvailable	Requested CVM method is not available at this time. Choose another CVM method. The end user should be presented with the option to choose from one of the other CVM methods.
400	invalidParameter	The input value for parameters is in invalid format or the authentication code is invalid - validate the input data.
409	expired	The authentication code has expired. The OTP code has expired and cannot be used any more. Obtain a new OTP value.
409	notAllowed	Retry limit exceeded and/or further operations for this card are no longer allowed. Contact your bank to resolve this issue.

This API also returns Standard Errors.

Validate OTP Examples

Validate OTP Examples

Request

Selected Headers:

```
"x-request-id": "gvC...yB"  
"Accept": "application/json"
```

Endpoint:

```
https://cert.api.visa.com/vts/provisionedTokens/58f...01/  
stepUpOptions/validateOTP?apikey=KJP...aQ
```

Request Body

```
{  
  "otpValue": "761110",  
  "date": "1516660442"  
}
```

Response

Selected Headers:

```
"x-response-id": "qhh...Or"
```

Response Body:

The response does not have a body.

Chapter 12

Validate Authentication Code

Validate Authentication Code Request

Endpoints

Resource Path

`vts/provisionedTokens/{vProvisionedTokenID}/stepUpOptions/validateAuthCode`

Sandbox Path

```
https://cert.api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}
/stepUpOptions/validateAuthCode?apiKey=key&relationshipID=id
```

Live Path

```
https://api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}  
/stepUpOptions/validateAuthCode?apiKey=key&relationshipID=id
```

URL Parameters

Parameter	Description
{vProvisionedTokenID}	<i>(Required)</i> Unique ID for the provision being confirmed. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.

Methods

POST

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as: <code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
relationshipID	(Optional) The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

Field	Description
issuerAuthCode	(Conditional) Value of the Authentication Code from the issuer received through redirect from mobile payment app.Encrypted BLOB and opaque to MWP. Format: Standard base64 encoding per RFC4648; maximum 4096 characters.
date	(Required) Timestamp when this auth code submitted by client. Format: Date. The UNIX_UTC_Timestamp is a UNIX Epoch Timestamp. The value is in seconds.
clientReferenceID	(Conditional) Unique ID for the transaction. It is required when the reasonCode is PAYMENT. This field should not be present for token provisioning. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.

Field	Description
encIssuerAuthCode	(Conditional) Value of the issuer auth code provided by issuer's mobile app, which is encrypted with Visa public key and signed with device key. The payload includes: • <code>authCode</code> that identifies the auth code value • <code>certID</code> that identifies the certificate ID; it is the first 4 bytes of the device integrity public key's modulus Format: JWE: RSAES-OAEP with SHA-256 as hash and MGF JWS: RSA-PSS with SHA-256 as the hash and the MGFLimit 8192 characters. Example: Unencrypted payload: <pre>{ "authCode": "...", "certID": "..."} </pre>
clientAppID	(Conditional) Unique identifier for the client application; required when the <code>relationshipID</code> query parameter is specified. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.

Validate Authentication Code Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

The response does not have a body.

Validate Authentication Code Errors

Error Code	Reason Code	Message
409	notAllowed	Further operations for this card are no longer allowed. Contact your bank to resolve this issue.

This API also returns Standard Errors.

Validate Authentication Code Examples

Validate Authentication Code Example

Request

Selected Headers:

```
"x-request-id": "gvC...yB"  
"Accept": "application/json"
```

Endpoint:

```
https://cert.api.visa.com/vts/provisionedTokens/b86...01/  
stepUpOptions/validateAuthCode?apikey=3TC...1Q
```

Request Body

```
{  
  "issuerAuthCode": "TUJ...==",  
  "date": "1441828471"  
}
```

Response

Selected Headers:

```
"x-response-id": "qhh...Or"
```

Response Body:

The response has no body.

Chapter 13

Get Step Up Options

Get Step Up Options Introduction

This API allows wallet provider to retrieve the available step-up options for a token. It retrieves the list of 0 or more step-up request options filtered by the specified query parameters. These options are dynamic and can change from one API call to another.

There are 2 methods for this API:

- `GET` - retrieves the the available step-up options by the specified query parameters
- `POST` - retrieves the the available step-up options by the specified query parameters and the cardholder's name and address, if available

Get Step Up Options Request

Endpoints

Resource Path

`vts/provisionedTokens/{vProvisionedTokenID}/stepUpOptions`

Sandbox Path

```
https://cert.api.visa.com/vts/provisionedTokens  
/{vProvisionedTokenID}/stepUpOptions?apiKey=key&platformType=type  
&clientReferenceId=id&reasonCode=code&relationshipID=id
```

Live Path

```
https://api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}  
/stepUpOptions?apiKey=key&platformType=type  
&clientReferenceId=id&reasonCode=code&relationshipID=id
```

URL Parameters

Parameter	Description
<code>{vProvisionedTokenID}</code>	<i>(Required)</i> Unique ID for the provision being confirmed. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.

Method

GET, POST

Required Headers

Header	Description
<code>x-request-id</code>	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
<code>x-pay-token</code>	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Field	Description
apiKey	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: Alphabetic, numeric; maximum 64 characters.
clientAppID	(Conditional) Unique identifier for the client application; required when the <code>relationshipID</code> query parameter is specified. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
clientReferenceId	(Conditional) Unique ID for a payment transaction, which is required unless the <code>reasonCode</code> is <code>PROVISIONING</code> in the Step Up Options method. Do not specify a value for token provisioning. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
platformType	(Conditional) The platform which is required for the NFC-SE presentation type. Note: <i>This parameter affects the <code>APP-TO-APP</code> step up fields in the response.</i> Format: It is one of the following values: • <code>ios</code> – iOS app • <code>ANDROID</code> – Android app • <code>WINDOWS</code> – Windows app • <code>WEB</code> – Browser-based app
reasonCode	(Optional) The reason for the step-up request. It is required for a payment transaction. Format: It is one of the following values: • <code>PAYMENT</code> • <code>CARDHOLDER_STEPUP</code> • <code>DEVICE_BINDING</code> • <code>PROVISIONING</code> (Default)
relationshipID	(Conditional) The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

The request for the GET method does not have a body. The request for the POST method optionally contains the following body:

Field	Description
<code>encBillingInfo</code>	<i>(Optional)</i> Encrypted billing information blob. Encrypted by JSON Web Encryption using the shared secret. For more information, see "JSON Web Encryption and Object Signing." Format: String. Maximum 4000 characters

Unencrypted Billing Info

Field	Description
<code>billingAddress</code>	<i>(Optional)</i> Billing address associated with the payment instruments. Format: An <code>address</code> structure.
<code>name</code>	<i>(Optional)</i> The full name on the Visa card. Must use UTF-8 characters. Format: String; minimum 0 and maximum 256 characters.
<code>email</code>	<i>(Optional)</i> Email address. Must use UTF-8 characters. Format: String; maximum 64 characters.
<code>phoneNumber</code>	<i>(Optional)</i> Phone number. Format: String; numeric; a valid phone number; maximum 18 digits.
<code>currencyCode</code>	<i>(Optional)</i> Currency code. Format: String; ISO 4217-format currency code (numeric)

Billing Address

Field	Description
<code>line1</code>	<i>(Optional)</i> Street 1 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.

line2	(Optional) Street 2 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line3	(Optional) Street 3 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line4	(Optional) Street 4 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line5	(Optional) Street 5 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
city	(Optional) The city associated with the enrolled payment instrument. Format: Alphabetic, single quote ('), hyphen (-), and whitespace.; maximum 100 characters.
state	(Optional) State or province code in ISO 3166-2 format, e.g. NY. Format: Alphabetic; maximum 3 characters.
country	(Optional) Country in ISO 3166-1 alpha-2 format, e.g. US. Format: Alphabetic; maximum 2 characters.
countryDigits	(Optional) ISO 3166-1 numeric Country Code. Format: Numeric; maximum 3 digits.
postalCode	(Optional) The postal code associated with the enrolled payment instrument. Note: This field does not support spaces (" ") in the middle of characters. Format: Alphabetic, numeric, e.g. spaces are not allowed; maximum 3–16 characters.

Get Step Up Options Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
<code>stepUpRequest</code>	Returns a list of step-up methods. In some business cases, it is possible to receive an empty array. This field is not populated if the reason code is <code>CARDHOLDER_STEPUP</code> or <code>DEVICE_BINDING</code>. If the call is approved without challenge (or declined), this field will be empty or not present. Format: An array of <code>stepUpRequest</code> structures.
<code>status</code>	The request result. Format: It is one of the following values: • <code>APPROVED</code>—Request approved, no step-up needed • <code>CHALLENGE</code>—Step-up requested • <code>DECLINED</code>—Request denied

Step-Up Request

`stepUpRequest` is conditional. It is not required for the Ecommerce token type. Items 2, 3, and 4 below are applicable to push provisioning only.

Note: This is only applicable when:

- TokenStatus is INACTIVE.
- Device ID used during push provisioning does not match the DeviceID used during enrollment.
- Issuer indicated a preference for IDnV in the JWE by setting isIDnV=Y.
- Push Provisioning expired.

Field	Description
method	ID&V step-up method. Format: It is one of the following values: • OTPEMAIL • OTPONLINEBANKING • OTPSMS • CUSTOMERSERVICE • APP-TO-APP • OUTBOUNDCALL
value	This could be a masked value depending on the ID&V step-up method. Email looks like 'ip****@email.com' bank_app looks like 'Mobile Banking App' customer_service looks like '1-800-555-1212' Note: This field will support internationalization using UTF-8 characters. Format: Alphanumeric, UTF-8 whitespace; maximum 64 characters.
source	The issuer source of contact for this ID&V step-up challenge. In case of OTP, the source contains the issuer contact that will reach out to the consumer. In the case of banking-app step-up, the source is a concatenation of Application ID (<i>id</i>) and Intent Handler (<i>h</i>). This field will be defined by the issuer. Note: Method handler of Banking App designed to respond to authentication code handling. For the Android platform this will be an explicit intent with fully name-spaced class name. Visa requires the issuer to use the a2a name for this intent handler. Example: "source": "{id} {h}", where <i>id</i> is required (size 60, [a-z,A-Z,0-9,_,.]), and <i>h</i> is optional (size 3, [a-z][A-Z][0-9]). Format: Alphanumeric; maximum 1024 characters.

Field	Description
identifier	Unique and opaque identifier for this method. A subsequent step-up request will only provide the identifier. The wallet provider must send this value "as is" to subsequent step-up related APIs. Format: Alphabetic, numeric, plus signs (+), slashes (/), and equal signs (=), spaces are not allowed; maximum 96 characters.
requestPayload	The request payload to be sent to the banking application on behalf of Visa. This field is opaque to wallet providers. Note: This is only present if the step-up type is app-to-app. Format: Base64 encoded string, maximum 1024 characters.
platformType	Platform type, which is present in cases where the Token Requestor has opted in to 3DS Web Step-Up. Format: It is one of the following values: • WEB
subMethod	Flavor of the step-up method. Format: It is one of the following values: • THREE-DS

Get Step Up Options Errors

Error Code	Reason Code	Message
409	notAllowed	The requested action is not allowed for a given token. Token status is invalid for this operation. Most likely, the token is in a state that is different from <code>INACTIVE</code>. Check the token details and perform the required/appropriate LCM action.

This API also returns Standard Errors.

Get Step Up Options Examples

Get Step-Up Options Examples

Request

Selected Headers:

```
"x-request-id": "gvC...yB"  
"Accept": "application/json"
```

Endpoint:

```
https://cert.api.visa.com/vts/provisionedTokens/817...01/  
stepUpOptions?apikey=0AF...-k
```

Request Body

The request does not have a body.

Response

Selected Headers:

```
"x-response-id": "qhh...Or"
```

Response Body:

```
{  
  "ignore00field": "6F7A0E6A",  
  "stepUpRequest": [  
    {  
      "method": "OTPSMS",  
      "value": "...",  
      "identifier": "YTY...DE="    },  
    {  
      "method": "OTPEMAIL",  
      "value": "...",  
      "identifier": "NmE...E="    },  
    {  
      "method" : "APP-TO-APP",  
      "value" : "Verify Online with Bank",  
      "source" : "https://....",  
      "identifier" : "NjZ...=",  
      "requestPayload" : "",  
      "platformType" : "WEB"    },  
  ],  
}
```

```
{
  "method": "PLEASE_IGNORE_ENUM_VALUE",
  "value": "1-800-555-6789",
  "identifier": "OUTBOUNDWDDFGFG33"
}
]
```

Chapter 14

Get Card Metadata

Get Card Metadata Introduction

Retrieves metadata associated with the PAN.

Get Card Metadata Request

Endpoints

Resource Path

`vts/panEnrollments/{vPanEnrollmentID}`

Sandbox Path

```
https://cert.api.visa.com/vts/panEnrollments/{vPanEnrollmentID}
?apiKey=key&platformType=type&relationshipID=id
```

Live Path

```
https://api.visa.com/vts/panEnrollments/{vPanEnrollmentID}  
?apiKey=key&platformType=type&relationshipID=id
```

URL Parameters

Parameter	Description
{vPanEnrollmentID}	<i>(Required)</i> Unique ID for the enrolled PAN. Format: String. Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.

Method

GET

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Field	Description
apiKey	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: Alphabetic, numeric; maximum 64 characters.
platformType	(Conditional) The platform which is required for the NFC-SE presentation type. Note: <i>This parameter affects the APP-TO-APP step up fields in the response.</i> Format: It is one of the following values: • <code>ios</code> – iOS app • <code>android</code> – Android app • <code>windows</code> – Windows app • <code>web</code> – Browser-based app
relationshipID	(Conditional) The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

None. The request does not have a body.

Get Card Metadata Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
<code>vPanEnrollmentID</code>	Unique ID for enrolled PAN. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
<code>coresidenceInfo</code>	Contains data about the card and whether or not it is co-badged. Format: A <code>coresidenceInfo</code> structure.
<code>paymentInstrument</code>	Format: A <code>paymentInstrument</code> structure.
<code>tokens</code>	A list of <code>vProvisionedTokenID</code> values and the state for each token. If the array is empty, it means that no tokens have been provisioned. Legacy API implementations will not return this array. Format: An array of provisioned tokens.
<code>cardMetaData</code>	Format: A <code>cardMetaData</code> structure.
<code>unsupportedPresentationTypes</code>	This field is to be used by wallet providers with more than one token type. This field indicates the presentation types that are supported by an issuer. Format: • If the issuer is not enrolled for HCE, this field will contain: - <code>NFC-HCE</code> - <code>INAPP</code> - <code>MERCHANT_PRESENTED_QR</code> • If the issuer has opted out of <code>ECOM</code>, this field will contain <code>ECOM</code> presentation types. • If the issuer has opted out of <code>ESE</code>, this field will contain <code>NFC-SE</code> presentation types. Note: During integration for multiple use cases, wallet providers must not sent unsupported presentation types in the provisioning request.

cardDetails	Card Issuer details provided based on Issuer consent. This is applicable for ECOM and COF token types only. Format: A cardDetails object.
encPaymentInstrument	BLOB field containing all payment instrument details (URL-safe base64 encoded) that represents the encrypted payload for payment instrument. It is only available if the issuer consented that the token requestor receive this information. Format: JSON Web Encrypted string using the shared secret. For more information, see "JSON Web Encryption and Object Signing." String Size: 8192.

Coresidence

Field	Description
cardImageMode	Identifies the card image usage for the alternate networks available to the wallet provider. Format: It is one of the following values: - SHARED—Single co-residence CardArt image provided by one of the networks. The wallet provider will use this image (card Data element having digitalCardArt as the contentType) across all co-residence networks. - PER_NETWORK—A CardArt image for co-residence per network. Wallet will use the CardArt image provided by the respective network.
alternateNetworks	Array of available alternate networks. Format: An array of alternateNetworks structures.

Coresidence Alternate Networks

Field	Description
name	Alternate network name. Format: Alphabetic, numeric; maximum 30 characters.
id	EMV® Sepcification RID, a <i>registered application provider identifier</i> (RID) which is issued by the ISO/IEC 7816–5 registration authority. Format: Alphabetic, numeric; maximum 30 characters.

Payment Instrument

Field	Description
last4	Last 4 digits of the account. Note: If an IBAN was provided in the provisioning request, this is the last 4 digits of the IBAN. Format: String, numeric; maximum 4 digits.
expirationDate	Payment instrument's expiration date. Format: An <code>expirationDate</code> structure.
accountStatus	<i>Future:</i> Is the account closed? To be returned with Get Card Metadata API only. Format: It is one of the following values: - Y – Account closed - N – Account not closed
isTokenizable	<i>Future:</i> Is this account eligible for tokenization? To be returned with Get Card Metadata API only. Format: It is one of the following values: - Y – Eligible - N – Not Eligible
indicators	These are indicators associated with the Payment Instrument. If the field is not returned, or empty, then there are no indicators associated with it. Expected values are: PRIVATE_LABEL - This Payment Instrument is for a Private Label PAN. Format: String
expDatePrintedInd	Indicates if the expiration date is printed on the payment instrument. Format: It is one of the following String values: - Y - Printed - N - Not printed
cvv2PrintedInd	Indicates if the CVV2 is printed on the payment instrument. Format: It is one of the following String values: - Y - Printed - N - Not printed
paymentAccountReference	Payment Account Reference (PAR) if available for this payment instrument. Format: String, alphabetic or numeric; maximum 29 characters.

Field	Description
<code>enabledServices</code>	Enabled services, such as scan-and-pay. Format: An <code>enabledServices</code> structure.
<code>verificationResults</code>	Payment instrument verification results. Present only for <code>ECOM</code> presentation type when the CVV2 and/or address fields appear in the provision request. Format: A <code>verificationResults</code> object.

Expiration Date

Field	Description
<code>month</code>	The month upon which the payment instrument is set to expire. Example: <code>"month": "9"</code> Format: String; numeric; max length 2 digits.
<code>year</code>	The year upon which the payment instrument is set to expire. Example: <code>"year": "2024"</code> Format: String; numeric; length 4 digits.

Enabled Services

Field	Description
<code>merchantPresentedQR</code>	Whether the issuer enabled the PAN for the scan-and-pay use case. Format: It is one of the following String values: • Y – Enabled • N – Disabled

Verification Results

Field	Description
cvv2VerificationCode	The CVV2 verification result. This field is present when CVV2 verification data is present in the request. Format: It is one of the following values: • M – verified, CVV2 data match • N – verified, negative data match • U – verification not possible (issuer does not participate) • D – verification not performed due to duplicate request
addressVerificationCode	The address verification result. This field is present when address verification data is present in the request. Format: It is one of the following values: • Y – verified, address and postal code data match • P – verified, partial match (either address or postal code data match, but no result from the other) • B – verified, partial match (either address or postal code data match, but the other could not be verified due to incompatible format) • N – verified, neither address nor postal code data match • U – verification not possible (issuer does not participate) • D – verification not performed due to duplicate request

Provisioned Tokens

Field	Description
vProvisionedTokenID	Unique ID for token. Format:String. Alphanumeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.6 characters.
tokenStatus	If a token was deleted, no information will be returned for the token. a Token status of <code>SUSPENDED</code> is not applicable to this API. Format: It is one of the following values: • INACTIVE • ACTIVE • SUSPENDED

Card Metadata

Field	Description
backgroundColor	Background color of the mobile wallet UI entry (“space”) for the card. Specified as CSS style hex format, e.g., 0x0e19d2. Format: CSS style hex format, e.g. 0x0e19d2; maximum 32 characters.
foregroundColor	Foreground color of the mobile wallet entry for the card. Specified as CSS style hex format, e.g., 0x0e19d2. Format: CSS style hex format. E.g. 0x0e19d2; maximum 32 characters.
labelColor	Label color of the mobile wallet entry for the card. Specified as CSS style hex format, e.g. 0x0e19d2. Format: Alphabetic, numeric; maximum 32 characters.
contactWebsite	Customer service website of the issuing bank. It is recommended that this be a specific landing page created for cards that are provisioned into a specific wallet program. Format: Alphabetic, numeric; maximum 256 characters. Note: This field will support internationalization using UTF-8 characters.
contactEmail	Customer service email address of the issuing bank. Format: CSS style hex format, e.g. 0x0e19d2; maximum 32 characters. Note: This field will support internationalization using UTF-8 characters.
contactNumber	Customer service phone number of the issuing bank. Format: Alphabetic, numeric.
contactName	Name of the issuing bank. It may be used in combination with other text strings, e.g. if <code>contactName</code> is “XYZ Bank”, it may be used to construct a string “Contact XYZ Bank” or “XYZ Bank Contact Information”. Format: Alphabetic, numeric, and spaces (); maximum 32 characters. Note: This field will support internationalization using UTF-8 characters.
bankAppName	Name of banking application for display. Format: Alphabetic, numeric; maximum 64 characters. Note: This field will support internationalization using UTF-8 characters.

bankAppAddress	The package name for the destination mobile application. Example: <code>com.sampleissuer.thebankapp</code> Format: Alphabetic, numeric, and periods (.); maximum 1024 characters.
privacyPolicyURL	URL pointing to issuing bank privacy policy. This refers to the issuer website. Format: Valid URL; maximum 128 characters.
termsAndConditionsURL	URL pointing to issuing bank Terms and Conditions for card. Format: Valid URL; maximum 128 characters.
termsAndConditionsID	Terms and Conditions identifier in GUID format. If the GUID is changed from the earlier version, the consumer must be presented a new Terms and Conditions at time of provisioning. Format: Alphabetic, numeric; maximum 32 characters.
shortDescription	A short description of the card to be used for notifications on the mobile device (on the lock screen), e.g. <code>The Bank Card</code>. Format: Alphabetic, numeric; maximum 32 characters. Note: This field will support internationalization using UTF-8 characters.
longDescription	A longer description of the card that could appear inside the mobile wallet e.g., “The Bank Card Platinum Rewards”. The <code>shortDescription</code> will be used if <code>longDescription</code> is not specified. Format: Alphabetic, numeric; maximum 64 characters. Note: This field will support internationalization using UTF-8 characters.
unsupportedPresentationTypes	The presentation types that are not supported such as “MSR” (for example). This is an advisory field to tell the wallet provider that they should not include the unsupported presentation type returned in this field in the provisioning request. Note: • If the issuer is not enrolled for HCE, this field will contain: – <code>NFC-HCE</code> – <code>INAPP</code> – <code>MERCHANT_PRESENTED_QR</code> • If the issuer has opted out of <code>ECOM</code>, this field will contain <code>ECOM</code> presentation types. • If the issuer has opted out of <code>ESE</code>, this field will contain <code>NFC-SE</code> presentation types. Format: An array; maximum 256 characters.

<code>unsupportedCardVerificationTypes</code>	The card verification types that are not supported such as "AVS" (for example). This is an advisory field to let the wallet provider know which card verification services are not supported for a given payment instrument. The wallet provider can use this information to relax any field validations on the Customer UI (such as Address). Format: An array; maximum 256 characters.
<code>cardData</code>	Card data. Format: An array of structures.
<code>issuerFlags</code>	Issuer flags. Format: An <code>issuerFlags</code> structure.

Card Data

Field	Description
<code>guid</code>	GUID for the content. Each <code>guid</code> can have an array of content attributes. Content array will carry metadata related to the content. Format: Alphabetic, numeric; maximum 32 characters.
<code>contentType</code>	Type for the content. Format: It is one of the following values: • <code>digitalCardArt</code> • <code>digitalCardArtBackground</code> • <code>cardSymbol</code> • <code>termsAndConditions</code>
<code>content</code>	Format: A <code>content</code> structure.

Content

Field	Description
<code>mimeType</code>	MIME type of the content. Format: It is one of the following values: • <code>image/png</code> • <code>image/pdf</code> • <code>text/plain</code> • <code>text/html</code>

width	Width of image, in pixels. Value between 1 and 4096 for image/png; the value is 0 for image/pdf and text/html. Format: String; maximum 32 characters.
height	Height of image, in pixels. Value between 1 and 4096; the value is 0 for image/pdf and text/html. Format: String; maximum 32 characters.

Issuer Flags

Field	Description
deviceBinding	Device binding. Format: It is one of the following values: • true • false
cardholderVerification	Whether the issuer participating in step-up flow. Format: It is one of the following values: • true • false
trustedBeneficiaryEnrollment	Whether this is a trusted beneficiary enrollment. Format: It is one of the following values: • true • false
delegatedAuthenticationSupported	Whether issuer supports delegated authentication. Format: It is one of the following values: • true • false
oboDeviceBinding	Whether issuer supports on behalf of device binding. Format: It is one of the following values: • true • false

Card Details

Field	Description
issuerName	Issuer name. Format: String; maximum 240 characters.
countryCode	Country code in ISO 3166-1 format Format: Numeric; 3 digits.
isoBIN	ISO BIN. Format: Numeric; 6 or 8 digits.
productIdCode	Product ID code, if available. Format: String; maximum 32 characters.
accountFundingSourceCode	Account funding source code, if available. Format: String; maximum 10 characters.

Encrypted Payment Instrument

Field	Description
accountNumber	PAN printed on the card. Format: Numeric; 12-19 digits.
cardIssuanceDate	Card issuance date, if available. Format: Numeric digits and hyphens; 10 characters in the form of CCYY-MM-DD
cardActivationDate	Card activation date, if available. Format: Numeric digits and hyphens; 10 characters in the form of CCYY-MM-DD

Get Card Metadata Errors

Error Code	Reason Code	Message
401	authError	API Credentials and the encryption keys used do not match, or the API Key was invalid or does not exist.
403	Declined	This card is considered not eligible for tokenization at this moment. Try to enroll/provision again for this PAN. If you encounter the same message after the new enrollment/provision request, ask the customer to either add a different card or to reach out to their issuer.
409	notAllowed	The requested action is not allowed for a given token. Token status is invalid for this operation. Check the token details and perform the required/appropriate LCM action.

This API also returns Standard Errors.

Get Card Metadata Examples

Get Card Metadata Example

Request

Endpoint:

```
https://cert.api.visa.com/vts/panEnrollments/  
e53...01?apikey=GPN...3I
```

Request Body:

The request does not have a body.

Response

```
{  
  "vPanEnrollmentID": "cf9...01",  
  "paymentInstrument": {  
    "expirationDate": {  
      "month": "...",  
      "year": "..."  
    },  
    "last4": "...",  
  },  
}
```

```

    "cvv2PrintedInd": "Y",
    "expDatePrintedInd": "Y",
    "paymentAccountReference": "V00...15",
    "enabledServices": {
      "merchantPresentedQR": "N"
    }
  },
  "tokens": [
    {
      "vProvisionedTokenID": "a74...01",
      "tokenStatus": "ACTIVE"
    }
  ],
  "cardMetaData": {
    "backgroundColor": "0x000000",
    "foregroundColor": "0xffffffff",
    "labelColor": "0xffffffff",
    "shortDescription": "TEST-VISA-Card",
    "cardData": [
      {
        "guid": "12a...e2",
        "contentType": "cardSymbol",
        "content": [
          {
            "mimeType": "image/png",
            "width": "100",
            "height": "100"
          }
        ]
      },
      {
        "guid": "fe4...1b",
        "contentType": "digitalCardArt",
        "content": [
          {
            "mimeType": "image/png",
            "width": "1536",
            "height": "969"
          }
        ]
      }
    ]
  },
  "unsupportedPresentationTypes": [
    "NFC-HCE",
    "INAPP",
    "QR_CONSUMER_DEVICE_HCE",
    "QR_CONSUMER_CLOUD",
    "MERCHANT_PRESENTED_QR",
    "MSR"
  ],
  "issuerFlags": {
    "deviceBinding": false,
    "cardholderVerification": false,
    "trustedBeneficiaryEnrollment": false,
    "delegatedAuthenticationSupported": true
  }
},
"cardDetails": {

```

```
"issuerName": "TEST CREDITBANK ",  
"countryCode": "276",  
"isoBIN": "..."  
},  
"encPaymentInstrument": "eyJ...ag"  
}
```

Chapter 15

Get Payment Data (Token & Cryptogram)—Given Token ID

Get Payment Data (Token & Cryptogram)—Given Token ID Introduction

This API allows a client wallet provider (WP) to request the cryptogram for a given token. It supports enhanced performance and allows for OTP validation. Given a token or `tokenID`, this API returns the token data and cryptogram. It is only applicable for Ecommerce.

Note: Previous releases of this API provided a separate QR version. It has been deprecated; use the standard version, which is version 2 (v2) described here for all features.

Get Payment Data (Token & Cryptogram)—Given Token ID Request

Endpoints

Resource Path

```
vts/v2/provisionedTokens/{vProvisionedTokenID}/paymentData
```

Sandbox Path

```
https://cert.api.visa.com/vts/v2/provisionedTokens/{vProvisionedTokenID}/paymentData?apiKey=key&relationshipID=id
```

Live Path

```
https://api.visa.com/vts/v2/provisionedTokens/{vProvisionedTokenID}/paymentData?apiKey=key&relationshipID=id
```

URL Parameters

Parameter	Description
{vProvisionedTokenID}	<i>(Required)</i> Unique ID for the provision being confirmed. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.

Methods

POST

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request.

Header	Description
	Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	(Required) A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients. Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query

Header	Description
	parameters, such as <code>apiKey=key</code> , and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
<code>apiKey</code>	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
<code>relationshipID</code>	(Optional) The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

Field	Description
<code>encryptionMetaData</code>	(Conditional) Cryptographic payload required for end-to-end encryption of sensitive fields. Format: Alphabetic, numeric; maximum 512 characters
<code>clientPaymentDataID</code>	(Required) Reference identifier defined by client. No uniqueness required. Source: Wallet Provider Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters
<code>paymentRequest</code>	(Required) Required for transaction information about the payment request. Format: A <code>paymentRequest</code> structure.

Field	Description
atc	(Conditional) Should be passed only for device bound tokens for in-app transactions. The ATC Counter is used as part of the input to the TAVV cryptogram. Each time this API is called, the ATC for the Token making the request will be increased by one. Note: ATC is only required for <code>tokenType</code> supporting in-app as <code>presentationType</code>. It should not be present for the e-commerce token. Format: Numeric; maximum 5 characters
riskData	(Optional) Risk data. Format: A <code>riskData</code> structure.
threeDsData	(Conditional) Submit only if <code>cryptogramType</code> is CTAVV. Format: A <code>threeDsData</code> structure.
cryptogramType	(Optional) Cryptogram Type. Format: It is one of the following values: • TAVV – Generate TAVV (default) • CTAVV – Generate CTAVV • DTVV– Generate DTVV. (Compatible only with ECOM presentation type.) The parameter should not be present for tokens supporting QR_CONSUMER_CLOUD presentation type. Note: Shared Secret is always used to encrypt the API response fields when the presentation type is: • ECOM • QR_CONSUMER_CLOUD • RECEIVE_ONLY
otpValue	(Conditional) Value of OTP entered by consumer. Required when the wallet provider (WP) determines that multi-factor authentication is needed for the transaction and initiates consumer authentication prior to calling the Get Payment Data API. Format: Numeric; maximum 8 digits
clientReferenceId	(Conditional) Unique ID for a payment transaction, which is required when the <code>reasonCode</code> is PAYMENT in the Step-Up Options method. Do not specify a value for token provisioning. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.

Field	Description
dynamicData	(Conditional) Dynamic data. - Required for Digital Authentication Framework (DAF). - Required if the <code>delegatedAuthenticationRequested</code> field is <code>true</code>. Format: A <code>dynamicData</code> structure.
sigDeviceDynamicData	(Conditional) The content of this field is a JSON Web Signature (JWS). Required for Digital Authentication Framework (DAF). - The header of the JWS has these elements: <code>"alg": "..."</code>—Algorithm used for signature <code>"kid": "1234"</code>—Token requestors's client device ID <code>"typ": "JOSE"</code> <code>"cty": "application/json"</code>—Content type of the content inside JWS <code>"iat": "147...38"</code>—Issued at the timestamp in UTC when the transaction was created and signed. - Construct the body as JSON consisting of these: <code>vProvisionedTokenID</code> - the nonce <code>clientDeviceID</code> - The Signature. When the signature is provided, use this algorithm: RSA-PSS with SHA-256 as the hash and the MGF. Set <code>"algo"</code> to <code>"PS256"</code>
encDeviceRiskData	(Optional) JWE/JWS. For more information, see "JSON Web Encryption and Object Signing." To be used to convey data related to risk decisions. Actual risk data elements will be shared separately outside of the technical specification. <code>riskData</code> will be encrypted using Visa Public Key and signed using Device Key. Format: Alphanumeric, maximum size: 10240 (Maximum number of Name/Value pairs allowed — 100.)

Field	Description
<code>encRiskData</code>	(Optional) JWE that contains data related to the risk decision; see "Risk Data Attributes" in the Digital Authentication Framework chapter for risk data attributes to encrypt. Maximum number of Name/Value pairs allowed — 100. Format: JSON Web Encrypted string using the shared secret; max size 10240. Note: The JWE header should include the <code>iat</code> timestamp to specify when the transaction was created: <pre>"iat": "1422779638"</pre> For more information, see "JSON Web Encryption and Object Signing."
<code>clientAppID</code>	(Required) Unique identifier for the client application. It is obtained from the wallet provider or is the API key of the token service provider (TR-TSP) making the call on behalf of a token requestor. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
<code>networkAuthenticationRequested</code>	(Conditional) Request a network-authenticated transaction. • Required to be <code>true</code> for Digital Authentication Framework (DAF). • For a Token Requester in India, <code>true</code> enables requesting a Network Authenticated transaction. Device Binding must be present, the transaction is a domestic India transaction, and the Token Requester is enrolled. Format: Boolean; default is <code>false</code>

Payment Request

Contains transaction information about the payment request.

Field	Description
<code>transactionType</code>	(Required) Transaction type. Format: It is one of the following values: • <code>ECOM</code> - e-commerce transaction type. To be used with web flows. • <code>RECURRING</code> - Recurring payment. • <code>POS</code> - Point of Sale transaction type for the <code>QR_CONSUMER_CLOUD</code> presentation type. • <code>AFT</code> - Account Fund Transfer

Risk Data

Field	Description
transactionInitiationType	(Optional) Format: It is one of the following values: • CONSUMER • MERCHANT
acquirerMerchantID	(Optional) Contains the acquirer assigned merchant identifier. Format: Alphabetic, numeric; maximum 36 characters.

Three DS Data

Field	Description
cryptogramData	(Required) CAVV and ECI. Format: A cryptogramData structure.

Cryptogram Data

Field	Description
cryptogram	(Required) CAVV-equivalent cryptogram. Format: Alphabetic, numeric; maximum 28 characters.
eci	(Required) EC indicator for liability shift. Format: Alphabetic, numeric; maximum 2 characters.

Dynamic Data

Field	Description
<code>authenticationAmount</code>	(Conditional) The authorization amount for the transaction. You must truncate minor digits according to Visa spec, based on the ISO spec. For example, for a dollar amount of \$456.37, set the value to 456. • Required for Digital Authentication Framework (DAF). • Required if the <code>delegatedAuthenticationRequested</code> field is <code>true</code>. Format: String; maximum 7 decimal digits.
<code>currencyCode</code>	(Conditional) This is an ISO 4217 currency code. • Required for Digital Authentication Framework (DAF). • Required if the <code>delegatedAuthenticationRequested</code> field is <code>true</code>. Format: String; 3 decimal digits.
<code>merchantIdentifier</code>	(Conditional) The Visa merchant identifier. • Required for Digital Authentication Framework (DAF). • Required if the <code>delegatedAuthenticationRequested</code> field is <code>true</code>. Format: A <code>merchantIdentifier</code> structure.
<code>authenticationFactorA</code>	(Conditional) This is the first authentication factor used to authenticate the user. • Required for Digital Authentication Framework (DAF); for more information, see "DAF Authentication Factor Requirements" in the Digital Authentication Framework chapter. • Required if the <code>delegatedAuthenticationRequested</code> field is <code>true</code>; for more information, see "Authentication Methods" below. Format: String; valid Consumer Device Cardholder Verification Method (CDCVM)

Field	Description
<code>authenticationFactorB</code>	(<i>Conditional</i>) This is the first authentication factor used to authenticate the user. - Required for Digital Authentication Framework (DAF); for more information, see "DAF Authentication Factor Requirements" in the Digital Authentication Framework chapter. - Required if the <code>delegatedAuthenticationRequested</code> field is <code>true</code>; for more information, see "Authentication Methods" below. Format: String; valid Consumer Device Cardholder Verification Method (CDCVM)
<code>delegatedAuthenticationRequested</code>	(<i>Optional</i>) This field indicates whether delegated authority is requested. The default is <code>false</code>. Format: It is one of the following values: <code>true</code> <code>false</code>

Merchant Identifier

For the `merchantIdentifier` structure, only the `aid` and `mid` values need be provided together. When `delegatedAuthenticationRequested` is `true`, you must provide at least one of `externalClientId`, `applicationUrl`, or `merchantName`; you can provide any combination of them.

Value	Description
<code>externalClientId</code>	(<i>Conditional</i>) The Merchant ID assigned by the client. Format: Base64URL-encoded; maximum size 96.
<code>aid</code>	(<i>Conditional</i>) The existing acquirer ID sent in combination with the merchant ID. If the <code>aid</code> is provided, the <code>mid</code> must also be provided. The two values in combination identify the merchant. Format: Alphanumeric; maximum size 36.
<code>mid</code>	(<i>Conditional</i>) The merchant ID sent in combination with the acquirer ID. If the <code>mid</code> is provided, the <code>aid</code> must also be provided. The two values in combination identify the merchant. Format: Alphanumeric; maximum size 36.

Value	Description
<code>applicationUrl</code>	(Conditional) The merchant's URL, Application Package ID, or Application Bundle ID. Required for Digital Authentication Framework (DAF) when the token type is HCE and for tokens with the <code>AUTO_FILL</code> presentation type option. Format: Base64URL-encoded string; maximum size 512.
<code>merchantName</code>	(Conditional) The merchant's name. Required for tokens with the <code>AUTO_FILL</code> presentation type option. Base64 encoded UTF8. Format: Base64URL-encoded string; maximum size 2048.

Authentication Methods

These are the supported Consumer Device Cardholder Verification Methods (CDCVM).

Important:

To use Digital Authentication Framework (DAF), you must specify authentication methods from those listed in the DAF Authentication Factor Requirements; see the Digital Authentication Framework chapter for more information.

Authentication Method	Description
00	No authentication method required.
01	Username/password
02	Passcode or password
Consumer Device Cardholder Verification Method (CDCVM)	
10	Passcode
11	Password
12	Pattern
13	Biometric fingerprint
14	Biometric facial recognition
15	Biometric iris recognition
16	Biometric voice recognition
17	Behavioral biometric
One Time Passcode (OTP)	
18	Device unlocked (CDCVM unknown)
30	SMS

Authentication Method	Description
31	Email
32	Hardware token without user verification
33	Hardware token with user verification
34	Soft token
35	Any other method
40	Knowledge Based Authentication (KBA)
41	Out of Band (OOB) Authentication
42	Local Authentication
FIDO	
50	FIDO bound token: FIDO device bound token was provided by the Token Requestor with user presence only – no user verification. Establishes possession.
51	FIDO bound token: FIDO device bound token was provided by the Token Requestor with biometric user verification. Establishes inherence and possession.
52	FIDO bound token: FIDO device bound token was provided by the Token Requestor with user verification via passcode, password, or pattern. Establishes knowledge and possession.
Other	
60	SE based token: Cryptogram generated from a SE device for a device bound token was provided. Establishes possession.
61	Device bound token: Device bound token (token reference) was provided by Token Requestor along with proof of device used for binding token. Establishes possession.

Get Payment Data (Token & Cryptogram)—Given Token ID Response

HTTP Response

HTTP Response	Description
201	Created

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
<code>vPaymentDataID</code>	Unique identifier for transaction set by VTS for the cryptogram that is generated. Format: Alphabetic, numeric; maximum 36 characters.
<code>encryptionMetaData</code>	Cryptographic payload required for end-to-end encryption of sensitive fields. Format: Alphabetic, numeric; between 1-512 characters.
<code>paymentInstrument</code>	Format: A <code>paymentInstrument</code> structure.
<code>tokenInfo</code>	Format: A <code>tokenInfo</code> structure.
<code>cryptogramInfo</code>	Format: A <code>cryptogramInfo</code> structure.
<code>qrInfo</code>	Object to hold the encrypted QR Code. This object will be returned for POS transaction type and tokens supporting the <code>QR_CONSUMER_CLOUD</code> presentation type. Format: A <code>qrInfo</code> structure.
<code>authenticationFlags</code>	Authentication flag status. Format: An <code>authenticationFlagStatus</code> array.

Authentication Flag Status

Field	Description
name	The authentication flag status name. Format: It is one of the following values: • <code>delegatedAuthority</code> • <code>DAF</code>
value	The edited value assigned by VTS. Format: It is one of the following values: • <code>true</code> • <code>false</code>
reasonCode	Reason code. Format: It is one of the following values: • 00 - The token requestor received the delegated authority (the <code>value</code> field is <code>true</code> for the associated <code>name</code> field containing <code>delegatedAuthority</code>) • The token requestor did not receive the delegated authority (the <code>value</code> field is <code>false</code> for the associated <code>name</code> field containing <code>delegatedAuthority</code>) for one of the following reasons: - 00 - The delegated authority is not qualified for this transaction; perhaps because the token requestor did not provide all required fields - 02 - The issuer is not participating in the delegated authority - 03 - The token requestor is not enabled for delegated authority

Payment Instrument

Field	Description
last4	Last 4 digit of the PAN. Format: Alphabetic, numeric; maximum 4 characters.
paymentType	The payment brand type. Format: A <code>paymentType</code> structure.
paymentAccountReference	(Optional) Payment Account Reference (PAR) if available for this payment instrument. Format: Alphanumeric; maximum 29 characters.

Payment Type

Field	Description
cardBrand	The brand associated with the card. Format: It is one of the following values: • VISA — Visa

Token Info

Field	Description
last4	Last 4 digits of the token. Format: Numeric; 4 digits.
expirationDate	Token expiration date in UTC format. Format: An <code>expirationDate</code> structure.
encTokenInfo	Encrypted token payload. For more information, see "JSON Web Encryption and Object Signing." This field is not returned for <code>NFC-SE</code> presentation types. Note: The unencrypted field contains a token as a 13-16 character string. Format: Alphabetic, numeric; maximum 4096 characters.

Expiration Date

Field	Description
month	The month upon which the payment instrument is set to expire. Example: "month": "9" Format: String; numeric; max length 2 digits.
year	The year upon which the payment instrument is set to expire. Example: "year": "2024" Format: String; numeric; length 4 digits.

Cryptogram Info

Field	Description
cryptogram	Base64-encoded cryptogram. This object is optional for POS transaction type and tokens supporting the QR_CONSUMER_CLOUD presentation type. Not present when <code>cryptogramType</code> is CTAVV or DTVV. Format: Alphabetic, numeric; maximum 28 characters.
eci	EC indicator for liability shift. Format: Alphabetic, numeric; maximum 2 characters.
atc	The ATC counter used as part of the input to the TAVV cryptogram. Format: String; maximum 5 digits.
dtvvCryptogram	DTVV value present in the response when the DTVV cryptogram type is requested. Format: Alphabetic, numeric; maximum 3 characters.
ctavvCryptogram	Base64 encoded CTAVV cryptogram. It is present in the response when the CTAVV cryptogram type is requested. Format: Base64 encoded; maximum 28 characters.
cryptogramExpirationDate	Cryptogram Expiration Date. When available, the date/time is always in GMT, and milliseconds are ignored. Format: CCYY-MM-DDThh:mm:ss.000Z; UTC Time and Date combination format (ISO 8601).

QR Info

Field	Description
encQrDataInfo	(Required) JWE encrypted QR Code object. For more information, see "JSON Web Encryption and Object Signing." Format: Alphabetic, numeric; maximum 4096 characters.

Unencrypted QR Code Info

Field	Description
qrData	(Required) Base64 encoded QR Code, as specified in "Visa QR Code Payment Specification", version 1.02 document.

Unencrypted encQrDataInfo

```
{  
  "qrData": "YQ1PB6AA...."  
}
```

Get Payment Data (Token & Cryptogram)—Given Token ID Errors

Error Code	Reason Code	Message
403	declined	This card is considered ineligible for tokenization at this moment.
409	notAllowed	Further operations for this card are no longer allowed. Contact your bank to resolve this issue.

This API also returns Standard Errors.

Get Payment Data (Token & Cryptogram)—Given Token ID Example

CTAVV Cryptogram Type

Request

Selected Headers:

```
"x-request-id": "sXX...Lw"  
"Accept": "application/json"  
"x-pay-token": "xv2...9c"
```

Endpoint:

```
https://cert.api.visa.com/vts/v2/provisionedTokens/  
{vProvisionedTokenID}/paymentData?apikey=key
```


Request Body

```
{
  "cryptogramType": "CTAVV",
  "clientPaymentDataID": "3242342",
  "threeDsData": {
    "cryptogramData": {
      "eci": "05",
      "cryptogram": "B5g...Q="
    }
  },
  "paymentRequest": {
    "transactionType": "ECOM"
  }
}
```

Response

Selected Headers:

```
"x-response-id": "sXX...Lw"
```

Response Body:

```
{
  "vPaymentDataID": "8d7..01",
  "paymentInstrument": {
    "last4": "1234",
    "paymentType": {
      "cardBrand": "VISA"
    }
  },
  "paymentAccountReference": "V00...44",
  "tokenInfo": {
    "encTokenInfo": "eyJ...Sw",
    "last4": "1234",
    "expirationDate": {
      "month": "12",
      "year": "2023"
    }
  },
  "cryptogramInfo": {
    "ctavvCryptogram": "ABE...4=",
    "eci": "05",
    "cryptogramExpirationDate": "2018-09-27T17:06:09.000Z"
  }
}
```

No Cryptogram Type

Request

```
{
  "encryptionMetaData": " base 64 encoded..",
  "clientPaymentDataID": "3242342",
  "paymentRequest": {
    "transactionType": "ECOM"
  },
  "atc": "12345"
}
```

Response

```
{
  "vPaymentDataID": "502...02",
  "paymentInstrument": {
    "last4": "0826",
    "paymentType": {"cardBrand": "VISA"},
    "paymentAccountReference": "V00...49"
  },
  "tokenInfo": {
    "encTokenInfo": "eyJ...AA",
    "last4": "9994",
    "expirationDate": {
      "month": "12",
      "year": "2022"
    }
  },
  "cryptogramInfo": {
    "cryptogram": "AgA...A=",
    "eci": "07"
  }
}
```

DTVV Cryptogram Type

Request

```
{
  "clientPaymentDataID": "3242342",
  "cryptogramType": "DTVV",
  "paymentRequest": {
    "transactionType": "ECOM"
  },
  "otpValue": "12345678"
}
```

Response

```
{
  "vPaymentDataID": "9d8...02",
  "paymentInstrument": {
    "last4": "0826",
    "paymentType": {"cardBrand": "VISA"},
    "paymentAccountReference": "V00...49"
  },
  "tokenInfo": {
    "encTokenInfo": "eyJ...pw",
    "last4": "9994",
    "expirationDate": {
      "month": "12",
      "year": "2022"
    }
  },
  "cryptogramInfo": {
    "dtvvCryptogram": "776",
    "eci": "07",
    "cryptogramExpirationDate": "2018-12-05T19:45:36.000Z"
  }
}
```

Get Payment Data Examples by Token Requestor

Request

Selected Headers:

```
x-request-id: alu...Go
x-pay-token: xv2:1567634919:8fd...ea
Accept: application/json
```

Endpoint:

```
https://cert.digital.visa.com/vts/v2/provisionedTokens/0d7...02/
paymentData?
apiKey=VZM...gw
```

TAVV Cryptogram Type Request Body:

```
{
  "cryptogramType": "TAVV",
  "clientPaymentDataID": "3242342",
  "paymentRequest": {
    "transactionType": "ECOM"
  }
}
```

```
}  
}
```

No Cryptogram Type Request:

```
{  
  "encryptionMetaData": " base 64 encoded..",  
  "clientPaymentDataID": "3242342",  
  "paymentRequest": {  
    "transactionType": "ECOM"  
  },  
  "atc": "12345",  
}
```

CTAVV Cryptogram Type Request:

```
{  
  "encryptionMetaData": " base 64 encoded..",  
  "clientPaymentDataID": "3242342",  
  "paymentRequest": {  
    "transactionType": "ECOM"  
  },  
  "atc": "12345",  
  "cryptogramType": "CTAVV",  
  "threeDsData": {  
    cryptogramData: {  
      "cryptogram": "SaD...pZ",  
      "eci": "05"  
    }  
  }  
}
```

DTVV Cryptogram Type Request:

```
{  
  "encryptionMetaData": " base 64 encoded..",  
  "clientPaymentDataID": "3242342",  
  "paymentRequest": {  
    "transactionType": "ECOM"  
  },  
  "atc": "12345",  
  "cryptogramType": "DTVV",  
}
```

Merchant Identifier Specified Request:

```
{  
  "clientPaymentDataID": "12345678",  
  "sigDeviceDynamicData": "eyJ...Ug",  
  "paymentRequest": {  
    "transactionType": "ECOM"  
  },  
  "dynamicData": {  
    "authenticationAmount": "7894564",  
    "authenticationFactorA": "10",  
    "authenticationFactorB": "61",  
  }  
}
```

```
"currencyCode": "840",
"merchantIdentifier": {
  "applicationUrl": "V1dXL1BTRDJAR1BELkNPTQ",
  "externalClientId": "Vm9...bg",
  "merchantName": "Q09NLkdPT0dMRS5VQkVSL1BBWU1FT1Q"
}
}
```

Response

Selected Headers:

```
x-response-id: alu...Go
```

TAVV Cryptogram Type Response Body:

```
{
  "vPaymentDataID": "58f...02",
  "cryptogramInfo": {
    "cryptogram": "AgA...A=",
    "eci": "07",
    "atc": "6"
  },
  "paymentInstrument": {
    "last4": "0818",
    "paymentType": {
      "cardBrand": "VISA"
    },
    "paymentAccountReference": "V00...62"
  },
  "tokenInfo": {
    "encTokenInfo": "eyJ...pg",
    "last4": "1767",
    "expirationDate": {
      "month": "12",
      "year": "2022"
    }
  }
}
```

Unencrypted encTokenInfo

```
"token": "409...59"
```

Authentication Flags Response Example

Response

```
{
  "vPaymentDataID": "1ff...01",
  "cryptogramInfo": {
    "cryptogram": "YwA...QA=",
    "eci": "07",
    "atc": "33"
  },
  "authenticationFlags": [
    {
      "name": "delegatedAuthority",
      "value": false,
      "reasonCode": "03"
    }
  ],
  "paymentInstrument": {
    "last4": "...",
    "paymentType": {
      "cardBrand": "VISA"
    }
  },
  "tokenInfo": {
    "encTokenInfo": "eyJ...kw",
    "last4": "...",
    "expirationDate": {
      "month": "...",
      "year": "..."
    }
  }
}
```

Get Payment Data Examples by Token Service Provider (TR-TSP)

Request

Selected Headers:

```
"x-request-id": "FbR...dB"
"Accept": "application/json",
"x-pay-token": "xv2:1592422058:240...aa"
```

Endpoint:

```
https://cert.visa.com/vts/v2/provisionedTokens/554...01/paymentData?apikey=WSV...bI&relationshipID=Trt...e1
```

TAVV Cryptogram Type Request Body:

```
{
  "cryptogramType": "TAVV",
  "clientPaymentDataID": "495...53",
  "clientAppID": "TR-TSP_key",
  "paymentRequest": {
    "transactionType": "ECOM"
  }
}
```

TAVV Cryptogram Type Request Body:

```
{
  "dynamicData": {
    "authenticationAmount": "7654321",
    "merchantIdentifier": {
      "applicationUrl": "aHR0cDovL3d3dy5WaXZMYWJzLmNvbQ==",
      "externalClientId": "N29...==",
      "merchantName": "Vml...=="
    },
    "delegatedAuthenticationRequested": true,
    "authenticationFactorA": "31",
    "currencyCode": "840",
    "authenticationFactorB": "32"
  },
  "cryptogramType": "TAVV",
  "clientPaymentDataID": "434...46",
  "riskData": {
    "acquirerMerchantID": "PSD...e1",
    "transactionInitiationType": "MERCHANT"
  },
  "clientAppID": "TR-TSP_key",
  "paymentRequest": {
    "transactionType": "ECOM"
  }
}
```

Network Authentication Request Examples

ECOM Request

```
{
  "paymentRequest": {
    "transactionType": "ECOM"
  },
}
```

```
"clientPaymentDataID": "3242342",
"cryptogramType": "TAVV",
"clientReferenceId": "testClientRefId111",
"otpValue": "12345",
"dynamicData": {
  "authenticationAmount": "1000.50",
  "currencyCode": "356"
},
"networkAuthenticationRequested": true
}
```

InApp Request

```
{
  "encryptionMetaData": "R4e...GN",
  "paymentRequest": {
    "transactionType": "ECOM"
  },
  "atc": "123",
  "clientPaymentDataID": "3242342",
  "cryptogramType": "TAVV",
  "clientReferenceId": "testClientRefId111",
  "otpValue": "12345",
  "dynamicData": {
    "authenticationAmount": "1000.50",
    "currencyCode": "356"
  },
  "networkAuthenticationRequested": true
}
```


Chapter 16

Get Payment Data for Autofill

Get Payment Data for Autofill Introduction

Given token or tokenID, this API returns the token data and cryptogram. It is only applicable for Ecommerce.

Get Payment Data for Autofill Request

Endpoints

Resource Path

```
vts/v2/provisionedTokens/{vProvisionedTokenID}/paymentData/autofill
```

Sandbox Path

```
https://cert.api.visa.com/vts/v2/provisionedTokens/{vProvisionedTokenID}/paymentData/autofill?apiKey=key&relationshipID=id
```

Live Path

```
https://api.visa.com/vts/v2/provisionedTokens/{vProvisionedTokenID}/paymentData/autofill?apiKey=key&relationshipID=id
```

URL Parameters

Parameter	Description
{vProvisionedTokenID}	<i>(Required)</i> Unique ID for the provision being confirmed. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.

Methods

POST

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request.

Header	Description
	Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	(Required) A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients. Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query

Header	Description
	parameters, such as <code>apiKey=key</code> , and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
<code>apiKey</code>	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
<code>relationshipID</code>	(Optional) The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

Field	Description
<code>clientPaymentDataID</code>	(Required) Reference identifier defined by client. No uniqueness required. Source: Wallet Provider Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters
<code>paymentRequest</code>	(Required) Required for transaction information about the payment request. Format: A <code>paymentRequest</code> structure.
<code>cryptogramType</code>	(Optional) Cryptogram Type. Format: It is one of the following values: • <code>DTVV</code>– Generate <code>DTVV</code>. (default)

Field	Description
dynamicData	(Conditional) Dynamic data. Required for tokens with the AUTO_FILL presentation type. Format: A dynamicData object.
sigDeviceDynamicData	(Conditional) The content of this field is a JSON Web Signature (JWS). Required for Digital Authentication Framework (DAF). - The header of the JWS has these elements: "alg": "..."—Algorithm used for signature "kid": "1234"—Token requestors's client device ID "typ": "JOSE" "cty": "application/json"—Content type of the content inside JWS "iat": "147...38"—Issued at the timestamp in UTC when the transaction was created and signed. - Construct the body as JSON consisting of these: - vProvisionedTokenID - the nonce - clientDeviceID - The Signature. When the signature is provided, use this algorithm: RSA-PSS with SHA-256 as the hash and the MGF. Set "algo" to "PS256"
clientAppID	(Conditional) Unique identifier for the client application. Required when the query parameter contains a relationship ID. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
deviceBindingInfoShort	(Conditional) Device binding information. Required for autofill when device binding is performed without signing. Format: A deviceBindingInfoShort structure.

Payment Request

Contains transaction information about the payment request.

Field	Description
transactionType	(Required) Transaction type. Format: It is one of the following values: • ECOM - e-commerce transaction type. To be used with web flows. • RECURRING - Recurring payment. • POS - Point of Sale transaction type for the QR_CONSUMER_CLOUD presentation type. • AFT - Account Fund Transfer

Dynamic Data

Field	Description
authenticationAmount	(Conditional) The authorization amount for the transaction. You must truncate minor digits according to Visa spec, based on the ISO spec. For example, for a dollar amount of \$456.37, set the value to 456. • Required for Digital Authentication Framework (DAF). • Required if the delegatedAuthenticationRequested field is true. Format: String; maximum 7 decimal digits.
currencyCode	(Conditional) This is an ISO 4217 currency code. • Required for Digital Authentication Framework (DAF). • Required if the delegatedAuthenticationRequested field is true. Format: String; 3 decimal digits.
merchantIdentifier	(Conditional) The Visa merchant identifier. • Required for Digital Authentication Framework (DAF). • Required if the delegatedAuthenticationRequested field is true. Format: A merchantIdentifier structure.

Field	Description
<code>authenticationFactorA</code>	(Conditional) This is the first authentication factor used to authenticate the user. • Required for Digital Authentication Framework (DAF); for more information, see "DAF Authentication Factor Requirements" in the Digital Authentication Framework chapter. • Required if the <code>delegatedAuthenticationRequested</code> field is <code>true</code>; for more information, see "Authentication Methods" below. Format: String; valid Consumer Device Cardholder Verification Method (CDCVM)
<code>authenticationFactorB</code>	(Conditional) This is the first authentication factor used to authenticate the user. • Required for Digital Authentication Framework (DAF); for more information, see "DAF Authentication Factor Requirements" in the Digital Authentication Framework chapter. • Required if the <code>delegatedAuthenticationRequested</code> field is <code>true</code>; for more information, see "Authentication Methods" below. Format: String; valid Consumer Device Cardholder Verification Method (CDCVM)
<code>delegatedAuthenticationRequested</code>	(Optional) This field indicates whether delegated authority is requested. The default is <code>false</code>. Format: It is one of the following values: • <code>true</code> • <code>false</code>

Merchant Identifier

Value	Description
<code>applicationUrl</code>	(Required) The merchant's URL, Application Package ID, or Application Bundle ID. Format: Base64URL-encoded string; maximum size 512.

Authentication Methods

These are the supported Consumer Device Cardholder Verification Methods (CDCVM).

Important:

To use Digital Authentication Framework (DAF), you must specify authentication methods from those listed in the DAF Authentication Factor Requirements; see the Digital Authentication Framework chapter for more information.

Authentication Method	Description
00	No authentication method required.
01	Username/password
02	Passcode or password
Consumer Device Cardholder Verification Method (CDCVM)	
10	Passcode
11	Password
12	Pattern
13	Biometric fingerprint
14	Biometric facial recognition
15	Biometric iris recognition
16	Biometric voice recognition
17	Behavioral biometric
One Time Passcode (OTP)	
18	Device unlocked (CDCVM unknown)
30	SMS
31	Email
32	Hardware token without user verification
33	Hardware token with user verification
34	Soft token
35	Any other method
40	Knowledge Based Authentication (KBA)
41	Out of Band (OOB) Authentication
42	Local Authentication
FIDO	
50	FIDO bound token: FIDO device bound token was provided by the Token Requestor with user presence only – no user verification. Establishes possession.

Authentication Method	Description
51	FIDO bound token: FIDO device bound token was provided by the Token Requestor with biometric user verification. Establishes inheritance and possession.
52	FIDO bound token: FIDO device bound token was provided by the Token Requestor with user verification via passcode, password, or pattern. Establishes knowledge and possession.
Other	
60	SE based token: Cryptogram generated from a SE device for a device bound token was provided. Establishes possession.
61	Device bound token: Device bound token (token reference) was provided by Token Requestor along with proof of device used for binding token. Establishes possession.

Device Binding Short Info

Field	Description
vProvisionedTokenID	(Required) Unique ID for the provisioned token for a token bound to the same device for autofill when device binding is performed without request signing. Format:String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.
clientDeviceID	(Required) Device ID of the provisioned token of the device for which the token is bound to the same device for autofill when device binding is performed without request signing. Format:String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.

Get Payment Data for Autofill Response

HTTP Response

HTTP Response	Description
201	Created

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
<code>vPaymentDataID</code>	Unique identifier for transaction set by VTS for the cryptogram that is generated. Format: Alphabetic, numeric; maximum 36 characters.
<code>encryptionMetaData</code>	Cryptographic payload required for end-to-end encryption of sensitive fields. Format: Alphabetic, numeric; between 1-512 characters.
<code>paymentInstrument</code>	Format: A <code>paymentInstrument</code> structure.
<code>tokenInfo</code>	Format: A <code>tokenInfo</code> structure.
<code>cryptogramInfo</code>	Format: A <code>cryptogramInfo</code> structure.
<code>qrInfo</code>	Object to hold the encrypted QR Code. This object will be returned for POS transaction type and tokens supporting the <code>QR_CONSUMER_CLOUD</code> presentation type. Format: A <code>qrInfo</code> structure.
<code>authenticationFlags</code>	Authentication flag status. Format: An <code>authenticationFlagStatus</code> array.

Authentication Flag Status

Field	Description
name	The authentication flag status name. Format: It is one of the following values: • <code>delegatedAuthority</code> • <code>DAF</code>
value	The edited value assigned by VTS. Format: It is one of the following values: • <code>true</code> • <code>false</code>
reasonCode	Reason code. Format: It is one of the following values: • 00 - The token requestor received the delegated authority (the <code>value</code> field is <code>true</code> for the associated <code>name</code> field containing <code>delegatedAuthority</code>) • The token requestor did not receive the delegated authority (the <code>value</code> field is <code>false</code> for the associated <code>name</code> field containing <code>delegatedAuthority</code>) for one of the following reasons: - 00 - The delegated authority is not qualified for this transaction; perhaps because the token requestor did not provide all required fields - 02 - The issuer is not participating in the delegated authority - 03 - The token requestor is not enabled for delegated authority

Payment Instrument

Field	Description
last4	Last 4 digit of the PAN. Format: Alphabetic, numeric; maximum 4 characters.
paymentType	The payment brand type. Format: A <code>paymentType</code> structure.
paymentAccountReference	(Optional) Payment Account Reference (PAR) if available for this payment instrument. Format: Alphanumeric; maximum 29 characters.

Payment Type

Field	Description
cardBrand	The brand associated with the card. Format: It is one of the following values: • VISA — Visa

Token Info

Field	Description
last4	Last 4 digits of the token. Format: Numeric; 4 digits.
expirationDate	Token expiration date in UTC format. Format: An <code>expirationDate</code> structure.
encTokenInfo	Encrypted token payload. For more information, see "JSON Web Encryption and Object Signing." This field is not returned for <code>NFC-SE</code> presentation types. Note: The unencrypted field contains a token as a 13-16 character string. Format: Alphabetic, numeric; maximum 4096 characters.

Expiration Date

Field	Description
month	The month upon which the payment instrument is set to expire. Example: "month": "9" Format: String; numeric; max length 2 digits.
year	The year upon which the payment instrument is set to expire. Example: "year": "2024" Format: String; numeric; length 4 digits.

Cryptogram Info

Field	Description
cryptogram	Base64-encoded cryptogram. This object is optional for POS transaction type and tokens supporting the QR_CONSUMER_CLOUD presentation type. Not present when <code>cryptogramType</code> is CTAVV or DTVV. Format: Alphabetic, numeric; maximum 28 characters.
eci	EC indicator for liability shift. Format: Alphabetic, numeric; maximum 2 characters.
atc	The ATC counter used as part of the input to the TAVV cryptogram. Format: String; maximum 5 digits.
dtvvCryptogram	DTVV value present in the response when the DTVV cryptogram type is requested. Format: Alphabetic, numeric; maximum 3 characters.
ctavvCryptogram	Base64 encoded CTAVV cryptogram. It is present in the response when the CTAVV cryptogram type is requested. Format: Base64 encoded; maximum 28 characters.
cryptogramExpirationDate	Cryptogram Expiration Date. When available, the date/time is always in GMT, and milliseconds are ignored. Format: CCYY-MM-DDThh:mm:ss.000Z; UTC Time and Date combination format (ISO 8601).

QR Info

Field	Description
encQrDataInfo	(Required) JWE encrypted QR Code object. For more information, see "JSON Web Encryption and Object Signing." Format: Alphabetic, numeric; maximum 4096 characters.

Unencrypted QR Code Info

Field	Description
qrData	(Required) Base64 encoded QR Code, as specified in "Visa QR Code Payment Specification", version 1.02 document.

Unencrypted encQrDataInfo

```
{  
  "qrData": "YQlPB6AA...."  
}
```

Get Payment Data for Autofill Errors

Error Code	Reason Code	Message
403	declined	This card is considered ineligible for tokenization at this moment.
403	deviceNotBound	Device binding is required for this token prior to a Get Payment Data request.
403	nonParticipatingMerchant	This merchant does not accept tokens via auto-fill. Ensure token credentials are not populated at this merchant.
409	notAllowed	Further operations for this card are no longer allowed. Contact your bank to resolve this issue.

This API also returns Standard Errors.

Get Payment Data for Autofill Example

Request

Selected Headers:

```
x-request-id=SaC...GA  
Accept=application/json
```

Endpoint:

```
POST https://cert.api.visa.com/vts/v2/provisionedTokens/  
{vProvisionedTokenID}/paymentData/autofill?apikey=key
```

Request Body

```
{
  "deviceBindingInfoShort": {
    "clientDeviceID": "lN7XoJ",
    "vProvisionedTokenID": "be3...01"
  },
  "dynamicData": {
    "merchantIdentifier": {
      "applicationUrl": "d3d...=="
    }
  },
  "sigDeviceDynamicData": "eyJ...3Dw",
  "cryptogramType": "DTVV",
  "clientPaymentDataID": "8f3...35",
  "paymentRequest": {
    "transactionType": "ECOM"
  }
}
```

Response

Selected Headers:

```
x-response-id=SaC...GA
```

Response Body:

```
{
  "vPaymentDataID": "96ef37e9215eee9161931ebac1edcf01",
  "cryptogramInfo": {
    "dtvvCryptogram": "692",
    "eci": "07",
    "cryptogramExpirationDate": "2022-04-11T18:20:29.000Z"
  },
  "paymentInstrument": {
    "last4": "6928",
    "paymentType": {
      "cardBrand": "VISA"
    }
  },
  "paymentAccountReference": "V00...86"
},
{
  "tokenInfo": {
    "encTokenInfo": "eyJ...ew",
    "last4": "7548",
    "expirationDate": {
      "month": "04",
      "year": "2022"
    }
  }
}
}
```

Unencrypted encTokenInfo

```
"token": "..."
```


Chapter 17

Get Transaction History

Get Transaction History Introduction

Obtains the specified transaction history. The maximum time range is 30 days, and the maximum transaction record count is 10. Transaction history is tied to a provisioned token; a wallet provider will need to manage association of the history with a consumer card in their system. The scope of the transaction history (this token only, or all transactions associated with the PAN) is determined by the combination of issuer and token requestor transaction history configurations.

Note:

PAN transactions will only be returned if both the issuer and the wallet provider are configured for PAN-level transaction history details.

The transaction history response is encrypted and should be decrypted on the device during user interaction.

Get Transaction History Request

Endpoints

Resource Path

vts/paymentTxns

Sandbox Path

```
https://cert.api.visa.com/vts/paymentTxns?apiKey=key&clientAppID=id  
&count=count&encryptionMetaData=meta_data&lastUpdatedTimestamp=timestamp  
&relationshipID=id&vProvisionedTokenID=id
```

Live Path

```
https://api.visa.com/vts/paymentTxns?apiKey=key&clientAppID=id  
&count=count&encryptionMetaData=meta_data&lastUpdatedTimestamp=timestamp  
&relationshipID=id&vProvisionedTokenID=id
```

Method

GET

Required Headers

Header	Description
x-request-id	(Required) Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	(Required) A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as: <code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: Alphabetic, numeric; maximum 64 characters.
clientAppID	(Conditional) Unique identifier for the client application; required when the <code>relationshipID</code> query parameter is specified. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
count	(Optional) Number of records to retrieve. Maximum is 10. If not specified, the maximum number of records will be returned, up to 10, inclusive. Format: Integer.
encryptionMetaData	(Required) Cryptographic payload required for end-to-end encryption of sensitive fields. To be used in messages participating in end-to-end encryption scheme. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 512 characters.
lastUpdatedTimestamp	(Optional) Last Updated Date Time as a UNIX Epoch timestamp. The value is in seconds. Maximum time range is 30 days. The maximum time range will be used if this parameter is not specified. Format: Numeric.
relationshipID	(Optional) The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.
vProvisionedTokenID	(Required) Unique ID for token. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.

Request Body

None. The request does not have a body.

Get Transaction History Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
<code>transactionScope</code>	Scope for the transaction detail; based on the combination of issuer and token requestor transaction history configurations. Format: It is one of the following values: • <code>TOKEN</code> - All of the reported transactions are for this token only. • <code>PAN</code> - The list of transactions can include transactions performed with the PAN itself and all tokens associated with the PAN. For more information, the <code>usingThisToken</code> field.
<code>encryptionMetaData</code>	Cryptographic payload for end-to-end encryption of sensitive fields. It is required for the <code>Opacity_A</code> protocol. Format: Alphabetic, numeric; maximum 512 characters.
<code>transactionDetails</code>	Format: An array of <code>transactionDetails</code> structures.

Transaction Details

Field	Description
encTransactionInfo	Encrypted blob of transaction detail record. The transaction detail records that make up this encrypted blob are defined in the rows that follow. For more information, see "JSON Web Encryption and Object Signing." Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
vProvisionedTokenID	Unique ID for token. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.
transactionID	Transaction identifier Format: Alphabetic, numeric; maximum 32 characters.
transactionType	Format: It is one of the following values: • 0 - Purchase • 1 - Refund • 3 - Cash Withdrawal • 4 - ATM Services • 5 - ATM Balance Inquiry • 6 - AFT • 7 - OCT String. Size: 1.
transactionDate	Transaction Date. Note: For settled transactions (transactionStatus = 4), the date will be provided, but time is not applicable and will always be 00:00:00. Format: String; date and time in Zulu format, e.g. <i>YYYY-MM-DDTHH:MM:SSZ</i>.
currencyCode	ISO 4217 format. Format: Alphabetic, numeric; maximum 4 characters.
merchantName	Merchant Name. Format: Alphabetic, numeric; maximum 32 characters.
amount	Amount. Format: Alphabetic, numeric; maximum 16 characters.

Field	Description
transactionStatus	Transaction status. Format: It is one of the following values: • 1 - Approved • 2 - Reversed • 3 - Declined • 4 - Settled
industryCatgCode	Industry category code. Format: Numeric.; maximum 4 digits.
industryCatgName	Industry category like Restaurant, Airline etc. Format: Alphabetic, numeric; maximum 32 characters.
industryCode	MCC Merchant code. Format: Numeric.; maximum 4 digits.
industryName	Industry name. Format: Alphanumeric; maximum 32 characters.
merchantZipCode	The postal code where the transaction was made. Format: Alphabetic, numeric; 3–16 characters.
merchantCity	The city where the transaction was made. Format: Alphabetic, numeric; maximum 100 characters.
usingThisToken	Returns whether transaction was made using the token referenced by the vProvisionedTokenID field in this API request. Format: It is one of the following values: • true — The transaction was made using the token. • false — The transaction was not made using the token.
atc	Application Transaction Counter (ATC) present if available for the transaction. Format: Alphabetic, numeric; maximum 5 characters.

Get Transaction History Errors

Error Code	Reason Code	Description
403	declined	This card is considered not eligible for tokenization at this moment. Try enroll/provision again for this PAN. If you encounter the same message even for the new enrollment/provision request then ask the customer to either add a different card or to reach out to their issuer.
409	notAllowed	The requested action is not allowed for a given token. Token status is invalid for this operation. Check the token details and perform the required/appropriate LCM action.

This API also returns Standard Errors.

Get Transaction History Examples

Get Transaction History Example

Request

Endpoint:

```
https://cert.api.visa.com/vts/  
paymentTxns?apikey=GPN...3I&vProvisionedTokenID=8ec...02
```

Request Body:

The request does not have a body.

Response

```
{"transactionDetails": [],  
 "transactionScope": "TOKEN"}
```

Opacity Get Transaction History Response

```
{  
  "transactionScope": "PAN",
```



```
"encryptionMetaData": "A6DF450A521F9F712DBAF. . .",  
"transactionDetails": [{  
  "encTransactionInfo": "eyJhbGciOiJBbmJU2R0NNS1ciL. . ."  
}]  
}
```

Chapter 18

Retrieve Transaction History

Retrieve Transaction History Introduction

Obtains the specified transaction history. The maximum time range is 30 days and maximum transaction record count is 10. A transaction history is tied to a provisioned token; a wallet provider will need to manage the association of these transaction details with the consumer PAN in their system. The transaction history response is encrypted and should be decrypted on the device during user interaction.

Retrieve Transaction History Request

Endpoints

Resource Path

vts/paymentTxns

Sandbox Path

```
https://cert.api.visa.com/vts/paymentTxns?apiKey=key  
&vProvisionedTokenID=id
```

Live Path

```
https://api.visa.com/vts/paymentTxns?apiKey=key  
&vProvisionedTokenID=id
```

Methods

POST

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	<i>(Required)</i> Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
relationshipID	<i>(Required)</i> The ID identifying the relationship between a token requester and the caller of this API, which is a token service provider (TR-TSP) with which the token requester has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.
vProvisionedTokenID	<i>(Required)</i> Unique ID for the provisioned token. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.

Request Body

Field	Description
count	<i>(Optional)</i> Number of records to retrieve. Maximum is 10. If not specified, the maximum number of records will be returned.
lastUpdatedTimestamp	<i>(Optional)</i> Last Updated Date Time. Maximum time range is 30 days. The maximum time range will be returned if this parameter is not specified.
channelSecurityContext	<i>(Conditional)</i> Present in case of Pan entry on the companion device when the companion device is a SE or HCE. Not Present in case of Provision with Pan Enroll ID. Device Signature and Encryption public certs signed but the wallet provider root private key.
clientAppID	<i>(Conditional)</i> Unique identifier for the client application. Required when the relationshipID query parameter is specified. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.

Channel Security Context

Field	Description
channelInfo	Security Channel used to communicate between the device and the Visa Token Service back-end. Format: A <code>channelInfo</code> structure.

<code>vtsCerts</code>	(Conditional) Visa certificate IDs. Format: An array of <code>vtsCerts</code> structures.
<code>deviceCerts</code>	(Conditional) Device certificates. Not required for autofill. Format: An array of <code>deviceCerts</code> structures.
<code>keyData</code>	(Conditional) Key data; required for OPACITY_A; otherwise, it is not used. In the enroll PAN and Provision with PAN APIs, <code>keyData</code> and <code>encryptionMetaData</code> must now be present together. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 512 characters.

Channel Info

Field	Description
<code>encryptionScheme</code>	(Conditional) The security scheme used to secure communication between the device and the Visa Token Service back-end. Note: For Secure Copy (SCP) encryption schemes, the encryption version and the type are concatenated to create a string in the form of <code>scp_version/type</code> as per Global Platform Amendment A. Format: It is one of the following values: • RSA_PKI • OPACITY_A • SCP_02/PULL

Visa Token Service Certs

Field	Description
<code>vCertificateID</code>	(Required) Unique ID for Visa Certificate. This will be given to the wallet provider during onboarding. Devices can use this certificate ID while communicating with Visa Token Service . This will be the first 4 bytes of the Visa Public Key. Format: String. Size: 32.
<code>certUsage</code>	(Conditional) The usage of this certificate. Format: It is one of the following values: • CONFIDENTIALITY • INTEGRITY

Device Certs

Field	Description
certUsage	<i>(Conditional)</i> The usage of this certificate. Format: It is one of the following values: • CONFIDENTIALITY • INTEGRITY • DEVICE_ROOT
certFormat	<i>(Conditional)</i> X509 certificate structure. Format: String [A-Za-z0-9]+. Size: 16.
certValue	<i>(Conditional)</i> In PEM format/Base64URL safe encoded. Format: String[A-Za-z0-9]+. Size: 8192.

Retrieve Transaction History Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
transactionScope	Scope for the transaction detail. Defaults to <code>TOKEN</code> in this release of VTS service Format: It is one of the following values: • <code>TOKEN</code> - All of the reported transactions are for this token only. • <code>PAN</code> - The list of transactions can include transactions from other tokens that are from the same PAN.
encryptionMetaData	Cryptographic payload for end-to-end encryption of sensitive fields. It is required for the <code>OPACITY_B</code> encryption scheme. Format: Alphabetic, numeric; maximum 512 characters.
transactionDetails	Format: A <code>transactionDetails</code> structure.

Transaction Details

Field	Description
encTransactionInfo	Encrypted blob of transaction detail record. The transaction detail records that make up this encrypted blob are defined in the rows that follow. For more information, see "JSON Web Encryption and Object Signing." Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
vProvisionedTokenID	Unique ID for token. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.
transactionID	Transaction identifier Format: Alphabetic, numeric; maximum 32 characters.
transactionType	Format: It is one of the following values: • 0 - Purchase • 1 - Refund • 3 - Cash Withdrawal • 4 - ATM Services • 5 - ATM Balance Inquiry • 6 - AFT • 7 - OCT String. Size: 1.

Field	Description
transactionDate	Transaction Date. Note: For settled transactions (transactionStatus = 4), the date will be provided, but time is not applicable and will always be 00:00:00. Format: String; date and time in Zulu format, e.g. <i>YYYY-MM-DDTHH:MM:SSZ</i> .
currencyCode	ISO 4217 format. Format: Alphabetic, numeric; maximum 4 characters.
merchantName	Merchant Name. Format: Alphabetic, numeric; maximum 32 characters.
amount	Amount. Format: Alphabetic, numeric; maximum 16 characters.
transactionStatus	Transaction status. Format: It is one of the following values: • 1 - Approved • 2 - Reversed • 3 - Declined • 4 - Settled
industryCatgCode	Industry category code. Format: Numeric.; maximum 4 digits.
industryCatgName	Industry category like Restaurant, Airline etc. Format: Alphabetic, numeric; maximum 32 characters.
industryCode	MCC Merchant code. Format: Numeric.; maximum 4 digits.
industryName	Industry name. Format: Alphanumeric; maximum 32 characters.
merchantZipCode	The postal code where the transaction was made. Format: Alphabetic, numeric; 3–16 characters.
merchantCity	The city where the transaction was made. Format: Alphabetic, numeric; maximum 100 characters.

Field	Description
usingThisToken	Returns whether transaction was made using the token referenced by the vProvisionedTokenID field in this API request. Format: It is one of the following values: • true — The transaction was made using the token. • false — The transaction was not made using the token.
atc	Application Transaction Counter (ATC) present if available for the transaction. Format: Alphabetic, numeric; maximum 5 characters.

Retrieve Transaction History Errors

Error Code	Reason Code	Message
403	declined	This card is considered not eligible for tokenization at this moment. Try enroll/provision again for this PAN. If you encounter the same message even for the new enrollment/provision request then ask the customer to either add a different card or to reach out to their issuer.
409	notAllowed	The requested action is not allowed for a given token. Token status is invalid for this operation. Check the token details and perform the required/appropriate LCM action.

This API also returns Standard Errors.

Retrieve Transaction History Examples

Retrieve Transaction History Example

Request

Selected Headers:

```
"x-request-id": "qhh...Or"  
"Accept": "application/json"  
"x-pay-token": "xv2...15"
```

Endpoint:

```
https://cert.api.visa.com/vts/paymentTxns?apikey=RlY...01
```

Request Body

The request does not have a body.

Response

Selected Headers:

```
"x-response-id": "qhh...Or"
```

Response Body:

```
{  
  "ignore00field": "9C7C8AF0",  
  "transactionScope": "PAN",  
  "transactionDetails": [  
    {  
      "encTransactionInfo": "eyJ...Qw"  
    },  
    {  
      "encTransactionInfo": "eyJ...rQ"  
    },  
    {  
      "encTransactionInfo": "eyJ...pQ"  
    },  
    {  
      "encTransactionInfo": "eyJ...Nw"  
    },  
    {  
      "encTransactionInfo": "eyJ...IA"  
    },  
  ],  
}
```

```
{
  "encTransactionInfo": "eyJ...uQ"
},
{
  "encTransactionInfo": "eyJ...sQ"
},
{
  "encTransactionInfo": "eyJ...wQ"
},
{
  "encTransactionInfo": "eyJ...zQ"
},
{
  "encTransactionInfo": "eyJ...MA"
}
]
```

Chapter 19

Get Content

Get Content Introduction

Retrieves the content resource associated with the card's GUID..

Get Content Request

Endpoints

Resource Path

`vts/cps/getContent/{guid}`

Sandbox Path

```
https://cert.api.visa.com/vts/cps/getContent/{guid}?apiKey=key
```

Live Path

```
https://api.visa.com/vts/cps/getContent/{guid}?apiKey=key
```

URL Parameters

Parameter	Description
{guid}	<i>(Required)</i> Unique ID for the content resource. Format: String. Alphabetic, numeric; maximum 32 characters.

Method

GET

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as: <code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Request Body

None. The request does not have a body.

Get Content Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
<code>altText</code>	Format: String. Size: 32.
<code>contentType</code>	Type for the content. Format: It is one of the following values: • <code>digitalCardArt</code> • <code>digitalCardArtBackground</code> • <code>cardSymbol</code> • <code>termsAndConditions</code> Size: 32.
<code>content</code>	Format: An array of <code>content</code> structures.

Content

Field	Description
contentType	MIME type of the content. Format: It is one of the following values: • image/png • image/pdf • text/plain • text/html
width	Width of image, in pixels. Value between 1 and 4096 for image/png; the value is 0 for image/pdf and text/html. Format: String; maximum 32 characters.
height	Height of image, in pixels. Value between 1 and 4096 for image/png; the value is 0 for image/pdf and text/html. Format: String; maximum 32 characters.
encodedData	Content data, base64 encoded. Format: Standard base64 Encoding per RFC4648; max 6MB.

Get Content Errors

This API also returns Standard Errors.

Get Content Examples

Get Content Example

Request

Selected Headers:

```
"x-request-id": "gvC...yB"  
"Accept": "application/json"
```

Endpoint:

```
https://cert.api.visa.com/vts/cps/getContent/a0b...7c?apiKey=GPN...3I
```

Request Body

The request does not have a body.

Response**Selected Headers:**

```
"x-response-id": "qhh...Or"
```

Response Body:

```
{
  "altText": "Front_Image",
  "contentType": "cardSymbol",
  "content": [
    {
      "mimeType": "image/png",
      "width": "200",
      "height": "200",
      "encodedData": "R0l...aP="
    }
  ]
}
```

Chapter 20

Get Token Status

Get Token Status Introduction

Retrieves status information for a token.

Get Token Status Request

Endpoints

Resource Path

`vts/provisionedTokens/{vProvisionedTokenID}`

Sandbox Path

```
https://cert.api.visa.com/vts/provisionedTokens/  
{vProvisionedTokenID}?apiKey=key&relationshipID=id
```

Live Path

```
https://api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}  
?apiKey=key&relationshipID=id
```

URL Parameters

Parameter	Description
{vProvisionedTokenID}	<i>(Required)</i> Unique ID for the provision being confirmed. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.

Method

GET

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as: <code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
<code>apiKey</code>	<i>(Required)</i> Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
<code>relationshipID</code>	<i>(Optional)</i> The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

None. The request does not have a body.

Get Token Status Response

Response Body

Field	Description
<code>deviceBindingInfoList</code>	A list of device binding information. Format: A <code>deviceBindingInfoList</code> structure.
<code>tokenInfo</code>	Token Info. Format: A <code>tokenInfo</code> structure.

Device Binding Info

Field	Description
clientDeviceID	Stable and unique identifier set by the wallet provider for the device. Format: String; max 24 characters.
deviceName	Name of device. Format: String.
status	Device binding info status. Format: It is one of the following values: • APPROVED • DECLINED • CHALLENGED • DELETED

Token Info

Field	Description
tokenStatus	Status of the token. Format: It is one of the following values: • INACTIVE • ACTIVE • SUSPENDED • DELETED
expirationDate	Token expiration date. Format: An <code>expirationDate</code> structure.
hceData	<code>hceData</code> is only available for device-bound tokens. Format: An <code>hceData</code> structure.

HCE Data

Field	Description
dynParams	(Required) Format: A <code>dynParams</code> structure.

Dynamic Parameters

Field	Description
api	Account parameter index used to generate cryptograms. Source: Generated by VTS and associated with token request/ replenishment. Format is defined by VCPCS 1.8. Format: String. Size: 8.
paramsStatus	A paramsStatus structure.

Parameter Status

Field	Description
status	The current status of the token-associated LUK. Format: It is one of the following values: • CURRENT • OBSOLETE
reason	LUK status. Format: It is one of the following values: • THRESHOLD_EXCEEDED • KEY_EXPIRED

Expiration Date

Field	Description
month	The month upon which the payment instrument is set to expire. Example: "month": "9" Format: String; numeric; max length 2 digits.
year	The year upon which the payment instrument is set to expire. Example: "year": "2024" Format: String; numeric; length 4 digits.

Get Token Status Errors

Error Code	Reason Code	Message
409	notAllowed	The requested action is not allowed for a given token. Token status is invalid for this operation. Check the token details and perform the required/appropriate LCM action.

This API also reports errors using a Standard Error Response structure.

Get Token Status Examples

Get Token Status Example

Request

Selected Headers:

```
x-pay-token: x:1570136317:65e9...f4
Content-Type: application/json
x-request-id: test11234
```

Endpoint:

```
https://cert.api.visa.com/vts/provisionedTokens/053...01?apikey=GPN...3I
```

Request Body:

The request does not have a body.

Response

Response Body:

```
{
  "deviceBindingInfoList": [
    {
      "clientDeviceID": "9C4...72",
      "deviceName": "...",
      "status": "CHALLENGED"
    },
    {
      "clientDeviceID": "9C4...71",
      "deviceName": "...",
      "status": "APPROVED"
    }
  ]
}
```

```
    },
    {
      "clientDeviceID": "9C4...12",
      "deviceName": "...",
      "status": "APPROVED"
    }
  ],
  "tokenInfo": {
    "tokenStatus": "ACTIVE",
    "ignore01field": "D2553045",
    "expirationDate": {
      "year": "2022",
      "month": "12"
    }
  }
}
```

Get Token Status Response for Obsolete Token

```
{
  "tokenInfo": {
    "tokenStatus": "ACTIVE",
    "expirationDate": {
      "month": "10",
      "year": "2018"
    }
  },
  "hceData": {
    "dynParams": {
      "api": "4646710",
      "paramStatus": {
        "status": "OBSOLETE",
        "reason": "THRESHOLD_EXCEEDED"
      }
    }
  }
}
```

Chapter 21

Token Life-Cycle Management— Suspend Token

Token Life-Cycle Management— Suspend Token Introduction

Changes the token status to `Suspend`. The result of this operation leads to a token status update. This changed token status should be propagated to the device.

Important:

This operation is not supported for e-commerce/card-on-file and e-commerce enabler tokens. Attempting to suspend the token, in this case, will result in an error.

Token Life-Cycle Management—Suspend Token Request

Endpoints

Resource Path

`vts/provisionedTokens/{vProvisionedTokenID}/suspend`

Sandbox Path

```
https://cert.api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}
/suspend?apiKey=key&relationshipID=id
```

Live Path

```
https://api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}
/suspend?apiKey=key&relationshipID=id
```

URL Parameters

Parameter	Description
<code>{vProvisionedTokenID}</code>	<i>(Required)</i> Unique ID for the provision being confirmed. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.

Methods

PUT

Required Headers

Header	Description
<code>x-request-id</code>	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
<code>x-pay-token</code>	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
<code>apiKey</code>	(<i>Required</i>) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
<code>relationshipID</code>	(<i>Optional</i>) The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

Field	Description
<code>updateReason</code>	(<i>Required</i>) Reason for the change. Format: An <code>updateReason</code> structure.

Update Reason

Field	Description
<code>reasonCode</code>	(<i>Required</i>) The reason for the current status of the token or token-associated account parameters. Format: It is one of the following string values: • <code>FRAUD</code> • <code>CUSTOMER_CONFIRMED</code> String; maximum 20 characters.
<code>reasonDesc</code>	(<i>Optional</i>) Reason description for the change in status. Format: Alphabetic, numeric; maximum 256 characters.

Token Life-Cycle Management—Suspend Token Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

The response does not have a body.

Token Life-Cycle Management—Suspend Token Errors

Error Code	Reason Code	Message
409	notAllowed	Further operations for this card are no longer allowed. Contact your bank to resolve this issue.

This API also returns Standard Errors.

Token Life-Cycle Management—Suspend Token Examples

Suspend Token Example

Request

Endpoint:

```
https://cert.api.visa.com/vts/provisionedTokens/  
8ec...02/suspend?apikey=GPN...3I
```

Request Body:

```
{  
  "updateReason": {  
    "reasonCode": "FRAUD",  
    "reasonDesc": "Unknown transactions"  
  }  
}
```

Response

The response does not have a body.

Chapter 22

Token Life-Cycle Management— Resume Token

Token Life-Cycle Management— Resume Token Introduction

Changes the token status to *Active*. The result of this operation leads to a token status update. This changed token status should be propagated to the device.

Important:

This operation is not supported for e-commerce/card-on-file and e-commerce enabler tokens. Attempting to resume the token, in this case, will result in an error.

Token Life-Cycle Management—Resume Token Request

Endpoints

Resource Path

`vts/provisionedTokens/{vProvisionedTokenID}/resume`

Sandbox Path

```
https://cert.api.visa.com/vts/provisionedTokens/  
{vProvisionedTokenID}/resume?apiKey=key&relationshipID=id
```

Live Path

```
https://api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}  
/resume?apiKey=key&relationshipID=id
```

URL Parameters

Parameter	Description
<code>{vProvisionedTokenID}</code>	<i>(Required)</i> Unique ID for the provision being confirmed. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36. characters.

Methods

PUT

Required Headers

Header	Description
<code>x-request-id</code>	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
<code>x-pay-token</code>	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
<code>apiKey</code>	(<i>Required</i>) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
<code>relationshipID</code>	(<i>Optional</i>) The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

Field	Description
<code>updateReason</code>	(<i>Required</i>) Reason for the change. Format: An <code>updateReason</code> structure.

Update Reason

Field	Description
<code>reasonCode</code>	(<i>Required</i>) The reason for the current status of the token or token-associated account parameters. Format: It is one of the following string values: • <code>FRAUD</code> • <code>CUSTOMER_CONFIRMED</code> String; maximum 20 characters.
<code>reasonDesc</code>	(<i>Optional</i>) Reason description for the change in status. Format: Alphabetic, numeric; maximum 256 characters.

Token Life-Cycle Management— Resume Token Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

The response does not have a body.

Token Life-Cycle Management— Resume Token Errors

Error Code	Reason Code	Message
409	notAllowed	Invalid token in request.

This API also returns Standard Errors.

Token Life-Cycle Management— Resume Token Examples

Resume Token Example

Request

Endpoint:

```
https://cert.api.visa.com/vts/provisionedTokens/  
8ec...02/resume?apikey=GPN...3I
```

Request Body:

```
{  
  "updateReason": {  
    "reasonCode": " CUSTOMER_CONFIRMED",  
    "reasonDesc": " Customer called"  
  }  
}
```

Response

The response does not have a body.

Chapter 23

Token Life-Cycle Management— Delete Token

Token Life-Cycle Management— Delete Token Introduction

Changes the status of a provisioned token to `Delete`. The result of this operation leads to a token status update. This changed token status should be propagated to the device. Once the token is deleted, you cannot modify the token status to another state. To re-activate a deleted token, you must re-provision the token (Provision Token—Given PAN Enrollment ID).

Token Life-Cycle Management—Delete Token Request

Endpoints

Resource Path

`vts/provisionedTokens/{vProvisionedTokenID}/delete`

Sandbox Path

```
https://cert.api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}/delete?apiKey=key&relationshipID=id
```

Live Path

```
https://api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}/delete?apiKey=key&relationshipID=id
```

URL Parameters

Parameter	Description
<code>{vProvisionedTokenID}</code>	<i>(Required)</i> Unique ID for the provision being confirmed. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.

Methods

PUT

Required Headers

Header	Description
<code>x-request-id</code>	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
<code>x-pay-token</code>	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
<code>apiKey</code>	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
<code>relationshipID</code>	(Optional) The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

Field	Description
<code>updateReason</code>	(Required) Reason for the change. Format: An <code>updateReason</code> structure.

Update Reason

Field	Description
<code>reasonCode</code>	(Required) The reason for the current status of the token or token-associated account parameters. Format: It is one of the following string values: • <code>FRAUD</code> • <code>CUSTOMER_CONFIRMED</code> String; maximum 20 characters.
<code>reasonDesc</code>	(Optional) Reason description for the change in status. Format: Alphabetic, numeric; maximum 256 characters.

Token Life-Cycle Management—Delete Token Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

The response does not have a body.

Token Life-Cycle Management—Delete Token Errors

Error Code	Reason Code	Message
409	notAllowed	Further operations for this card are no longer allowed. Contact your bank to resolve this issue.

This API also returns Standard Errors.

Token Life-Cycle Management—Delete Token Examples

Delete Token Example

Request by Token Requestor

Endpoint:

```
https://cert.api.visa.com/vts/provisionedTokens/  
8ec...02/delete?apikey=GPN...3I
```

Request Body:

```
{  
  "updateReason": {  
    "reasonCode": "FRAUD",  
    "reasonDesc": " Unknown transactions"  
  }  
}
```

Request by Token Service Provider (TR-TSP)

Endpoint:

```
https://cert.api.visa.com/vts/provisionedTokens/  
8ec...02/delete?apikey=GPN...3I&relationshipID=111...39
```

Request Body:

```
{  
  "updateReason": {  
    "reasonDesc": "Customer informed",  
    "reasonCode": "LOST PHONE"  
  }  
}
```

Response

The response does not have a body.

Chapter 24

Get PAN Data

Get PAN Data Introduction

Retrieves PAN data, which allows you to determine whether a PAN is eligible for tokenization. It supports Private Label, V PAY, Electron, and e-Commerce requirements. For PANs that are eligible for tokenization, the `expDatePrintedInd` and `cvv2PrintedInd` fields in the response will advise a Wallet Provider whether or not to request Expiration Date and/or CVV2 from the cardholder prior to calling Enroll PAN or provisioning the token.

Note:

If the PAN is not currently eligible for tokenization, the response will be HTTP Error Code, Reason Code: 403, Declined.

Get PAN Data Request

Endpoints

Resource Path

`vts/panData`

Sandbox Path

`https://cert.api.visa.com/vts/panData?apiKey=key`

Live Path

`https://api.visa.com/vts/panData?apiKey=key`

Methods

POST

Required Headers

Header	Description
<code>x-request-id</code>	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
<code>x-pay-token</code>	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.

Request Body

Field	Description
encPaymentInstrument	(Required) BLOB field (URL safe base 64 encoded) containing the encrypted payload for payment instrument using JSON Web Encryption by the shared secret. For more information, see "JSON Web Encryption and Object Signing." The caller only has access to the <code>accountNumber</code> (PAN), so only the PAN will be passed in this field. Format: Base64 encoded string, per RFC4648, with the following exceptions: hyphens (-) replace plus signs (+) and underscores (_) replace forward slash (/) characters, maximum 8192 characters.

Get PAN Data Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
paymentInstrument	Unencrypted payment instrument. expirationDate of the payment instrument will not be populated for the Get PAN Data Response. Format: A paymentInstrument structure.

Payment Instrument

Field	Description
last4	Last 4 digits of the PAN associated with the enrolled account. Note: This will be the last 4 digits of the IBAN if an IBAN was provided in the provisioning request. Source: client-provided during enrollment request. Format: Numeric; maximum 4 digit.
indicators	These are indicators (Boolean values) associated with the Payment Instrument. If the array is not returned, or empty, then there are no indicators associated with it. Expected values are: PRIVATE_LABEL - This Payment Instrument is for a Private Label PAN. Format: String array.
expDatePrintedInd	(Optional) Indicates if the expiration date is printed on the payment instrument. Format: It is one of the following values: - Y - Printed - N - Not printed
cvv2PrintedInd	(Optional) Indicates if the CVV2 is printed on the payment instrument. Format: It is one of the following values: - Y - Printed - N - Not printed

Get PAN Data Errors

Error Code	Reason Code	Description
403	declined	This card is considered not eligible for tokenization at this moment. Try enroll/provision again for this PAN. If you encounter the same message, even for the new enrollment/provision request, ask the customer to either add a different card or reach out to their issuer.

This API also returns Standard Errors.

Get PAN Data Examples

RSA-PKI Get Pan Data

JSON RSA-PKI Get PAN Data Request

```
{
  "encPaymentInstrument": "eyJ...iw"
}
```

JSON RSA-PKI Get PAN Data Response

```
{
  "paymentInstrument": {
    "indicators": [
      "PRIVATE_LABEL"
    ],
    "last4": "7819",
    "cvv2PrintedInd": "N",
    "expDatePrintedInd": "N"
  }
}
```

Opacity Get PAN Data

JSON OPACITY Get PAN Data Request

```
{
  "encPaymentInstrument": "eyJhbGciOiJBbmJU2R0NNS ..."
}
```

JSON OPACITY Get PAN Data Response

```
{
  "paymentInstrument": {
    "last4": "2750",
    "cvv2PrintedInd": "Y",
    "expDatePrintedInd": "Y"
  }
}
```

Unencrypted Payment Instrument Example

This example shows the contents of the payment instrument before encryption.

```
{
  "accountNumber": "4622943127x11584",
  "expirationDate": {
    "month": "12",
    "year": "2022"
  },
  "name": "vts",
  "provider": {
    "clientAppID": "payapp01",
    "isIDnV": true,
    "clientWalletProvider": "400100xxxx5",
    "intent": "PUSH_PROV_MOBILE",
    "issuerClientInformation": {
      "paymentAccountReference": "V001001301801xxxxxxxxxxxx5106",
      "clientWalletAccountID": "userWalletAcId",
      "clientDeviceID": "userDeviceid",
      "phoneNumber": "650-555-1212",
      "contactEmail": "user@email.com",
      "name": "TokenRequesterName",
      "source": "ISSUER"
    }
  },
  "passcode": {
    "maxRetries": "3",
    "value": "123456789012345",
    "type": "PASSCODE"
  },
  "returnURI": "issuer/provided/return/uri"
}
```

```
{  
  },  
  "billingAddress": {  
    "zip": "94404",  
    "state": "CA",  
    "line1": "line1",  
    "country": "US",  
    "city": "A City"  
  },  
  "cvv2": "123"  
}
```

Chapter 25

Confirm Token Association

Confirm Token Association Introduction

The Confirm Token Association API is used to confirm that an email address and/or the last 6 digits of a PAN match the information associated and stored with a token in VTS and VIP. Confirm Token Association addresses the need to confirm the relationship between an existing token and the elements that were used to provision it. To verify a customer, token requestors must send a token along with any preferred combination of optional parameters used in provisioning. The provision elements for confirmation are restricted to the email address associated with the cardholder and the last 4 or last 6 digits of a cardholder's PAN. The following examples represent the request combinations that a token requestor can currently send to confirm associations:

- Token + Email Address
- Token + Email Address + last 4 of cardholder PAN
- Token + Email + last 6 of cardholder PAN

- Token + Last 4 of cardholder PAN
- Token + last 6 of cardholder PAN

Confirm Token Association Request

Endpoints

Resource Path

`vts/provisionedTokens/{vProvisionedTokenID}/confirmAssociation`

Sandbox Path

```
https://cert.api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}
/confirmAssociation?apiKey=key
```

Live Path

```
https://api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}
/confirmAssociation?apiKey=key
```

URL Parameters

Field	Description
<code>{vProvisionedTokenID}</code>	<i>(Required)</i> Unique ID for the provisioned token being confirmed. Format: String. Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.

Methods

POST

Required Headers

Header	Description
<code>x-request-id</code>	<i>(Required)</i> Unique ID for the API request.

Header	Description
	Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	(Required) A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients. Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query

Header	Description
	parameters, such as <code>apiKey=key</code> , and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.

Request Body

Field	Description
<code>clientWalletAccount EmailAddress</code>	(Conditional) Client provided email address linked to their wallet account login. It is used for customer service operations as well as risk operations. The email address needs to be provided if <code>lastDigits</code> is not provided. It can also be provided with <code>lastDigits</code> for a stronger match. - This field will support internationalization using UTF-8 characters. - A match on <code>clientWalletAccountEmailAddress</code> is dependent on this attribute's presence in VTS. It is an optional parameter during enrollment. Format: Alphabetic, numeric; maximum 48 characters.
<code>lastDigits</code>	(Conditional) Last 4 to 6 digits of the PAN provided associated with the Token. The last 4 to 6 digits need to be provided if the <code>clientWalletAccountEmailAddress</code> is not provided. It can also be provided with the <code>clientWalletAccountEmailAddress</code> for a stronger match. Format: Alphabetic, numeric; 4–6 characters.

Confirm Token Association Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Confirm Token Association Response

Field	Description
<code>associationConfirmed</code>	Indicates whether or not the association between the input parameters supplied by the API consumer and the token could be confirmed. Format: It is one of the following values: • YES • NO This is only indicative. The entire PAN is not provided, so only the <code>clientWalletAccountEmailAddress</code> can be completely verified. The last 4 to 6 digits of the PAN can be verified against the tokenized PAN. This API does not give a 100% guarantee that the entire PAN will match.

Confirm Token Association Errors

The following errors are specific to the Confirm Token Association API. This API also returns Standard Errors.

Error Code	Reason Code	Message
400	inputValidationError	Returned if validation failed for either <code>clientWalletAccountEmailAddress</code> or <code>lastDigits</code> .
401	AuthError	Returned if authorize <code>vProvisionedTokenId</code> as belonging to the Token Requestor fails.

This API also returns Standard Errors.

Confirm Token Association Examples

Confirm Token Association Example

Request

Selected Headers:

```
"x-request-id": "gvC...yB"  
"Accept": "application/json"
```

Endpoint:

```
https://cert.api.visa.com/vts/provisionedTokens/f5e...02/  
confirmAssociation?apiKey=GPN...3
```

Request Body

```
{  
  "lastDigits": "x10xx6"  
}
```

Response

Selected Headers:

```
"x-response-id": "qhh...Or"
```

Response Body:

```
{  
  "associationConfirmed": "YES"  
}
```

Chapter 26

Re-Perso Token Data

Re-Perso Token Data Introduction

Requests VTS to update personalization profile data and static data for existing and new features for provisioned tokens. It enables the device to retrieve new perso profile data without requiring the cardholder to delete and re-provision their card again.

Re-Perso Token Data Request

Endpoints

Resource Path

`vts/provisionedTokens/{vProvisionedTokenID}/reperso`

Sandbox Path

```
https://cert.api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}  
/reperso?apiKey=key
```

Live Path

```
https://api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}  
/reperso?apiKey=key
```

URL Parameters

Field	Description
{vProvisionedTokenID}	(Required) Unique ID for the provisioned token. Format: String. Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.

Methods

POST

Required Headers

Header	Description
x-request-id	(Required) Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	(Required) A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.

Request Body

Field	Description
clientAppID	(Required) Unique identifier for the client application. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
clientDeviceID	(Conditional) Stable and unique identifier set by wallet provider for the device. By default, this field should be considered required. It is optional only for the following presentation types: <code>ECOM</code>, <code>MERCHANT_PRESENTED_QR</code>, <code>QR_CONSUMER_CLOUD</code>, and <code>RECEIVE_ONLY</code>. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; 24 characters.
clientWalletAccountID	(Required) Client-provided consumer ID that identifies the wallet account holder entity. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 24 characters.
ssdData	(Conditional) Applicable for NFC-SE use cases only. Container for all data specific to the SD in the chip. If presentation type is NFC-SE, SSD Data needs to be provided. Format: An <code>ssdData</code> structure.
channelSecurityContext	(Conditional) Present only when using <code>OPACITY</code>. Format: A <code>channelSecurityContext</code> structure.
vNotificationID	(Required) Reperso Notification ID Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
atc	(Conditional) Should be passed for device bound tokens, if possible. Required if full reperso is requested. Ignored for eSE tokens. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 5 characters.

Field	Description
<code>fullReperso</code>	(Optional) Whether to return full reperso information. Ignored for eSE tokens. Format: It is one of the following values: • <code>true</code> - Return the LUK if allowed • <code>false</code> - Do not return the LUK (default)
<code>seData</code>	(Conditional) This field is needed when the caller asks for <code>seData</code> instead of APDU. Format: An <code>seData</code> object.

SSD Data Reperso

Field	Description
<code>VMPAVersion</code>	(Required) Denotes the version of the VMPA on the chip from where the provisioning request originates. This is a hex string value. Format: Alphabetic, numeric; maximum 128 characters.
<code>aidArray</code>	(Required) Each element contains an AID + Applet sequence number. Format: Array. Size: 20 (each array element).
<code>sequenceCounter</code>	(Required) Sequence counter for the SCP key in the SD, Format: Numeric; maximum 6 digits. It is a hex-encoded integer pad left with 0. Note: For SCP02, the max size is 4. For SCP03, the max size is 6.

Channel Security Context

Field	Description
<code>channelInfo</code>	Security Channel used to communicate between the device and the Visa Token Service back-end. Format: A <code>channelInfo</code> structure.
<code>vtcCerts</code>	(Conditional) Visa certificate IDs. Format: An array of <code>vtcCerts</code> structures.

deviceCerts	(Conditional) Device certificates. Not required for autofill. Format: An array of deviceCerts structures.
keyData	(Conditional) Key data; required for OPACITY_A; otherwise, it is not used. In the enroll PAN and Provision with PAN APIs, keyData and encryptionMetaData must now be present together. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 512 characters.

Channel Info

Field	Description
encryptionScheme	(Conditional) The security scheme used to secure communication between the device and the Visa Token Service back-end. Note: For Secure Copy (SCP) encryption schemes, the encryption version and the type are concatenated to create a string in the form of scp_version/type as per Global Platform Amendment A. Format: It is one of the following values: • RSA_PKI • OPACITY_A • SCP_02/PULL

Visa Token Service Certs

Field	Description
vCertificateID	(Required) Unique ID for Visa Certificate. This will be given to the wallet provider during onboarding. Devices can use this certificate ID while communicating with Visa Token Service . This will be the first 4 bytes of the Visa Public Key. Format: String. Size: 32.
certUsage	(Conditional) The usage of this certificate. Format: It is one of the following values: • CONFIDENTIALITY • INTEGRITY

Device Certs

Field	Description
certUsage	<i>(Conditional)</i> The usage of this certificate. Format: It is one of the following values: • CONFIDENTIALITY • INTEGRITY • DEVICE_ROOT
certFormat	<i>(Conditional)</i> X509 certificate structure. Format: String [A-Za-z0-9]+. Size: 16.
certValue	<i>(Conditional)</i> In PEM format/Base64URL safe encoded. Format: String[A-Za-z0-9]+. Size: 8192.

SE Data

Field	Description
VMPAVersion	<i>(Required)</i> This field denotes the version of the VMPA on the chip from where the provisioning request originates. Format: Hex string; maximum 128 characters.

Re-Perso Token Data Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
paymentInstrument	Last 4 digits and expiration date of the PAN associated with the enrolled account. Format: A <code>paymentInstrument</code> structure.
tokenInfo	Data associated with the token. Format: A <code>tokenInfo</code> structure.
vProvisionedTokenID	Provisioned ID for a token. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.

Payment Instrument

Field	Description
last4	Last 4 digits of the PAN associated with the enrolled account. Note: This will be the last 4 digits of the IBAN if an IBAN was provided in the provisioning request. Source: client-provided during enrollment request. Format: Numeric; maximum 4 digits.
expirationDate	Payment instrument expiration date. Format: An <code>expirationDate</code> structure.
indicators	These are indicators (Boolean values) associated with the Payment Instrument. If the array is not returned, or empty, then there are no indicators associated with it. Expected values are: PRIVATE_LABEL - This payment instrument is for a Private Label PAN. Format: String array.
expDatePrintedInd	(Optional) Indicates if the expiration date is printed on the payment instrument. Format: It is one of the following values: - Y - Printed - N - Not printed

Field	Description
cvv2PrintedInd	(Optional) Indicates if the CVV2 is printed on the payment instrument. Format: It is one of the following values: • Y - Printed • N - Not printed
paymentAccountReference	(Optional) Payment Account Reference (PAR) if available for this payment instrument. Format: Alphanumeric; maximum 29 characters.

Expiration Date

Field	Description
month	The month upon which the payment instrument is set to expire. Example: "month": "9" Format: String; numeric; max length 2 digits.
year	The year upon which the payment instrument is set to expire. Example: "year": "2024" Format: String; numeric; length 4 digits.

Token Info

Field	Description
tokenStatus	Token status. If a token was deleted, no information will be returned for the token. Format: It is one of the following values: • INACTIVE • ACTIVE • SUSPENDED
tokenRequestorID	Unique ID that identifies the entity for which the token request is being conducted. Format: Numeric; maximum 11 digits.
tokenReferenceID	Unique ID associated with a single token. The ID is created upon initial provisioning of a token to a device. Format: Alphanumeric; maximum 32 characters.
last4	Last 4 digits of the token. Format: Numeric; maximum 4 digits.

Field	Description
expirationDate	Token expiration date in UTC format. Format: An <code>expirationDate</code> structure.
appPrgrmID	Visa proprietary data element identifying the application program ID of the card application; not applicable to NFC-SE. When personalized, the application program ID is returned in the FCI issuer discretionary data of the select response (tag 'BF0C'). qVSDC readers that support Dynamic Reader Limits (DRL) functionality examine the application program ID to determine the reader limit set to apply. Format: Alphabetic, numeric; 10–32 characters.
encTokenInfo	Encrypted token payload. For more information, see "JSON Web Encryption and Object Signing." This field is not returned for <code>NFC-SE</code> presentation types. Note: The unencrypted field contains a token as a 13-16 character string. Format: Alphabetic, numeric; maximum 4096 characters.
hceData	HCE data. Only applicable for NFC-HCE use case; not applicable for NFC-SE use case. Format: An <code>hceData</code> structure.
mst	MST data. Only applicable for NFC-HCE use case when the <code>presentationType</code> is <code>MST</code> ; not applicable for NFC-SE use case. Format: An <code>mst</code> structure.
seCardPerso	APDU commands. Present in the case of NFC-SE use cases except when APDU script is handled by another entity. Format: An APDU JSON structure.
sePersoData	Perso application data. Present when APDU script is handled by another entity. Format: An <code>sePersoData</code> structure.
atc	Transaction counter; present in HCE tokens. Format: Numeric; 5 digits.

Expiration Date

Field	Description
<code>month</code>	The month upon which the payment instrument is set to expire. Example: "month": "9" Format: String; numeric; max length 2 digits.
<code>year</code>	The year upon which the payment instrument is set to expire. Example: "year": "2024" Format: String; numeric; length 4 digits.

HCE Data

Field	Description
<code>dynParams</code>	HCE data dynamic parameters. Format: A <code>dynParams</code> structure.
<code>staticParams</code>	HCE data static parameters. Format: A <code>staticParams</code> structure.

Dynamic Parameters

Field	Description
<code>encKeyInfo</code>	Encrypted new, limited-use key for a given provision token ID. This value is used for cryptogram generation and is updated during replenishment requests. It is encrypted using the wallet provider's preferred encryption scheme (for example, _A). Format: Alphabetic, numeric; maximum 32 characters. For more information, see "JSON Web Encryption and Object Signing."
<code>maxPmts</code>	Maximum number of payments per provisioned token before replenishment is required Source: VTS. Format: Numeric; maximum 99999 characters.
<code>api</code>	Account parameter index used to generate cryptograms. Source: Generated by VTS and associated with token request/replenishment. Format is defined by VCPCS 1.8. Format: Alphabetic, numeric; maximum 8 characters.

Field	Description
sc	Sequence counter, which is used to generate cryptograms. Tracked in SDK, the sequence counter will be incremented upon replenishment call. Source: VTS generated. Format: Alphabetic, numeric; maximum 65535 characters.
keyExpTS	Expiration timestamp associated with the LUK time-to-live. Source: VTS. Format: Numeric, UTC Format in epoch seconds.
dki	Derived key indicator defining the key index used to calculate the cryptogram. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 2 characters.

Static Parameters

Field	Description
aidInfo	Format: An aidInfo structure.
kernelIdentifier	Returned for Durbin (US Debit). Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 8 characters.
cardHolderNameVCPCS	Cardholder name per ISO 7813. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 26 characters.
pdol	List of terminal/reader- related data objects (tags and lengths) requested by the card to be transmitted in the GET PROCESSING OPTIONS command. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 48 characters.
countrycode5F55	Returned for Durbin (US Debit). Source: Issuer Configuration File. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 4 characters.

Field	Description
issuerIdentificationNumber	Returned for Durbin (US Debit). Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Numeric; maximum 6 digits.
msdData	Format: A <code>MsdData</code> structure.
qVSDCData	Format: A <code>qVSDCData</code> structure.
track2DataDec	Format: A <code>track2DataDec</code> structure.
track2DataNotDec	Format: A <code>track2DataNotDec</code> structure.

Note:

Provisioning response payload related to qVSDC and MSD profiles. Paywave elements have conditional use, qVSDC response will have track2datanotdec whereas MSD/qVSDC will have msdData and track2datadec fields populated.

MSD Data

Field	Description
aip	Application Interchange Profile that contains information about the technology the card supports, such as MSD or qVSDC. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; 2 hexadecimal digits. For hex representation, two characters are used to represent one hex byte.
afl	Application file locator that indicates the location (SFI, range of records) of the AEFs related to a given application. As with all data elements returned in the GPO response, the card application supports personalization of different AFL data elements for use in the following GPO responses: • qVSDC Offline GPO Response (if offline implemented) • qVSDC Online with ODA GPO Response • qVSDC Online/Decline without ODA GPO Response • MSD CVN17 with ODA GPO Response • MSD CVN17 without ODA GPO Response • MSD Legacy GPO Response Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 8 characters.

qVSDC Data

Field	Description
ctq	Card transaction identifiers used by the device to determine the card CVM requirements, issuer preferences and card capabilities. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 4 characters.
ced	Customer exclusive data that is transmitted to issuer. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 64 characters.
ffi	Form factor indicating the kind of consumer payment device and contactless interface over which the transaction was conducted. This information is made available to the issuer. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 8 characters
auc	Application usage control which is an ISO 3166 code identifying the issuer's country. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 4 characters.
psn	PAN sequence number used in encryption. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Numeric; maximum 2 digits.
cvn	Most significant nibble: '4': this IAD format. Least significant nibble: '3': Limited Use Key Cryptogram. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 2 characters.
digitalWalletID	VTS-defined unique ID that identifies the wallet provider entity from which the external request is being sent. This ID will be provided to the wallet provider during onboarding to VTS. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 8 characters.

Field	Description
countryCode	An ISO 3166 code identifying the issuer's country. Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 4 characters.
cid	Indicates the type of cryptogram (TC, ARQC, or AAC) returned by the card and the actions to be performed by the reader. Source: Issuer configuration file. Format: Alphabetic, numeric; maximum 2 characters.
qVSDCWithoutODA	Format: A qVSDCWithoutODA structure.

qVSDC Without ODA

Field	Description
aip	Application Interchange Profile which contains information about the technology the card supports, such as MSD or qVSDC. Format: Alphabetic, numeric; maximum 4 characters.
afl	Application File Locator, which indicates the location (SFI, range of records) of the AEFs related to a given application. As with all data elements returned in the GPO response, the card application supports personalization of different AFL data elements for use in the following GPO responses: • qVSDC Offline GPO Response (if offline implemented) • qVSDC Online with ODA GPO Response • qVSDC Online/Decline without ODA GPO Response • MSD CVN17 with ODA GPO Response • MSD CVN17 without ODA GPO Response • MSD Legacy GPO Response Format: Alphabetic, numeric; maximum 8 characters.

Track2 Data Dec

Field	Description
svcCode	Service code that identifies the components necessary to compile Track 2 equivalent data per <i>Visa Contactless Payment Specification</i> . Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 3 characters.

Track2 Data Not Dec

Field	Description
svcCode	Service code that identifies the components necessary to compile Track 2 equivalent data per the <i>Visa Contactless Payment Specification</i> . Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; maximum 3 characters.
pinVerField	Necessary to compile Track 2 Equivalent data for the <i>Visa Payment Contactless Specification</i> Format: Alphabetic, numeric; maximum 5 characters.
track2DiscData	Track 2 disc data. Discretionary Data (Partly ICCV data). Source: Issuer configuration file. This data is configured by the issuer during onboarding to VTS. Format: Alphabetic, numeric; 8-13 characters.

MSR

Field	Description
svcCode	Service code which identifies the components necessary to compile Track 2 equivalent data per the <i>Visa Contactless Payment Specification</i> . Format: Alphabetic, numeric; maximum 3 characters.
cvv	Card verification value. Format: Numeric; 3-4 digits.

APDU JSON

This is the `seCardPerso` structure.

Field	Description
appletInstanceAID	Full AID of the card instance or application. Format: String; max 20 characters.
priorityCode	Priority for the AID. Format: String; max 4 characters.
apduPerso	APDU commands. Format: An array of <code>ApduLines</code> structures.

APDU Lines

Field	Description
apduID	SerialID for each APDU command. Format: String; max 20 characters.
apdu	APDU command. Format: String; max 4 characters.

Perso Application Data

Field	Description
persoApplicationData	AID data. Format: An array of <code>PersoElement</code> structures.

Perso Element

Field	Description
dataTag	EMV/VMPA tag. Format: String; max 4 characters.
dataName	Data element name. Format: String; max 512 characters.
dataValue	Perso data element. Format: String; max 1024 characters.

ODA Data

This data will only be returned for NFC-HCE use cases. ODA data for NFC-SE will be part of the `seCardPerso` script. Will be returned if the token is enabled for Offline Data Authorization (ODA).

Field	Description
appFileLocator	Application File Locator. Format: Alphabetic, numeric; maximum 10 characters.
appProfile	Application Interchange Profile. Format: Alphabetic, numeric; maximum 5 characters.

caPubKeyIndex	Certificate Authority Public Key Index. Index to identify the payment system public key used for dynamic data authentication (for ODA). Format: Alphabetic, numeric; maximum 3 characters.
iccPubKeyCert	Integrated Circuit Card (ICC) Public Key Certificate.
tokenBinPubKeyCert	Token-BIN Public Key certificate. This is also known as Issuer Public Key Certificate.
encicccPrivateKey	JWE/JWS formatted encrypted data. Format: For more information, see "JSON Web Encryption and Object Signing."

Re-Perso Token Data Errors

The following errors are specific to the Re-Perso Token Data API only. This API also returns Standard Errors.

Error Code	Reason Code	Message
409	notAllowed	The requested action is not allowed for a given token. Token status is invalid for this operation. Check the token details and perform the required/appropriate LCM action.

Re-Perso Token Data Examples

JSON Re-Perso Token Data Request

```
{
  "clientWalletAccountID": "4vmt597",
  "clientDeviceID": "PA4FG5G73KX2",
  "clientAppID": "YourCompanyABC"
}
```

JSON Re-Perso Token Data Response

```
{
  "paymentInstrument": {
    "expirationDate": {
      "month": "12",
      "year": "2017"
    }
  }
}
```

```

    },
    "last4": "6548",
    "cvv2PrintedInd": "Y",
    "expDatePrintedInd": "Y"
  },
  "tokenInfo": {
    "hceData": {
      "staticParams": {
        "aidInfo": [{
          "qVSDCData": {
            "qVSDCWithODA": {
              "afl": "10020400",
              "aip": "2040"
            },
            "ctq": "0100",
            "ffi": "238C0000",
            "auc": "0080",
            "psn": "00",
            "cvn": "43",
            "digitalWalletID": "000000000",
            "countryCode": "0840",
            "cid": "80",
            "qVSDCWithoutODA": {
              "afl": "08030300",
              "aip": "0040"
            }
          }
        ]
      },
      "appPrgrmID": "1008400840",
      "kernelIdentifier": "03",
      "cardHolderNameVCPCS": "",
      "pdol": "9F6...04",
      "countrycode5F55": "5553",
      "issuerIdentificationNumber": "405954",
      "track2DataDec": {
        "svcCode": "201"
      },
      "msdData": {
        "aip": "00C0",
        "afl": "08010100"
      },
      "ODADData": {
        "appFileLocator": "10020400",
        "appProfile": "2040",
        "caPubKeyIndex": "89",
        "encicccPrivateKey": "...",
        "icccPubKeyCert": {
          "certificate": "...",
          "exponent": "03",
          "remainder": "",
          "expirationDate": {
            "month": "06",
            "year": "2017"
          }
        },
        "tokenBinPubKeyCert": {
          "certificate": "...",
          "exponent": "03",
          "remainder": "tYo...E=",

```

```
    "expirationDate": {
      "month": "12",
      "year": "2023"
    }
  },
  "applicationLabel": "555...74",
  "aid": "A00000000980840",
  "CVMrequired": "N",
  "priority": "01",
  "cap": "0000D800"
},
{
  "qVSDCData": {
    "qVSDCWithODA": {
      "afl": "10020400",
      "aip": "2040"
    },
    "ctq": "0100",
    "ffi": "238C0000",
    "auc": "0080",
    "psn": "00",
    "cvn": "43",
    "WalletID": "00000000",
    "countryCode": "0840",
    "cid": "80",
    "qVSDCWithoutODA": {
      "afl": "08030300",
      "aip": "0040"
    }
  },
  "appPrgrmID": "1008400840",
  "kernelIdentifier": "03",
  "cardHolderNameVCPCS": "",
  "pdol": "9F6...04",
  "countrycode5F55": "5553",
  "issuerIdentificationNumber": "405954",
  "track2DataDec": {
    "svcCode": "201"
  },
  "msdData": {
    "aip": "00C0",
    "afl": "08010100"
  },
  "ODADData": {
    "appFileLocator": "10020400",
    "appProfile": "2040",
    "caPubKeyIndex": "89",
    "encicccPrivateKey": "...",
    "iccPubKeyCert": {
      "certificate": "...",
      "exponent": "03",
      "remainder": "",
      "expirationDate": {
        "month": "06",
        "year": "2017"
      }
    }
  }
},
```

```
    "tokenBinPubKeyCert": {  
      "certificate": "...",  
      "exponent": "03",  
      "remainder": "tY...E=",  
      "expirationDate": {  
        "month": "12",  
        "year": "2023"  
      }  
    },  
    },  
    "applicationLabel": "566...74",  
    "aid": "A0000000031010",  
    "CVMrequired": "N",  
    "priority": "01",  
    "cap": "0000D800"  
  }  
}  
},  
"tokenRequestorID": "40...52",  
"tokenReferenceID": "DNI...90",  
"tokenStatus": "ACTIVE",  
"last4": "0750",  
"expirationDate": {  
  "month": "12",  
  "year": "2020"  
}  
},  
"vProvisionedTokenID": "74f...01"  
}
```

eSE Reperso for SCP02 Example

Request

Request URL

```
https://api.visa.com/vts/provisionedTokens/910...01/reperso?  
apikey=H68...18
```

Request Body

```
{  
  "clientAppID": "testappid",  
  "clientDeviceID": "r9...H",  
  "vNotificationID": "9a9...01",  
  "clientWalletAccountID": "ON...T",  
  "ssdData": {  
    "aidArray": [  
      "A00...01"  
    ],  
    "sequenceCounter": "0001",  
    "keyVersionNumber": "01",  
    "vCertificateID": "c12...52",  
  }  
}
```



```
"sdScript": "820...EA",
"CASDCert": "7F2...00",
"VMPPAVersion": "313...30"
},
"fullReperso": false
}
```

Response

Response Body

```
{
  "tokenInfo": {
    "tokenRequestorID": "400...51",
    "tokenStatus": "ACTIVE",
    "tokenReferenceID": "DNI...58",
    "last4": "3608",
    "expirationDate": {
      "year": "2026",
      "month": "12"
    },
  },
  "seCardPerso": [
    {
      "appletInstanceAID": "A00...01",
      "priorityCode": "01",
      "apduPerso": {
        "apduLines": [
          {
            "apduID": "01",
            "apdu": "00A...00"
          },
          {
            "apduID": "02",
            "apdu": "805...00"
          },
          {
            "apduID": "03",
            "apdu": "848...F3"
          },
          ...
          {
            "apduID": "22",
            "apdu": "84E...82"
          }
        ]
      }
    }
  ],
  "presentationType": [
    "INAPP",
    "NFC-SE"
  ],
  "paymentInstrument": {
    "expirationDate": {
      "month": "12",

```

```
"ignore02field": "...",
"year": "2023"
},
"last4": "7524",
"cvv2PrintedInd": "Y",
"ignore01field": "...",
"expDatePrintedInd": "Y",
"paymentAccountReference": "V00...86",
"enabledServices": {
  "merchantPresentedQR": "N"
}
},
"ignore00field": "...",
"vProvisionedTokenID": "910...01"
}
```

Chapter 27

Validate JWE

Validate JWE Introduction

Validates the encrypted payment instrument JWE received from an issuer.

Validate JWE Request

Endpoints

Resource Path

`vts/validateJWE`

Sandbox Path

`https://cert.api.visa.com/vts/validateJWE?apiKey=key`

Live Path

`https://api.visa.com/vts/validateJWE?apiKey=key`

Methods

POST

Required Headers

Header	Description
x-request-id	(Required) Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	(Required) A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients. Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists.

Header	Description
	Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <i>shared_secret</i> is the private key value associated with your API key, which is available in your Visa Developer Center account, <i>resource_path</i> is the API name / URI, <i>query_string</i> contains query parameters, such as <code>apiKey=key</code>, and <i>request_body</i> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: Alphabetic, numeric; maximum 64 characters.

Request Body

Field	Description
encPaymentInstrument	(Required) BLOB field containing all payment instrument details (URL-safe base64 encoded) that represents the encrypted payload for a payment instrument. Format: JSON Web Encrypted string using the shared secret. For more information, see "JSON Web Encryption and Object Signing." String Size 8192.
clientAppID	(Required) Unique identifier for the client application. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.

Unencrypted Payment Instrument

Field	Description
accountNumber	<i>(Required)</i> PAN of the card to be enrolled and provisioned Format: Numeric; 13-19 digits.
cvv2	<i>(Optional)</i> CVV2 value associated with the PAN on the card. Populate this field based on the response from Get PAN Data, which indicates whether this information is printed on the card. Note: For CVV2 to be considered during token provisioning, the CVV2 must also be provided within the encRiskDataInfo structure of the Provision Token request. Format: Numeric; 3-4 digits.
name	<i>(Optional)</i> The full name on the Visa card associated with the enrolled payment instrument. Format: Alphabetic and spaces (); maximum 256 characters.
expirationDate	<i>(Optional)</i> Payment instrument's expiration date. Populate based on the response from getPanData, which indicates whether this information is printed on the card. Format: An expirationDate structure.
billingAddress	<i>(Optional)</i> Billing address associated with the payment instrument Format: A billingAddress structure.
provider	<i>(Conditional)</i> Information about the provider of the payment instrument and the context in which it is provided. Required if the panSource is ISSUER_PUSH_PROVISION. Format: A provider structure.

Expiration Date

Note:

The expiration date is optional; however, all fields must have a value if the date is provided.

Field	Description
month	<i>(Required)</i> The month the Visa card is set to expire. Format: Numeric; 2 digits.
year	<i>(Required)</i> The year the Visa card is set to expire. Format: Numeric; 4 digits.

Billing Address

Field	Description
line1	(Optional) Street 1 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line2	(Optional) Street 2 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line3	(Optional) Street 3 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line4	(Optional) Street 4 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line5	(Optional) Street 5 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
city	(Optional) The city associated with the enrolled payment instrument. Format: Alphabetic, single quote ('), hyphen (-), and whitespace.; maximum 100 characters.
state	(Optional) State or province code in ISO 3166-2 format, e.g. NY. Format: Alphabetic; maximum 3 characters.
country	(Optional) Country in ISO 3166-1 alpha-2 format, e.g. US. Format: Alphabetic; maximum 2 characters.

countryDigits	<i>(Optional)</i> ISO 3166-1 numeric Country Code. Format: Numeric; maximum 3 digits.
postalCode	<i>(Optional)</i> The postal code associated with the enrolled payment instrument. Note: This field does not support spaces (" ") in the middle of characters. Format: Alphabetic, numeric, e.g. spaces are not allowed; maximum 3–16 characters.

Payment Instrument Provider

This object allows the payment instrument provider to specify the intent under which the PAN should be used for token provisioning. Some of the fields provided here will be matched with the corresponding fields from the API payload. This structure applies only to Visa Card Enrollment Hub (VCEH) or legacy direct push provisioning.

Field	Description
intent	(Required) The intent of the encryptor, which specifies what the encryptor of the data trying to do. Format: It is one of the following values: PUSH_PROV_MOBILE - Push the PAN data to a Token Requestor to provision a token for the card holder on a specific device for a specific wallet/account. The push initiating application and the push receiving (token requestor) application are accessible on the same device. PUSH_PROV_CROSS_USER - Push the PAN data to a Token Requestor to provision a token for a secondary/authorized user. The Issuer is providing the PAN for the purpose of provisioning a token for an authorized consumer that is different from the push initiating card holder. This is only applicable to only to Visa Card Enrollment Hub (VCEH). PUSH_PROV_CROSS_DEVICE - Push the PAN data to a Token Requestor to provision a token for the cardholder on a different device. The issuer is providing the PAN for the purpose of provisioning a token for the consumer of a token requestor application accessible on a device that is different from the device from which the push is being initiated. This is only applicable to only to Visa Card Enrollment Hub (VCEH). PUSH_PROV_ONFILE - Push the PAN data to a Card On File merchant/Token Requestor to provision a token for the cardholder. The issuer is providing the PAN for the purpose of provisioning a token for the consumer into a card on file merchant. The push initiating application and the push receiving application(token requestor) are accessible on the same device.
clientWalletProvider	(Required) Token requestor ID. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed. Size: maximum 50 characters.
clientWalletAccountID	(Optional) Client provided consumer ID that identifies the Wallet Account Holder entity. It must have the same value as clientWalletAccountID field from the API payload. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed. Size: maximum 24 characters.
clientAppID	(Conditional) Unique Identifier for the client application, used to provide some of the encrypted values. Required if intent is PUSH_PROV_MOBILE. Example: Issuer's AppID used to select the PAN and the wallet. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed. Size: maximum 36 characters.

isIDnV	(Optional) Specifies whether the issuer wants ID&V to be performed. Format: It is one of the following values: • true - perform ID&V • false - do not perform ID&V
issuerAccountID	(Optional) Uniquely represents “pushing” account from issuer system. May be different from PAN holder account. It applies only to Visa Card Enrollment Hub (VCEH). Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 24 characters.
issuerClientInformation	(Optional) Client information shared from issuer to token requestor from information captured in the issuer systems about the PAN's owner. It applies only to Visa Card Enrollment Hub (VCEH). This information will be decrypted and shared with the token requestor. Format: A <code>clientInformation</code> structure.
passcode	(Optional) OTP generated by the issuer. It applies only to Visa Card Enrollment Hub (VCEH). Added to Enroll PAN and Provision + PAN data. Format: A <code>passcode</code> structure.
returnURIType	(Optional) The type of URI for the return app. It applies only to Visa Card Enrollment Hub (VCEH). Format: It is one of the following values: • IOS - iOS app • ANDROID - Android app • WEB - browser-based app
returnURI	(Optional) URI provided by the issuer to the token requestor to return control to the issuer app. This can be an app or web URL. Format: Alphanumeric. Size: maximum 128 characters.

issuerTraceID	(Optional) An ID, preferably a Globally Unique Identifier (GUID), provided by the issuer in order to query token status after sending the JWE to a token requestor. It applies only to Visa Card Enrollment Hub (VCEH). Format: Alphanumeric. Size: maximum 32 characters.
isTsAndCsAccepted	(Optional) This field indicates whether or not the customer has already accepted the issuer's Terms and Conditions. Note: If this value is set to <code>true</code>, the wallet provider can opt to not display the Terms and Conditions to the customer. They would submit the Terms and Conditions GUID to Visa. The default value for this field is <code>false</code>. Format: It is one of the following values: • <code>true</code> • <code>false</code>

Passcode

Passcode fields are only applicable to Visa Card Enrollment Hub (VCEH).

Field	Description
type	(Required) Passcode type. Format: It is one of the following values: • <code>PASSCODE</code>
value	(Required) The OTP / Passcode / Passphrase generated by the user or issuer that should be validated by Visa prior to PAN Enrollment. Format: Alphanumeric, maximum 50 characters.
passCodeConversationID	(Conditional) This is returned when <code>passcode</code> is required. The <code>passCodeConversationID</code> should be returned to Visa by the token requestor in the subsequent Enroll PAN call. This attribute should not be sent by the issuer. If the issuer sends a value for this attribute in the encrypted payment instrument, it should be ignored. Format: Alphanumeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 32 characters
maxRetries	(Optional) The number of times the OTP can be retried before a risk code is sent to VIP indicating that the OTP did not match. This value is provided by the issuer when sending the payment instrument to the wallet provider. It should not be populated by the wallet provider during enrollment. If the wallet provider sends this attribute in the request, it is rejected (invalid request – bad parameter). Format: String. Digits [0-9].

Client Information

Client information applies only to Visa Card Enrollment Hub (VCEH).

Field	Description
source	(Optional) The source of the information. Format: It is one of the following values: • ISSUER • TOKEN_REQUESTOR
clientWalletAccountID	(Optional) Client provided consumer ID that identifies the wallet account holder entity. It must have the same value as clientWalletAccountID field from the API payload. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 24 characters.
firstName	(Required) First name of the token requestor client that received the token. The issuer will populate with the information they have for the client. Format: Alphanumeric; between 2–80 characters, inclusive. Escape Characters, including tab entries, are not supported.
middleName	(Optional) Middle name of the token requestor client that received the token. The issuer will populate with the information they have for the client. Format: Alphanumeric; between 2–80 characters, inclusive. Escape Characters, including tab entries, are not supported.
lastName	(Required) Last name of the token requestor client that received the token. The issuer will populate with the information they have for the client. Format: Alphanumeric; between 2–80 characters, inclusive. Escape Characters, including tab entries, are not supported.
issuerAccountID	(Optional) The issuer account ID provided by the issuer to the token requestor. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 24 characters.
clientDeviceID	(Conditional) It is a stable and unique identifier set by the wallet provider for the device. This field is required when it is returned to the issuer by the token requestor. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 24 characters.

Field	Description
tokenReferenceID	<i>(Conditional)</i> It is a unique ID associated with a single token. The ID is created upon initial provisioning of a token to a device. This field is required when returned to the issuer by the token requestor. Format: Alphanumeric; maximum 32 characters.
paymentAccountReference	<i>(Optional)</i> Payment Account Reference (PAR) if available for this payment instrument. Format: Alphanumeric; maximum 29 characters.
phoneNumber	<i>(Conditional)</i> Either phone number or email must be passed when intent is <code>PUSH_PROV_CROSS_DEVICE</code> or <code>PUSH_PROV_CROSS_USER</code>. Mobile phone number of the client as per issuer records. Format: A valid phone number; maximum 32 bytes.
contactEmail	<i>(Conditional)</i> Either phone number or email must be passed when intent is <code>PUSH_PROV_CROSS_DEVICE</code> or <code>PUSH_PROV_CROSS_USER</code>. Email address of client as per issuer records. Format: A valid email address; maximum 32 bytes. Note: This field will support internationalization using UTF-8 characters.
country	<i>(Optional)</i> The client's country code. Format: An alpha 2 country code (ISO 3166-1 alpha-2).
sessionIdentifier	<i>(Conditional)</i> Optional session identifier that can be used by the issuer to uniquely identify the cardholder. This field is required if IBAN and Push Provisioning and no cardholder name was provided. Format: Alphanumeric, numeric, and underscores (_), e.g. spaces are not allowed; maximum 64 characters.
locale	<i>(Required)</i> Language in which the app communicates with the customer. Based on ISO format for language (ISO 639-1) and alpha 2 country code (ISO 3166-1 alpha-2). The language and country should be separated using an underscore (_). e.g. en_US. Format: Alphanumeric, underscores (_), e.g. spaces are not allowed; maximum 5 characters.
campaignID	<i>(Optional)</i> Cobrand campaign identifier. Format: Alphanumeric, numeric, hyphens (-), and underscores (_); maximum 100 characters.

Field	Description
campaignDescription	<i>(Optional)</i> Cobrand campaign description. Format: Alphabetic, numeric, hyphens (-), and underscores (_); maximum 250 characters.
userPersonalID	<i>(Optional)</i> An identifier that uniquely identifies a person. Format: Alphabetic or numeric; maximum 15 characters.

Validate JWE Response

Response Body

Field	Description
status	Whether the validation succeeded. Format: It was one of the following values: • OK — Validation succeeded • FAILED — Validation failed
isPassCodeRequired	Whether OTP step up is required. Format: It is one of the following values: • Y — OTP required. • N — OTP not required.
clientInformation	Client information, as provided by the issuer. This information will be decrypted and sent back to the token requestor. Note: Contains PII information, so decryption means that Visa decrypts it using the issuer key and then encrypts it with the token requestor's key before returning to the token requestor. Format: A <code>clientInformation</code> structure.
returnURIType	The type of URI for the return app. It applies only to Visa Card Enrollment Hub (VCEH). Format: It is one of the following values: • IOS - iOS app • ANDROID - Android app • WEB - browser-based app

Field	Description
returnURI	URI provided by the issuer to the token requestor to return control to the issuer app. This can be an app or web URL. Format: Alphanumeric. Size: maximum 128 characters.
isTsAndCsAccepted	This field indicates whether or not the customer has already accepted the issuer's Terms and Conditions. Note: If this value is set to <code>true</code> , the wallet provider can opt to not display the Terms and Conditions to the customer. They would submit the Terms and Conditions GUID to Visa. The default value for this field is <code>false</code> . Format: It is one of the following values: • <code>true</code> • <code>false</code>

Validate JWE Errors

the section on Standard Errors.

Validate JWE Examples

JSON Request

```
POST https://cert.api.visa.com/vts/validateJWE?apiKey=U...Y HTTP/1.0
Content-Type: application/json; charset=UTF-8
Content-Length: 1634
x-pay-token: xv2:1516869972:1bf783aa9369246aa792c576cadffc3b02a72e6a64b...
x-request-id: muj-1516869970
```

```
{
  "encPaymentInstrument" : "eyJhbGciOiJIbmJU2R0NNS1ci...347Udw",
  "clientAppID" : "WPappId"
}
```

JSON Response

```
{
  "isPASSCODERequired" : "Y",
  "status" : "OK",
  "clientInformation" : "eyJhbGciOiJIbmJU2R0NNS1ciLCJpdjI6Ij1...",
  "passCodeConversationID" : "f8bf42626a06eef8daxxxxxxxxxxb802"
}
```

Validate JWE VCEH Example

Request

Selected Headers:

```
x-pay-token: xv2:153...db  
x-request-id: muj...19
```

Endpoint:

```
https://cert.api.visa.com/vts/validateJWE?apikey=U8...vLY
```

Request Body:

```
{  
  "encPaymentInstrument": "eyJ...sg",  
  "clientAppID": "Clienttest"  
}
```

Unencrypted Payment Instrument Example

This is the contents of the payment instrument before encryption.

```
{  
  "accountNumber": "4622943127x11584",  
  "expirationDate": {  
    "month": "12",  
    "year": "2022"  
  },  
  "name": "vts",  
  "provider": {  
    "clientAppID": "payapp01",  
    "isIDnV": true,  
    "clientWalletProvider": "400100xxxx5",  
    "intent": "PUSH_PROV_MOBILE",  
    "issuerClientInformation": {  
      "paymentAccountReference": "V001001301801xxxxxxxxxxxxx5106",  
      "clientWalletAccountID": "userWalletAcId",  
      "clientDeviceID": "userDeviceid",  
      "phoneNumber": "650-555-1212",  
      "contactEmail": "user@email.com",  
      "name": "TokenRequesterName",  
      "source": "ISSUER"  
    },  
    "passcode": {  
      "maxRetries": "3",  
      "value": "123456789012345",  
      "type": "PASSCODE"  
    },  
    "returnURI": "issuer/provided/return/uri"  
  },  
}
```



```
"billingAddress": {
  "zip": "94404",
  "state": "CA",
  "line1": "line1",
  "country": "US",
  "city": "A City"
},
"cvv2": "123"
}
```

Response

```
{
  "isPassCodeRequired": "Y",
  "status": "OK",
  "clientInformation": "eyJ...SQ",
  "passCodeConversationID": "34d...01",
  "ignore00field": "9B103B3D",
  "returnURIType": "ANDROID",
  "returnURI": "aHR...Ju",
  "isTsAndCsAccepted": "Y"
}
```

Decrypted Client Information:

```
{
  "source": "ISSUER",
  "firstName": "Firstname",
  "middleName": "Middlename",
  "lastName": "Lastname",
  "issuerAccountID": "TEST1",
  "phoneNumber": "650-555-1212",
  "contactEmail": "username@email.com",
  "country": "US",
  "locale": "us_EN"
}
```

JWE encrypted encPaymentInstrument

The `encPaymentInstrument` is JWE-encrypted, which includes:

```
{
  "accountNumber": "...",
  "cvv2": "758",
  "name": "BillEvans",
  "expirationDate": {
    "month": "10",
    "year": "2015"
  },
  "billingAddress": {
    "line1": "801MetroCenterBlvd",
    "line2": null,

```

```
    "city": "FosterCity",  
    "state": "CA",  
    "country": "US",  
    "postalCode": "94404"  
  },  
  "provider": {  
    "clientAppID": "APP1",  
    "clientDeviceID": "DEiOiJBMjU2R0NNNS1ciLCJiI",  
    "clientWalletProvider": "sampleIssuer",  
    "clientWalletAccountID": "AiOiJBMjU2R0NNNS1ciLCJiI6",  
    "Intent": "PUSH_PROV_MOBILE",  
    "isIDnV": false  
  }  
}
```

Chapter 28

Generate Keys

Generate Keys Introduction

Provides the ephemeral keys needed to set up end-to-end encryption between an embedded Secure Element (eSE) device and Visa Token Service. You must call this API after calling Enroll Device.

Generate Keys Request

Endpoints

Resource Path

`vts/panEnrollments/{vPanEnrollmentID}/generateKeys`

Sandbox Path

`https://cert.api.visa.com/vts/panEnrollments/{vPanEnrollmentId}/
generateKeys?apikey=key`

Live Path

`https://api.visa.com/vts/panEnrollments/{vPanEnrollmentId}/generateKeys?
apikey=key`

URL Parameters

Parameter	Description
<code>{vPanEnrollmentID}</code>	(Required) Unique ID for the enrolled PAN. Format: String. Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.

Methods

PUT

Required Headers

Header	Description
<code>x-request-id</code>	(Required) Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
<code>x-pay-token</code>	(Required) A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.

Request Body

Field	Description
clientDeviceID	(Required) This is a stable and unique identifier set by the wallet provider for the device. Format: String. Alphabetic, numeric, hyphens (-) and underscores (_), e.g. spaces are not allowed; 24 characters.
vCertificateID	(Required) This is a Unique ID for Visa Certificate — INTEGRITY. This is given to the wallet provider during onboarding. Devices can use this certificate ID while communicating with Visa Token Services. This will be the first 4 bytes of the Visa Public Key. This is the key Visa Token Services will use to sign the ephemeral public key certificate. Format: Maximum 32 characters.
clientWalletAccountID	(Required) Client-provided consumer ID that identifies the wallet account holder entity. Format: String. Alphabetic, numeric, hyphens (-) and underscores (_), e.g. spaces are not allowed; maximum 24 characters.

Generate Keys Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
<code>ephemeralVisaPublicCert</code>	JWS using the key supplied in the request. The public key is in the Global Platform format. This public key is wrapped into JWS. For new integrations use <code>ephemeralVisaPublicKey</code> parameter. Format: String. base64 encoded; maximum 8192 characters.
<code>ephemeralVisaPublicKey</code>	JWS using the key supplier in the request. The public key is in the Global Platform format. Use this parameter for new integrations. Format: String. base64 encoded; maximum 8192 characters.
<code>vEphemeralVisaPublicKeyID</code>	Key ID generated by Visa Token Services. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 36 characters.

Generate Keys Errors

This API returns Standard Errors.

Generate Keys Examples

Generate Keys Request

```
{
  "clientDeviceID": "DeviceIDZiIG3a0DgF4mQfkC",
  "vCertificateID": "d7df019e",
  "clientWalletAccountID": "Rwt3tJek_k1JxivcwbPHjKDk"
}
```

Generate Keys Response

```
{  
  "vEphemeralVisaPublicKeyID": "fd447de827d4e28b3300a211a79bbd01",  
  "ephemeralVisaPublicCert": "eyJhbGciOiJSUzI1NiIsIm12IjoiIi..."  
}
```


Chapter 29

Initiate Re-Perso

Initiate Re-Perso Introduction

Enables an issuer to re-issue a token without consumer intervention; for example, if a PAN changes; the app can call this API to re-issue the tokens associated with that PAN and associate them with the new PAN.

Initiate Re-Perso Request

Endpoints

Resource Path

`/vts/provisionedTokens/{vProvisionedTokenID}/repersoFlow`

Sandbox Path

`https://cert.api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}/repersoFlow?apikey=key`

Live Path

`https://cert.api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}/repersoFlow?apikey=key`

URL Parameters

Parameter	Description
<code>{vProvisionedTokenID}</code>	(Required) Unique ID for the provision being confirmed. Format: String. Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.

Methods

POST

Required Headers

Header	Description
<code>x-request-id</code>	(Required) Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
<code>x-pay-token</code>	(Required) A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	<i>(Required)</i> Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.

Request Body

None.

Initiate Re-Perso Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
vNotificationID	Re-perso notification ID. Applicable only to re-perso; otherwise, it is not needed. Format: String. [A-Za-z0-9-]+; maximum 36 characters.
deviceId	Device ID. Format: String

Initiate Re-Perso Errors

This API returns Standard Errors.

Initiate Re-Perso Examples

Initiate Re-Perso Request

```
POST https://cert.api.visa.com  
/vts/provisionedTokens/{vProvisionedTokenID}/  
repersoFlow?apiKey="..."
```

Initiate Re-Perso Response

```
{  
  "vNotificationID": "3f0...01",  
  "deviceId": "..."  
}
```

Chapter 30

Get Task Status

Get Task Status Introduction

This API is used to get the status of a task.

Get Task Status Request

Endpoints

Resource Path

`vts/tasks/{vTaskID}/status`

Sandbox Path

`https://cert.api.visa.com/vts/tasks/{vTaskID}/status?apiKey=key`

Live Path

`https://api.visa.com/vts/tasks/{vTaskID}/status?apiKey=key`

Method

GET

URL Parameters

Parameter	Description
vTaskID	<i>(Conditional)</i> This parameter is populated for the <code>TOKEN_CREATED</code> notification in the case of async provisioning flow. Format: String. Alphabetic, numeric; 40 characters.

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	<i>(Required)</i> Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.

Request Body

None. The request does not have a body.

Get Task Status Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
taskStatus	The task status. Format: It is one of the following values: - PENDING—The task is in progress. - COMPLETE—The task has completed.

Get Task Status Errors

This API returns Standard Errors.

Get Task Status Example

Request

Selected Headers:

```
"x-request-id": "gvC...yB"  
"Accept": "application/json"
```

Endpoint:

```
https://cert.visa.com/vts/tasks/8f8...01/status?apikey=ASV...fg
```

Request Body

The request does not have a body.

Response

Response Body:

```
{  
  "taskStatus" : "PENDING"  
}
```

Chapter 31

Get Async Token Provisioning Result

Get Async Token Provisioning Result Introduction

This API is used to get the result of Async Token Provisioning.

Get Async Token Provisioning Result Request

Endpoint

Resource Path

`vts/asyncprovision/tasks/{vTaskID}/result`

Sandbox Path

`https://cert.api.visa.com/vts/asyncprovision/tasks/{vTaskID}/result?apiKey=key`

Live Path

`https://api.visa.com/vts/asyncprovision/tasks/{vTaskID}/result?apiKey=key`

Methods

POST

URL Parameters

Parameter	Description
<code>vTaskID</code>	<i>(Conditional)</i> This parameter is populated for the <code>TOKEN_CREATED</code> notification in the case of async provisioning flow. Format: String. Alphabetic, numeric; 40 characters.

Required Headers

Header	Description
<code>x-request-id</code>	<i>(Required)</i> Unique ID for the API request.

Header	Description
	Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	(Required) A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients. Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query

Header	Description
	parameters, such as <code>apiKey=key</code> , and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
<code>apiKey</code>	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.

Request Body

None. The request does not have a body.

Get Async Token Provisioning Result Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
taskData	The task data. Format: It is a Task Provision Data object.

Task Provision Data

Field	Description
vPanEnrollmentID	Unique ID to identify this PAN enrollment. Another provision token API call on the same PAN is not guaranteed to return the same vPanEnrollmentID. Note: This is the only additional parameter returned as this API response, compared to the Provision Token—Given PAN Enrollment ID API response. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
coresidenceInfo	Contains data about the card and whether or not it is co-badged. This information is used by Wallet Providers who support coresidenceInfo to determine the alternate networks available to use with this PAN. Format: A coresidenceInfo structure.
vProvisionedTokenID	Unique ID for token. Format: Alphabetic, numeric and hyphens (-), e.g. spaces are not allowed; maximum characters 36 characters.
paymentInstrument	Last 4 digits and expiration date of the PAN associated with the enrolled account. Format: A PaymentInstrument structure.
tokenInfo	Data associated with the token. Format: A tokenInfo structure.

Field	Description
<code>stepUpRequest</code>	Step-up request. This field is not required for the <code>ECOM</code> (ecommerce) token type. Note: This is only applicable when: 1. <code>tokenStatus</code> is <code>INACTIVE</code> 2. The <code>deviceID</code> used during push provisioning does not match the <code>deviceID</code> used during enrollment. 3. The issuer indicates a preference for <code>IDnV</code> in the JWE by setting <code>isIDnV=Y</code>. 4. Push Provisioning is required. Numbers 2, 3, and 4 are only applicable for push provisioning. Format: A <code>StepUpRequest</code> structure.
<code>encryptionMetaData</code>	Cryptographic payload required for end-to-end encryption of sensitive fields. Format: Alphabetic, numeric; maximum 512 characters.
<code>ODADData</code>	<code>ODADData</code> will be returned if the token can be used for Offline Data Authorization (ODA). If <code>ODADData</code> is returned, then all fields in the <code>ODADData</code> structure. Format: An <code>ODADData</code> structure.
<code>encTokenRequestorClientInformation</code>	Visa to populate and return in case of push provisioning. The content of this field is the encrypted client information object; BLOB (url safe, base64-encoded). This object is created and encrypted by Visa and should be returned to the user by the token requestor when token provisioning concludes, as per the VCEH protocol. Format: JSON Web Encryption using the shared secret. Contains <code>ClientInformation</code> structure; size 8192.
<code>cardMetaData</code>	Consumer contact information, label foreground and background colors, terms and conditions, etc. Format: A <code>cardMetaData</code> structure.

Coresidence

Field	Description
cardImageMode	Identifies the card image usage for the alternate networks available to the wallet provider. Format: It is one of the following values: • <code>SHARED</code>—Single co-residence CardArt image provided by one of the networks. The wallet provider will use this image (card Data element having digitalCardArt as the contentType) across all co-residence networks. • <code>PER_NETWORK</code>—A CardArt image for co-residence per network. Wallet will use the CardArt image provided by the respective network.
alternateNetworks	Array of available alternate networks. Format: An array of <code>alternateNetworks</code> structures.

Coresidence Alternate Networks

Field	Description
name	Alternate network name. Format: Alphabetic, numeric; maximum 30 characters.
id	EMV® Sepcification RID, a <i>registered application provider identifier</i> (RID) which is issued by the ISO/IEC 7816–5 registration authority. Format: Alphabetic, numeric; maximum 30 characters.

Card Metadata

Field	Description
backgroundColor	Background color of the mobile wallet UI entry (“space”) for the card. Specified as CSS style hex format, e.g., 0x0e19d2. Format: CSS style hex format, e.g. 0x0e19d2; maximum 32 characters.
foregroundColor	Foreground color of the mobile wallet entry for the card. Specified as CSS style hex format, e.g., 0x0e19d2. Format: CSS style hex format. E.g. 0x0e19d2; maximum 32 characters.
labelColor	Label color of the mobile wallet entry for the card. Specified as CSS style hex format, e.g. 0x0e19d2. Format: Alphabetic, numeric; maximum 32 characters.

<code>contactWebsite</code>	Customer service website of the issuing bank. It is recommended that this be a specific landing page created for cards that are provisioned into a specific wallet program. Format: Alphabetic, numeric; maximum 256 characters. Note: This field will support internationalization using UTF-8 characters.
<code>contactEmail</code>	Customer service email address of the issuing bank. Format: CSS style hex format, e.g. 0x0e19d2; maximum 32 characters. Note: This field will support internationalization using UTF-8 characters.
<code>contactNumber</code>	Customer service phone number of the issuing bank. Format: Alphabetic, numeric.
<code>contactName</code>	Name of the issuing bank. It may be used in combination with other text strings, e.g. if <code>contactName</code> is “XYZ Bank”, it may be used to construct a string “Contact XYZ Bank” or “XYZ Bank Contact Information”. Format: Alphabetic, numeric, and spaces (); maximum 32 characters. Note: This field will support internationalization using UTF-8 characters.
<code>bankAppName</code>	Name of banking application for display. Format: Alphabetic, numeric; maximum 64 characters. Note: This field will support internationalization using UTF-8 characters.
<code>bankAppAddress</code>	The package name for the destination mobile application. Example: <code>com.sampleissuer.thebankapp</code> Format: Alphabetic, numeric, and periods (.); maximum 1024 characters.
<code>privacyPolicyURL</code>	URL pointing to issuing bank privacy policy. This refers to the issuer website. Format: Valid URL; maximum 128 characters.
<code>termsAndConditionsURL</code>	URL pointing to issuing bank Terms and Conditions for card. Format: Valid URL; maximum 128 characters.
<code>termsAndConditionsID</code>	Terms and Conditions identifier in GUID format. If the GUID is changed from the earlier version, the consumer must be presented a new Terms and Conditions at time of provisioning. Format: Alphabetic, numeric; maximum 32 characters.

shortDescription	A short description of the card to be used for notifications on the mobile device (on the lock screen), e.g. The Bank Card. Format: Alphabetic, numeric; maximum 32 characters. Note: This field will support internationalization using UTF-8 characters.
longDescription	A longer description of the card that could appear inside the mobile wallet e.g., "The Bank Card Platinum Rewards". The shortDescription will be used if longDescription is not specified. Format: Alphabetic, numeric; maximum 64 characters. Note: This field will support internationalization using UTF-8 characters.
unsupportedPresentationTypes	The presentation types that are not supported such as "MSR" (for example). This is an advisory field to tell the wallet provider that they should not include the unsupported presentation type returned in this field in the provisioning request. Note: • If the issuer is not enrolled for HCE, this field will contain: – NFC-HCE – INAPP – MERCHANT_PRESENTED_QR • If the issuer has opted out of ECOM, this field will contain ECOM presentation types. • If the issuer has opted out of ESE, this field will contain NFC-SE presentation types. Format: An array; maximum 256 characters.
unsupportedCardVerificationTypes	The card verification types that are not supported such as "AVS" (for example). This is an advisory field to let the wallet provider know which card verification services are not supported for a given payment instrument. The wallet provider can use this information to relax any field validations on the Customer UI (such as Address). Format: An array; maximum 256 characters.
cardData	Card data. Format: An array of structures.
issuerFlags	Issuer flags. Format: An issuerFlags structure.

Issuer Flags

Field	Description
deviceBinding	Device binding. Format: It is one of the following values: • true • false
cardholderVerification	Whether the issuer participating in step-up flow. Format: It is one of the following values: • true • false
trustedBeneficiaryEnrollment	Whether this is a trusted beneficiary enrollment. Format: It is one of the following values: • true • false
delegatedAuthenticationSupported	Whether issuer supports delegated authentication. Format: It is one of the following values: • true • false
oboDeviceBinding	Whether issuer supports on behalf of device binding. Format: It is one of the following values: • true • false

Card Data

Field	Description
guid	GUID for the content. Each <code>guid</code> can have an array of content attributes. Content array will carry metadata related to the content. Format: Alphabetic, numeric; maximum 32 characters.

contentType	Type for the content. Format: It is one of the following values: • digitalCardArt • digitalCardArtBackground • cardSymbol • termsAndConditions
content	Format: A content structure.

Payment Instrument

Field	Description
last4	Last 4 digits of the account. Note: If an IBAN was provided in the provisioning request, this is the last 4 digits of the IBAN. Format: String, numeric; maximum 4 digits.
expirationDate	Payment instrument's expiration date. Format: An expirationDate structure.
accountStatus	<i>Future:</i> Is the account closed? To be returned with Get Card Metadata API only. Format: It is one of the following values: • Y – Account closed • N – Account not closed
isTokenizable	<i>Future:</i> Is this account eligible for tokenization? To be returned with Get Card Metadata API only. Format: It is one of the following values: • Y – Eligible • N – Not Eligible
indicators	These are indicators associated with the Payment Instrument. If the field is not returned, or empty, then there are no indicators associated with it. Expected values are: • PRIVATE_LABEL - This Payment Instrument is for a Private Label PAN. Format: String

Field	Description
expDatePrintedInd	Indicates if the expiration date is printed on the payment instrument. Format: It is one of the following String values: • Y - Printed • N - Not printed
cvv2PrintedInd	Indicates if the CVV2 is printed on the payment instrument. Format: It is one of the following String values: • Y - Printed • N - Not printed
paymentAccountReference	Payment Account Reference (PAR) if available for this payment instrument. Format: String, alphabetic or numeric; maximum 29 characters.
enabledServices	Enabled services, such as scan-and-pay. Format: An <code>enabledServices</code> structure.
verificationResults	Payment instrument verification results. Present only for <code>ECOM</code> presentation type when the CVV2 and/or address fields appear in the provision request. Format: A <code>verificationResults</code> object.

Expiration Date

Field	Description
month	The month upon which the payment instrument is set to expire. Example: "month": "9" Format: String; numeric; max length 2 digits.
year	The year upon which the payment instrument is set to expire. Example: "year": "2024" Format: String; numeric; length 4 digits.

Enabled Services

Field	Description
merchantPresentedQR	Whether the issuer enabled the PAN for the scan-and-pay use case. Format: It is one of the following String values: • Y – Enabled • N – Disabled

Verification Results

Field	Description
cvv2VerificationCode	The CVV2 verification result. This field is present when CVV2 verification data is present in the request. Format: It is one of the following values: • M – verified, CVV2 data match • N – verified, negative data match • U – verification not possible (issuer does not participate) • D – verification not performed due to duplicate request
addressVerificationCode	The address verification result. This field is present when address verification data is present in the request. Format: It is one of the following values: • Y – verified, address and postal code data match • P – verified, partial match (either address or postal code data match, but no result from the other) • B – verified, partial match (either address or postal code data match, but the other could not be verified due to incompatible format) • N – verified, neither address nor postal code data match • U – verification not possible (issuer does not participate) • D – verification not performed due to duplicate request

Token Info

Field	Description
encTokenInfo	Encrypted token payload. For more information, see "JSON Web Encryption and Object Signing." This field is not returned for NFC-SE presentation types. Note: The unencrypted field contains a token as a 13-16 character string. Format: Alphabetic, numeric; maximum 4096 characters.
tokenRequestorID	Unique ID that identifies the entity for which the token request is being conducted. Format: Numeric; maximum 11 digits.
tokenReferenceID	Unique ID associated with a single token. The ID is created upon initial provisioning of a token to a device. Format: Alphabetic, numeric; maximum 32 characters.
tokenStatus	Token status. If a token was deleted, no information will be returned for the token. Format: It is one of the following values: • INACTIVE • ACTIVE • SUSPENDED
last4	Last 4 digits of the token. Format: Numeric; maximum 4 digits.
expirationDate	Token expiration date in UTC format. Format: An <code>expirationDate</code> structure.

ODA Data

This data will only be returned for NFC-HCE use cases. ODA data for NFC-SE will be part of the `seCardPerso` script. Will be returned if the token is enabled for Offline Data Authorization (ODA).

Field	Description
appFileLocator	Application File Locator. Format: Alphabetic, numeric; maximum 10 characters.
appProfile	Application Interchange Profile. Format: Alphabetic, numeric; maximum 5 characters.

caPubKeyIndex	Certificate Authority Public Key Index. Index to identify the payment system public key used for dynamic data authentication (for ODA). Format: Alphabetic, numeric; maximum 3 characters.
iccPubKeyCert	Integrated Circuit Card (ICC) Public Key Certificate.
tokenBinPubKeyCert	Token-BIN Public Key certificate. This is also known as Issuer Public Key Certificate.
enciccPrivateKey	JWE/JWS formatted encrypted data. Format: For more information, see "JSON Web Encryption and Object Signing."

Get Async Token Provisioning Results Errors

This API returns Standard Errors.

Get Async Token Provisioning Example

Request

Selected Headers:

```
x-pay-token: xv2:155...3c
x-request-id: muj-1554325495
```

Endpoint:

```
https://cert.api.visa.com/vts/asyncprovision/tasks/000...602/result?
apikey=2Y2...Ds
```

Request Body

The request does not have a body.

Response

Response Body:

```
{
  "taskData": {
    "taskProvisionData": {
      "vPanEnrollmentID": "35f...01",
      "vProvisionedTokenID": "cb4...01",
      "paymentInstrument": {
        "expirationDate": {
          "month": "..",
          "year": "...."
        },
        "last4": "....",
        "cvv2PrintedInd": "Y",
        "expDatePrintedInd": "Y",
        "paymentAccountReference": "V00...26",
        "enabledServices": {
          "merchantPresentedQR": "N"
        }
      },
      "tokenInfo": {
        "encTokenInfo": "eyJ...KA",
        "tokenRequestorID": "400...D3",
        "tokenReferenceID": "DNI...D2",
        "tokenStatus": "ACTIVE",
        "last4": "...",
        "expirationDate": {
          "month": "..",
          "year": "...."
        }
      }
    },
    "cardMetaData": {
      "backgroundColor": "0xc40d3c",
      "foregroundColor": "0xffffffff",
      "labelColor": "0xffffffff",
      "contactWebsite": "https://www.....php",
      "contactNumber": "1-888-888-1234",
      "contactName": "Test ...",
      "shortDescription": "... Debit Card",
      "longDescription": "... Debit Card",
      "cardData": [
        {
          "guid": "aee96f492c...5d94fce",
          "contentType": "cardSymbol",
          "content": [
            {
              "mimeType": "image/png",
              "width": "100",
              "height": "100"
            }
          ]
        }
      ],
      {
        "guid": "423768cbb05...8fad2b6",
        "contentType": "digitalCardArt",

```

```
        "content": [  
            {  
                "mimeType": "image/png",  
                "width": "1536",  
                "height": "969"  
            }  
        ]  
    },  
    ],  
    "bankAppName": "...",  
    "bankAppAddress": "com.....android",  
    "issuerFlags": {  
        "deviceBinding": true,  
        "cardholderVerification": true,  
        "trustedBeneficiaryEnrollment": false,  
        "delegatedAuthenticationSupported": true  
    }  
}  
}  
}
```

Chapter 32

Submit Job

Submit Job Introduction

This API creates a job.

Submit Job Request

Submit Job Endpoint

Resource Path

`vts/jobs`

Sandbox Path

`https://cert.api.visa.com/vts/jobs?apiKey=key&relationshipID=id`

Live Path

`https://api.visa.com/vts/jobs?apiKey=key&relationshipID=id`

Methods

POST

Required Headers

Header	Description
<code>x-request-id</code>	(Required) Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
<code>x-pay-token</code>	(Required) A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients. Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists.

Header	Description
	Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code>
Accept	where <i>shared_secret</i> is the private key value associated with your API key, which is available in your Visa Developer Center account, <i>resource_path</i> is the API name / URI, <i>query_string</i> contains query parameters, such as <i>apiKey=key</i>, and <i>request_body</i> is a JSON structure representing the request without extra whitespace. (Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
relationshipID	(Optional) The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

Field	Description
jobType	(Required) Hardcode to VTS_BULK_PROCESSING Format: Alphabetic, numeric.
jobSubType	(Required) Hardcode to PROVISION_TOKEN_WITH_PAN Format: Alphabetic, numeric.
clientRequestId	(Required) An ID generated by the client. it is used to uniquely identify a client request for a particular job type and job sub type. Format: Alphabetic, numeric; between 16-150 characters.

Submit Job Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
id	The job ID assigned by VTS, which you specify as a query parameter in subsequent API calls for this job. Format: String; alphanumeric.

Submit Job Errors

This API returns Standard Errors.

Submit Job Examples

Request

Selected Headers:

```
"x-pay-token": "xv2:1615398425:cba...ea"
```

Endpoint:

```
GET https://cert.api.visa.com/vts/jobs?apikey=ASV...fg
```

Request Body

```
{
  "jobType": "VTS_BULK_PROCESSING",
  "jobSubType": "PROVISION_TOKEN_WITH_PAN",
  "clientRequestId": "Xly...gI"
}
```

Response

Response Body:

```
{
  "id": "8f8...01"
}
```


Chapter 33

Get Job Status

Get Job Status Introduction

This API gets the status of a previously submitted job.

Get Job Status Request

Get Job Status Endpoint

Resource Path

`vts/jobs/{jobId}/status`

Sandbox Path

`https://cert.api.visa.com/vts/jobs/{jobId}/status?
apiKey=key&relationshipID=id`

Live Path

`https://api.visa.com/vts/jobs/{jobId}/status?apiKey=key&relationshipID=id`

URL Parameters

Parameter	Description
jobId	<i>(Required)</i> The Job ID assigned by VTS. Format: String; alphabetic, numeric, and hyphens (-)

Method

GET

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	<i>(Required)</i> Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
relationshipID	<i>(Optional)</i> The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

None. The request does not have a body.

Get Job Status Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
status	The execution status of the job. Format: It is one of the following values: • ACCEPT—The job was accepted. • PROGRESS—The job execution is in progress. • COMPLETE—The job has successfully completed its execution. You can call the Get Job Results API to get the result of the execution • FAIL—The job execution has failed. You can call the Get Job Results API to get the failure details. Note: Any other status should be interpreted as PROGRESS.

Get Job Status Errors

This API returns Standard Errors.

Get Job Status Examples

Request

Selected Headers:

```
"x-pay-token": "xv2:1615420033:bf8...ae"
```

Endpoint:

```
GET https://cert.api.visa.com/vts/jobs/8f8...01/status?apikey=ASV...fg
```

Request Body

The request does not have a body.

Response

Response Body:

```
{  
  "status": "COMPLETE"  
}
```

Chapter 34

Upload Job Data

Upload Job Data Introduction

This API is used to upload job data. The data size for a simple upload is limited to 5 MB.

Upload Job Data Request

Endpoint Upload Job Data

Resource Path

`vts/jobs/{jobId}/data`

Sandbox Path

`https://cert.api.visa.com/vts/jobs/{jobId}/data?
apiKey=key&relationshipID=id`

Live Path

`https://api.visa.com/vts/jobs/{jobId}/data?apiKey=key&relationshipID=id`

URL Parameters

Parameter	Description
jobId	(Required) The Job ID assigned by VTS. Format: String; alphabetic, numeric, and hyphens (-)

Methods

PUT

Required Headers

Header	Description
x-request-id	(Required) Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	(Required) A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
relationshipID	(Optional) The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

Field	Description
provisioningData	(Required) Provisioning data. Format: A Provisioning Data array; maximum 5MB.
rowCount	(Optional) The number of provision requests. Format: Numeric.
clientAppID	(Conditional) A unique identifier for the client application. Note: This field is required unless it is being transferred via Submit Job. Format: Alphabetic, numeric, and hyphens (-), e. g. spaces are not allowed; maximum 36 characters.
relationshipID	(Conditional) The ID identifying the relationship between a token requestor and the caller of this API; required if the caller is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Provisioning Data

Field	Description
rowId	(Required) This is unique per job. Format: An identifier; maximum characters 36.
clientAppID	(Conditional) A unique identifier for the client application. Note: This field is required unless it is being transferred via Submit Job. Format: Alphabetic, numeric, and hyphens (-), e. g. spaces are not allowed; maximum 36 characters.
clientWalletAccountID	(Required) Client-provided consumer ID that identifies the wallet account holder entity. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 24 characters
location	(Optional) The location of the device. Information is given with latitude and longitude. Format: It is this structure: +xxx.xxxx/-xxx.xxxx Note: Precision is rounded to the nearest whole digit; for example, +37/-121. Maximum 25 characters.
clientWalletAccountEmailAddressHash	(Required) Hash of email address linked to their wallet account login. Hash algorithm to be provided by Visa. base64 URL-encoded. Format: Base64 encoded string, per RFC4648, with the following exceptions: hyphens (-) replace plus signs (+) and underscores (_) replace forward slash (/) characters; maximum 48 characters.
protectionType	(Optional) Describes the token credential protection on the device. Format: It is one of the following values: • SOFTWARE • TEE • SE • CLOUD - default
presentationType	(Optional) Token presentation mode. Format: ECOM is the only accepted presentation type for Upload Job Data Note: Shared Secret is always used to encrypt the API response fields when the presentation type is ECOM.

Field	Description
accountType	(Conditional) An attribute that describes the type of account. Applicable in e-commerce use cases. Format: It is one of the following values: • GUEST • WALLET
locale	(Required) Language in which the app communicates with the customer. Based on ISO format for language (ISO 639-1) and alpha 2 country code (ISO 3166-1 alpha-2). The language and country should be separated using an underscore (_). e.g. en_US. Format: Alphabetic; underscores (_), e.g. spaces are not allowed; maximum 5 characters.
panSource	(Required) Source of the payment instrument (PAN). This enum value specifies the source and the method used to provide the payment instrument data. Format: It is one of the following values: • ONFILE – The wallet provider already has the PAN data for the customer in their records. • MANUALLYENTERED – The customer is expected to type the card number or capture a picture of the card. • ISSUER_PUSH_PROVISION – The PAN data was provided by the issuer of the PAN. • TOKEN_FOR_TOKEN – Token was used for source of the PAN data.
encPaymentInstrument	(Required) BLOB field containing all payment instrument details (URL-safe base64 encoded) that represents the encrypted payload for a payment instrument. Format: JSON Web Encrypted string using the shared secret. For more information, see "JSON Web Encryption and Object Signing." String Size 8192.

Encrypted Payment Instrument

Field	Description
accountNumber	(Conditional) PAN of the card to be enrolled and provisioned. This field is required if an IBAN is not provided. Format: Numeric; 13-19 digits.
iban	(Conditional) This field is the IBAN of the card to be enrolled and provisioned. This field is required if a PAN is not provided. Format: Alphabetic, numeric; maximum 34 characters

Field	Description
token	(Conditional) EMV Co Token number. This is required if there is no <code>accountNumber</code> or <code>iban</code>. Format: Alphabetic, numeric; 12–19 characters..
cvv2	(Optional) CVV2 value associated with the PAN on the card. Note: For CVV2 to be considered during token provisioning, the CVV2 must also be provided within the <code>encRiskDataInfo</code> structure of the Provision Token request. Format: Numeric; 3-4 digits.
name	(Conditional) The full name on the Visa card associated with the enrolled payment instrument. This field is required if: • IBAN is provided AND no <code>sessionIdentifier</code> is provided for push provisioning. • IBAN is provided for normal pull provisioning This field is optional if a PAN is provided. Format: Alphanumeric, including spaces; maximum 256 characters.
expirationDate	(Conditional) Payment instrument's expiration date. This field is required unless the expiration date is not printed on the card. Format: An <code>expirationDate</code> structure.
billingAddress	(Optional) Billing address associated with the payment instrument. Format: A <code>billingAddress</code> structure.
provider	(Conditional) Information about the provider of the payment instrument and the context in which it is provided. This field is required if <code>panSource</code> equals <code>ISSUER_PUSH_PROVISION</code>. Format: A <code>provider</code> structure.

Upload Job Data Response

HTTP Response

HTTP Response	Reason Code	Description
200	Success	Data sent in this request was ignored – previously uploaded
202	Accepted	The server will generate 202 to indicate it has accepted the data.
400	Bad Request, <code>incorrectTransmissionType</code>	The data transmission field specified as part of job creation does not match the actual one used as part of the Upload Job Data API call.
400	Bad Request, <code>dataAlreadyUploaded</code>	This error occurs after the server has received the complete set of data. From this point on, any request from the client to upload the data again, or part of the data, will be considered an error.
404	<code>notFound</code>	Indicates that this job cannot be used any more. The upload timeout has been reached. The <code>jobId</code> is invalid.

Response Body

The response does not have a body.

Upload Job Data Errors

This API returns Standard Errors.

Upload Job Data Examples

Request

Selected Headers:

```
"Accept": "application/json"
"x-pay-token": xv2:1615398589:853...e3
```

Endpoint:

```
PUT https://cert.visa.com/vts/jobs/8f8...01/data?apikey=ASV...fg
```

Request Body

```
{
  "rowCount": 1,
  "clientAppID": "TestTRTSP",
  "relationshipID": "00123456-40010081022",
  "provisioningData": [
    {
      "clientWalletAccountID": "400...22",
      "clientWalletAccountEmailAddressHash": "VW6..._4",
      "protectionType": "CLOUD",
      "accountType": "WALLET",
      "locale": "en_US",
      "encPaymentInstrument": "eyJ...Dg",
      "panSource": "ONFILE",
      "presentationType": [
        "ECOM"
      ],
      "rowId": "0"
    }
  ]
}
```

Response

Response Body:

The response does not have a body.

Chapter 35

Get Job Results

Get Job Results Introduction

This API is used to get the results of the completed job.

Get Job Results Request

Endpoint Get Job Results

Resource Path

`vts/jobs/{jobId}/result`

Sandbox Path

`https://cert.api.visa.com/vts/jobs/{jobId}/result?
apiKey=key&relationshipID=id`

Live Path

`https://api.visa.com/vts/jobs/{jobId}/result?apiKey=key&relationshipID=id`

URL Parameters

Parameter	Description
jobId	<i>(Required)</i> The Job ID assigned by VTS. Format: String; alphabetic, numeric, and hyphens (-)

Method

GET

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	<i>(Required)</i> Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
relationshipID	<i>(Optional)</i> The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

None. The request does not have a body.

Get Job Results Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
header	A <code>TokenProvisioningBatchJobSummary</code> required object.
details	It is an array of <code>TokenProvisioningBatchJobDetails</code> required. Format: Maximum size: 5,000 bytes. Note: Actual array size will be smaller.

Token Provisioning Batch Job Summary

Field	Description
successCount	The number of successful token provisioning(s). Format: Numeric.
failureCount	The number of failed token provisioning(s). Format: Numeric.
correlationID	The Correlation ID. Format: String. Alphabetic, numeric; maximum 36 characters.
relationshipID	If applicable, the ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Token Provisioning Batch Job Details

Field	Description
vPanEnrollmentID	This is a unique ID for an enrolled PAN. Another provision token API call on the same PAN is not guaranteed to return the same vPanEnrollmentID. Note: This is the only parameter returned as this API response, compare to Provision Token—Given PAN Enrollment ID API response. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
vProvisionedTokenID	Unique ID for token. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.
paymentInstrument	Summary of the Payment Instrument. Format: A paymentInstrument structure.
tokenInfo	Token information. Format: A tokenInfo structure.
rowId	This row ID is unique per completed job. Format: Identifier; maximum 36 characters.
x-req-id	This is an internal field. Format: Identifier; between 1–36 characters.

Payment Instrument

Field	Description
last4	Last 4 digits of the account. Note: If an IBAN was provided in the provisioning request, this is the last 4 digits of the IBAN. Format: String, numeric; maximum 4 digits.
expirationDate	Payment instrument's expiration date. Format: An expirationDate structure.
accountStatus	<i>Future:</i> Is the account closed? To be returned with Get Card Metadata API only. Format: It is one of the following values: - Y – Account closed - N – Account not closed

Field	Description
isTokenizable	<i>Future:</i> Is this account eligible for tokenization? To be returned with Get Card Metadata API only. Format: It is one of the following values: • Y – Eligible • N – Not Eligible
indicators	These are indicators associated with the Payment Instrument. If the field is not returned, or empty, then there are no indicators associated with it. Expected values are: • PRIVATE_LABEL - This Payment Instrument is for a Private Label PAN. Format: String
expDatePrintedInd	Indicates if the expiration date is printed on the payment instrument. Format: It is one of the following String values: • Y - Printed • N - Not printed
cvv2PrintedInd	Indicates if the CVV2 is printed on the payment instrument. Format: It is one of the following String values: • Y - Printed • N - Not printed
paymentAccountReference	Payment Account Reference (PAR) if available for this payment instrument. Format: String, alphabetic or numeric; maximum 29 characters.
enabledServices	Enabled services, such as scan-and-pay. Format: An <code>enabledServices</code> structure.
verificationResults	Payment instrument verification results. Present only for <code>ECOM</code> presentation type when the CVV2 and/or address fields appear in the provision request. Format: A <code>verificationResults</code> object.

Expiration Date

Field	Description
month	The month upon which the payment instrument is set to expire. Example: "month": "9" Format: String; numeric; max length 2 digits.
year	The year upon which the payment instrument is set to expire. Example: "year": "2024" Format: String; numeric; length 4 digits.

Enabled Services

Field	Description
merchantPresentedQR	Whether the issuer enabled the PAN for the scan-and-pay use case. Format: It is one of the following String values: • Y – Enabled • N – Disabled

Token Info

Field	Description
encTokenInfo	Encrypted token payload. For more information, see "JSON Web Encryption and Object Signing." This field is not returned for NFC-SE presentation types. Note: The unencrypted field contains a token as a 13-16 character string. Format: Alphabetic, numeric; maximum 4096 characters.
tokenRequestorID	Unique ID that identifies the entity for which the token request is being conducted. Format: Numeric; maximum 11 digits.
tokenReferenceID	Unique ID associated with a single token. The ID is created upon initial provisioning of a token to a device. Format: Alphabetic, numeric; maximum 32 characters.

Field	Description
tokenStatus	Token status. If a token was deleted, no information will be returned for the token. Format: It is one of the following values: • INACTIVE • ACTIVE • SUSPENDED
last4	Last 4 digits of the token. Format: Numeric; maximum 4 digits.
expirationDate	Token expiration date in UTC format. Format: An <code>expirationDate</code> structure.

Expiration Date

Field	Description
month	The month upon which the payment instrument is set to expire. Example: "month": "9" Format: String; numeric; max length 2 digits.
year	The year upon which the payment instrument is set to expire. Example: "year": "2024" Format: String; numeric; length 4 digits.

Get Job Results Errors

This API returns Standard Errors.

Get Job Results Example

Request

Selected Headers:

```
"x-request-id": "gvC...yB"  
"Accept": "application/json"
```


Endpoint:

```
https://cert.visa.com/vts/jobs/8f8...01/result?apikey=ASV...fg
```

Request Body

The request does not have a body.

Response**Response Body:**

```
{
  "header": {
    "successCount": 3,
    "failureCount": 0,
    "correlationID": "153...RM"
  },
  "details": [
    {
      "vProvisionedTokenID": "ala...01",
      "vPanEnrollmentID": "5af...01",
      "ignore00field": "2D335699",
      "paymentInstrument": {
        "expirationDate": {
          "year": "2020",
          "month": "12"
        },
        "verificationResults": {
          "cvv2VerificationCode": "D",
          "addressVerificationCode": "D"
        },
        "last4": "7202",
        "cvv2PrintedInd": "Y",
        "ignore01field": "51D6132A",
        "expDatePrintedInd": "Y",
        "paymentAccountReference": "V00...09",
        "enabledServices": {
          "merchantPresentedQR": "N"
        }
      },
      "tokenInfo": {
        "encTokenInfo": "eyJ...kg",
        "tokenRequestorID": "400...44",
        "tokenReferenceID": "DNI...269",
        "tokenStatus": "ACTIVE",
        "last4": "0006",
        "expirationDate": {
          "month": "12",
          "year": "2020"
        }
      },
      "exchangeID": "15..._26",
      "rowId": "1"
    },
    {

```

```

    "vProvisionedTokenID": "471...01",
    "vPanEnrollmentID": "b77...01",
    "ignore00field": "50E...A2",
    "paymentInstrument": {
      "expirationDate": {
        "year": "2020",
        "month": "12"
      },
      "verificationResults": {
        "cvv2VerificationCode": "D",
        "addressVerificationCode": "D"
      },
      "last4": "9346",
      "cvv2PrintedInd": "Y",
      "ignore01field": "3926F1E9",
      "expDatePrintedInd": "Y",
      "paymentAccountReference": "V00...54",
      "enabledServices": {
        "merchantPresentedQR": "N"
      }
    },
    "tokenInfo": {
      "encTokenInfo": "eyJ...1w",
      "tokenRequestorID": "400...44",
      "tokenReferenceID": "DNI...36",
      "tokenStatus": "ACTIVE",
      "last4": "0002",
      "expirationDate": {
        "month": "12",
        "year": "2020"
      }
    },
    "exchangeID": "1537245533441_28",
    "rowId": "2"
  },
  {
    "vProvisionedTokenID": "cec...01",
    "vPanEnrollmentID": "8dd...01",
    "ignore00field": "9E923D92",
    "paymentInstrument": {
      "expirationDate": {
        "year": "2020",
        "month": "12"
      },
      "verificationResults": {
        "cvv2VerificationCode": "D",
        "addressVerificationCode": "D"
      },
      "last4": "9520",
      "cvv2PrintedInd": "Y",
      "ignore01field": "23E3318D",
      "expDatePrintedInd": "Y",
      "paymentAccountReference": "V00...37",
      "enabledServices": {
        "merchantPresentedQR": "N"
      }
    },
    "tokenInfo": {

```

```
    "encTokenInfo": "eyJ...ZA",
    "tokenRequestorID": "400...44",
    "tokenReferenceID": "DNI...86",
    "tokenStatus": "ACTIVE",
    "last4": "0005",
    "expirationDate": {
      "month": "12",
      "year": "2020"
    },
    "exchangeID": "15..._29",
    "rowId": "3"
  }
]
```

Chapter 36

Request Device Token Binding

Request Device Token Binding Introduction

This API is used to initiate the Device Binding request.

Request Device Token Binding Request

Endpoint

Resource Path

`vts/provisionedTokens/{vProvisionedTokenID}/deviceBinding`

Sandbox Path

```
https://cert.api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}/  
deviceBinding?apiKey=key&relationshipID=id
```

Live Path

```
https://api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}/  
deviceBinding?apiKey=key&relationshipID=id
```

URL Parameters

Parameter	Description
<code>{vProvisionedTokenID}</code>	<i>(Required)</i> Unique ID for the provision being confirmed. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.

Methods

POST

Required Headers

Header	Description
<code>x-request-id</code>	<i>(Required)</i> Unique ID for the API request.

Header	Description
	Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	(Required) A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients. Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: - Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> - Resource path (API name). - This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. - Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query

Header	Description
	parameters, such as <code>apiKey=key</code> , and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
relationshipID	(Optional) The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

Field	Description
platformType	(Conditional) The step-up option app-to-app details in the response is based on the platform type. This field is required for mobile devices to support app-to-app. Format: It is one of the following values: • iOS – iOS app • ANDROID – Android app • WINDOWS – Windows app • WEB – Browser-based app
sigBindingRequest	(Conditional) JWS, which is required for all cases except autofill. It contains the following fields: 1. The header of the JWS has these elements: • "alg": "..."—Algorithm used for signature • "kid": "1234"—Token requestors's client device ID • "typ": "JOSE" • "cty": "application/json"—Content type of the content inside JWS • "iat": "147...38"—Issued at the timestamp in UTC when the transaction was created and signed. 2. Construct the body as JSON consisting of these: • clientDeviceID • clientReferenceID • vProvisionedTokenID • nonce 3. The Signature. It is always signed with primary. When the signature is provided, use this algorithm: RSA-PSS with SHA-256 as the hash and the MGF. Set "algo" to "PS256" Format: Binding Request.
merchantIdentifier	(Conditional) The Visa merchant identifier. This field is required if <code>delegatedAuthenticationRequested</code> equals <code>true</code>. Format: A <code>merchantIdentifier</code> structure.

Field	Description
encDeviceRiskDataInfo	(Optional) JWE/JWS to be used to convey data related to risk decision. Actual risk data elements will be shared separately outside of the technical specification. Note: The maximum number of name/value pairs allowed = 100. The <code>riskData</code> structure will be encrypted using Visa public key and sign using Device Key. Format: String. Maximum 10240 characters. If the binding request is submitted without signature, it must be encrypted using the shared secret. For more information, see "JSON Web Encryption and Object Signing."
clientWalletAccountEmail AddressHash	(Required) Hash of email address linked to their wallet account login. Hash algorithm to be provided by Visa. base64 URL-encoded. Format: Base64 encoded string, per RFC4648, with the following exceptions: hyphens (-) replace plus signs (+) and underscores (_) replace forward slash (/) characters; maximum 48 characters.
encBillingInfo	(Optional) Encrypted billing information blob. Encrypted by JSON Web Encryption using the shared secret. For more information, see "JSON Web Encryption and Object Signing." Format: String. Maximum 4000 characters
clientDeviceID	(Conditional) Device identifier. Required if <code>sigBindingRequest</code> is not specified for autofill. This field is required when it is returned to the issuer by the token requestor. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 24 characters.
clientReferenceID	(Conditional) Unique ID for binding. Required if <code>sigBindingRequest</code> is not specified for autofill. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters
suppressStepUp	(Optional) Whether the Token Requestor requests that step up be suppressed when the Issuer enabled this feature for a given Token Requestor. Format: It is one of the following values: • true • false (default)
clientAppID	(Conditional) Unique identifier for the client application; required when the <code>relationshipID</code> query parameter is specified. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.

Merchant Identifier

For the `merchantIdentifier` structure, only the `aid` and `mid` values need be provided together. When `delegatedAuthenticationRequested` is `true`, you must provide at least one of `externalClientId`, `applicationUrl`, or `merchantName`; you can provide any combination of them.

Value	Description
<code>externalClientId</code>	<i>(Conditional)</i> The Merchant ID assigned by the client. Format: Base64URL-encoded; maximum size 96.
<code>aid</code>	<i>(Conditional)</i> The existing acquirer ID sent in combination with the merchant ID. If the <code>aid</code> is provided, the <code>mid</code> must also be provided. The two values in combination identify the merchant. Format: Alphanumeric; maximum size 36.
<code>mid</code>	<i>(Conditional)</i> The merchant ID sent in combination with the acquirer ID. If the <code>mid</code> is provided, the <code>aid</code> must also be provided. The two values in combination identify the merchant. Format: Alphanumeric; maximum size 36.
<code>applicationUrl</code>	<i>(Conditional)</i> The merchant's URL, Application Package ID, or Application Bundle ID. Required for Digital Authentication Framework (DAF) when the token type is HCE and for tokens with the <code>AUTO_FILL</code> presentation type option. Format: Base64URL-encoded string; maximum size 512.
<code>merchantName</code>	<i>(Conditional)</i> The merchant's name. Required for tokens with the <code>AUTO_FILL</code> presentation type option. Base64 encoded UTF8. Format: Base64URL-encoded string; maximum size 2048.

Risk Data

Field	Description
ip4address	(Optional) IP address of the device requesting token provisioning in dot-decimal notation consisting of four decimal numbers, each ranging from 0 to 255, separated by dots. Each part represents a group of 8 bits (an octet) of the address. Example: "ip4address": "89.24.143.232" Format: Numeric and periods (.); maximum 15 characters.
location	(Optional) Location of the device with up to 4-digits of precision when it is being provisioned. Information will be supplied with latitude/longitude; for example +xxx.xxxx/-xxx.xxxx. Precision may be rounded to the nearest whole digit; for example, +37/-121. Format: Alphanumeric; maximum 25 characters.
locationSource	(Optional) Source used to identify consumer location. Format: It is one of the following values: • WIFI • CELLULAR • GPS • OTHER

Binding Request

Field	Description
clientDeviceID	(Required) Device Identifier. Format: String. Alphabetic, numeric, hyphens (-) and underscores (_), e.g. spaces are not allowed; 24 characters.
clientReferenceID	(Required) Unique ID for binding. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
vProvisionedTokenID	(Required) Unique ID for Provision that is being confirmed. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
nonce	(Required) Nonce. Format: Numeric; maximum 5 digits.

Unencrypted Billing Info

Field	Description
billingAddress	<i>(Optional)</i> Billing address associated with the payment instruments. Format: An <code>address</code> structure.
name	<i>(Optional)</i> The full name on the Visa card. Must use UTF-8 characters. Format: String; minimum 0 and maximum 256 characters.
email	<i>(Optional)</i> Email address. Must use UTF-8 characters. Format: String; maximum 64 characters.
phoneNumber	<i>(Optional)</i> Phone number. Format: String; numeric; a valid phone number; maximum 18 digits.
currencyCode	<i>(Optional)</i> Currency code. Format: String; ISO 4217-format currency code (numeric)

Billing Address

Field	Description
line1	<i>(Optional)</i> Street 1 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line2	<i>(Optional)</i> Street 2 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line3	<i>(Optional)</i> Street 3 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.

line4	(Optional) Street 4 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line5	(Optional) Street 5 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
city	(Optional) The city associated with the enrolled payment instrument. Format: Alphabetic, single quote ('), hyphen (-), and whitespace.; maximum 100 characters.
state	(Optional) State or province code in ISO 3166-2 format, e.g. NY. Format: Alphabetic; maximum 3 characters.
country	(Optional) Country in ISO 3166-1 alpha-2 format, e.g. US. Format: Alphabetic; maximum 2 characters.
countryDigits	(Optional) ISO 3166-1 numeric Country Code. Format: Numeric; maximum 3 digits.
postalCode	(Optional) The postal code associated with the enrolled payment instrument. Note: This field does not support spaces (" ") in the middle of characters. Format: Alphabetic, numeric, e.g. spaces are not allowed; maximum 3–16 characters.

Request Device Token Binding Response

HTTP Response

HTTP Response	Description
201	Created

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
<code>stepUpRequest</code>	Returns a list of step-up methods. In some business cases, it is possible to receive an empty array.
<code>status</code>	The request result. Format: It is one of the following values: • APPROVED • CHALLENGE • DECLINED

Step-Up Request

`stepUpRequest` is conditional. It is not required for the Ecommerce token type. Items 2, 3, and 4 below are applicable to push provisioning only.

Note: This is only applicable when:

- `TokenStatus` is `INACTIVE`.
- Device ID used during push provisioning does not match the `DeviceID` used during enrollment.
- Issuer indicated a preference for IDnV in the JWE by setting `isIDnV=Y`.
- Push Provisioning expired.

Field	Description
method	ID&V step-up method. Format: It is one of the following values: • OTPEMAIL • OTPONLINEBANKING • OTPSMS • CUSTOMERSERVICE • APP-TO-APP • OUTBOUNDCALL
value	This could be a masked value depending on the ID&V step-up method. Email looks like 'ip****@email.com' bank_app looks like 'Mobile Banking App' customer_service looks like '1-800-555-1212' Note: This field will support internationalization using UTF-8 characters. Format: Alphanumeric, UTF-8 whitespace; maximum 64 characters.
source	The issuer source of contact for this ID&V step-up challenge. In case of OTP, the source contains the issuer contact that will reach out to the consumer. In the case of banking-app step-up, the source is a concatenation of Application ID (<i>id</i>) and Intent Handler (<i>h</i>). This field will be defined by the issuer. Note: Method handler of Banking App designed to respond to authentication code handling. For the Android platform this will be an explicit intent with fully name-spaced class name. Visa requires the issuer to use the a2a name for this intent handler. Example: "source": "{id} {h}", where <i>id</i> is required (size 60, [a-z,A-Z,0-9,_,.]), and <i>h</i> is optional (size 3, [a-z][A-Z][0-9]). Format: Alphanumeric; maximum 1024 characters.
identifier	Unique and opaque identifier for this method. A subsequent step-up request will only provide the identifier. The wallet provider must send this value "as is" to subsequent step-up related APIs. Format: Alphabetic, numeric, plus signs (+), slashes (/), and equal signs (=), spaces are not allowed; maximum 96 characters.

Field	Description
requestPayload	The request payload to be sent to the banking application on behalf of Visa. This field is opaque to wallet providers. Note: This is only present if the step-up type is app-to-app. Format: Base64 encoded string, maximum 1024 characters.
platformType	Platform type, which is present in cases where the Token Requestor has opted in to 3DS Web Step-Up. Format: It is one of the following values: • WEB
subMethod	Flavor of the step-up method. Format: It is one of the following values: • THREE-DS

Request Device Token Binding Errors

This API returns Standard Errors.

Request Device Token Binding Example

Request

Selected Headers:

```
x-pay-token: xv2:1570681853:5c1...22
x-request-id: Test1570681848
```

Endpoint:

```
https://cert.api.visa.com/vts/provisionedTokens/932...02/deviceBinding?
apikey=QHF...ZU
```

Request Body

```
{
  "clientWalletAccountEmailAddressHash": "zmI...Yg",
```



```
"platformType": "ANDROID",  
"sigBindingRequest": "eyJ...Kg"  
}
```

Unencrypted Binding Request (in sigBindingRequest)

```
{  
  "clientDeviceID": "Z_w...gj",  
  "clientReferenceID": "ewr...pA",  
  "vProvisionedTokenID": "d47...01",  
  "nonce": "84039"  
}
```

Response

Selected Headers:

```
x-response-id: Test1570681848
```

Response Body:

```
{  
  "stepUpRequest": [  
    {  
      "method": "OTPSMS",  
      "value": "...",  
      "source": "https://otpsms.com",  
      "identifier": "ZDY...I="    },  
    {  
      "method": "OTPEMAIL",  
      "value": "...",  
      "source": "https://otpemail.com",  
      "identifier": "MjZ...I="    },  
    {  
      "method": "APP-TO-APP",  
      "value": "device Name test",  
      "source": "https://android.com|a2a",  
      "identifier": "ZTB...I="    },  
    {  
      "method" : "APP-TO-APP",  
      "value" : "Verify Online with Bank",  
      "source" : "https://....",  
      "identifier" : "NjZ...=",  
      "requestPayload" : "",  
      "platformType" : "WEB"    },  
    {  
      "method": "OUTBOUNDCALL",  
      "value": "device Name test",  
      "source": "https://outboundCall.com",  
      "identifier": "MWZ...I="    }  
  ]  
}
```

```
] ,  
  "status": "CHALLENGE"  
}
```

Chapter 37

Delete Device Binding

Delete Device Binding Introduction

This API allows wallet providers to delete a device binding.

Delete Device Binding Request

Endpoint

Resource Path

`vts/provisionedTokens/{vProvisionedTokenID}/deviceBinding/{clientDeviceID}`

Sandbox Path

```
https://cert.api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}/  
deviceBinding/{clientDeviceID}?apiKey=key  
&clientAppID=appId&relationshipID=id
```

Live Path

```
https://api.visa.com/vts/provisionedTokens/{vProvisionedTokenID}/  
deviceBinding/{clientDeviceID}?apiKey=key  
&clientAppID=appId&relationshipID=id
```

URL Parameters

Parameter	Description
{vProvisionedTokenID}	<i>(Required)</i> Unique ID for the provision being confirmed. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.
{clientDeviceID}	<i>(Required)</i> Identifier for the device. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 24.

Methods

DELETE

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	<i>(Required)</i> Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
clientAppID	<i>(Conditional)</i> Unique identifier for the client application; required when the <code>relationshipID</code> query parameter is specified. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
relationshipID	<i>(Optional)</i> The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

None. The request does not have a body.

Delete Device Binding Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

The response does not have a body.

Delete Device Binding Errors

This API returns Standard Errors.

Delete Device Binding Example

Request

Selected Headers:

```
x-request-id: YCi...hw  
x-pay-token: xv2:1572997384:135...56
```

Endpoint:

```
https://cert.digital.visa.com/vts/provisionedTokens/642...02/  
deviceBinding/-Z9...H1?apiKey=QHF...ZU
```

Request Body:

None

Response

Selected Headers:

```
x-response-id: YCi...hw
```

Response Body:

```
{ }
```

Chapter 38

Get Nonce

Get Nonce Introduction

Returns a Visa-supplied nonce. Call Get Nonce before calling Enroll PAN for a non-EMVCo-certified terminal when using either the `CHIP_DIP` or `CONTACTLESS_TAP` PAN source.

Get Nonce Request

Endpoints

Resource Path

`vts/nonce`

Sandbox Path

`https://cert.api.visa.com/vts/nonce?apikey=key&relationshipID=id`

Live Path

`https://api.visa.com/vts/nonce?apikey=key&relationshipID=id`

Methods

POST

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as: <code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
<code>apiKey</code>	<i>(Required)</i> Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
<code>relationshipID</code>	<i>(Optional)</i> The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

Field	Description
<code>clientAppID</code>	<i>(Required)</i> Unique identifier for the client application. It is obtained from the wallet provider. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.

Get Nonce Response

HTTP Response

HTTP Response	Description
200	Success

HTTP Response Header

An `x-response-id` header that contains the unique ID for the API response.

Response Body

Field	Description
sigNonce	A signed payload blob that contains the nonce. Format: JWS signed by Visa over the blob, which is a Base64URL encoded string, per RFC4648. The encoding is the same as Base64, but uses '-' character instead of '+' and '_' character instead of '/' character [A-Za-z0-9=-_]+; maximum length 1024 characters

Signed Payload for Nonce (sigNonce)

Field	Description
nonce	Nonce, which is a random number. Format: String; maximum 8 characters; 4 hexadecimal digits.
clientAppID	Unique Identifier for the client application. It is obtained from the wallet provider. Format: String; UUID, maximum length 36 characters
clientId	Unique ID of the client allowed to use the nonce. Format: String; UUID, maximum length 36 characters

Get Nonce Errors

This API returns Standard Errors.

Get Nonce Example

Response

Response Body:

```
{
  "sigNonce": "eyJ...pw"
}
```

Decoded JWE Body:

```
{  
  "nonce": "3DE52ADB",  
  "clientAppID": "...",  
  "clientId": "69e...01"  
}
```

Chapter 39

Onboarding

About the Onboarding API

A token service provider (TR-TSP) can onboard a token requestor (TR) into VTS by calling the onboarding API.

To onboard a token requestor, the token service provider must call the onboarding API and provide the required information to uniquely identify the token requestor.

Onboarding Request

Path and Endpoints

Resource Path

`vts/merchant-api-ic/onboarding` — Sandbox

`vts/merchant-api/onboarding` — Live

Sandbox

```
https://cert.api.visa.com/vts/merchant-api-ic/onboarding?  
apikey=key&createApiKey=false
```

Live

```
https://api.visa.com/vts/merchant-api/onboarding?  
apikey=key&createApiKey=false
```

Method

POST

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as: <code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
<code>apikey</code>	(Required) Public API key, which is different from the shared secret. Format: Alphanumeric; maximum 49 characters Example: <code>apikey=...</code>
<code>createApiKey</code>	(Required) Whether to create a new API key for the client. Note: Do not set to <code>true</code>. Format: One of the following values: • <code>false</code> - Do not create an API key Example: <code>createApiKey=false</code>

Client Request Body

Field	Description
<code>serviceContext</code>	(Required) Service context. Format: It is one of the following values: • <code>TOKENIZATION</code>
<code>dunsNumber</code>	(Optional) Dun and Bradstreet number. Format: Numeric; 9 decimal digits
<code>companyPrimaryTradeName</code>	(Optional) Trade name of token requestor, such as a doing-business-as (DBA) name. If not specified, the legal name (<code>companyPrimaryLegalName</code>) is used by default. Format: Alphanumeric, excepting semi-colon, percent sign, and parentheses; maximum 75 characters Example: <pre>"companyPrimaryTradeName": "MyMerchant.com"</pre>

Field	Description
companyPrimaryLegalName	(Required) Legal name of token requestor. Some processors may not support a 75-character legal name. You cannot exceed the limit imposed by your processor. Format: Alphanumeric, excepting semi-colon, percent sign, and parentheses; maximum 75 characters Example: <pre>"companyPrimaryLegalName" : "My Merchant"</pre>
primaryWebsiteURL	(Required) The URL for your token requestor's site for handling VTS transactions. Format: Alphanumeric, valid URL; maximum 100 characters Example: <pre>"primaryWebsiteURL" : "http://www.myMerchant.com"</pre>
businessIdentificationType	(Conditional) The Identifier associated with the business type; required unless both <code>acquirerId</code> and <code>acquirerMerchantId</code> are provided. Format: See Business Identification Types and Values for a complete list of business identification Types for all countries. Example: "businessIdentificationType": "EIN"
businessIdentificationValue	(Conditional) The value associated with the business identifier type; required unless both <code>acquirerId</code> and <code>acquirerMerchantId</code> are provided. Format: See Business Identification Types and Values for a complete list of business identification values for all countries Example: "businessIdentificationValue" : "..."
companyAddress1	(Optional) First line of the token requestor's primary address. Format: Alphabetic, numeric, or the following characters: spaces, ' (single quote), # (pound-sign or hash), , (comma), _ (underscore), : (colon), / (forward slash), and - (hyphen); maximum 140 characters Example: "companyAddress1" : "1 Main Street"
companyAddress2	(Optional) Second line of the token requestor's primary address. It can be empty. Format: Alphabetic, numeric, or the following characters: spaces, ' (single quote), # (pound-sign or hash), , (comma), _ (underscore), : (colon), / (forward slash), and - (hyphen); maximum 140 characters Example: "companyAddress2" : ""

Field	Description
<code>companyCity</code>	<i>(Required)</i> City in the token requestor's primary address. Format: Alphabetic, numeric, or the following characters: spaces, ' (single quote), . (period), and – (hyphen); maximum 100 characters Example: "companyCity" : "San Francisco"
<code>companyStateProvinceCode</code>	<i>(Optional)</i> State or province code associated with the physical address in the specified country. The <code>companyStateProvinceCode</code> for US must be a standard 2-characters code. Format: The suffix of an ISO-3166-2 country code, corresponding to the <code>companyCountryCode</code> maximum 2 characters Example: "companyStateProvinceCode" : "CA"
<code>companyPostalCode</code>	<i>(Optional)</i> Postal code associated with the primary address of token requestor, such as a ZIP code. Format: Depends on <code>companyStateProvinceCode</code>, maximum 8 characters Example: "companyPostalCode" : "94301"
<code>companyCountryCode</code>	<i>(Required)</i> ISO-3166-1 alpha-2 standard country associated with the token requestor's primary address. Format: Alphabetic; maximum 2 characters Example: "companyCountryCode" : "US"
<code>companyPhone</code>	<i>(Optional)</i> Token requestor's primary phone number. Format: Numeric or hyphens, parentheses, period, or plus sign, valid for the <code>companyCountryCode</code>; maximum 16 characters Example: "companyPhone" : "4155551212"
<code>primaryContactFirstName</code>	<i>(Optional)</i> First name of token requestor's primary contact person. Format: Alphabetic or the following characters: spaces, ' (single quote), ` (back tick), ~ (tilde), " (double quote), . (period), and – (hyphen); maximum 256 characters Example: "primaryContactFirstName" : "John"
<code>primaryContactLastName</code>	<i>(Optional)</i> Surname of token requestor's primary contact person. Format: Alphabetic or the following characters: spaces, ' (single quote), ` (back tick), ~ (tilde), " (double quote), . (period), and – (hyphen); maximum 256 characters Example: "primaryContactLastName" : "Smith"

Field	Description
primaryContactEmail	(Required) Email address of token requestor's primary contact. Format: Alphanumeric, valid email address; maximum 256 characters Example: "primaryContactEmail": "jsmith@mymerchant888.com"
tokenRequestorServiceTypes	(Optional) Additional token requestor service types. Note: By default, the "Purchase" service type is always present. Format: Array of strings, which can include the following values: "Visa Direct"
relationships	(Optional) Relationship to the token requestor being onboarded, which is by externalClientId. If not specified, Visa creates the externalClientId for you, which is available in the response. Format: An array of Relationships structures.
acquirerIdentifiers	(Conditional) Acquirer identifiers; required if businessIdentificationType and businessIdentificationValue are not provided. Format: An Acquirer Identifier structure
trSettings	(Optional) Token requestor settings.
tokenizationConfiguration	(Required) Tokenization configuration. Format: A Tokenization Configuration structure

Business Identification Types and Values

Business Identification Type	Business Identification Value Description
ABN	Australian business number Format: 11 numeric digits
ACN	Australian company number Format: 9 numeric digits
BID	Business identification number; all countries. Note: You must contact your Visa representative before using BID for businessIdentificationType Format: 8 numeric digits
BIN	Kazakhstan business identification number Format: 12 numeric digits

Business Identification Type	Business Identification Value Description
BIR	Philippines certificate of registration Format: 12 numeric digits; hyphen-separated Example: 123-456-789-012
BN	Canadian business number Format: 9 numeric digits
BRN	South Korea business registration number Format: 10 numeric digits; hyphen-separated Example: 123-45-67890
BRNO	Malaysia only Format: Up to 10 numeric digits followed by a hyphen and a single character or 12 numeric digits. Example: 123456789012
CL	Commercial License Number Qatar Format: 4–6 numeric digits
CLN	Company License Number Kuwait Format: 5–7 numeric digits
CN	Japan corporate number Format: 13 numeric digits; leading digit is a non-zero check digit for the following 12 digits
CNPJ	Brazil only Format: 14 alphanumeric characters, identified by <i>n</i> in the example, and punctuation characters (period, slash, and hyphen). Example: nn.nnn.nnn/nnnn-nn
CR	Company Registration Number Saudi Arabia Format: 10–12 numeric digits
CUIT	Argentina only Format: 11 numeric digits
EDRPOU	Ukraine - Ukrainian corporations Format: 8 numeric digits - Ukrainian individual entrepreneurs Format: 10 numeric digits

Business Identification Type	Business Identification Value Description
EIN	US Federal tax ID (US only) Format: 9 numeric digits
HKBR	Hong Kong only Format: 11 numeric digits and a hyphen Example: 12345678-123
NATIONAL_ID	Dominican Republic (DO) Cedula de Identidad (Merchant Owner ID) Format: DO + 11 digits Example: DO01234567890
NIB	Indonesia business identification number Format: 13 numeric digits
NID	South Africa national identification number Format: 13 numeric digits
NZBN	New Zealand business number Format: 13 numeric digits
PAN	Indian business identification number Format: 10 alphanumeric characters
PAN	Belarus payer account number Format: 9 numeric digits
RFC	Mexico only Format: 12 alphanumeric characters for corporations; 13 alphanumeric characters for individuals.
RNC	Dominican Republic (DO) Registro Nacional de Contribuyentes (National Contributors Number) Format: DO + 9 digits Example: DO123456789
RUC	Peru only Format: 11 numeric digits
RUT	Chile and Columbia only Format: 9 numeric digits; for Chile; 10 numeric digits for Columbia
SIN	Canadian social insurance number Format: 9 numeric digits
SSN	US social security number (US only) Format: 9 numeric digits

Business Identification Type	Business Identification Value Description
TC	Vietnam tax code Format: 10 or 13 numeric digits
UEN	Singapore only Format: 9 or 10 alphanumeric characters
USCI	China united social credit code Format: 18 alphanumeric characters
VAT	Austria, Belgium, Bulgaria, Croatia, Czech Republic, Denmark, Estonia, Finland, France, Germany, Greece, Hungary, Ireland, Isle of Man, Italy, Latvia, Lithuania, Luxembourg, Malta, Netherlands, Norway, Poland, Portugal, Republic of Cyprus, Romania, Saudi Arabia, Slovak Republic, Slovenia, South Africa, Spain, Sweden, Switzerland, Thailand, Turkey, Ukraine, United Arab Emirates, United Kingdom See the formats below.

VAT Countries and Formats

Country / Code	VAT Value Format
Austria / AT	Format: AT + 9 characters in which the first character is always a 'U' Example: ATU12345678
Belgium / BE	Format: BE + 10 characters. Prefix with zero '0' if the customer provides a 9 digit VAT number. Example: BE1234567890
Bulgaria / BG	Format: BG + 9 or 10 characters Example: BG123456789 or BG0123456789
Croatia / HR	Format: HR + 11 characters. Example: HR01234567891
Czech Republic / CZ	Format: CZ + 8, 9 or 10 characters. Example: CZ12345678, CZ123456789, or CZ0123456789
Denmark / DK	Format: DK + 8 characters Example: DK12345678
Estonia / EE	Format: EE + 9 characters Example: EE123456789
Finland / FI	Format: FI + 8 characters Example: FI12345678
France / FR	Format: FR + 11 characters Example: FR12345678901

Country / Code	VAT Value Format
Germany / DE	Format: DE + 9 characters Example: DE123456789
Greece / GR	Format: GR + 9 characters Example: GR123456789
Hungary / HU	Format: HU + 8 characters Example: HU12345678
Ireland / IE	Format: IE + 8 numeric or alphabetic characters Example: IE1234567X
Isle of Man / GB	Format: GB + 9 or 12 digits; first 2 digits must be 00 Example: GB003456789
Italy / IT	Format: IT + 11 characters Example: IT12345678901
Latvia / LV	Format: LV + 11 characters Example: LV12345678901
Lithuania / LT	Format: LT + 9 or 12 characters Example: LT123456789 or LT123456789012
Luxembourg / LU	Format: LU + 8 characters Example: LU12345678
Malta / MT	Format: MT + 8 characters Example: MT12345678
Netherlands / NL	Format: NL + 12 characters of which the tenth character is always B. Example: NL123456789B01
Norway / NO	Format: NO + 9 digits and the letters + MVA Example: NO123456789MVA
Poland / PL	Format: PL + 10 alphanumeric characters Example: PL1234567890
Portugal / PT	Format: PT + 9 characters Example: PT123456789
Republic of Cyprus / CY	Format: CY + 9 characters. The last character must always be a letter. Example: CY12345678X
Romania / RO	Format: RO + 2 to 10 characters. Example: RO12, RO123, RO1234, RO12345, RO123456, RO1234567, RO12345678, RO123456789, or RO1234567890

Country / Code	VAT Value Format
Saudi Arabia / SA	Format: 15 numeric digits Example: 123456789012345
Slovak Republic / SK	Format: SK + 10 characters Example: SK1234567890
Slovenia / SI	Format: SI + 8 characters Example: SI12345678
South Africa / ZA	Format: 10 or 11 digits Example: 1234567890 or 12345678901
Spain / ES	Format: ES + 9 numeric digits Example: ES123456789
Sweden / SE	Format: SE + 12 digits Example: SE123456789012
Switzerland / CH	Format: CH + 9 digits + MWST, TVA, or IVA Note: • German part: MWST • French part: TVA • Italian part: IVA Example: CH123456789MWST, CH123456789TVA, or CH123456789IVA
Thailand	Format: 13 numeric digits Example: 1234567890123
Turkey / TR	Format: TR + 10 characters Example: TR1234567890
Ukraine / UA	Format: 9 or 12 digits Example: 123456789 or 123456789012
United Arab Emirates / AE	Format: 15 digits Example: 123456789012345
United Kingdom / GB	Format: GB + 9 digits Example: GB123456789

Acquirer Identifier

Field	Description
<code>acquirerId</code>	(Conditional) Acquirer ID; required unless both <code>businessIdentificationType</code> and <code>businessIdentificationValue</code> are provided. Format: String; numeric, max 11 digits Example: "acquirerId" : "..."
<code>acquirerMerchantId</code>	(Conditional) Acquirer merchant ID; required unless both <code>businessIdentificationType</code> and <code>businessIdentificationValue</code> are provided. Format: String; alphanumeric, max 25 characters Example: "acquirerMerchantId" : "..."

Token Requestor Settings

Field	Description
<code>panLifecycleUpdateParticipationType</code>	(Optional) Specifies whether the token requestor should be enrolled in Visa Digital Credential Updater (VDCU). If not specified, VDCU defaults apply. Format: String. It is one of the following values: • "optin" - enrolled in VDCU • "optout" - not enrolled in VDCU

Tokenization Configuration

Field	Description
<code>channelSecurityContext</code>	(Optional) Channel security context; default is shared secret. Format: String; max length 13 characters. Example: "channelSecurityContext" : "SHARED_SECRET"
<code>tokenizationProfiles</code>	(Required) List of profiles. Format: A <code>tokenizationProfiles</code> structure.

Tokenization Profiles

Field	Description
profileName	<i>(Required)</i> Profile name. Format: Alphanumeric; maximum 50 characters
applicationId	<i>(Required)</i> Application ID. Unique Identifier; same as the wallet provider's <code>clientAppID</code> . Format: Alphabetic, numeric, and hyphens (-), e. g. spaces are not allowed; maximum 36 characters.
postbackURL	<i>(Optional)</i> Postback URL. Format: Alphanumeric; maximum 50 characters
enforceDeviceBindingForCrypto	<i>(Optional)</i> Whether to enforce device binding for cryptograms. Format: It is one of the following values: • true • false

Relationships

Field	Description
externalClientId	<i>(Optional)</i> The relationship ID; if not specified, a Visa supplied ID appears in the <code>externalClientId</code> field of the response. This will become the relationship ID to use when calling VTS APIs. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 100 characters.

Onboarding Response

HTTP Responses

HTTP Response	Description
201	Token requestor (TR) successfully created and onboarded
400	Validation error
403	You are not authorized to create a token requestor (TR)
500	Internal error

HTTP Response Headers

A `Location` header that contains the URL of the newly created DPA.

Response Body

Field	Description
<code>dunsNumber</code>	Dun and Bradstreet number. Format: Numeric; 9 decimal digits
<code>companyPrimaryTradeName</code>	Trade name of token requestor, such as a doing-business-as (DBA) name. If not specified, the legal name (<code>companyPrimaryLegalName</code>) is used by default. Format: Alphanumeric, excepting semi-colon, percent sign, and parentheses; maximum 75 characters Example: <pre>"companyPrimaryTradeName" : "MyMerchant.com"</pre>
<code>companyPrimaryLegalName</code>	Legal name of token requestor. Some processors may not support a 75-character legal name. You cannot exceed the limit imposed by your processor. Format: Alphanumeric, excepting semi-colon, percent sign, and parentheses; maximum 75 characters Example: <pre>"companyPrimaryLegalName" : "My Merchant"</pre>
<code>primaryWebsiteURL</code>	The URL for your token requestor's site for handling VTS transactions. Format: Alphanumeric, valid URL; maximum 100 characters Example: <pre>"primaryWebsiteURL" : "http://www.myMerchant.com"</pre>
<code>businessIdentificationType</code>	The Identifier associated with the business type, if provided. Format: See Business Identification Types and Values in the request for a complete list of business identification types for all countries. Example: <code>"businessIdentificationType": "EIN"</code>

Field	Description
<code>businessIdentificationValue</code>	The value associated with the business identifier type, if provided. Format: See Business Identification Types and Values in the request for a complete list of business identification values for all countries Example: "businessIdentificationValue" : "..."
<code>companyAddress1</code>	First line of the token requestor's primary address. Format: Alphabetic, numeric, or the following characters: spaces, ' (single quote), # (pound-sign or hash), , (comma), _ (underscore), : (colon), / (forward slash), and – (hyphen); maximum 140 characters Example: "companyAddress1" : "1 Main Street"
<code>companyAddress2</code>	Second line of the token requestor's primary address. It can be empty. Format: Alphabetic, numeric, or the following characters: spaces, ' (single quote), # (pound-sign or hash), , (comma), _ (underscore), : (colon), / (forward slash), and – (hyphen); maximum 140 characters Example: "companyAddress2" : ""
<code>companyCity</code>	City in the token requestor's primary address. Format: Alphabetic, numeric, or the following characters: spaces, ' (single quote), . (period), and – (hyphen); maximum 100 characters Example: "companyCity" : "San Francisco"
<code>companyStateProvinceCode</code>	State or province code associated with the physical address in the specified country. The <code>companyStateProvinceCode</code> for US must be a standard 2-characters code. Format: The suffix of an ISO-3166-2 country code, corresponding to the <code>companyCountryCode</code> maximum 2 characters Example: "companyStateProvinceCode" : "CA"
<code>companyPostalCode</code>	Postal code associated with the primary address of token requestor, such as a ZIP code. Format: Depends on <code>companyStateProvinceCode</code>, maximum 7 characters Example: "companyPostalCode" : "94301"
<code>companyCountryCode</code>	ISO-3166-1 alpha-2 standard country associated with the token requestor's primary address. Format: Alphabetic; maximum 2 characters Example: "companyCountryCode" : "US"

Field	Description
<code>companyPhone</code>	Token requestor's primary phone number. Format: Numeric or hyphens, parentheses, period, or plus sign, valid for the <code>companyCountryCode</code>; maximum 16 characters Example: "companyPhone" : "4155551212"
<code>primaryContactFirstName</code>	First name of token requestor's primary contact person. Format: Alphabetic or the following characters: spaces, ' (single quote), ` (back tick), ~ (tilde), " (double quote), . (period), and – (hyphen); maximum 256 characters Example: "primaryContactFirstName" : "John"
<code>primaryContactLastName</code>	Surname of token requestor's primary contact person. Format: Alphabetic or the following characters: spaces, ' (single quote), ` (back tick), ~ (tilde), " (double quote), . (period), and – (hyphen); maximum 256 characters Example: "primaryContactLastName" : "Smith"
<code>primaryContactEmail</code>	Email address of token requestor's primary contact. Format: Alphanumeric, valid email address; maximum 256 characters Example: "primaryContactEmail": "jsmith@mymerchant888.com"
<code>relationships</code>	Relationship to the token requestor being onboarded, which is by <code>externalClientId</code>. Format: An array of Relationships structures.
<code>acquirerIdentifiers</code>	Acquirer identifiers, if provided. Format: An array of Acquirer Identifier structures
<code>tokenRequestorServiceTypes</code>	Token requestor service types. Format: Array of strings, which can include the following values: • "Purchase" • "Visa Direct"
<code>tokenizationConfiguration</code>	Tokenization configuration. Format: A Tokenization Configuration structure

The following fields may be returned in the response; however, they are internal and may be removed in a future release. Do not use these fields:

- `id`
- `enablePortalAccess`

- status
- enablePANAccess
- permissionGroups
- createdBy
- clientAuthenticationLevel
- isFavorite
- permissionPolicies
- promoteVdpApp
- cloudServices
- associatedVdpEnvironments
- integrationWalletData structure and all of its fields

Tokenization Configuration

Field	Description
tokenRequestorId	Token requestor ID. Format: Numeric; maximum 11 digits.
channelSecurityContext	Channel security context. Format: String; max length 13 characters. Example: "channelSecurityContext" : "SHARED_SECRET"
tokenizationProfiles	List of profiles. Format: A tokenization profile structure.

Tokenization Profiles

Field	Description
profileName	Profile name. Format: Alphanumeric; maximum 50 characters
applicationId	Application ID. . Unique Identifier; same as the wallet provider's <code>clientAppID</code> . This is the ID to use for <code>clientAppID</code> when calling VTS APIs. Format: Alphabetic, numeric, and hyphens (-), e. g. spaces are not allowed; maximum 36 characters.

Field	Description
postbackURL	Postback URL. Format: Alphanumeric; maximum 50 characters
enforceDeviceBinding ForCrypto	Whether to enforce device binding for cryptograms. Format: It is one of the following values: • true • false

Relationships

Field	Description
externalClientId	The relationship ID for the entity being onboarded. This is the relationship ID to use when calling VTS APIs. Format: Alphanumeric, hyphens (-), and underscores (_), e.g., spaces are not allowed; maximum 100 characters

Acquirer Identifier

Field	Description
acquirerId	Acquirer ID, if provided. Format: String; numeric, max 11 digits Example: "acquirerId" : "..."
acquirerMerchantId	Acquirer merchant ID, if provided. Format: String; alphanumeric, max 25 characters Example: "acquirerMerchantId" : "..."

Onboarding Errors

The format of onboarding errors is as follows:

```
{
  "responseStatus": {
    "code": 2940,
    "message": "Invalid Request",
    "severity": "ERROR",
    "details": "There is an onboarding error please reach out to Visa",
    "status": "BAD_REQUEST",
    "additionAttributes": {},
    "info": "http://api.visa.com/docs/errors/2940"
```



```
}  
}
```

Use the `responseStatus.code` field to drive your error handling logic:

Code	Description
2098	Validation failed or not enough information provided
2100	Profile not found
2101	Entity not found
2103	Agreements not accepted
2104	No entity relationship is found for relationship ID
2105	User not found
2106	Incomplete or missing request header
2107	API key not found
2108	Profile already exists
2109	Invalid token requestor
2110	Cannot create relationship; inactive entity
2111	Entity must be active to create relationship
2112	Invalid actor
2113	Relationship already exists
2114	Relationship name already exists
2115	Invalid entity type
2116	Relationship does not exist
2117	You are not allowed to onboard
2118	You are not allowed to onboard using self service
2119	Entity onboarding not allowed
2120	More than one default profile found
2122	Not Authorized; valid Subject not present in the Header
2123	Cannot update inactive entity
2125	Cannot activate/deactivate self
2126	DFM not allowed
2127	User with this email address already exists
2133	User not found

Code	Description
2134	Entity's system not found
2141	Cloud application profile not found
2142	Cloud-based profile exists with appld for entity
2143	App does not exist with external appld
2144	API key does not exist
2183	Invalid acquirer ID profile
2185	Invalid profile
2191	Cloud-based profile already exists
2192	Entity is not enrolled for CBP service
2193	Entity is not enrolled for system service
2500	General service error
2812	Invalid MCC input found, rejecting request.
2940	Invalid Request
2945	Invalid Request. Invalid TR TSP Onboarding Request, multiple API Key observed.

Note:

Other errors are internal.

Onboarding Examples

Request

```
{
  "serviceContext": "TOKENIZATION",
  "companyPrimaryLegalName": "OnnboardingTest-Test9",
  "primaryWebsiteURL": "...",
  "status": "ACTIVE",
  "companyCity": "...",
  "companyCountryCode": "US",
  "companyStateProvinceCode": "CA",
  "primaryContactFirstName": "...",
  "primaryContactLastName": "...",
  "primaryContactEmail": "...",
  "businessIdentificationType": "EIN",
  "businessIdentificationValue": "...",
  "tokenRequestorServiceTypes" : ["Visa Direct"],
```

```
"tokenizationConfiguration": {
  "tokenizationProfiles": [
    {
      "profileName": "partnername",
      "applicationId": " partnername "
    }
  ]
}
```

Response

Note:

Not all fields are available for use; some are internal.

```
{
  "id": "659...02",
  "primaryContactFirstName": "...",
  "primaryContactLastName": "...",
  "primaryContactEmail": "...",
  "enablePortalAccess": false,
  "primaryWebsiteURL": "...",
  "companyPrimaryLegalName": "OnboardingTest-Test9",
  "companyPrimaryTradeName": "OnboardingTest-Test9",
  "companyCity": "...",
  "companyStateProvinceCode": "CA",
  "companyCountryCode": "US",
  "businessIdentificationType": "EIN",
  "businessIdentificationValue": "...",
  "status": "ACTIVE",
  "enablePANAccess": false,
  "created": "2021-01-07",
  "apiKeys": [],
  "profiles": [],
  "relationships": [
    {
      "externalClientId": "100...57",
      "permissionGroups": []
    }
  ],
  "acquirerIdentifiers": [],
  "trSettings": {
    "panLifecycleUpdateParticipationType": "optin"
  },
  "tokenizationConfiguration": {
    "channelSecurityContext": "SHARED_SECRET",
    "tokenRequestorId": "40000000057",
    "tokenizationProfiles": [
      {
        "profileName": "Test1",
        "applicationId": "Test1"
      }
    ]
  },
  "createdBy": "...",
```

```
"clientAuthenticationLevel": "XV2_SEG",
"isFavorite": false,
"associatedVdpEnvironments": [
  "CERT"
],
"permissionPolicies": [],
"promoteVdpApp": "CERT",
"integrationWalletData": {
  "trTSPClientId": "619...01",
  "wpClientId": "155...02",
  "trTspId": null,
  "trTSPCompanyPrimaryLegalName": "...",
  "wpCompanyPrimaryLegalName": "..."
},
"tokenRequestorServiceTypes": [
  "Purchase",
  "Visa Direct"
],
"cloudServices": [
  "CBP"
]
}
```

Chapter 40

Get Push Enrollment Info

About the Get Push Enrollment Info API

Given a sessionID in pushData received from the VCEH web proxy, the Get Push Enrollment Info API returns user information from the issuer.

Get Push Enrollment Info Request

Path and Endpoints

Resource Path

`vts/pushCardEnrollment`

Sandbox

```
https://cert.api.visa.com/vts/pushCardEnrollment/{sessionId}/info?  
&apikey=key
```

Live

```
https://api.visa.com/vts/pushCardEnrollment/{sessionId}/info?  
&apikey=key
```

URL Parameters

Parameter	Description
{sessionId}	<i>(Required)</i> The session ID in <code>pushData</code> received from the VCEH web proxy. Format: String; maximum 32 characters.

Method

POST

Required Headers

Header	Description
x-request-id	<i>(Required)</i> Unique ID for the API request. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
x-pay-token	<i>(Required)</i> A token identifying the transaction and its contents. The token expires in 480 seconds (8 minutes) for all clients.

Header	Description
	Format: Alphanumeric; 256 characters in the form of <code>x-pay-token: xv2:UTC_Timestamp:HMAC-SHA256_hash</code>, where: • <code>UTC_Timestamp</code> is a UNIX Epoch timestamp, in seconds • <code>HMAC-SHA256_hash</code> is an HMAC-SHA256 hash using the shared secret associated with the API key and the following unseparated items: 1. Timestamp from the transaction; exactly the same as:<code>UTC_Timestamp</code> 2. Resource path (API name). 3. This HTTPS request's query string, if it exists: Note: To create the query string, concatenate all query string components (names and values) as UTF-8 characters, which are URL-encoded per RFC 3986. Hex characters must be uppercase. Multiple parameters must be sorted using lexicographic byte ordering and separated from each other by an ampersand (&) character (ASCII code 38). Parameter names are separated from their values by the = character (ASCII character 61), which must be present even if the value is empty. "Unreserved" characters specified in Section 2.3 of RFC 3986, including dash (-), dot (.), underscore (_), and tilde (~), should not be URL-encoded. 4. Complete request body, when a request body exists. Example: <code>x-pay-token: xv2:1440199445:4a3e6ea1d2...</code> which is concatenated as follows: <code>x-pay-token: xv2: + UTC_Timestamp + : + HMAC-SHA256_hash(shared_secret, (UTC_Timestamp + resource_path + query_string + request_body))</code> where <code>shared_secret</code> is the private key value associated with your API key, which is available in your Visa Developer Center account, <code>resource_path</code> is the API name / URI, <code>query_string</code> contains query parameters, such as <code>apiKey=key</code>, and <code>request_body</code> is a JSON structure representing the request without extra whitespace.
Accept	(Required) Acceptable response format. Format: Must include <code>application/json</code> Example: <code>Accept: application/json</code>

Query Parameters

Parameter	Description
apiKey	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.

Request Body

Field	Description
clientAppID	(Required) Unique identifier for the client application. Source: Wallet provider. Format: Alphabetic, numeric, and hyphens (-), e. g. spaces are not allowed; maximum 36 characters

Get Push Enrollment Info Response

Response Body

Field	Description
encInfo	Encrypted information. For more information, see "JSON Web Encryption and Object Signing." Format:String; maximum 8096 characters.

Unencrypted Info

Field	Description
userInformation	User information provided by issuer, if available. Format: A <code>clientInformation</code> structure.
billingInformation	An array of billing information for payment instruments. Format: An array of 10 or fewer <code>billingInformation</code> structures.

Billing Information

Field	Description
ref	Reference to the payment instrument; same as the value in enrollment info. Format: String; maximum 256 characters.
address	Billing address associated with the payment instruments. Format: An <code>address</code> structure.

Billing Address

Field	Description
line1	Street 1 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line2	Street 2 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line3	Street 3 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line4	Street 4 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
line5	Street 5 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; maximum 140 characters.
city	The city associated with the enrolled payment instrument. Format: Alphabetic, single quote ('), hyphen (-), and whitespace.; maximum 100 characters.

state	State or province code in ISO 3166-2 format, e.g. NY. Format: Alphabetic; maximum 3 characters.
country	Country in ISO 3166-1 alpha-2 format, e.g. US. Format: Alphabetic; maximum 2 characters.
countryDigits	ISO 3166-1 numeric Country Code. Format: Numeric; maximum 3 digits.
postalCode	The postal code associated with the enrolled payment instrument. Note: This field does not support spaces (" ") in the middle of characters. Format: Alphabetic, numeric, e.g. spaces are not allowed; maximum 3–16 characters.

Get Push Enrollment Info Example

Request Body

```
{
  "clientAppID": "...",
}
```

Response Body

```
{
  "encInfo": "eyJ...RA"
}
```

Decrypted encInfo:

```
{
  "userInformation": {
    "firstName": "...",
    "middleName": "...",
    "lastName": "...",
    "contactEmail": "...",
  },
  "billingInformation": [
    {
      "ref": "v00...01",
      "address": {
        "city": "FosterCity",
        "line1": "...",
        "postalCode": "94404",
        "state": "CA",
        "country": "US"
      }
    }
  ]
}
```

```
    }  
  },  
  {  
    "ref": "v00...01",  
    "address": {  
      "city": "FosterCity",  
      "line1": "...",  
      "postalCode": "94404",  
      "state": "CA",  
      "country": "US"  
    }  
  }  
]  
}
```

Chapter 41

Status Notifications

Status Notifications Summary

Use status notifications to update your MAP. When you receive a notification, call the appropriate VTS API to receive information about the change. Visa Token Service sends notifications to your MAP if the MAP is signed up to receive them, meaning that a post back URL is specified in the VTS Profile. Notifications indicate AAM events (Replenish, API Expiration, Transaction Logs), Transaction History, and LCM events (token status changed).

Note:

Notifications are asynchronous, meaning that they may be delivered at any time. Their arrival is not synchronized with an API call and could occur much later, depending on Visa's system-wide load.

The Wallet Provider needs to configure the following settings in the VTS profile:

Profile Name*	
Countries Of Operation	App ID*
	Type or Paste; comma, space delimited string
Postback URL	Access Token Expiration Time*
	43200
	(Number in Minutes, 43200 minutes = 30 days)
Encryption API Key	
Select API Key 	
If an API Key is not selected a new one will be created and assigned to this field	
☐ Token Level Transaction History	
☐ PAN Enrollment Billing Address	

- **Post-back URL** of your MAP (MAPNotificationBaseURL) to receive notifications coming from Visa Token Service.
- Transaction History scope, which must be TOKEN.

Encryption API Key

Contact your Visa Digital Solutions representative to configure the notification type (token life cycle, PAN enrollment, etc.).

A notification is a REST (HTTPS `POST` message) call from VTS to your post-back URL. It consists of headers, query parameters, and a request body, which is determined by the kind of event. The notification API uses the same `x-pay-token` as calls to VTS. The client-specific API key issued during onboarding will be passed as a query parameter.

The `apiName` follows the slash after the `MAPNotificationBaseURL` base URL. VTS provides the following notifications; the `apiName` is shown in parentheses:

- Token Status Updated (`provisionedToken`)

```
POST
https://MAPNotificationBaseURL/provisionedToken?
apiKey=key&eventType=type
```

- Payment Transactions (`paymentTxns`)

```
POST
https://MAPNotificationBaseURL/paymentTxns?apiKey=key
```

- Card Metadata Update (`panMetadata`)

```
POST
https://MAPNotificationBaseURL/panMetadata?apiKey=key
```

- Token Push Provisioning (`tokenPushProvisioning`)

```
POST
https://MAPNotificationBaseURL/tokenPushProvisioning?apiKey=key
```

- Task Status Event (`panMetadata`)

```
POST
https://MAPNotificationBaseURL/taskStatus?
apiKey=key&eventType=type&taskType=type
```

- Bulk Provisioning Job Completion (`bulkProvisionedToken`)

```
POST
https://MAPNotificationBaseURL/bulkProvisionedToken?
apiKey=key&eventType=type
```

SSL Certificate Requirements

To receive notifications, you must

- Specify a fully-qualified postback URL on which you will listen for notifications from Visa.
- Provide certificates on this host. VTS needs your client's root and to know of any intermediary certificates.

- White list the following Visa IP addresses for Sandbox and Production.

```
198.241.168.15  
198.241.162.104
```

- Open firewall (layer 3 or 7) to receive notifications from the white-listed Visa IP addresses.

Task Status Event

This event is a status notification for async task-related events.

Endpoint

```
https://MAPNotificationBaseURL/taskStatus?  
apiKey=key&eventType=type&taskType=type
```

Query Parameters

Parameter	Description
taskType	Task type. Format: It is one of the following values: • TOKEN_PROVISION
eventType	Event type. Format: It is one of the following values: • TASK_COMPLETED

Request Body

Field	Description
date	Event creation timestamp in UTC format. Format: Date. The UNIX_UTC_Timestamp is a UNIX Epoch Timestamp. The value is in seconds.
vTaskID	(Conditional) This field is populated for the <code>TOKEN_CREATED</code> notification in the case of async provisioning flow. Format: String. Alphabetic, numeric; 40 characters.
relationshipID	If applicable, the ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

To retrieve the details associated with this event, invoke the Get Async Token Provisioning Result API.

Task Status Event Example

Notification For Task-Related Events

```
{
  "date": 1627928004,
  "vTaskID": "000...02"
}
```

The following notification event shows a relationship ID:

```
{
  "date": 1661807833,
  "relationshipID": "100...56",
  "vTaskID": "000...02"
}
```

Token Status Updated Event

To respond to this event, call the Get Token Status API. If a `KEY_STATUS_UPDATED` event occurs, the recommended action is to call the Replenish API.

Notification for token related events include Token Deleted, Suspended, Resumed, Token Expired, and Token Reperso Advice events: (eventType=TOKEN_CREATED, TOKEN_STATUS_UPDATED)

Endpoint

`https://MAPNotificationBaseURL/provisionedToken?apiKey=key&eventType=type`

Query Parameters

Parameter	Description
apiKey	Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: Alphabetic, numeric; maximum 64 characters.
eventType	Event type. Format: It is one of the following values: • TOKEN_CREATED • TOKEN_STATUS_UPDATED • KEY_STATUS_UPDATED • TOKEN_REPERSO_ADVICE • TOKEN_DEVICE_BINDING_APPROVED • TOKEN_DEVICE_BINDING_DECLINED • TOKEN_DEVICE_BINDING_DELETED • CARDHOLDER_STEPUP_APPROVED • CARDHOLDER_STEPUP_DECLINED

Request Body

Field	Description
date	Event creation timestamp in UTC format. Format: Date. The UNIX_UTC_Timestamp is a UNIX Epoch Timestamp. The value is in seconds.
api	Account Parameter Index (YHHHCC) value. Applicable for key status update - Replenishment. This data element is utilized during cryptogram generation for KEY_STATUS_UPDATED event. Format: Alphabetic, numeric; maximum 8 characters.

Field	Description
vProvisionedTokenID	Unique ID for token. Format: String. Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36.
vNotificationID	Reperso Notification ID. Present in case of reperso advice. Applicable for Reperso. Format: String. Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36.
clientReferenceID	Client reference identifier. Format: String. Alphabetic, numeric, and hyphens (-); maximum 80.
deviceBindingInfo	Information regarding device binding. Applicable for device binding.
relationshipID	If applicable, the ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.
vPanEnrollmentID	Unique ID for the enrolled PAN associated with the <code>TOKEN_REPERSO_ADVICE</code> event type. It is applicable only for the SCP03 implementation with Generate Keys API. Format: String. Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36.

Token Status Updated Event Examples

Notification For TOKEN_STATUS_UPDATED Events

```
{
  "date": 1661807833,
  "relationshipID": "100...16",
  "vProvisionedTokenID": "aa3...01"
}
```

Notification For TOKEN_REPERSO_ADVICE Events

```
{
  "date":1688425701,
  "vProvisionedTokenID":"a98...01",
  "vNotificationID":"ead...01",
}
```

```
"vPanEnrollmentID": "ae3...01"  
}
```

Payment Transaction Event

To respond to this event, call the Get Transaction History API.

Endpoint

```
https://MAPNotificationBaseURL/paymentTxns?apiKey=key
```

Query Parameters

Parameter	Description
apiKey	<i>(Required)</i> Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.
clientAppID	<i>(Conditional)</i> Unique identifier for the client application; required when the <code>relationshipID</code> query parameter is specified. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
relationshipID	<i>(Optional)</i> The ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Request Body

Field	Description
date	Event creation timestamp in UTC format. Format: Date. The UNIX_UTC_Timestamp is a UNIX Epoch Timestamp. The value is in seconds.
vProvisionedTokenID	Unique ID for token. Format: String. Alphabetic, numeric [A-Z][a-z][0-9,-], and hyphens (-), e.g. spaces are not allowed; maximum 36.

Payment Transaction Event Examples

```
{
  "vProvisionedTokenID": "adc...a9",
  "date": 1423248912
}
```

Card Metadata Update Event

To respond to this event, call the Get Card Metadata API.

Endpoint

```
https://MAPNotificationBaseURL/panMetadata?apiKey=key
```

Query Parameters

Parameter	Description
apiKey	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphabetic, numeric; maximum 64 characters.

Request Body

Field	Description
vPanEnrollmentID	Unique ID for enrolled PAN. Format: Alphabetic, numeric, and hyphens (-), e.g. spaces are not allowed; maximum 36 characters.
date	Event creation timestamp in UTC format. Format: Date. The UNIX_UTC_Timestamp is a UNIX Epoch Timestamp. The value is in seconds.
relationshipID	If applicable, the ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Card Metadata Update Event Examples

```
{
  "vPanEnrollmentID": "bf90a0edbb3c4fc09344120a3a91e355",
  "date": 1423248912
}
```

Token Push Provisioning Event

This event notifies you that the issuer has pushed a card enrollment. Respond to this event by calling the Enroll PAN method using the `paymentInstrumentReference`, then call Token Push Provisioning Given Enrollment ID to provision the token.

Endpoint

`https://MAPNotificationBaseURL/tokenPushProvisioning?apiKey=key`

Query Parameters

Parameter	Description
apiKey	(Required) Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: String. Alphanumeric; maximum 64 characters.

Request Body

Field	Description
paymentInstrumentReference	(Deprecated) A reference to the payment instrument or PAN provided by the issuer. Format: Alphanumeric; maximum 32 characters.
enrollments	An array of enrollment payloads, which identify the token service provider and contains an opaque payload from the TSP. Format: An array of 10 or fewer enrollment payloads.
expireTimestamp	The date time that the <code>paymentInstrumentReference</code> expires, after which a request based on the <code>paymentInstrumentReference</code> will fail. Format: Date. The <code>UNIX_UTC_Timestamp</code> is a UNIX Epoch Timestamp. The value is in seconds.
isPassCodeRequired	Whether OTP step up is required. Format: It is one of the following values: • Y — OTP required. • N — OTP not required.
clientInformation	Encrypted client information, as provided by the issuer. This information is encrypted using the token requestor's key. Format: String; maximum 7,000 characters.
passCodeConversationID	Passcode conversation ID, which is required when calling Enroll PAN if OTP step up is required. Format: Alphanumeric, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; maximum 32 characters

Enrollment Payload

Field	Description
TSP	Token service provider. Format: It is one of the following values: • V — Visa • MC — Mastercard
ref	Reference to the payment instrument. Format: String; maximum 256 characters.

Token Push Provisioning Notification Examples

Request

Selected Headers:

```
x-request-id: dd4...59
X-PAY-TOKEN: xv2:1570095241:eb8...d2
```

Endpoint:

```
https://MAPNotificationBaseURL/tokenPushProvisioning?apiKey=null
```

Request Body

```
{
  "date": 1570095241,
  "paymentInstrumentReference": "8fc...02",
  "expireTimestamp": 1570959238,
  "isPassCodeRequired": "Y",
  "clientInformation": "eyJ...kw",
  "passCodeConversationID": "746...02"
}
```

Response

Response Body:

None

Bulk Provisioning Job Completion

Token requestors integrated for bulk tokenization will be notified of job completion so that the initial job request results can be fetched. Token requestors must support this additional notification type when integrating the Bulk Provisioning APIs.

Endpoint

```
https://MAPNotificationBaseURL/bulkProvisionedToken?  
apiKey=key&eventType=type
```

Query Parameters

Parameter	Description
apiKey	Client-specific API key issued during onboarding. This will be passed as a query parameter. Format: Alphabetic, numeric; maximum 64 characters.
eventType	Event type. Format: It is one of the following values: PROVISION_TOKEN_WITH_PAN_JOB_COMPLETED

Request Body

Field	Description
date	Event creation timestamp in UTC format. Format: Date. The UNIX_UTC_Timestamp is a UNIX Epoch Timestamp. The value is in seconds.
jobId	The Job ID assigned by VTS. Format: String; alphabetic, numeric, and hyphens (-)

Field	Description
status	Bulk notification status. Format: It is one of the following values: • COMPLETE
relationshipID	If applicable, the ID identifying the relationship between a token requestor and the caller of this API, which is a token service provider (TR-TSP) with which the token requestor has a relationship. The relationship is defined at the time of onboarding; contact your Visa representative for more information. Format: String. Alphanumeric; maximum 100 characters.

Bulk Provisioning Job Completion Example

Notification For Batch Job-Related Events

```
Notification for batch job status (JOB_COMPLETED)
{
  "date": 1423248912,
  "jobId": , "344...01"
}
```

Chapter 42

VCEH Web Proxy

About the VCEH Web Proxy

VCEH Push Card Enrollment to Token Requestor's App

VCEH launches the token requestor's app through the URI scheme registered by the token requestor. The request transfers provision enrollment and associated client information from the issuer app to token requestor's app on the same device through `pushData` parameter.

```
[TR URI scheme]?pushData="JWS payload"
```

The `pushData` parameter's value is a JWS blob wrapping payment instrument enrollments and user information.

JWS Header

- "cty": "JOSE"
- "kid": *Visa's integrity key*
- "iat": *timestamp*. Time (in UTC) when the JWS was issued; expressed in UNIX epoch time (seconds since 1 January, 1970).

JSON Body

Field	Description
enrollments	An array of enrollment payloads, which identify the token service provider and contains an opaque payload from the TSP. Format: An array of 10 or fewer enrollment payloads.
OTP	Whether or not a passcode is required. Format: It is one of the following values: • Y — passcode required • N — passcode not required
sessionID	A reference to client info from the issuer, which can be used to retrieve all client info sent by the issuer. Format: String; maximum 32 characters.
exp	The date time that the push data expires, after which client info will not be available. If the <code>paymentInstrumentReference</code> is received with the Enroll PAN or provision token APIs after the expiration, the request will fail due to expired payment instrument reference. Format:
encInfo	Encrypted information; JWE blob encrypted with shared secret. For more information, see "JSON Web Encryption and Object Signing." It contains the user's email address and/or phone number, if provided by the issuer. Format: String; maximum 256 characters.

Enrollment Payload

Field	Description
tsp	Token service provider. Format: It is one of the following values: • v — Visa • MC — Mastercard
ref	Reference to the payment instrument. Format: String; maximum 256 characters.

Unencrypted `encInfo`

Field	Description
email	User's email address, if provided by the issuer. Format: String; maximum 32 characters.
phone	User's phone number, if provided by the issuer. Format: String; maximum 32 characters.

Token Requestor's App Returns to VCEH

The token requestor's app launches the default web browser on the user's device with a request to redirect the user back to VCEH with provision status:

```
GET [VCEH_URL]/return#results=JWS<<JWE payload>>
```

where [VCEH_URL] is

```
https://vceh.consumerapi.digital.visa.com/vceh
```

JWS Header

Field	Description
iat	(Required) Timestamp (in UTC) when JWE was issued, expressed in UNIX epoch time (seconds since 1 January, 1970). Format: String; numeric digits

alg	(Required) Description of the algorithm used to encrypt the randomly generated symmetric Content Encryption Key (CEK). Format: Value is PS256.
kid	(Required) Key identifier that identifies the key used to encrypt the CEK, which is the token requestor's API key. Format: String; max length 64 characters.

JWE Header

Field	Description
iat	(Required) Timestamp (in UTC) when JWE was issued, expressed in UNIX epoch time (seconds since 1 January, 1970). Format: String; numeric digits
alg	(Required) Description of the algorithm used to encrypt the randomly generated Content Encryption Key (CEK). Format: It is one of the following values: A256GCMKW - symmetric CEK
kid	(Required) Key identifier that identifies the key used to encrypt the CEK, which is the token requestor's API key. Format: String; max length 64 characters.
exp	(Optional) Expiration time of the payload. This field is optional and should be evaluated in conjunction with the ttl default defined with use case. exp can be used to reduce the default ONLY. If exp represents a time greater than iat + ttl then iat + ttl will be used. Format: yyyy-MM-ddTHH:mm:ss.SSSZ.
iss	(Required) Message originator's client ID, which is the issuer's vClientID of the issuer that has been registered with VDCS. Format: String; alphanumeric; max length 64 characters.
aud	(Required) Desired recipient, which is a token requestor's vClientID value that has been registered with VCEH. Format: String; alphanumeric; max length 256 characters.

JWE Body

Field	Description
enrollments	An array of enrollment payloads, which identify the token service provider and contains an opaque payload from the TSP. Format: An array of 10 or fewer enrollment payloads.
sessionID	Session ID from the <code>pushData</code> parameter. Format: String; maximum 32 characters.

TR Enrollment Results

Field	Description
tsp	Token service provider. Format: It is one of the following values: • <code>V</code> — Visa • <code>MC</code> — Mastercard
ref	Reference to the payment instrument. Format: String; maximum 256 characters.
status	Enrollment status. Format: It is one of the following values: • <code>PENDING</code> • <code>SUCCESS</code> • <code>CANCEL</code>

VCEH Web Proxy Examples

VCEH to Token Requestor

Endpoint:

```
https://<TR URI scheme>/?pushData=eyJ...fQ
```

Decrypted Push Data:

```
{
  "enrollments": [
    {
      "tsp": "V",
```

```
    "ref": "v00...01"
  },
  {
    "tsp": "V",
    "ref": "v00...01"
  },
  {
    "tsp": "V",
    "ref": "v00...01"
  },
  {
    "tsp": "MC",
    "ref": "EHM2Dey41C"
  },
  {
    "tsp": "MC",
    "ref": "ezXP3hJfUc"
  },
  {
    "tsp": "MC",
    "ref": "GAZXTdU95Z"
  }
],
"otp": "N",
"exp": 1616828433,
"sessionID": "v00...01",
"encInfo": "eyJ...zw"
}
```

Decrypted encInfo:

```
{
  "email": "...",
  "phone": "..."
}
```

Token Requestor to VCEH

Endpoint:

```
https://<VCEH_URL>/vceh/return#results=eyJ...6Q
```

JWE Payload:

```
{
  "enrollments": [
    {
      "ref": "v005a925418883239c6b233a58d9a301",
      "tsp": "V",
      "status": "SUCCESS"
    },
    {
      "ref": "v001e9cc28a9b554609a2d1f6d20c101",
      "tsp": "V",
      "status": "PENDING"
    }
  ]
}
```

```
    },
    {
      "ref": "v008c415d75d9d490da226e6270eb001",
      "tsp": "V",
      "status": "CANCEL"
    },
    {
      "ref": "EHM2Dey41C",
      "tsp": "MC",
      "status": "SUCCESS"
    },
    {
      "ref": "ezXP3hJfUc",
      "tsp": "MC",
      "status": "PENDING"
    },
    {
      "ref": "GAZXTdU95Z",
      "tsp": "MC",
      "status": "CANCEL"
    }
  ],
  "sessionID": "v00...01"
}
```


Chapter 43

Digital Authentication Framework (DAF)

Risk Data Attributes

Risk data attributes are encrypted into the `encRiskData` field in the Get Payment API request. All required fields must be included for qualification by the Digital Authentication Framework. Token requestors should send optional and conditional fields when available for accurate risk scoring.

Field	Description
<code>ip4address</code>	(Required) IP address of the device on which purchase occurred, in dot-decimal notation consisting of four decimal numbers, each ranging from 0 to 255, separated by dots. Each part represents a group of 8 bits (an octet) of the address. Example: <code>"ip4address": "89.24.143.232"</code> Format: Numeric and periods (.); maximum 15 characters.

clientDeviceID	(Required) Stable device identification set by wallet provider. Format: Alphabetic, numeric, hyphens (-), and underscores (_), e.g. spaces are not allowed; 24 characters.
location	(Optional) Location of the device on which purchase occurred, up to 4-digits of precision. Information will be supplied with latitude/longitude. Recommended for Digital Authentication Framework (DAF) if token requester has consent from the consumer to share geolocation information. Example: +xxx.xxxx/-xxx.xxxx. Precision can be rounded to the nearest whole digit. Example:+37/-121. Format: Alphanumeric; maximum 25 characters.
deliveryMethod	(Optional) Delivery method. Format: It is one of the following values: • 01 - Ship to cardholder's billing address. • 02 - Ship to another verified address on file with merchant. • 03 - Ship to address that is different than the cardholder's billing address. • 04 - "Ship to Store" or pick up at local store. (Store address must be populated in shipping address fields.) • 05 - Digital goods (includes online services, electronic gift cards and redemption codes). • 06 - Travel and Event tickets, not shipped. • 07 - Other, such as gaming, digital services not shipped, e-media subscriptions, etc.
devicePhoneNumber	(Optional) Phone number of the device on which the purchase occurred. Format: Valid phone number; maximum 15 characters, which are digits and '+', '-', '.', '(', ')'
clientWalletAccountEmailAddress	(Required) Client-provided email address linked to their wallet account login. Used for consumer service operations as well as risk operations. Required by Digital Authentication Framework (DAF). Format: Valid email address; UTF-8, maximum 48 characters.
clientWalletAccountEmail AddressHash	(Optional) Hash of email address linked to their wallet account login. Hash algorithm to be provided by Visa. base64 URL-encoded. Format: Base64 encoded string, per RFC4648, with the following exceptions: hyphens (-) replace plus signs (+) and underscores (_) replace forward slash (/) characters; maximum 48 characters.

walletAccountFirstCreated	(Optional) Number of days since the consumer created the wallet account or started using the wallet. Format: String; maximum 4 digits.
walletAccountLastChanged	(Optional) Number of days since the consumer's last account change. Format: String; maximum 4 digits.
billingZip	(Conditional) The postal code associated with the payment instrument. Required for countries where billing address information is available, such as Australia, Canada, UK, US, etc. Note: This field does not support spaces (" ") in the middle of characters. Format: Alphabetic, numeric, e.g. spaces are not allowed; maximum 3-16 characters.
billingLine1	(Conditional) Street 1 on billing address for the payment instrument. Required for countries where billing address information is available, such as Australia, Canada, UK, US, etc. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; UTF-8, maximum 140 characters.
billingLine2	(Optional) Street 2 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; UTF-8, maximum 140 characters.
billingLine3	(Optional) Street 3 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; UTF-8, maximum 140 characters.
billingLine4	(Optional) Street 4 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; UTF-8, maximum 140 characters.
billingLine5	(Optional) Street 5 on billing address for the payment instrument. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; UTF-8, maximum 140 characters.

billingCity	(Conditional) The city associated with the payment instrument. Required for countries where billing address information is available, such as Australia, Canada, UK, US, etc. Format: Alphabetic, single quote ('), hyphen (-), and whitespace; UTF-8, maximum 100 characters.
billingState	(Conditional) State or province code in ISO 3166-2 format, e.g. NY. Required for countries where billing address information is available, such as Australia, Canada, UK, US, etc. Format: Alphabetic; maximum 3 characters.
billingCountry	(Conditional) Country in ISO 3166-1 alpha-2 format, e.g. US. Required for countries where billing address information is available, such as Australia, Canada, UK, US, etc. Format: Alphabetic; maximum 2 characters.
shippingZip	(Conditional) The postal code associated with the shipping address. Required if deliveryMethod is 02, 03 or 04. Note: This field does not support spaces (" ") in the middle of characters. Format: Alphabetic, numeric, e.g. spaces are not allowed; maximum 3–16 characters.
shippingLine1	(Conditional) Street 1 of shipping address. Required if deliveryMethod is 02, 03 or 04. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; UTF-8, maximum 140 characters.
shippingLine2	(Optional) Street 2 of shipping address. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; UTF-8, maximum 140 characters.
shippingLine3	(Optional) Street 3 of shipping address. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; UTF-8, maximum 140 characters.
shippingLine4	(Optional) Street 4 of shipping address. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; UTF-8, maximum 140 characters.

shippingLine5	(Optional) Street 5 of shipping address. Format: Alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace; UTF-8, maximum 140 characters.
shippingCity	(Conditional) The city associated with the shipping address. Required if <code>deliveryMethod</code> is 02, 03 or 04. Format: Alphabetic, single quote ('), hyphen (-), and whitespace; UTF-8, maximum 100 characters.
shippingState	(Conditional) State or province code in ISO 3166-2 format, e.g. NY, or XX if a state is not applicable for the address. Required if <code>deliveryMethod</code> is 02, 03 or 04. Format: Alphabetic; maximum 3 characters.
shippingCountry	(Conditional) Country in ISO 3166-1 alpha-2 format, e.g. US. Required if <code>deliveryMethod</code> is 02, 03 or 04. Format: Alphabetic; maximum 2 characters.

DAF Authentication Factor Requirements

DAF requires local authentication of the cardholder using Visa-approved CDCVM methods. One authentication method must be from the *possession* (P) category and another from the *knowledge* (K) or *inherence* (I) category. You can specify the method codes in the `authenticationFactorA` and `authenticationFactorB` fields. The table below lists the authentication methods accepted by DAF.

Method Code	Method Description	DAF Category	DAF 2nd Factor Category, if required
01	Username/password	K	P
02	Passcode or password	K	P
Consumer Device Cardholder Verification Method (CDCVM)			
10	Passcode	K	P
11	Password	K	P
12	Pattern	K and P	Not req'd
13	Biometric fingerprint	I and P	Not req'd

Method Code	Method Description	DAF Category	DAF 2nd Factor Category, if required
14	Biometric facial recognition	I and P	Not req'd
15	Biometric iris recognition	I and P	Not req'd
16	Biometric voice recognition	I and P	Not req'd
17	Behavioral biometric	I and P	Not req'd
One Time Passcode (OTP)			
18	Device unlocked (CDCVM unknown)	I or K	P
30	SMS	P	K
31	Email	K	P
40	Knowledge Based Authentication (KBA)	K	P
FIDO			
50	FIDO bound token: FIDO device bound token was provided by the Token Requestor with user presence only – no user verification. Establishes possession.	I and P	Not req'd
51	FIDO bound token: FIDO device bound token was provided by the Token Requestor with biometric user verification. Establishes inherence and possession.	I and P	Not req'd
52	FIDO bound token: FIDO device bound token was provided by the Token Requestor with user verification via passcode, password, or pattern. Establishes knowledge and possession.	K and P	Not req'd
Other			
60	SE based token: Cryptogram generated from a SE device for a device bound token was provided. Establishes possession.	P	Not req'd
61	Device bound token: Device bound token (token reference) was provided by Token Requestor along with proof of device used for binding token. Establishes possession.	P	I or K

Appendix A

ID&V Risk Data Attributes

ID&V Risk Data Attributes

RiskData attributes are packaged as `NAME: VALUE` pairs and passed as a single field, encrypted using Shared Secret encryption scheme, and encoded using JWE. Though technically optional, it is best practice to include all available attributes to provide the highest possible assurance to issuers and increase the chances of successful ID&V.

Field	Size	Type	Description
cvv2	4 [0-9]	String	CVV2 value associated with the 16-digit PAN. Applicable for use cases when cvv2 is sent in the clear. The risk parameters should make sure to pass only one of clear cvv2 or encrypted cvv2 in the risk blob. When both are present, the request would fail in VTS as an invalid request. For CVV2 to be considered during token provisioning, cvv2 or encCvv2 must be provided here.

encCvv2	2048	String	Encrypted CVV2 value associated with the 16- digit PAN. Device keys should be used for encryption. For RSA PKI encryption, the encryption should be represented as JWE/JWS. For Opacity A encryption, the encryption should be represented as JWE. Applicable for use cases when cvv2 is sent encrypted. The risk parameters should make sure to pass only one of clear cvv2 or encrypted cvv2 in the risk blob. When both are present, the request would fail in VTS as an invalid request. For CVV2 to be considered during token provisioning, cvv2 or encCvv2 must be provided here.
paymentInstrument. expirationDate.month	2	String	This field is conditional. Sending this field is required only when the provisioning token is not working because of a bad expiration date provided during PAN enroll API. This field must be sent with paymentInstrument.expirationDate.year.
paymentInstrument. expirationDate.year	4	String	This field is conditional. Sending this field is required only when the provisioning token is not working because of a bad expiration date provided during PAN enroll API. This field must be sent with paymentInstrument.expirationDate.month.
billingZip	3-16 [A-Z][a-z] [0-9]	String	Billing zip of the consumer at the time of provisioning a card.
line1	1-32	String	Street address of the consumer at the time of provisioning a card.
line2	1-32	String	Line 2 of street address of the consumer at the time of provisioning a card.
city	1-100	String	City for the consumer's address at the time of provisioning a card.
state	2	String	State for the consumer's address at the time of provisioning a card; in ISO 3166-2 format, e.g. NY.
country	3-16	String	Country for the consumer at the time of provisioning a card; ISO 3166-1 alpha-2 format, e.g. US.
deviceSerialNumber	24 [A-Z][a-z] [0-9]	String	Serial number of the mobile device.
deviceIMEI	1-24 [A-Z][a-z] [0-9]	String	Hardware ID of the device.

deviceBluetoothMac	1-24 [0-9,A-Z, a- z,;+,/,=]	String	MAC address for Bluetooth.
deviceTimeZone	5 [A-Z][a-z]	String	TZ_Abbreviation. Example: PDT, MSR.
deviceTimeZoneSetting	1 [0-9]	String	One of the two possible values: Enum 1 = NetworkSet, 2= ConsumerSet
seID	24 [A-Z][a-z] [0-9]	String	Hardware ID of SE (Secure Execution Environment)
androidId	24 [0-9,A-Z, a-z,+,/,=]	String	DEPRECATED – Use OSID instead. Android ID to allow building velocity and risk rules.
OSID	24 [0-9,A-Z, a-z,+,/,=]	String	OS ID to allow building velocity and risk rules.
simSerialNumber	24 [A-Z][a-z] [0-9]	String	Sim Serial number of the mobile device.
walletProviderPANAge	4 [0-9999]	String	Length of time in days user's PAN has been on file for that user. If the number of days surpass this, it will stay at 9999.
daysSinceConsumerData LastAccountChange	4 [0-9999]	String	Number of days since account settings were changed, (e.g. password change). If the number of days surpass this, it will stay at 9999.
walletAccount FirstCreated	4 [0-9999]	String	Number of days since user created a wallet account or started using their wallet.
userAccountFirstCreated	4 [0-9]	String	Number of days since a user account for this user exists (i.e., how long have you known this user?).
walletAccount HolderCardNameMatch	1	String	Name of the account holder and name of cardholder match. Y or N for match.
accountHolderName	24 [A-Z][a-z]	String	Account holder's name.
accountToDevice BindingAge	4 [0-9999]	String	Number of days this device used by this account. If the number of days surpass this, it will stay at 9999

provisioningAttemptsOnDevicein24hours	2 [0-256]	String	Number of provisioning attempts on this device within 24 hours. If the number of days surpass this, it will stay at 256.
distinctCardholderNames	2 [0-9]	String	Number of distinct cardholder names used in provisioning from this device.
deviceCountry	2	String	Country of device at time of provisioning. Country in ISO 3166-1 alpha-2 format.
walletAccountCountry	2	String	Country of account holder. Country in ISO 3166-1 alpha-2 format.
suspendedCardsInAccount	2 [0-99]	String	Number of cards suspended in account. If the number of days surpass this, it will stay at 99
daysSinceLastAccountActivity	4 [0-9999]	String	days since last activity on account. If the number of days surpass this, it will stay at 9999.
deviceLostMode	4 [0-9999]	String	Number of days. It can be up to 9999 days.
numberOfTransactionsInLast12months	4 [0-9999]	String	Number of transaction on this account in the last 12 months. If the number of transaction surpass this limit, it will stay at 9999.
numberOfActiveTokens	2 [0-99]	String	Number of active tokens on this account. If the number of tokens surpass this limit, it will stay at 99.
deviceWithActiveTokens	2 [0-99]	String	Number of devices for this user with same token. If the number of tokens surpass this limit, it will stay at 99.
activeTokenOnAllDevicesForAccount	4 [0-9999]	String	Number of active tokens for this user across all devices. If the number of tokens surpass this limit, it will stay at 9999.
billingCountryCode	2	String	Country of cardholder. Country in ISO 3166-1 alpha-2 format.
accountScore	2	Digits	Wallet Provider Account Score. Score value between 1 to 5 indicates confidence level of the account where 5 is most confidence on the account
deviceScore	2	Digits	Wallet Provider Device Score. Score value between 1 to 5 indicates confidence level of the device where 5 is more confidence on the device.
deviceID	24 [A-Z][a-z] [0-9,-,_,]		Stable and unique identifier set for the device. Note: This field applies to all token types.

deviceName	256	String.	A readable name for the device. Ideally, this name would be entered by the consumer. It can be used to identify the device in Customer Support calls. Base64 URL-encoded since some platforms allow emoticons for the device name. This field applies to all token types.
deviceNumber	32 bytes	String.	The client's valid mobile phone number, per issuer records.
deviceType	2-17 [A-Z][a-z] [0-9,-_]	String.	Device type It is one of the following values: • PHONE • TABLET • PHONE_TABLET • PC • WATCH • TV • WEARABLE_DEVICE • AUTOMOBILE_DEVICE • HOUSEHOLD_DEVICE This field applies to all token types.
consumerIdentifier	88	String.	Market-specific consumer identification; maximum 88 bytes sent as a base64-encoded string.

Appendix B

Issuer Support for App-to-App Step Up Mode

Issuer Support for App-to-App Step Up Mode

Wallet providers will receive mobile banking application metadata through the enrollment and provisioning API responses when applicable. This will enable the wallet provider to associate the issuer's mobile banking application to the card being added to the wallet.

There are two use cases that may return mobile banking information. The generic Mobile Banking Application information will be passed in the enroll PAN response and mobile banking application details for performing app-to-app step-up will be provided in the Provision Token responses if the issuer supports an app-to-app authentication method.

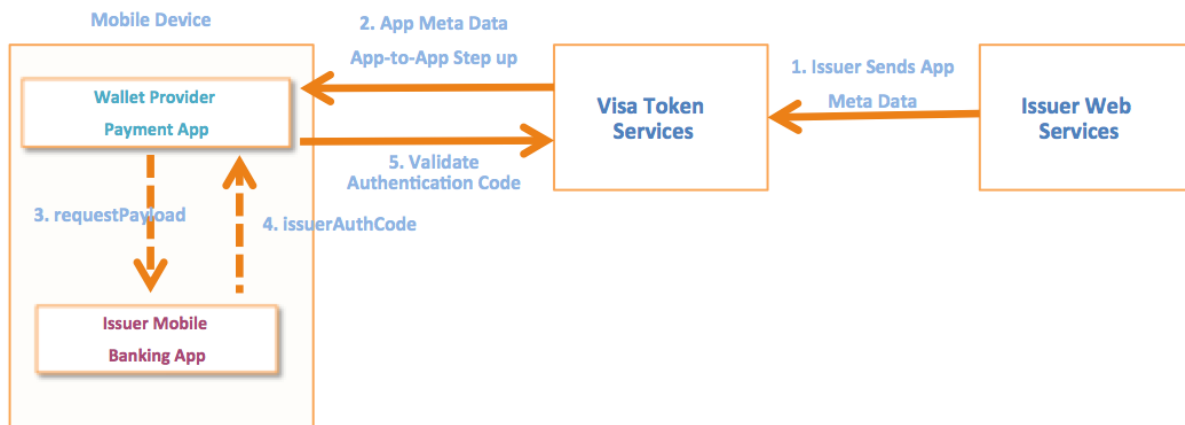
This appendix will provide details on the API response related to app-to-app and how the wallet provider should use the content to establish connectivity to the Mobile Banking Application for app-to-app step-up.

The following is an example of the response JSON that would be returned in an issuer supports app-to-app as a step-up method.

```
{
  "method": "APP-TO-APP",
  "value": "BANKAPP",
  "identifier": "APP2APPSDFD1234F4",
  "source": "com.wf.sampleIssuermobile|a2a",
  "requestPayload":
  "6C51CAE8CF7DFF6C51CAE8CF7DFFBFBFD536BEFC97BBF41BFBFD536BEFC97BBF41"
}
```

Issuers will be required to provide necessary mobile app meta data and encrypted payload to assist Wallet Providers in building the user experience. the `stepUpRequest.appMetaData.requestPayload` in Step Up Request.

To securely authenticate the consumer's step up method, issuers will also construct secure code to be passed to the issuer mobile app by the wallet provider app:



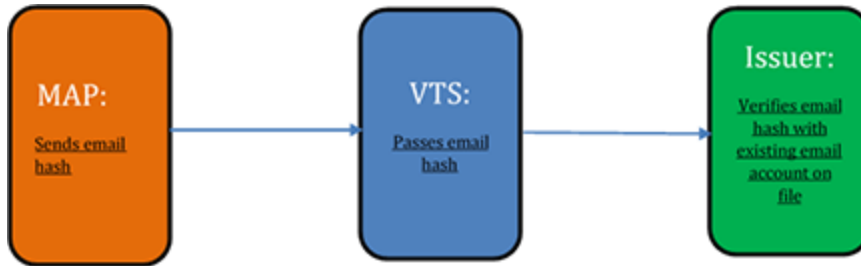
In response, the issuer mobile app will return `issuerAuthCode`. The issuer `authCode` is a BLOB encrypted using Triple DES symmetric key which contains the PAN and the consumer name. For additional information, Bank App based Step Up v1.1.

Appendix C

Email Hashing Requirements

Email Hashing Requirements

Due to privacy restriction policies, the email address of a consumer should not be disclosed to parties that don't already possess it. Emails are classified as PII (Personally Identifiable Information) and, as such, their confidentiality must be guarded. Some parties still need an identifier that is passed around to enable some use cases. Using a digest of the email address is an acceptable method of not disclosing the email address of a consumer. The following diagram shows the intended use case for sharing email confidentially for verification:



- MAP: Is the entity that does not wish to disclose the email address to VTS (or any entity)
- VTS: A pass through that forwards the email digest to the issuer
- Issuer: Is the entity that knows the email address, and performs a cryptographic hash operation to validate the hash passed to it by VTS

There are certain conditions the digest or hash algorithm needs to meet:

- Theoretically infeasible to reverse the digested email and extract the clear text email address
- Resist rainbow pre-image attacks
- SHA2-256
- Hash (email + f(email))
- f = 1000 iterations of Hash (email).

The formula is: SHA (SHA (email) || SHA (1000 times) (email)); where “||” indicates a concatenation operation. We only care about preserving the clear text of the email address. Although typically we would care about the two properties of a hash algorithm listed below:

1. Preimage resistant: For a given H in the output space of the Hash function, it is computationally infeasible to find any message M with $H(M) = H$
2. Second-preimage resistant: For a given message M1, it's computationally infeasible to find a second message M2 $\neq$ M1 with $H(M1) = H(M2)$.

None of the above properties are of concern because our goal is only to preserve the confidentiality of the email address, and we are not concerned with work factor of finding a collision.

Example -

Suppose the email address is: *person@corp.com*

So, in the code snippet below, the variable **email** = person@corp.com

```
1.String getEmailHashAlgorithmValue(String email) {  
    // Use a consistent case, e.g. lower case  
2. String currentHash = email.toLowerCase();  
3.     for (int i=0; i<1000; ++i) {  
4.         currentHash = SHA256(currentHash);  
5.     }  
}
```

```
6.  return SHA256(SHA256(email) + currentHash);  
7. }
```

References:

- 1) Attacks on Cryptographic Hashes in Internet Protocols: <https://tools.ietf.org/html/rfc4270>
- 2) Rainbow tables: http://en.wikipedia.org/wiki/Rainbow_table
- 3) GPU usage: <http://on-demand.gputechconf.com/gtc/2014/presentations/S4507-evaluation-of-parallel-hashing-techniques.pdf>
- 4) Tools: <http://www.cryptohaze.com/gpurainbowcracker.php>

Appendix D

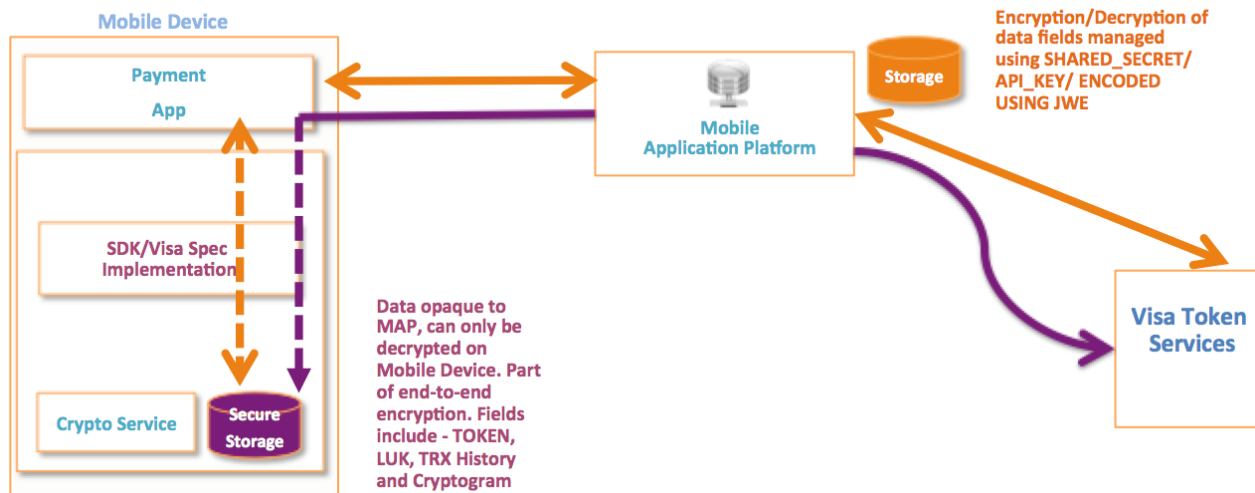
Encrypting and Decrypting Data

Data Encryption and Decryption Summary

VTs supports two security models to share encrypted data with MAP and Mobile Device:

- Shared secret/API Key based JWE encoded data encryption
- Wallet provider specified data encryption scheme used for end-to-end encryption, e.g. – RSA PKI and OPACITY_A.
- Key material (public key, vCertificateID) required for end-to-end data encryption will be shared via Visa Developer Center
- Key material for devices participating in data-in-transit protection will be exchanged via “Enroll Device” API, functional APIs will be carrying `encryptionMetaData` payload
- Visa uses JSON Web Encryption (JWE) to encode sensitive information. Typically, all encrypted input parameters should be constructed before sending in an API request. Tokens, LUKs, shared secret, and consumer account numbers (PANs) must never be transmitted unencrypted.

Also as a general practice, PCI DSS and other security guidelines must be applied when handling PAN data.



Field Level Encryption/Decryption

D = Device	Device encrypts
M = MAP	Device decrypts
V= VTS	MAP can encrypt
Encryption Scheme (ES)	MAP can decrypt
Shared Secret	
Data-in-Transit (RSA PKI or OPACITY_A)	

API	Req/Res	Attribute Field	D	M	ES
Enroll PAN	Request	encPayment Instrument	Device encrypts	MAP can encrypt MAP can decrypt	Could be encrypted using 1 or 2. Depending on the implementation model.
	Response	N/A			

Visa Digital Solutions – API Reference Guide
Encrypting and Decrypting Data

Enroll Device	Request	N/A, This API sets <code>channelSecurityContext</code> for E2E encryption scheme for a particular device.			
	Response	N/A			
Get PAN Enrollment & Card Meta Data	Request	N/A			
	Response	N/A			
Provision Token (PAN Enrollment ID)	Request	<code>encRiskDataInfo</code>		MAP encrypts	1
	Response	<code>encTokenInfo</code>	Device decrypts	MAP can decrypt*	1
		<code>encKeyInfo</code>	Device decrypts	MAP cannot decrypt	2
Provision Token (PAN)	Request	<code>encPaymentInstrument</code>	Device encrypts	MAP can encrypt MAP can decrypt	Could be encrypted using 1 or 2. Depending on the implementation model.
	Request	<code>encRiskDataInfo</code>		MAP encrypts	1
	Response	<code>encTokenInfo</code>	Device decrypts	MAP can decrypt*	1
		<code>encKeyInfo</code>	Device decrypts	MAP cannot decrypt	2
Confirm Provisioning	Request	N/A			
	Response	N/A			
Submit ID&V Step up method	Request	N/A			
	Response	N/A			
Validate OTP	Request	N/A			
	Response	N/A			
Validate Authentication Code	Request	N/A			
	Response				

Visa Digital Solutions - API Reference Guide
Encrypting and Decrypting Data

Payment Data (PAN Enroll ID)	Request	N/A			
	Response	encTokenInfo	Device decrypts	MAP can decrypt*	1
Payment Data	Request	encRiskDataInfo		MAP encrypts	1
	Response	encTokenInfo	Device decrypts	MAP can decrypt*	1
Notifications	Request	N/A			
	Response	N/A			
Notifications Transaction History	Request	N/A			
	Response	encTransaction Info	Device can decrypt	MAP can decrypt* (Requires business agreement)	
Card Metadata Update	Request	N/A			
	Response	N/A			
AAM Replenish	Request	N/A			
	Response	encKeyInfo	Device can decrypt	MAP cannot decrypt	2
AAM Delete	Request	N/A			
AAM Confirm Replen- ishment	Request	N/A			
	Response	N/A			
Life Cycle Management Suspend	Request	N/A			
	Response	N/A			
Life Cycle Management Resume	Request	N/A			
	Response	N/A			

Life Cycle Management	Request	N/A			
Delete					
	Response	N/A			

Note:

Ability of MAP to decrypt Token information is conditional and based on the implementation model used by Wallet Provider.

Encryption

VTs API uses following naming convention for fields requiring encryption:

```
"enc<FIELD>info" : "JWE ( ... ) "
```

The JSON Web Encryption (JWE) content could be either:

- signed or encrypted using the shared secret that was provided to you at the time of onboarding, or
- encrypted using end-to-end encryption scheme as specified in the channelSecurityContext field of Enroll Device API. To identify which fields qualify for data-in-transit protection scheme, follow the Field Level Encryption/Decryption table above.

Visa uses the compact serialization style. Base64url encoded and separated by ".".

JWE and JWS Details and Composition

Compact serialization scheme is followed for JWE.

Sample JWE Using API Key/Shared Secret

JWE Header

```
"header": {
  "alg": "A256GCMKW",
  // Encryption algorithm to be used for encryption of CEK
  "iv": <"SizeofIVistobe96bit.">,
  // IV to be used for encryption of CEK
  "tag": <"128bitvalue">,
  // HMAC generated from applying AES-256-GCM-KW to the CEK
  "kid": "50charAPIKey", // API key
  "channelSecurityContext": "SHARED_SECRET",
  "enc": "AGCM256",
  "iat": "1429837145", // iat is mandatory for push provisioning.
  // It is also conditionally required when the exp header is present.
  "exp": "1429835524" //Expiration time of the payload.
}
```

```
// This field is optional and should be evaluated
// in conjunction with the ttl default.
// Exp can be used to reduce the default ONLY.
//If exp represents a time greater than iat + ttl,
// iat + ttl will be used.
"iss": "12345678" // Message originator client id
// Issuer when sending payment instrument.
// Token requestor when sending response.
"aud": "128827728" //Client ID of desired recipient
// Token requestor when sending payment instrument.
// Issuer when sending response.
}
```

Note:

If the provider of the JWE is the issuer of the PAN, Visa requires an additional set of header fields to be provided.

JWE Body

```
"encrypted_key": "UghIOgu ... MR4gp_A=" , //base64 encoded form.
// CEK encrypted using A256GCMKW (alg) algorithm and the CEK IV
"iv": "AxY8DctDa...GlsbGljb3RoZQ=",
// base64 encoded form. IV for the text encryption.
// Size of IV is to be 96 bit
"ciphertext": "KDlTthhZTGufMY.....xPSUrfmqCHXaI9wOGY=",
// Base64 encoded form.
// Encrypted blob generated using the AES-GCM encryption
// (enc) of the text to encrypt
"tag": "Mz-VPPyU4...RlcuYv1IwIvzw=" // base64 encoded form.
// HMAC generated using
// the AES-GCM encryption of the text to encrypt.
// The size of the tag is to be 128 bits.
```

Note:

The JWE Protected Header is input as the AAD (Additional Authenticated Data) parameter of the authenticated encryption (AES-GCM) of the “text to encrypt”.

JWE Composition

```
BASE64URL (UTF8 (JWE Header)) || '.' ||
BASE64URL (JWE Encrypted Key) || '.' ||
BASE64URL (JWE IV) || '.' ||
BASE64URL (JWE Ciphertext) || '.' ||
BASE64URL (JWE Authentication Tag)
```

Note:

JWE/JWS specification requires BASE64URL encoding with NO padding.

General Approach for JSON Web Encryption using API Key/Shared Secret

1. Visa uses the compact serialization style. (elements separated by ".").
2. All fields are base 64 – Url Safe encoded.
3. Visa will use a CEK – Content Encryption Key – 256bits size.
4. You must use an AES-GCM-256KW algorithm and an Initialization Vector (IV) for encryption of content encryption key. Size of IV is to be 96 bit.
5. You must use an AES-GCM-256 algorithm and an Initialization Vector (IV) for the encryption of the payment instrument. Size of IV is to be 96 bit.
6. An authentication tag will be generated as additional output from the AES-GCM-256 encryption. Size of this field is 128 bits.
7. All string-to-byte and vice-versa conversions are done with UTF-8 charset.

Note:

Refer to the JWE section for a more detailed overview of the complete specification.

<https://tools.ietf.org/html/draft-ietf-jose-json-web-encryption-40>

Steps for Creating Encrypted Data by Applying JSON Web Encryption on Plaintext Data

1. Get the shared secret for the API key.
2. Create a SHA-256 digest of the shared secret.
3. Generate a random Content Encryption Key (CEK)
4. Encrypt the CEK for the recipient using the shared secret digest (32 byte value), the 96 bit random IV, and the algorithm AGCM256-KW specified in `alg` element. Base64UrlSafe encode to produce an Encrypted-KEY.
5. Generate a random IV of length 96 bit. Base64-UrlSafe encode to produce Payload-IV.
6. Encrypt plaintext data using the CEK and Payload-IV, and the algorithm AGCM256 specified in the `enc` section to form the ciphertext and the Payload Tag data.
7. Base64-UrlSafe encode the encrypted bytes to produce ciphertext.
8. Base64-UrlSafe encode the Tag data to produce TAG.
9. Base64-UrlSafe encode the entire JWE Header JSON containing the encryption parameters used in clear text as shown, to produce the Encoded-HEADER:

```
|| \.' ||  
BASE64URL (JWE Encrypted Key) || \.' ||  
BASE64URL (JWE IV) || \.' ||  
BASE64URL (JWE Ciphertext) || \.' ||  
BASE64URL (JWE Authentication Tag).
```

Note:

The encrypted CEK and IV fields are represented as empty.

Decryption

- Base64-UrlSafe decode the E-Header field. Get the API key and the algorithms to use for decryption.
- Base64-UrlSafe decode the E-Key field to get the encrypted E-key (encrypted CEK). Decrypt the encrypted E-key field using Shared secret and IV for CEK and encryption algorithm as specified in alg element (AGCM256-KW). Get the Content Encryption Key (CEK)
- Base64-UrlSafe decode the E-IV for use in ciphertext decryption.
- Base64-UrlSafe decode the E-Authentication Tag for use in ciphertext decryption.
- Base64-UrlSafe decode the ciphertext field. Decrypt the Ciphertext using CEK, IV and authentication Tag and AAD (header field) and encryption algorithm as specified in enc element (AGCM256), to get clear JSON string.

Sample JWE Using Opacity A

JWE Header

```
"header":
{
  {
    "enc": "AGCM256", //Encryption algorithm to be used for encryption of CEK
    "typ": "JOSE",
    "kid": "1234", //clientIdDeviceID
    "channelSecurityContext": "Opacity_A",
    "iat": "1429837145" // Time in UTC when the JWE was issued.
  }
}
```

JWE Body

```
"ciphertext": "KDlTthhZTGufMY.....xPSUrfmqCHXaI9wOGY=",
// Base64 encoded form.
// Encrypted blob generated using the AES-GCM encryption (enc)
// of the text to encrypt
"iv": "AxY8DctDa...GlsbGljb3RoZQ=", // base64 encoded form.
// IV for the text encryption. Size of IV is to be 96 bit
"tag": "Mz-VPPyU4...RlcuYv1IwIvzw=" // base64 encoded form.
// HMAC generated using the AES-GCM encryption of the text to encrypt.
// The size of the tag is to be 128 bits.
```


JWE Composition

```
BASE64URL (UTF8 (JWE Header)) || '.' ||  
BASE64URL (JWE Encrypted Key) || '.' ||  
BASE64URL (JWE IV) || '.' ||  
BASE64URL (JWE Ciphertext) || '.' ||  
BASE64URL (JWE Authentication Tag)
```

General approach for JSON Web Encryption Using Opacity A

1. Visa uses the compact serialization style. (elements separated by ".")
2. All fields are base 64 – Url Safe encoded.
3. You must use AES-GCM-256 algorithm for encryption of payment instrument. Initialization Vector (IV) is used for encryption. Use an Initialization Vector (IV) for encryption. Size of IV is to be 96 bit.
4. Authentication Tag will be generated as an additional output of the AES-GCM-256 encryption. Size of this field is 128 bits.
5. JWE header is not used to pass as AAD (Additional Authentication Data) for the AES-GCM-256 operation.
6. All String to byte and vice-versa conversions are done with UTF-8 charset.
7. Data Classification Tag Scheme – When the text is encrypted, a tag of size 1 byte is added in the front of the text and then encrypted. This tag is to flag the type of data being encrypted. If the tag is LUK, it implies this data is super sensitive and should not be decrypted to clear outside the secure zone.
8. When the text is decrypted to clear, the first byte needs to be extracted and inspected to determine the data type. Based on the data type, it can be handled as super sensitive or otherwise.

Note:

Refer to the JWE section for a more detailed overview of the complete specification.

<https://tools.ietf.org/html/draft-ietf-jose-json-web-encryption-40>

Steps for Creating Encrypted Data by Applying JSON Web Encryption on Plaintext Data

1. Generate the ECDH and perform the KDF operation to arrive at the secret key for the encryption.
2. Pad the plaintext in front with 1 byte of data using the data classification tag.
3. Generate a random IV of length 96 bit. Base64-UrlSafe encode to produce E-IV.
4. Encrypt plaintext data using the derived Secret key, and Payload-IV, and the algorithm AGCM256 specified in the enc section to form the ciphertext and the Payload Tag data.

5. Create the JWE header using *enc*, *typ*, *kid* and *channelSecurityContext* parameters as shown above in the example.
6. Base64-UrlSafe encode the encrypted bytes to produce Ciphertext.
7. Base64-UrlSafe encode the Tag data to produce TAG.
8. Base64-UrlSafe encode the entire JWE Header JSON containing the encryption parameters used in clear text as shown, to produce the Encoded-HEADER:

```
|| \.' ||  
BASE64URL (JWE IV) || \.' ||  
BASE64URL (JWE Ciphertext) || \.' ||  
BASE64URL (JWE Authentication Tag).
```

The encrypted CEK and IV fields are represented as empty.

Decryption

1. Base64-UrlSafe decode the E-Header field. Get the kid identifier (device Id) and the algorithms to use for decryption.
2. Fetch the seeding keys for ECDH operation and perform ECDH and KDF operation to arrive at the Secret key
3. Base64-UrlSafe decode the E-Authentication Tag for use in ciphertext decryption.
4. Base64-UrlSafe decode the E-IV for use in ciphertext decryption.
5. Base64-UrlSafe decode the E-CipherText field to get the ciphertext. Decrypt the cipher text using Secret key, and authentication Tag and encryption algorithm as specified in alg element (AGCM256).

Sample JWE Using RSA PKI

Compact serialization scheme is followed for both JWE and JWS.

JWE Header

```
"header":  
{  
  {  
    "alg": "RSA1_5", //Encryption algorithm to be used for encryption of CEK  
    "kid": "1234", //clientDeviceID  
    "typ": "JOSE",  
    "channelSecurityContext": "RSA_PKI",  
    "enc": "A128GCM", // Encryption algorithm to be used for encryption of  
    text to encrypt  
    "iat": "1429837145" // Time in UTC when the JWE was issued.  
  }  
}
```

JWE Body

```
"encrypted_key": "Ugh...A=" , // Base64 encoded form.  
// CEK encrypted using RSA-OAEP-256 algorithm  
"iv": "AxY...ZQ=", // base64 encoded form .  
// IV for the text encryption. Size of IV is to be 96 bit  
"ciphertext": "KdL...GY=", // Base64 encoded form.  
// Encrypted blob generated using the AES-GCM encryption(enc)  
// of the text to encrypt  
"tag": "Mz-...zw=" // Base64 encoded form .  
// HMAC generated using the AES-GCM encryption of the text to encrypt.  
// The size of the tag is to be 128 bits.
```

For RSA_PKI, during encryption, VTS prefixes one of the following values to the tag field, which indicates the kind of encrypted data:

- 21 – PAN data
- 22 – LUK data
- 23 – Token data
- 24 – Transaction history data
- 25 – Cryptogram data
- 26 – ICC private key data
- 27 – QR code data
- 28 – Cardholder PII data

Note:

When making wallet-to-VTS calls using the the RSA_PKI encryption scheme, the request must be prefixed with the tag value prior to encryption. The prefix should be represented in byte, e.g; 21 will be 0x15.

JWE Composition

```
BASE64URL (UTF8 (JWE Header)) || '.' ||  
BASE64URL (JWE Encrypted Key) || '.' ||  
BASE64URL (JWE IV) || '.' ||  
BASE64URL (JWE Ciphertext) || '.' ||  
BASE64URL (JWE Authentication Tag)
```

General approach for JSON Web Encryption Using PKI

1. Visa uses the compact serialization style. (elements separated by ".").
2. All fields are base 64 – Url Safe encoded.
3. Visa will use the hybrid encryption scheme. This means using an RSA 2048 key for encryption of a random symmetric key. The random symmetric key is then used to encrypt the text.

4. You must use RSA1_5 algorithm for encryption of random symmetric key
5. You must use AES-GCM-128 algorithm for encryption of text and Use an Initialization Vector (IV) for encryption. Size of IV is to be 96 bit.

Note: JWE header is not used to pass as AAD (Additional Authentication Data) for the AES-GCM-128 operation.

6. An authentication tag will be generated as additional output from the AES-GCM-128 encryption. Size of this field is 128 bits.
7. All string-to-byte and vice-versa conversions are done with UTF-8 charset.

Note:

Refer to the JWE section for a more detailed overview of the complete specification.

<https://tools.ietf.org/html/draft-ietf-jose-json-web-encryption-40>

Steps for Creating Encrypted Data by Applying JSON Web Encryption on Plaintext Data

1. Get the RSA 2048 public Key.
2. Generate a random symmetric Key (RSK)
3. Encrypt the RSK using the RSA 2048 key using the algorithm specified in alg(RSA1_5)
4. Generate a random IV of length 96 bit. Base64-UrlSafe encode to produce E-IV.
5. Encrypt plaintext data using the RSK and Payload-IV, and the algorithm AGCM128 specified in the enc section to form the ciphertext and the Payload Tag data.
6. Base64-UrlSafe encode the encrypted bytes to produce E-ciphertext.
7. Base64-UrlSafe encode the Tag data to produce E-TAG.
8. Base64-UrlSafe encode the RSK to produce E-Encrypted Key.
9. Base64-UrlSafe encode the entire JWE Header JSON containing the encryption parameters used in clear text as shown, to produce the Encoded-HEADER:

```
|| \.' ||  
BASE64URL (JWE Encrypted Key) || \.' ||  
BASE64URL (JWE IV) || \.' ||  
BASE64URL (JWE Ciphertext) || \.' ||  
BASE64URL (JWE Authentication Tag).
```

Decryption

1. Get the RSA 2048 private Key.
2. Base64-UrlSafe decode the E-Header field. Get the kid (deviceId) and the algorithms to use for decryption.

3. Base64-UrlSafe decode the E-Key field to get the encrypted E-key (Random Symmetric Key). Decrypt the encrypted E-key field using RSA 2048 private key with encryption algorithm as specified in alg element (RSA1_5). Get the RSK in clear.
4. Base64-UrlSafe decode the E-IV for use in ciphertext decryption.
5. Base64-UrlSafe decode the E-Authentication Tag for use in ciphertext decryption.
6. Base64-UrlSafe decode the ciphertext field. Decrypt the ciphertext using RSK and IV and authentication Tag and encryption algorithm as specified in enc element (AGCM128).

Sample JWS Using RSA PKI

JWS Header

```
"header": {  
  {  
    "alg": "RS256",  
    //Encryptionalgorithmusedforsignature"kid": "1234",  
    //clientDeviceID"typ": "JOSE",  
    "cty": "JWE"//ContenttypeofthecontentinsideJWS  
  }  
}
```

JWS Payload and Signature

The JWS payload will be the JWE which is going to be signed.

A sample JWS signature looks like this:

```
"signature": "rDx...Pw"
```

JWS Composition

```
BASE64URL (UTF8 (JWS Protected Header)) || '.' ||  
BASE64URL (JWS Payload) || '.' ||  
BASE64URL (JWS Signature)
```

General approach for JSON Web Signature Using PKI

- Visa uses the compact serialization style. (elements separated by ".")
- All fields are base 64 – Url Safe encoded.
- You must use RSASSA-PKCS1-v1_5 (RS256 in terms of JWS algorithm notation) algorithm for signature.
- All String to byte and vice-versa conversions are done with UTF-8 charset.

Steps for Creating a Signature by Applying JSON Web Signature on JWE

1. Get the RSA 2048 private Key.
2. Sign the JWE (JWS Payload) using the RSA 2048 private key using the algorithm specified in alg (RS256) to get the JWS Signature.
3. JWS Signing Input: ASCII (BASE64URL (UTF8 (JWS Protected Header)) || '.' || BASE64URL (JWS Payload))
4. Package the header using alg, kid, cty and typ params as shown in the above example for JWS.
5. Base64-UrlSafe encode the JWS Signature to produce E-Signature.
6. Base64-UrlSafe encode the JWE to produce E-JWS Payload.
7. Base64-UrlSafe encode the entire JWS Header JSON containing the encryption parameters used in clear text as shown, to produce the Encoded-HEADER:

```
|| \.' ||  
BASE64URL (JWE or the JWS payload) || \.' ||  
BASE64URL (JWS Signature) || \.' ||
```

Signature Validation

Validate the JWS Signature against the JWS Payload using RSA 2048 public key with encryption algorithm as specified in alg element (RS256):

1. Get the RSA 2048 public Key.
2. Base64-UrlSafe decode the E-Header field. Get the kid (deviceId) and the algorithms to use for signature validation.
3. Base64-UrlSafe decode the E-Signature field to get the JWS Signature.
4. Base64-UrlSafe decode the E-JWS Payload to get the JWS Payload.

Sample JWE Using RSA PKI2

Compact serialization scheme is followed for both JWE and JWS.

JWE Header

```
"header":  
{  
  {  
    "alg":"RSA-OAEP-256", //Encryption algorithm to be used for encryption  
    of CEK  
    "kid":"1234", // Visa cert ID  
    "typ" : "JOSE",  
    "channelSecurityContext" : "RSA_PKI2",
```

```
"enc": "AGCM128", // Encryption algorithm to be used for encryption of
text to encrypt
"iat": "1429837145" // Time in UTC when the JWE was issued.
}
}
```

JWE Body

```
"encrypted_key": "Ugh..._A=" , // Base64 encoded form.
// CEK encrypted using RSA-OAEP-256 algorithm
"iv": "AxY...ZQ=", // base64 encoded form .
// IV for the text encryption. Size of IV is to be 96 bit
"ciphertext": "KDl...GY=", // Base64 encoded form.
// Encrypted blob generated using the AES-GCM encryption(enc)
// of the text to encrypt
"tag": "Mz-...zw=" // Base64 encoded form .
// HMAC generated using the AES-GCM encryption of the text to encrypt.
// The size of the tag is to be 128 bits.
```

JWE Composition

```
BASE64URL (UTF8 (JWE Header)) || '.' ||
BASE64URL (JWE Encrypted Key) || '.' ||
BASE64URL (JWE IV) || '.' ||
BASE64URL (JWE Ciphertext) || '.' ||
BASE64URL (JWE Authentication Tag)
```

General approach for JSON Web Encryption Using PKI2

1. Visa uses the compact serialization style. (elements separated by ".").
2. All fields are base 64 – Url Safe encoded.
3. Visa will use the hybrid encryption scheme. This means using an RSA 2048 key for encryption of a random symmetric key. The random symmetric key is then used to encrypt the text.
4. You must use RSA-OAEP-256 algorithm for encryption of random symmetric key
5. You must use AES-GCM-128 algorithm for encryption of text and Use an Initialization Vector (IV) for encryption. Size of IV is to be 96 bit.
6. An authentication tag will be generated as additional output from the AES-GCM-256 encryption. Size of this field is 128 bits.
7. All string-to-byte and vice-versa conversions are done with UTF-8 charset.

Note:

Refer to the JWE section for a more detailed overview of the complete specification.

<https://tools.ietf.org/html/draft-ietf-jose-json-web-encryption-40>

Steps for Creating Encrypted Data by Applying JSON Web Encryption on Plaintext Data

1. Get the RSA 2048 public Key.
2. Generate a random symmetric Key (RSK)
3. Encrypt the RSK using the RSA 2048 key using the algorithm specified in alg(RSA-OAEP-256)
4. Generate a random IV of length 96 bit. Base64-UrlSafe encode to produce E-IV.
5. Encrypt plaintext data using the RSK and Payload-IV, and the algorithm AGCM128 specified in the enc section to form the ciphertext and the Payload Tag data.
6. Base64-UrlSafe encode the encrypted bytes to produce E-ciphertext.
7. Base64-UrlSafe encode the Tag data to produce E-TAG.
8. Base64-UrlSafe encode the RSK to produce E-Encrypted Key.
9. Base64-UrlSafe encode the entire JWE Header JSON containing the encryption parameters used in clear text as shown, to produce the Encoded-HEADER:

```
|| \.' ||  
BASE64URL (JWE Encrypted Key) || \.' ||  
BASE64URL (JWE IV) || \.' ||  
BASE64URL (JWE Ciphertext) || \.' ||  
BASE64URL (JWE Authentication Tag).
```

Decryption

1. Get the RSA 2048 private Key.
2. Base64-UrlSafe decode the E-Header field. Get the kid (deviceId) and the algorithms to use for decryption.
3. Base64-UrlSafe decode the E-Key field to get the encrypted E-key (Random Symmetric Key). Decrypt the encrypted E-key field using RSA 2048 private key with encryption algorithm as specified in alg element (RSA-OAEP-256). Get the RSK in clear.
4. Base64-UrlSafe decode the E-IV for use in ciphertext decryption.
5. Base64-UrlSafe decode the E-Authentication Tag for use in ciphertext decryption.
6. Base64-UrlSafe decode the ciphertext field. Decrypt the ciphertext using RSK and IV and authentication Tag and encryption algorithm as specified in enc element (AGCM128).

Sample JWS Using RSA PKI2

JWS Header

```
"header": {  
  {  
    "alg": "PS256",  
    //Encryptionalgorithmusedforsignature"kid": "1234",  
    //clientDeviceID"typ": "JOSE",  
    "cty": "JWE"//ContenttypeofthecontentinsideJWS  
  }  
}
```

JWS Payload and Signature

The JWS payload will be the JWE which is going to be signed.

A sample JWS signature looks like this:

```
"signature": "rDx...Pw"
```

JWS Composition

```
BASE64URL (UTF8 (JWS Protected Header)) || '.' ||  
BASE64URL (JWS Payload) || '.' ||  
BASE64URL (JWS Signature)
```

General approach for JSON Web Signature Using PKI2

- Visa uses the compact serialization style. (elements separated by ".")
- All fields are base 64 – Url Safe encoded.
- You must use RSASSA-PSS (PS256 in terms of JWS algorithm notation) algorithm for signature.
- All String to byte and vice-versa conversions are done with UTF-8 charset.

Steps for Creating a Signature by Applying JSON Web Signature on JWE

1. Get the RSA 2048 private Key.
2. Sign the JWE (JWS Payload) using the RSA 2048 private key using the algorithm specified in alg (PS256) to get the JWS Signature.
3. JWS Signing Input: ASCII (BASE64URL (UTF8 (JWS Protected Header)) || '.' || BASE64URL (JWS Payload))
4. Package the header using alg, kid, cty and typ params as shown in the above example for JWS.

5. Base64-UrlSafe encode the JWS Signature to produce E-Signature.
6. Base64-UrlSafe encode the JWE to produce E-JWS Payload.
7. Base64-UrlSafe encode the entire JWS Header JSON containing the encryption parameters used in clear text as shown, to produce the Encoded-HEADER:

```
|| \.' ||  
BASE64URL (JWE or the JWS payload) || \.' ||  
BASE64URL (JWS Signature) || \.' ||
```

Signature Validation

Validate the JWS Signature against the JWS Payload using RSA 2048 public key with encryption algorithm as specified in alg element (PS256):

1. Get the RSA 2048 public Key.
2. Base64-UrlSafe decode the E-Header field. Get the kid (deviceId) and the algorithms to use for signature validation.
3. Base64-UrlSafe decode the E-Signature field to get the JWS Signature.
4. Base64-UrlSafe decode the E-JWS Payload to get the JWS Payload.

Appendix E

Opacity A End-to-End Encryption Scheme

Opacity A End-to-End Encryption Scheme

This appendix describes the end to end encryption solution using OPACITY_A between Visa and wallet provider partner devices across proxies and intermediaries. It provides details of the cryptographic concepts and steps as well as the acceptable and recommended algorithm for the cryptographic steps based on the SP-800-56A standard:<http://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-56Ar2.pdf>. Specifically, you must implement a Diffie-Hellman algorithm to encode shared secrets and decode shared secrets sent to you from Visa Token Services. The following information may be useful when implementing the algorithm.

Acronyms

Term	Definition
Q_ec	Elliptic curve public key generated on the client device.
d_ec	Client ephemeral Elliptic curve private key, matched with the corresponding ephemeral public key: Q_ec.
KDF	Key Derivation Function.
ECDH	Elliptic Curve Deffie-Hellman.

Request Encryption

During the *request encryption*, the following sequence of cryptographic steps are used to derive Shared Key, which can be used for encryption. This sequence of steps has to be performed at both the client side and the server side for each call from client side to server side or vice-versa.

1. EC Key Generation - This is the first step which happens. An Elliptic curve key pair is generated at the client side – public key Q_ec and private key d_ec.
2. ECDH - During this step, the Visa public key and client d_ec are used as per ECDH protocol to derive a secret key Z.
3. KDF - During this step, key derivation functions on Z are performed to generate a single Shared Key: SK.
4. AEAD - During this step, the shared key is used to perform authenticated encryption on the "text to encrypt."

Request Decryption

The generated Q_ec is transmitted to the server side along with the encrypted text. During the *request decryption*, the following set of steps are performed on the server side during decryption of the request.

1. ECDH - During this step, the Visa private key and client Q_ec are used as per ECDH protocol to derive a secret key Z.
2. KDF - During this step, key derivation functions on Z are performed to generate a single Shared Key: SK.
3. AEAD - During this step, the shared key is used to perform authenticated decryption on the "text to encrypt."

Response Encryption

During the *response encryption*, the server performs following steps as below 3a-3c. Alternatively, the shared key: SK can be retained from the request decryption process, and just step 3c below needs to be performed to encrypt the “text to encrypt”. The server sends the Q_ec back to the client in the encryptionmetadata field.

1. ECDH - During this step, the Visa private key and client Q_ec are used as per ECDH protocol to derive a secret key Z.
2. KDF - During this step, key derivation functions on Z are performed to generate a single Shared Key: SK.
3. AEAD - During this step, the shared key is used to perform authenticated encryption on the “text to encrypt.”

Response Decryption

During the *response decryption*, the client retrieves the d_ec it had retained from the request encryption process, and performs similar steps as below 4a-4c. To perform this step, it has to retain the d_ec which was generated initially till the time it is about to decrypt the response.

1. ECDH - During this step, the Visa public key and client d_ec are used as per ECDH protocol to derive a secret key Z.
2. KDF - During this step, key derivation functions on Z are performed to generate a single Shared Key: SK.
3. AEAD - During this step, the shared key is used to perform authenticated decryption on the “text to encrypt.”

Additional Information

- **Off-band provisioning of Visa public key.** Visa public keys are provided to the wallet provider as out of band provisioning step. Real-time transmission of Wallet Provider public key. WP public key is transmitted real time during the business communication between client and the server. It uses the functional request (the business API itself) to do this, and does not require and an additional call.
- **Encryption metadata.** VTS APIs which transmit sensitive data (PAN, Token, and LUK) across between WP and server will have a field called `encryptionMetadata`. This field is used to pass metadata related to the encryption between WP and the VTS server in real time.
- **ECC public keys.** They are 64 bytes and required.

- **Retention of Qec for the response processing.** The client side needs to briefly store the Q_{ec} and d_{ec} generated during request encryption step until it is about to start response decryption.
- **JWE use and representation.** The cipher text and encryption artifacts are to be represented as JWE when sent to VTS APIs.
- **Z Key.** Z is generated as output of the ECDH operation using the Visa public key and EC device private key; size is 64 bytes. Only the x-coordinate is used, which is 32 bytes.
- **KDF hash function.** Visa uses a 1 step KDF operation per SP-800-56A. The key derivation emits only one shared key (SK). There is no other info. The pseudo code follows:

```
SK = SHA256 ('00 00 00 01', X co-ordinate of Z)
```

Appendix F

RSA PKI Encryption Requirements

RSA PKI Encryption Introduction

The RSA_PKI encryption requirements specify how you must implement a secure solution for securely transmitting sensitive data between Visa and a device using an RSA_PKI encryption scheme.

Mutual SSL Support

- Mutual SSL will be supported between Wallet Provider MAP and VTS. The use of mutual SSL also requires an authentication scheme.
- The UserID/Password (VOL Credentials that are requested when the wallet provider onboards for SSL communications) and the DN entry will be provided to the wallet provider during onboarding.
- The wallet provider SSL certificate must have this DN entry in the certificate.

- For SSL communications, the UserID/password must be passed in the HTTP header.
- The subjectDN entry in the certificate must be validated with the DN entry against the UserID.

Signature for Sensitive Data Transmission Between Visa and the Device

- The encryption scheme requires using digital signatures and signature validation for sensitive data and the classification of the keys based on encryption and signature operations.
- The following steps are required to enable the signature operations (mainly passing the asymmetric key pair to the other party):
 1. The wallet provider's device passes a public key to be used for signature validation to Visa during enroll device API call.
 2. Visa passes the public key to be used for validating Visa signatures to the wallet provider out-of-band. This is embedded in the wallet provider devices. The keys are not unique to the device.
 3. The wallet provider must provide a wallet provider Root Certificate in the form of out of band provisioning. This will be used to validate the signature on the device certificates passed to Visa.

Symmetric Encryption

Visa requires symmetric encryption support for use in the Hybrid encryption scheme.

Hybrid Encryption

Hybrid encryption scheme refers to encryption pattern as defined in PKCS#7, where a combination of asymmetric and symmetric encryption is used. A random symmetric key is generated and the symmetric key is encrypted using asymmetric encryption. Thereafter, the symmetric key is used to encrypt the "text to encrypt".

Data Classification

- Visa would require using hybrid encryption scheme for Data-in-Transit. Visa's Data-in-Transit encryption model uses hybrid encryption.
- Data-in-Transit encryption scheme - VTS would support asymmetric keys for securing Data-in-Transit. This will be specific to the device and passed by wallet provider to VTS in the enroll device API.
- The device would create multiple symmetric keys based on data classification types. The association of the symmetric keys with the data classification type is expressed in the cipher text in the form of DER encoding (Type-Length-Value).
- Wallet provider would have the option to use the same symmetric keys for DS, or use a different symmetric/asymmetric key for DS.

Cryptographic Algorithms

The following cryptographic algorithms are supported for the specified use cases:

Supported Algorithm	Use Case
RSA1_5	Asymmetric encryption and signature.
AES-256-GCM	Symmetric encryption inside device.
AES-256-GCM	Symmetric encryption for server-side encryption using API key/Shared Secret.
AES-256-GCM-KW	AES symmetric encryption key wrapping (used in server-side encryption using API key/SS).

Wallet Provider Root Certificate

Wallet provider would need to provide a wallet provider Root Certificate in the form of out of band provisioning. This will be used to validate the signature on the device certificates passed to Visa.

Transmission of Keys Related to Encryption in the Server APIs

Wallet Provider would need to pass the asymmetric key(s) using the server APIs. Wallet provider would pass this asymmetric key (in the form of certificate) to Visa in the Enroll Device API.

Note:

This API can be called again, as required.

Visa-Provided Certificates

Visa will provide a certificate to Wallet Provider to be put in the devices for signature validation. While sending sensitive data to Wallet Provider device, VTS will encrypt with the Wallet Provider provided device key and sign using Visa private key. The corresponding signature on the sensitive data must be validated using Visa provided certificate on the device.

Sensitive Data Between MAP and VTS

The use of API key/Shared secret scheme for encryption of sensitive data between MAP and VTS is required.

Mac of the TVL

Visa specification requires the Transaction Verification Log (TVL) needs to be MAC'ed using the LUK to show proof of possession of the LUK. The Mac of the TVL is validated by the Visa during a replenish LUK request.

Appendix G

Notes on JWE

Notes on JWE

JWE needs to be protected against replay attack and needs to be Integrity Protected across all encryption schemes. Thus, the JWE must be a short lived artifact and enforce no use after TTL expiry policy on it. This provides mitigation within the short time interval.

- Using an `iat` timestamp in the header indicates that a JWE can only be valid till an `x` time interval after it's created.
- The JWE header fields are not encrypted; however, fields are integrity protected by the use of a JWE protected header as AAD for the Authenticated encryption (AEAD) operation.
- The JWE's time-to-live is only a few minutes. The JWE header will be checked for `created timestamp + JWE_TTL <= current timestamp` (All timestamps represented in UTC epoch notation). If already expired or if created timestamp is missing, API returns with an error message associated with a 400 HTTPS response.

- For asynchronous provisioning, if a Token Requestor (TR) utilizes custom logic to determine an expired JWE, it is recommended that the TR wait up to 30 minutes prior to expiring the JWE to account for cases where task notification may be delayed.
- In the situation that the JWE is processed but the response does not reach the requesting client due to timeout, following client behavior is expected:
 - Client retries the same message and gets an error because of JWE TTL expiry.
 - Client retires with a new created timestamp in the JWE. This JWE is successfully handled in VTS.
- Tamper protection of timestamp in the JWE header and the JWE is guaranteed by signature operation on the JWE. The signature is packaged as JWS.
- Replay attack protection of the JWE is partially mitigated by the timestamp expiry, by limiting the usage of the JWE to a short time interval.
- For consistency, VTS will add timestamp field in all the JWE headers. It is up to the client to implement the replay attack protection as per Visa guidance (for TTL duration), if the use case merits it.
- VTS will inspect if the JWE has timestamp.
 - If the JWE does not have timestamp field `iat`, the JWE will not be validated for timestamp.
 - If the JWE does have a timestamp field `iat`, VTS will validate the timestamp field against the TTL.
- After the agreed cutover timeline has passed, all JWE not having the created timestamp (`iat`) field will fail. To avoid this situation, clients must provide the (`iat`) field.
- The following example shows a JWE header:

```
"header":
{
{
// Encryption algorithm to be used for encryption of CEK:
"alg":"RSA-OAEP-256",
"kid":"1234", //clientDeviceID
"typ": "JOSE",
"channelSecurityContext": "RSA_PKI",
// Encryption algorithm to be used
// for encryption of text to encrypt:
"enc":"AGCM256"
"iat": "1429837145" // Time in seconds in UTC when the JWE was issued.
}
}
```

- The JWE body is not impacted by the header. The JWE BNF is as follows:

```
BASE64URL (UTF8 (JWE Header)) || '.' ||
BASE64URL (JWE Encrypted Key) || '.' ||
BASE64URL (JWE IV) || '.' ||
```

```
BASE64URL (JWE Ciphertext) || '.' ||  
BASE64URL (JWE Authentication Tag)
```

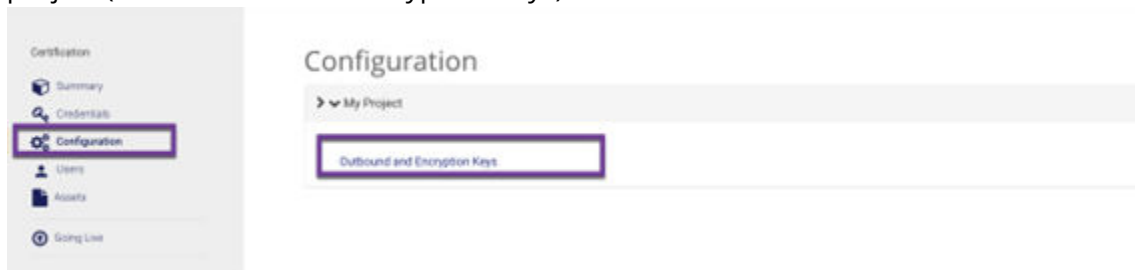
Appendix H

Frequently Asked Questions about JWE and JWS

- **WHERE CAN I FIND MY PROJECT ENCRYPTION KEY AND SHARED SECRET FOR ENCRYPTION AND DECRYPTION?**

An Encryption (API) key and shared secret can be obtained from your Visa Developer Portal (VDP) Project.

To view the project's Encryption Keys, click on the "Configuration" tab within the VDP project (→ Outbound and Encryption Keys)



If you don't have an API Key, you can create one by clicking on the **Create New** button.

- **WHERE CAN I FIND MY PROJECT ENCRYPTION KEY ID AND ENCRYPTION CERTIFICATE?**

- For Visa Token Service (VTS), contact your Visa Representative.
- If your project requires MLE (Message Level Encryption), please see the following article on VDP, which explains on how to set up MLE and obtain your credentials: [How to run MLE Sample Code Project](#).

- **WHAT ARE THE POSSIBLE REASONS FOR JWE ENCRYPTION FAILURES?**

Visa responds with an HTTP 400 status code for the following reasons:

- Incorrect JWE string being sent in the request which Visa cannot parse and decrypt
- Incorrect Encryption kid or API Key value used in the JWE Header.
- Incorrect shared secret was used for Symmetric Encryption and decryption
- Incorrect certificate (signing cert instead of encryption cert) was used for Asymmetric Encryption.
- Incorrect private key used for signing the JWE or decrypting the JWE.
- Mismatched alg value with different enc algorithms in JWE.
- The JWE Header field such as Content Encrypted Key (CEK), Initialization Vector, Ciphertext and Authentication Tag were not Base64URL encoded.
- Incorrect values in the header fields like Initialization Vector or Authorization Tag.

- **WHAT ARE THE POSSIBLE REASONS FOR JWS VERIFICATION FAILURES?**

Visa responds with an HTTP 400 status code for the following reasons:

- Incorrect JWS Payload being send in the request which Visa cannot parse or verify the signature
- Incorrect signing kid value in the JWS Header. For example: Encryption API Key or an Encryption Certificate ID was parsed instead of the Signing API Key or Signing Certificate ID
- Incorrect shared secret or different private key was used to sign the payload to create the JWS
- An unsupported library other than nimbus libraries being used
- Other algorithms were used.

- **WHAT ARE THE RECOMMENDED ALGORITHMS?**

- JWE header contains the **alg**: AGCM256KW for Symmetric encryption algorithm is **A256GCM** for Shared Secret (See corresponding API Specifications)
- JWE header contains the **alg**: RSA_OAEP_256 for Asymmetric algorithm is **A256GCM** for RSA PKI (See corresponding API Specifications)
- JWS header should be HS256 for Shared Secret (Symmetric) and PS256 algorithm for RSA PKI (Asymmetric). (See corresponding API Specifications)

- **HOW DO YOU VERIFY WHETHER YOUR CERTIFICATE OR PUBLIC KEY MATCHES YOUR PRIVATE KEY?**

Use OpenSSL to compute the hash of the modulus of the certificate or public key and private key. The computed SHA256 hash of the modulus in each should be the same.

- Compute the modulus of the certificate or private key:

```
opensslx509 -modulus -noout -in <CERT_OR_PUBLIC_KEY_PEM_FILE> | openssl sha256
```

- Compute the modulus of the private key:

```
opensslrsa -modulus -noout -in <PRIVATE_KEY_PEM_FILE> | openssl sha256
```


Appendix I

API Key Separation for Signing and Encryption

API Key Separation for Signing and Encryption

Visa supports the separation and classification of the signing (authentication) and encryption keys to comply with the PCI-DSS specifications. This is applicable to VTS clients.

An API key can be either of the following:

1. Default key - Support existing clients for backward compatibility. To be deprecated in the future.
2. Inbound authentication key - A client can use this key to create x-pay-token:xv2 (sign) for placing API calls to Visa (Inbound to Visa)
3. Outbound Authentication Key - Visa systems can use this key to sign the outbound post-back messages to the client's systems.
4. Data encryption Key: This key will be used to encrypt the data that is transmitted between the client's system and Visa systems.

Clients would have three pool of keys/secrets. Each pool can have one to multiple API key/shared secret.

1. Inbound authentication key – API key/shared-secret for x-pay-token (version 2: xv2). Clients will use this API-key/secret to create HMAC-SHA256 digests by following x-pay-token (version 2: xv2) protocol.
2. Data encryption key - For data encryption in the payload. There will be no separation of API keys for inbound or outbound APIs.
 - a. Inbound APIs to VTS - A client can select any encryption key for the encryption of sensitive data passed to VTS.
 - b. Outbound APIs from VTS - Clients should have an encryption key defined in the client profile during onboarding with VTS. If encryption key has not been defined in the profile, VTS will select an encryption key from the pool of encryption key.
3. Outbound authentication key - The API-key/shared-secret for x-pay-token (v2) for Notification APIs from VTS.

A key that is not a default key can have only one restriction. Restrictions can be enabled by contacting Visa customer support. A key cannot be used for both inbound and outbound, for example. Once a key is restricted to a specific usage, that usage cannot be changed or modified. For example, an inbound authentication key can never become an outbound authentication key or an encryption key. If the client desires a new key, they must create new one.

Versioning of x-pay-token. X-pay-token is supported in 2 versions: x and xv2 supporting the SHA2 and HMAC-SHA2 algorithms respectively. X version of the x-pay-token is supported only for backward compatibility and it will be deprecated.

Inbound authentication. Clients can indicate their preference for x-pay-token version for the inbound APIs, by contacting visa customer support. VTS will validate the indicated version of x-pay-token for inbound authentication.

- If the indicated preference is xv2, only xv2 version will be supported.
- If there is no indication, both x and xv2 version will be supported until x version is deprecated.

Outbound authentication. Clients can indicate their preference for x-pay-token version for the post-back APIs, by contacting visa customer support during onboarding in the portal. VTS will create the indicated version of x-pay-token for post message back to clients.

- If the indicated preference is xv2, only xv2 version will be supported.
- If there is no indication, both x and xv2 version will be supported until x version is deprecated.

Key usage error scenarios:

If an API key is used improperly in terms of API key classification, the API request would be rejected with an error response by VTS.

1. If an encryption key is used for authentication, “Unauthorized” response will be sent to client.
2. If an authentication key is used for encryption, the API request would be rejected as “bad request” by VTS.
3. Client should reject if the API-key in query parameter is not for the outbound purpose in their key management systems.
4. Visa would reject x-pay-token with version x, if the client is configured for xv2.
5. For the post-back APIs, client should reject if the indicated version of the x-pay-token is xv2 and if it receives x-pay-token with version x.

Appendix J

Replenish FAQ

When is the Replenish API called?

1. In response to VTS notifications.
2. While reaching device TTL.
3. While reaching device NOT.

What does the device do when making a request for a new LUK?

1. Send SC (Sequence Counter) – Send the current SC on the device.
2. Send API (Account Parameter Index) – Send the current API on the device.

3. Send TVL (Transaction Verification Log) - Max transactions that can be sent from the device is 100. Minimum transactions that can be sent in TVL is 0.
4. Send MAC of the TVL - Send MAC of the TVL using LUK from the device. Calculate MAC (API + SC + latest TVL record) using LUK. In case the number of transaction records in TVL is zero, calculate MAC (API + SC).

What does the device do after getting a new LUK?

After successful receive and storage:

1. Validate that the new API is different and a later version than the existing API.
2. Increment the SC by 1.
3. Store the latest API received in the response as the current API.
4. Old TVLs should be deleted from the device.
5. Call the Confirm Replenishment API.

What happens if the device requests a new LUK and does not receive it (either because of request not reaching VTS, or response not reaching back to device)?

1. Send current SC on the device. (It is not to be incremented since the new LUK did not reach the device.)
2. Send the current API on the device. (At this point, the VTS may have a different latest API than the current API on the device)
3. Send TVL (Transaction Verification Log) - Max transactions that can be sent from the device is 100. Minimum transactions that can be sent in TVL is 0.
4. Send MAC of the TVL - Send MAC of the TVL using LUK from the device. Calculate MAC (API + SC + latest TVL record) using LUK. In case the number of transaction records in TVL is zero, calculate MAC (API + SC).

What is the function of confirmReplenishment API?

Visa requires a confirmation from the device that the replenishment was successful. After calling the confirmReplenishment API, the old LUK cannot be used by the device. However, the device can use the new LUK for making payments even though the confirmReplenishment API has not been called yet.

Appendix K

What's New in Prior Versions

What's New in Version 23.08

This document describes enhancements to public features for issuers, wallet providers, merchants and developers.

Code changes

- Added the `appliedPrograms` field to the Provision Token—Given PAN Data API response, which is present when a program is applied to a token, such as when VCES applies a new expiration date.

Documentation changes

- Clarified that the `tokenRequestorServiceTypes` field in the Onboarding API request always contains "Purchase" as the default value. Specifying another value for this field adds it to the array; for example, you can add "Visa Direct" as an additional service type, in which case the service types will be "Purchase" and "Visa Direct".
- Removed the documented restriction on spaces in the `deviceModel` field.
- Identified the max size of the `encodedData` field as 6MB.

What's New in Version 23.07

This document describes enhancements to public features for issuers, wallet providers, merchants and developers.

Code changes

- Added a `tokenRequestorServiceTypes` field to the Onboarding API request and response.

Documentation changes

- Updated Onboarding API example.
- Added a `vPanEnrollmentID` field to the Token Status Updated Event notification for the `TOKEN_REPERSO_ADVICE` event type.
- Provided an example request for the `TOKEN_REPERSO_ADVICE` event type.
- Full reperso is not applicable for eSE tokens in the Re-Perso Token Data API--the value of the `fullReperso` field is ignored.
- The `atc` field is ignored for eSE tokens in the Re-Perso Token Data API.
- The maximum size of the `companyPostalCode` in the Onboarding API is 8 characters.
- Replaced "JSON Re-Perso Token Data With SE Data Request" section with "eSE Reperso for SCP02 Example".
- Added a topic to the introduction, "JSON Web Encryption and Object Signing (JWE/JWS)" that points to relevant information regarding encryption and decryption information of fields prefixed with `enc`.
- Added an appendix, "Frequently Asked Questions about JWE and JWS."

- For card data content, the values of `width` and `height` will be 0 for `image/pdf` and `text/html` MIME types.
- Added business identification types and values for the following countries and geographic areas: Isle of Man, Kazakhstan, South Africa, Australia, Philippines, Vietnam, South Korea, China, Thailand, Indonesia, Belarus, and Japan.

What's New in Version 23.06

This document describes enhancements to public features for issuers, wallet providers, merchants and developers.

Code changes

- Added the `subMethod` field to the `stepUpRequest` structure.

Documentation changes

- Removed `clientAppID` as a query parameter in the Request Token Device Binding API and added it as a field in the request body.

What's New in Version 23.02

Code changes

- Added `CHIP_DIP` and `CONTACTLESS_TAP` enumerations to the PAN source in the Enroll PAN API request.
- Added `encChipData` and `sigNonce` fields to the Enroll PAN API request.
- Added the Get Nonce API to provide a Visa-supplied nonce that is required when enrolling a PAN from a non-EMVCo-certified terminal using either the `CHIP_DIP` or `CONTACTLESS_TAP` PAN source.

Documentation changes

- Added 403 (`cardVerificationNotSupported`) to the list of standard errors.
- Changed the format of `currencyCode` in `dynamicData` to be a 3-digit string.
- Corrected `clientAppId` to `clientAppID` in the query parameters for Request Device Token Binding, Delete Device Binding, and Get Transaction History API requests; `clientAppID` is preferred, although `clientAppId` may continue to work.

What's New in Version 23.01

Code changes

- For Digital Authentication Framework (DAF) risk data attributes `shippingState` and `billingState`, a value of `XX` is allowed when a state is not applicable for the address.

Documentation changes

- Removed the `encodedData` field from the `content` structures in the Get Card Metadata and Enroll PAN API responses.
- Added `relationshipID` query parameter to Enroll Device API request.
- Provided values returned in the `reasonCode` field of the `authenticationFlags` structure in the Get Payment Data API response.
- The `deliveryMethod` risk data attribute is optional for qualification by the Digital Authentication Framework.

What's New in Version 22.10

This document describes enhancements to public features for issuers, wallet providers, merchants and developers.

Code changes

- Added the `relationshipID` query parameter to the Enroll PAN and Provision Token—Given PAN Enrollment ID APIs.

Documentation changes

- Added Enroll PAN by TR-TSP and Provision Token—Given PAN Enrollment ID by TR-TSP examples.
- Removed the statement "Tokens are always bound to known consumers (identified by the wallet provider in `clientWalletAccountID`)" from the introduction to the Provision Token—Given PAN Enrollment ID API.
- Added information about the asynchronous nature of notifications from Visa.
- Changed documentation for the client app ID query parameter name to `clientAppID`; `clientAppId` continues to work where it worked previously.

- The `clientWalletAccountID` is required in API requests, including those affecting card-on-file, AKA credentials-on-file, tokens.
- Changed the length of the `acquirerId` to max 11 digits.

What's New in Version 22.09

This document describes enhancements to public features for issuers, wallet providers, merchants and developers.

Code changes

- The `clientWalletAccountID` is conditionally required in API requests. It is not required for card-on-file, AKA credentials-on-file, tokens. Previously, it was required for all token types.

Documentation changes

- Updated VCEH JWE and JWS Headers to match VCEH JWE and JWS Headers in the JSON version for Issuers (Visa Token Service Issuer API Specifications REST JSON).
- In the Token Requestor's app return to VCEH, the VCEH URL is

```
https://vceh.consumerapi.digital.visa.com/vceh
```
- Removed suspending and resuming tokens as examples of token management actions performed by card-on-file/ECOM clients in "Card-On-File, ECOM, and HCE Use Cases."
- Added the `date` and `relationshipID` field descriptions to the Task Status Event notification.
- The `dynamicData` structure in the Get Payment Data (Token & Cryptogram)—Given Token ID API request is conditional; required for Digital Authentication Framework (DAF) or if the `delegatedAuthenticationRequested` field is `true`.
- Added the `clientAppID` query parameter to the Request Device Token Binding API.
- Identified the max size of the `encodedData` field as 20480 bytes.
- Clarified that the `cryptogram` field in the `cryptogramData` structure for 3D Secure in the Get Payment Data (Token & Cryptogram)—Given Token ID API request is required.
- Added additional information about forming the `sigBindingRequest` field contents in the Request Device Token Binding API's request.

What's New in Version 22.08

Code changes

- The Suspend Token and Resume Token VTS Token Lifecycle Management APIs are no longer supported for e-commerce/card-on-file and e-commerce enabler tokens; calling Suspend Token or Resume Token on these kinds of tokens will result in an error.

Documentation changes

- Removed some non-required headers from API examples.
- Moved the "Provision with Given PAN (async=true)" example to the "Provision Token—Given PAN Data" chapter.
- Removed the `async` query parameter from the "Get Async Token Provisioning Result" chapter.
- The default `protectionType` in the provisioning data of an Upload Job Data API request is `CLOUD`.
- Added an optional `riskData` structure to the Get Payment Data (Token & Cryptogram)—Given Token ID request.
- Added Unencrypted `encRiskDataInfo` example to Provision Token—Given PAN Enrollment ID API.
- Revised examples for Token Status Event and Token Status Event Updated Event notifications.
- Added the `clientAppID` query parameter to the Payment Transaction Event notification.
- The `deliveryMethod` risk data attribute is required for qualification by the Digital Authentication Framework.

What's New in Version 22.04

This document describes enhancements to public features for issuers, wallet providers, merchants and developers.

Changes to support autofill

- Added the Get Payment Data for Autofill API. It is a different endpoint than the existing Get Payment Data API.
- The `deviceCerts` field is not required for autofill in the `channelSecurityContext` structure of the Enroll Device API request.

- The `presentationType` enum supports `AUTO_FILL`.
- The `sigBindingRequest` field is not required in the Request Device Token Binding API for autofill. In this case, `encDeviceRiskDataInfo` must be encrypted using the shared secret.
- Added the following autofill-required fields to the Request Device Token Binding API request: `clientDeviceID` and `clientReferenceID`.
- In the `merchantIdentifier` structure, the `applicationUrl` field is required when the `presentationType` is `AUTO_FILL`. It accepts a merchant URL, Application Package ID, or Application Bundle ID.

Other API changes

- Added the following query parameters to the Get Transaction History API: `relationshipID` and `clientAppID`.
- Added the `relationshipID` query parameter and `clientAppID` request field to the Retrieve Transaction History API.
- Added the `relationshipID` query parameter to the Payment Transaction Event notification.
- The maximum size of the `applicationUrl` field is 512 Base64Url-encoded characters.

Documentation changes

- In the `tokenizationConfiguration` structure of the Onboarding API, the `channelSecurityContext` field is a string.
- Provided additional clarification about the formatting of the `authenticationAmount` field in the `dynamicData` structure.
- Updated Provision with Given PAN (`async=true`) example to remove invalid value in `location` field.
- Clarified that the update reason in the Token Life-Cycle Management—Resume Token Request is a string.
- In the `cardMetaData` field description, removed the indication that the `issuerFlags` field is an array and adjusted the format of the `cardData` field to indicate it is an array of structures.
- Clarified that `presentationType` is an array in the Provision Token Given PAN Data API request.
- Added `async` query parameter to Get Async Token Provisioning Result API.
- Clarified that the `async` query parameter in the Provision Token—Given PAN Data API defaults to `false`.

- For Digital Authentication Framework (DAF), in the Get Payment Data APIs, `sigBindingRequest` is always required.
- VAT business ID for Ireland is IE + 8 numeric or alphabetic characters. e.g. IE1234567X.
- Provided additional information about FIDO authentication factors in you can set in the Get Payment Data APIs and updated the DAF categories for method 10, Device unlocked (CDCVM unknown); it is inherence or knowledge, and possession, not all. Clarified that for methods 15 and 16, biometric iris or voice recognition, a DAF 2nd factor category is not required.
- Adjusted `paymentInstrument` structure description in the Get Card Metadata API response to include additional fields.

What's New in Version 22.01

- Added `clientAppID` as a request field or query parameter to the following APIs: Submit ID&V Step-Up Method, Validate OTP, Validate Authentication Code, Get Step Up Options, and Delete Device Binding. It is required when `relationshipID` is specified.
- Modified the business identification type for Canada (BN) and Malaysia (BRNO), and adjusted the formats of the VAT values for Spain and Ireland.
- Updated the introduction to the Enroll Device API.
- Added example of the unencrypted `encRiskDataInfo` to the Provision Token—Given PAN Enrollment ID and Provision Token—Given PAN Data APIs.
- Updated format information for the `acquirerId` and `acquirerMerchantId` fields in the Onboarding API.
- Adjusted references to shared secret in `encBillingInfo`, `encPaymentInstrument`, and `encTokenRequestorClientInformation` field descriptions to be consistent with the shared secret description in other fields.

What's New in Version 21.10

- Version 1 (QR version) of GetPaymentData is deprecated and should not be used by new clients. Version 2 (the Standard version) now supports all V1 capabilities. The V1 format is no longer referenced as Version 1 is no longer necessary.
- Added `POST` method to Get Step Up Options API, which allows selection on cardholder's name, email address, phone number, and/or billing address in addition to existing query parameters.
- Added enum value `HOUSEHOLD_DEVICE` to `deviceType` field.

- Added `encBillingInfo` field to the Request Device Token Binding API.
- Added `countryDigits` field to the Billing Address structure.

The following changes were made to the documentation:

- The billing address line fields allow only alphabetic, numeric, and the following alphanumeric characters: period (.), comma (,), single quote ('), hyphen (-), pound sign or hash tag (#), forward slash (/) and whitespace. The billing address `city` field allows only the following characters: alphabetic, single quote ('), hyphen (-), and whitespace.
- Added `relationshipID` query parameter to the following APIs: Submit ID&V Step-Up Method, Validate OTP, Validate Authentication Code, Get Step Up Options, Request Device Token Binding, and Delete Device Binding.
- The `vTaskID` has been added to the Provision Token Given PAN Data API response.
- VAT business ID for IE followed by 8 alphanumeric characters. e.g., IE1234567X
- Changed enum value `NFC_HCE` to `NFC-HCE` for the `unsupportedPresentationTypes` field.
- Removed redundant Opacity example for Reperso Token Data API.
- Added `status` field to the Get Step Up Options API response.

What's New in Version 21.08

- The CNPJ business identification type for Brazil onboarding is 14 alphanumeric characters, identified by `n` in the example, and punctuation characters (period, slash, and hyphen): `nn.nnn.nnn/nnnn-nn`.
- RSA PKI2 is described in "Encrypting and Decrypting Data," It is similar to the VTS implementation of RSA PKI, but supports PS256 for JWS and is more fully compliant with RFC 7615 and 7616. Additionally for RSA PKI2, the JWE encrypted blob created by VTS no longer uses the first byte as a content type tag; it is not present in the blob.
- Added `fullReperso` and `seData` fields to Reperso request.
- Examples in "Notification For Task-Related Events" have been revised.

What's New in Version 21.07

- Added platformType to APP-TO-APP Step-Up-Request in Provision Token—Given PAN Enrollment ID response example, and added APP-TO-APP Step-Up-Request to Get Step Up Options and Request Device Token Binding APIs.
- Added the relationshipID query parameter to the following APIs: Submit Job, Get Job Status, Upload Job Data, and Get Job Results.
- Added DECLINED value and removed PENDING value from the device binding info status field
- .
- Modified resource path and Sandbox endpoint for Onboarding API and adjusted value descriptions for several business identification types and VAT value formats.
- Clarified the use of externalClientId and applicationId fields when calling VTS APIs after onboarding.
- Added clientReferenceID field to Token Status Event requests.
- Added error code 2945 to Onboarding API.
- For client information name fields (firstname, middlename, lastname), escape characters, including tab entries, are not supported.

What's New in Version 21.05

This version is primarily a maintenance release. Changes to documentation include:

- Added business identification types and formats to the onboarding API for additional countries.
- Added status field to bulk provisioning job completion notification.
- Added requirement that Accept headers should be provided for every API request.
- Added tokenInfo.sePersoData and related fields to the Re-Perso Token Data API response.
- Removed riskData structure from the Get Payment Data request; it is not used.
- Clarified that the deviceId, deviceName, and deviceType ID&V risk data attributes apply to all token types.
- Changed endpoints and resource path for the Onboarding API.

What's New in Version 21.04

This version is primarily a maintenance release. Changes to documentation include:

- Changed endpoints and resource path for the Onboarding API
- Additional prefixes to data tags were documented for RSA PKI encryption by VTS
- The `presentationTypes` field was renamed to `presentationType`
- Documented `deviceBindingInfoList` at the root of the Get Token Status response, rather than inside the Token Info structure of the response
- Removed the duplicate `DEVICE_BINDING` enum in the `reasonCode` field description

What's New in Version 21.03

This version is primarily a maintenance release.

What's New in Version 21.02

- A transaction counter (`atc`) is present in the Token Info structure of an HCE provisioning response.
- Added `relationshipID` to the following status event notifications, when applicable: token status, card metadata update, and job status. The ID identifies the relationship between a token requestor and a token service provider (TR-TSP).
- Added VCEH web proxy interfaces to push card enrollment to a token requestor's app and return the enrollment status.
- Added `GetPushEnrollmentInfo` API
- Updated Token Push Provisioning event notification to provide enrollment payloads.

What's New in Version 21.01

- An onboarding API was added to enable a token service provider (TR-TSP) to onboard a token requestor (TR) into VTS.

What's New in Version 20.10

- `clientReferenceId` is a supported field in the standard version (v2) request of the Get Payment Data API.
- Added `networkAuthenticationRequested` field in the Get Payment Data request to enable a Token Requestor in India to request a Network Authenticated transaction.

What's New in Version 20.09

- Added `platformType` to the `stepUpRequest` structure; available for selected use cases.

What's New in Version 20.08

- In Get Payment Data (Token & Cryptogram)—Given Token ID, the QR version is deprecated for all non-QR cryptogram cases; use the Standard (v2) version instead.

What's New in Version 20.07

- Added `relationshipID` to the Provision Token Given PAN Data, Get Payment Data (v2), Get Card Meta Data, Get Token Status, and Suspend/Resume/Delete Token APIs. The ID identifies the relationship between the token requestor and the caller, which is a token service provider (TR-TSP). The relationship is defined at the time of onboarding; contact your Visa representative for more information.
- Added to Get Payment Data (Token & Cryptogram)—Given Token ID API, which specifies the API key of the token service provider (TR-TSP) making the call on behalf of a token requestor; required when the query parameter contains a relationship ID.

What's New in Version 20.03

- In the `reasonCode` field, added `PROVISIONING`, `CARDHOLDER_STEPUP`, `DEVICE_BINDING` to enum and removed `TRUSTED_LISTING_ENROLLMENT`
- Get Step Up
 - In request, removed request parameter `merchantID`
 - In response, added `status` of type `RequestResult`
- Validate OTP
 - In request, made `otpValue` field conditional; added `clientReferenceId` and `encOtpValue` fields
- Validate Auth Code
 - In request, made `issuerAuthCode` field conditional; added `clientReferenceId` and `encIssuerAuthCode` fields
- Get Card Meta Data
 - In response, under Card Meta Data added `issuerFlags` structure
- Get Token Status
 - In response, added `deviceBindingInfoList` structure; removed `cardholderVerificationStatus` field
- Token Status Event Notification
 - Added new notification types
 - Added `deviceBindingInfo` structure
- Added new API, Delete Device Binding
- Added new API, Request Device Token Binding

What's New in Version 20.01

Version 20.01 is primarily a maintenance release.

What's New in Version 19.10

To enable Visa as a Hop, the Enroll PAN and Provision Token APIs have been updated to include a new field `paymentInstrumentReference` that enables issuer applications to securely push cards to wallets and card-on-file merchant applications to be tokenized and accessed on different devices or by additional authorized users.

What's New in Version 19.09

This document describes enhancements to public features for issuers, wallet providers, merchants and developers.

Version 19.09 is primarily a maintenance release.

Note:

The sandbox endpoint is <https://cert.api.visa.com>.

What's New in Version 19.08

New APIs:

- Provision Token—Given Another Token
- Request Token Device Binding
- Delete Device Binding

Changes to APIs

- Added `seData` structure to Provision Token—Given Pan Enrollment ID.
- Added `cardMetaData` structure to Get Async Token Provisioning Result.
- Added a new version of Get Payment Data, v2.
- Validate OTP Request
 - Added `clientReferenceID`
 - Added `encOTPvalue`
 - `otpValue` became Conditional
- Validate Authentication Code Request
 - Added `clientReferenceID`
 - Added `encIssuerAuthCode`
 - `issuerAuthCode` became Conditional
- Get Step Up Methods request `reasonCode` ENUM supports these new values:
 - `TOKEN_PROVISIONING`
 - `CARDHOLDER_STEPUP`
 - `DEVICE_BINDING`
 - `TRUSTED_LISTING_ENROLLMENT`
- Get Step Up Methods Response
 - Added new element `issuerFlags` in the `cardMetaData` object
- Get Token Status Response
 - Added `deviceBindingInfoList`
 - Added `cardholderVerificationStatus`

Note:

Also, changed all occurrences of Network Hub Push Provisioning (NHPP) to Visa Card Enrollment Hub (VCEH).

What's New in Version 19.06

Version 19.06 was primarily a maintenance release.

What's New in Version 19.04

In version 19.04:

- Added Get Task Status API
- Added Get Async Token Provisioning Result API
- Added Task Status Event to Status Notifications

Additional Features for Cloud Token Framework (CTF)

In version 19.04:

- Added Provision Token—Given Another Token API
- Added Request Device Token Binding API
- Added Delete Device Binding API

What's New in Version 19.03

This version was primarily a maintenance release.

What's New in Version 19.02

- Added support for International Bank Numbers (IBAN)
In this version, an `iban` field has been added to the encrypted payment instrument.

What's New in Version 19.01

In version 19.01:

- Added `servicesToEnroll` and `encClientInfo` to the request and added `servicesEnrollmentStatus` to the response of the following APIs.
 - Provision Token—Given PAN Data
 - Provision Token—Given PAN Enrollment ID
- Added `unsupportedPresentationTypes` to the Enroll PAN API.
- Added `Job Status Event` to Status Notifications.

What's New in Version 6.8

In version 6.8:

- Added API Submit Job
- Added API Get Job Status
- Added API Upload Job Data
- Added API Get Job Results
- In Provision Token Given PAN Data and Provision Token Given PAN Enrollment Id
 - Added `QR_CONSUMER_DEVICE_HCE` under Presentation Type Enum
 - Added `qrConsumerDeviceData` under Static Parameters
- In Re-perso Token Data
 - Added `qrConsumerDeviceData` under Static Parameters Reperso

What's New In Version 6.7

In version 6.7:

New Presentation Type: Device-bound Consumer Presented QR HCE

This feature enables wallet providers in the United States and Asia Pacific to support consumer presented quick-response (QR) code proximity payments using a device-bound HCE token. This will be supported via a new Presentation Type: `QR_CONSUMER_DEVICE_HCE`.

QR_CONSUMER_DEVICE_HCE may be provisioned with the following Presentation Type combinations:

- QR_CONSUMER_DEVICE_HCE, NFC_HCE, MSR, and INAPP
- QR_CONSUMER_DEVICE_HCE, NFC_HCE, and INAPP
- QR_CONSUMER_DEVICE_HCE, NFC_HCE, and MSR
- QR_CONSUMER_DEVICE_HCE, and NFC_HCE
- QR_CONSUMER_DEVICE_HCE, and INAPP
- QR_CONSUMER_DEVICE_HCE

This is an HCE token; therefore:

- LUK is managed by the device,
- ATC is managed by the device, and
- Cryptogram and QR are generated by the device.

Other Changes

- Added `intent` field to Enroll PAN and Provision Token—Given PAN Data.
- New enum value, `AFT`, added to the Get Payment Data request's Payment Request.
- Payment Instrument object may have a new element, `verificationResults`, in Provision Token—Given PAN Data and Provision Token—Given PAN Enrollment ID.
- New fields `line3`, `line4`, and `line5` have been added to the billing address.
- New fields `campaignID`, `campaignDescription`, and `userPersonalID` have been added to Client Information.

What's New in Version 6.6

In version 6.6:

For the following APIs:

- Provisioned Token Given PAN Enrollment ID
- Provisioned Token Given PAN Data
- Get Payment Data

Shared Secret is used to encrypt the API response fields when the presentation type is:

- ECOM
- QR_CONSUMER_CLOUD
- RECEIVE_ONLY

What's New in Version 6.5

In version 6.5:

Added Card Verification Value (CVV2) Match and Address Verification Results for Card-on-File and e-Commerce Enabler Token Provisioning

- `verificationResults` field and structure added to Provision Token Given PAN Enrollment ID and Provision Token Given PAN Data APIs

What's New in Version 6.4

For more flexible Embedded Secure Element (eSE) integrations, the Elliptic Curve Cryptography(ECC) feature introduces support for Amd A: Scenario #3 ECKA as an alternative to the pull-based approach with P-256 ECC curve support and SCP-03 with AES-128. These APIs have been modified to support the SCP_03/ECKA encryption scheme:

- Enroll Device
- Provision Token—Given PAN Enrollment ID
- Reperso Token Data

Additionally, one new API is available to support this use case:

- Generate Keys.

Token Requestors using SCP_03/ECKA encryption scheme must call the Generate Keys API between the Enroll PAN and Provision Token—Given PAN Enrollment ID APIs.

Also included in this release:

- Added support for end-to-end reperso notifications.
- DTVV cryptography type added to Get Payment Data Given Token ID.
- Maintenance fixes.

What's New in Version 6.3

In version 6.3:

- Error code `cardVerificationFailed` added to Standard Errors.
- JWE Sample Header updated.

In Visa Visa Card Enrollment Hub (VCEH):

- Get Token Requestor Metadata API provides issuers with a filtered list of token requestors.
- A client information structure is specified during enroll PAN, along with other parameters. The client information is returned during provisioning.

In addition, a consumer identifier has been added to the list of ID&V risk data attributes and the Validate JWE API validates the issuer's payload, which contains the encrypted payment instrument.

VCEH Support

This version provides support for the Visa Visa Card Enrollment Hub (VCEH) protocol.

- Get Token Requestor Metadata API provides issuers with a filtered list of token requestors.
- A client information structure is specified during enroll PAN, along with other parameters. It is returned during provisioning.

What's New in Version 6.1

Version 6.1 was primarily a maintenance release.

What's New in Version 6.0

This version was primarily a maintenance release.

What's New in Version 5.8

- This version includes the addition of `paymentAccountReference` to the following APIs:
 - Enroll PAN
 - Provision Token—Given PAN Enrollment ID
 - Provision Token—Given PAN
 - Get Card Metadata
 - Get Payment Data
 - Re-perso Token Data
- `customerIdentificationValue` field added to `endRiskDataInfo`.
- `co-badge/co-residence` support added to `EnrollPAN`.
- `Validate JWE` API added.

What's New in Version 5.5

The `aidInfo` structure in the responses of `Provision Token—Given PAN Enrollment ID` and `Provision Token—Given PAN Data` have been extended to provide additional information.

What's New in Version 5.4

Version 5.4 was primarily a maintenance release.

What's New in Version 5.3

Version 5.3 was primarily a maintenance release.

What's New in Version 5.2

Version 5.2 was primarily a maintenance release.

New API

Retrieve Transaction History API.

New APIs

Re-Perso Token Data

Other

Get Transaction History

Added `atc` field to the response.

What's New in Version 4.9

New APIs

- Confirm Token Association
- With this release, a new API, Confirm Token Association has been built to address the need to confirm the relationship between an existing token and the elements used to provision it.
- In order to verify a customer, token requestors must send a token along with any preferred combination of optional parameters used in provisioning. Initially, the provision elements for confirmation will be restricted to an email address and the last 4–6 digits of a cardholder's PAN. The following examples represent the request combinations that a token requestor can currently send to confirm associations:
 - Token + Email Address
 - Token + Email Address + last 4 of cardholder PAN
 - Token + Email Address + last 6 of cardholder PAN
 - Token + last 4 of cardholder PAN
 - Token + last 6 of cardholder PAN
- This new API will provide a response back to the token requestor confirming (or otherwise) the token to the element's relationship as used during provisioning.

Changes Across Multiple APIs

- appPrgmID field name changed to appPrgrmID. Size of field changed from 5–16 to 10–32.
- pdol field size changed from 33 to 48.
- aid field size changed from 5–16 to 10–32.

Provision Token—Given PAN Enrollment ID

- Changed ECOM etc. to CBP etc.

Confirm Provisioning

- Added reperso field to request body.

Get Payment Data (Token & Cryptogram)—Given Token ID

- Added threeDsData field.
- Added cryptogramType field and ENUM values.
 - TAVV
 - HYBRID
 - DTVV
- Added ThreeDsData table.
- Added Get Payment Risk Data table.

What's New in Version 4.7

Payment Account Reference (PAR) has been added to enable merchants and acquirers to consistently link transactions initiated with payment tokens to a Primary Account Number (PAN) even after the account number has been tokenized. It is returned in the payment instrument structure for the following APIs:

- Enroll PAN
- Provision Token—Given PAN Enrollment ID
- Provision Token—Given PAN Data
- Get Card Metadata
- Get Payment Data (Token & Cryptogram)—Given Token ID

What's New in Version 4.6

This document describes enhancements to public features for issuers, wallet providers, merchants and developers.

- This version includes additional error reason codes and descriptions for all APIs.
- The AFT transaction type has been added to the Get Payment Data API.

What's New in Versions 3.8 to 4.5

- The Get PAN Data API is now available and allows the token requestor or wallet provider to determine if a PAN is eligible for tokenization. This API has no impact to existing implementations and calling this API is optional.
- Added Retrieve Transaction History API.
- In Get Transaction History
 - Changed transactionType from String length 1 to string 10.
 - Changed transactionStatus from String length 1 to String 10.
- In Provision Token given PAN Enrollment ID
 - Changed CASD Cert Size from 4K to 20K and changed description.
- Changed sdScript description to indicate that it is a HEX String.
- Changed VMPAVersion description to indicate that it is a HEX String.
-
- Cleaned up request and response samples.
- In Provision Token Given PAN Enrollment ID
 - Changed CASDCert length from 4k bytes to 4k String
 - Removed seData object
- In SSDData
 - Updated keyVersionNumber, sequenceCounter and CASDCert as hex.
- For apduReasonCode and apduPerso
 - Max array size set to 30
- In Provision Token given PAN Enrollment ID
 - Added seData object

- In Enroll Device
 - cryptoMetaData removed for SCP channel. Not needed
- In platformType enum
 - change MICROSOFT to WINDOWS
- in Provision token
 - rename cryptoMetaData to channelSecurityContext
 - ssdData.VMPAVersion is now required and no longer conditional.
 - ssdData.sdScript became conditional and is no longer optional.
 - ssdData.CASDCert is now conditional. Size increases to 4K.
 - sddData.vCertificateID is now conditional and no longer required.
- In provision token response
 - sdPersoScript renamed to seCardPerso
 - seCardPerso type is Array of APDU JSON
 - Size changed to 12 K
 - Definition of APDU added
- In Confirm Provisioning
 - failureReason became conditional and is no longer optional.
 - apduReasonCode attribute was added to the request payload.
 - apduReasonCode object definition added.
- In provision token
 - Changed asrpd tag to use string value instead of array name value pairs.
- Rename sdKeys to sdScript
- Change scpChannelSecurityContext object name to cryptoMetaData
- In Provision Token requests
 - Added platformType to request
- Change sdScript name to sdKeys in all requests.
- Change sdScript to sdPersoScript in all responses and move into TokenInfo object.
- In *Enroll Device* API
 - Using single encryptionScheme.
 - Decided to use scpChannelSecurityContext in place of CryptoMetadata

- In enroll device
 - Added additional device types and added "OTHER" as osType.
 - Added SCP_02/PULL to encryptionScheme.
- In enroll PAN
 - Added platformType and channelSecurityContext to request body
 - Added aidInfo to response.
- In Provision API's
 - Added sddData to request body and defined SDD Data object.
 - Added sdPersonalizationScript to response body.
- In Confirm Provisioning
 - Changes "api" to conditional in request
- In Get Stepup Methods and get card metadata
 - Added platform type as query parameter
- In provision token
 - Changed asrpd tag to use string value instead of array name value pairs.
- Add getPanData API
- In Payment Instrument
 - Add expDatePrintedInd as Y|N field as optional field
 - Add cvv2PrintedInd as Y|N field as optional field
- In aidInfo
 - Added 9F0A token personalization tag as asrpd.
- In provision token
- Made issuerAuthCode conditional.
- In *Enroll PAN* and *Provision Token Given PAN Data* APIs, made clientDeviceID field conditional from optional and clarified the condition under which it is not required. This change is only at the API specification level and did not change the business logic of these APIs.
- Add getPanData API
- In Payment Instrument
 - Add expDatePrintedInd as Y|N field as optional field
 - Add cvv2PrintedInd as Y|N field as optional field

- In aidInfo
 - Added 9FOA token personalization tag as asrpd.
- In provision token
- Made issuerAuthCode conditional.
- Modified *PAN Source Enum*
 - Renamed "ISSUER" value to "ISSUER_PUSH_PROVISION".
 - Removed "MOBILEAPP" *PAN Source Enum* value.
- Changed "provider" field in: *Encrypted Payment Instrument* to conditional and described the condition.
- Added some more clarifications and references for the "ISSUER" value in *PAN Source Enum*.
- in *Enroll PAN and Provision Token Given PAN Data APIs*
 - changed the size of clientWalletProvider the field from 24 to 50
 - clarified the value of the field to be the Token Requestor Id
- In *Get Payment Data (token & cryptogram) – Given Token ID* API response, clarified the "Visa QR Code Payment Specification", version to 1.02. US34483
- In *Get Payment Data (token & cryptogram) – Given Token ID* API response.
 - Renamed encQr field to encQrDataInfo. : *QR Info*
 - Renamed qrCode to qrData. : *Unencrypted QR code*
- In *Provision Token Given PAN Enrollment ID* and *Provision Token Given PAN Data* API requests, removed the size for presentationType field.
- Extracted a separate *PAN Source Enum* and described different values.
- Added common declaration for enum default size of ASCII 255 characters.
- Clarified *Payment Instrument* type and removed the extra one from *Provision Token Given PAN Enrollment ID* and *Provision Token Given PAN Data APIs*.

Revision History

- Version 3.6, January 26, 2016
- Version 3.9, May 10, 2016
- Version 4.2, August 15, 2016
- Version 4.5, December 27, 2016
- Version 4.6, January 24, 2017
- Version 4.7, February 24, 2017
- Version 4.9, March 31, 2017
- Version 5.0, May 2, 2017
- Version 5.1, May 24, 2017
- Version 5.2, June 28, 2017
- Version 5.3, July 31, 2017
- Version 5.4, August 23, 2017
- Version 5.5, September 27, 2017
- Version 5.6, November 1, 2017
- Version 5.8, January 26, 2018
- Version 6.0, March 16, 2018
- Version 6.1, May 2, 2018
- Version 6.2, May 25, 2018
- Version 6.3, June 19, 2018
- Version 6.4, July 25, 2018
- Version 6.5, August 22, 2018
- Version 6.6, September 26, 2018
- Version 6.7, October 31, 2018
- Version 6.8, December 5, 2018
- Version 19.01, January 23, 2019

- Version 19.02, March 12, 2019
- Version 19.03, March 27, 2019
- Version 19.04, April 16, 2019
- Version 19.05, May 29, 2019
- Version 19.06 June 26, 2019
- Version 19.08, August 21, 2019
- Version 19.09, September 26, 2019
- Version 19.10, October 30, 2019
- Version 20.01, January 22, 2020
- Version 20.03, March 25, 2020
- Version 20.04, May 21, 2020
- Version 20.06, June 24, 2020
- Version 20.07, July 22, 2020
- Version 20.08, August 26, 2020
- Version 20.09, September 29, 2020
- Version 20.10, November 4, 2020
- Version 20.11, December 4, 2020
- Version 21.01, January 27, 2021
- Version 21.02, February 25, 2021
- Version 21.03, March 31, 2021
- Version 21.04, May 5, 2021
- Version 21.05, June 9, 2021
- Version 21.07, July 28, 2021
- Version 21.08, August 25, 2021
- Version 21.10, November 3, 2021
- Version 22.01, January 26, 2022
- Version 22.02, February 23, 2022
- Version 22.04, May 5, 2022
- Version 22.08, August 24, 2022
- Version 22.09, September 28, 2022
- Version 22.10, November 2, 2022

- Version 23.01, January 31, 2023
- Version 23.02, February 22, 2023
- Version 23.03, April 11, 2023
- Version 23.05, June 6, 2023
- Version 23.06, July 5, 2023
- Version 23.07, August 1, 2023
- Version 23.08, August 31, 2023
- Version 23.09, September 26, 2023